

大模型和智能体的算法和工程实践

强化学习、长上下文记忆与智能体系统部署

技术报告（扩展修订版）

许达

未来院三室

April 9, 2026

摘要

当前大语言模型正在从“会生成文本”转向“能完成复杂任务”。这一跃迁带来三个同时上升的要求：更深的推理能力、更长程的上下文与记忆管理能力，以及在真实环境中调用工具、受权限约束执行任务的能力。传统监督微调在人类标注成本、能力上限和分布外泛化上都存在明显瓶颈；强化学习及其相关训练范式，则为模型通过反馈信号持续改进、突破示范上界提供了现实路径。本报告围绕这一主线，尝试把**训练算法、模型架构、推理系统基础设施和智能体工程**四个通常分散讨论的层面统一起来。

在训练算法层面，报告系统梳理策略梯度方法（PPO、GRPO、REINFORCE、RLOO）、偏好优化方法（DPO、Online DPO、KTO、ORPO、SimPO）、自我训练与迭代改进方法（STaR、Quiet-STaR、ReST、ReSTEM、V-STaR、Expert Iteration）、采样筛选方法（拒绝采样微调、RAFT）以及过程奖励方法（PRM、ORM、Math-Shepherd、PRIME）的数学原理、目标函数和工程实现。DeepSeek-R1与OpenAI o1两个代表案例则分别展示了“训练时强化学习扩展”和“推理时计算扩展”这两条能力提升路线。

在模型与系统基础设施层面，报告详细讨论Transformer骨干、自注意力、RoPE/ALiBi位置编码、GQA/MQA/MLA、交叉注意力、多模态统一表示，以及KV Cache、FlashAttention、PagedAttention、Mamba、RWKV、RetNet、Titans、TurboQuant等决定长上下文处理效率与推理吞吐的关键算法。核心目标不是简单罗列模块，而是说明**为什么这些结构会成为今天工业级推理系统的真正瓶颈与杠杆点**。

在智能体工程层面，报告将控制层（harness）视为一类可被形式化分析的系统对象，分别拆解OpenClaw、Claude Code与ChatGPT agent背后的执行抽象、权限模型、工具调用方式与产品化差异，并把审批、隔离、回滚、监控和灰度发布等问题纳入统一的运行时视角。与此相联系，报告提出PACE、TRACE、CRAFT三项原创算法提案，尝试把强化学习从“优化文本输出”进一步推进到“优化权限约束执行、上下文编辑与工件化协作”。

在前沿参照系层面，报告还将GPT-5、o系列、Gemini 3.0 Pro等闭源系统的技术分析，与OpenAI、Anthropic、Google DeepMind的工程组织方式放在同一章中对照，强调今天一线模型能力差距越来越体现为**软件工程闭环、评测基础设施、部署纪律与安全分层能力**。需要特别说明的是，文中关于PACE、TRACE、CRAFT的“原创”仅指其问题定义、状态变量、动作空间和目标函数由本文提出，并不意味着它们已经被公开基准充分验证为成熟方法。报告最后据此给出面向未来三至五年的研究判断与实践建议。

目录

1 引言	11
1.1 研究背景与动机	11
1.2 研究进展时间线	11
1.3 问题定义、目标读者与写作边界	12
1.4 证据类型与阅读提醒	13
1.5 本报告结构	13
1.6 全局结构总表	14
1.7 阅读路径建议	14
1.8 跳读原则与最小先修关系	15
2 强化学习基础理论	15
2.1 马尔可夫决策过程	15
2.2 部分可观测马尔可夫决策过程 (POMDP)	16
2.3 策略梯度方法	16
2.4 值函数估计	16
2.5 广义优势估计 (GAE)	17
2.6 信任域方法	17
2.7 重要性采样与Off-Policy学习	17
2.8 自然梯度与Fisher信息矩阵	17
3 大模型核心架构：Transformer、注意力机制与序列模型	18
3.1 Transformer统一骨干与多模态建模基础	18
3.1.1 自注意力机制的数学原理	18
3.1.2 位置编码：从正弦函数到旋转嵌入 (RoPE) 与ALiBi	19
3.1.3 分组查询注意力 (GQA)、多查询注意力 (MQA) 与DeepSeek 多头潜在注意力 (MLA)	21
3.1.4 层归一化变体：LayerNorm、Pre-LN、RMSNorm与DeepNorm	22
3.1.5 前馈网络 (FFN) 的设计原理	23
3.1.6 交叉注意力机制与条件信息注入	24
3.1.7 模态对齐技术：从表示空间到训练目标	24
3.1.8 Transformer如何成为统一多模态基础架构	25
3.2 长上下文、缓存与前沿系统算法	26
3.2.1 KV Cache的数学原理、内存分析与工程优化	26
3.2.2 StreamingLLM、SnapKV与选择性KV驱逐	27
3.2.3 Infini-attention：压缩记忆与无限上下文	28
3.2.4 隐式注意力的统一数学框架：从线性RNN到状态空间模型	28
3.2.5 Mamba与选择性状态空间模型 (S6)：完整数学推导	29
3.2.6 残差注意力：RealFormer的logit级跳连	32
3.2.7 微软差分注意力 (Differential Attention)：消除注意力噪声	32
3.2.8 RetNet：保留机制、多尺度衰减与训练-推理双范式	34
3.2.9 Google Titans：长期记忆作为测试时可学习模块	36
3.2.10 TurboQuant：极限压缩下的无偏内积估计	37
3.2.11 其他关键系统算法：FlashAttention、Mamba与PagedAttention	38
3.2.12 线性注意力：核函数近似与关联记忆的统一	38
3.2.13 RWKV：纯线性复杂度的可扩展语言模型	39

3.2.14 稀疏注意力与滑动窗口：局部-全局混合策略	40
3.2.15 推理速度与成本：投机解码、MoE细粒化与专用芯片	41
3.2.16 长记忆挑战的两条技术路线综述	43
4 强化学习算法	44
4.1 PPO算法	45
4.1.1 算法动机	45
4.1.2 数学推导	45
4.1.3 完整目标函数	45
4.1.4 算法流程	46
4.1.5 PPO的变体与改进	46
4.1.6 PPO在LLM中的特殊考虑	47
4.2 GRPO算法	47
4.2.1 算法动机	47
4.2.2 数学推导	47
4.2.3 与PPO的对比	48
4.2.4 算法流程	48
4.2.5 GRPO的理论分析	48
4.2.6 GRPO变体	49
4.3 直接偏好优化 (DPO)	49
4.3.1 核心思想	49
4.3.2 DPO目标函数	49
4.3.3 DPO vs RLHF	50
4.4 基于AI反馈的强化学习 (RLAIF)	50
4.4.1 核心框架	50
4.4.2 评判模型设计	50
4.4.3 RLAIF的变体	51
4.5 奖励函数设计	51
4.5.1 可验证奖励	51
4.5.2 格式奖励	51
4.5.3 长度惩罚	51
4.5.4 复合奖励	52
4.5.5 过程奖励与结果奖励的数学分析	52
4.5.6 奖励塑形 (Reward Shaping)	52
4.6 代表案例：DeepSeek-R1纯RL训练范式	52
4.6.1 项目概述	53
4.6.2 训练流程	53
4.6.3 DeepSeek-R1-Zero消融实验	54
4.6.4 性能表现	55
4.6.5 蒸馏到小模型	55
4.7 代表案例：OpenAI o1推理时计算扩展	56
4.7.1 官方披露信息	56
4.7.2 技术推测	56
4.7.3 推理时计算扩展	57
4.7.4 o1与DeepSeek-R1的对比	57
4.8 深度推理范式：System 2思维、过程奖励与树搜索	57
4.8.1 System 1与System 2：直觉与深思的二元结构	58
4.8.2 测试时计算扩展：推理作为第三扩展轴	58

4.8.3	过程奖励模型 (PRM) : 步骤级监督信号	58
4.8.4	蒙特卡洛树搜索 (MCTS) : 结构化推理探索	59
5	其他主要研究方法 with 前沿系统算法	60
5.1	Constitutional AI (Anthropic)	60
5.1.1	核心思想	60
5.1.2	宪法原则示例	60
5.1.3	优势与局限	60
5.2	自博弈与迭代改进	61
5.2.1	基本流程	61
5.2.2	代表工作	61
5.3	Google推理研究	61
5.3.1	Chain-of-Thought Prompting	61
5.3.2	Self-Consistency	61
5.4	STaR: 自教推理器 (Self-Taught Reasoner)	62
5.4.1	核心方法	62
5.4.2	Rationalization的作用	62
5.5	Quiet-STaR: 静默思考	62
5.5.1	核心创新	62
5.5.2	训练目标	63
5.6	ReST: 强化自我训练	63
5.6.1	核心方法	63
5.6.2	与其他方法的关系	63
5.6.3	理论分析	63
5.7	ReST-EM: 期望最大化自我训练	64
5.7.1	EM框架	64
5.7.2	与RLHF的联系	64
5.8	Expert Iteration (ExIt)	64
5.8.1	算法思想	64
5.8.2	在LLM中的应用	64
5.8.3	搜索策略	65
5.9	自奖励语言模型 (Self-Rewarding Language Models)	65
5.9.1	核心创新	65
5.9.2	自评Prompt设计	65
5.9.3	迭代训练	65
5.10	REINFORCE及其变体	66
5.10.1	基础REINFORCE	66
5.10.2	RLOO (REINFORCE Leave-One-Out)	66
5.10.3	与GRPO的关系	66
5.11	RAFT: 奖励排序微调	66
5.11.1	算法流程	66
5.11.2	与拒绝采样的区别	67
5.12	拒绝采样微调 (RSF)	67
5.12.1	方法细节	67
5.12.2	在数学推理中的应用	67
5.13	V-STaR: 验证器增强的自我训练	67
5.13.1	方法改进	67
5.13.2	验证器训练	67

5.13.3联合训练	68
5.14过程奖励模型（PRM）与结果奖励模型（ORM）	68
5.14.1 Let's Verify Step by Step	68
5.14.2 Math-Shepherd	68
5.14.3 PRM vs ORM对比	69
5.15 PRIME：隐式过程奖励	69
5.15.1 核心思想	69
5.15.2 实现方法	69
5.16 Online DPO与迭代偏好优化	69
5.16.1 Online DPO	69
5.16.2 IPO (Identity Preference Optimization)	70
5.17 WizardMath与WizardCoder系列	70
5.17.1 Evol-Instruct方法	70
5.17.2 RLEIF (RL from Evol-Instruct Feedback)	70
5.18 Qwen系列数学与代码模型	70
5.18.1 Qwen2.5-Math	70
5.18.2 训练流程	71
5.19 Marco-o1与MCTS增强推理	71
5.19.1 MCTS在LLM中的应用	71
5.19.2 与其他方法结合	71
5.20 Kimi k1.5与长上下文RL	71
5.20.1 长上下文强化学习	71
5.20.2 技术特点	72
5.21 主要RL训练方法综合对比	72
5.22 其他偏好优化与代码推理方法	72
5.22.1 SPIN: Self-Play Fine-Tuning	72
5.22.2 Iterative DPO	72
5.22.3 KTO: Kahneman-Tversky Optimization	73
5.22.4 ORPO: Odds Ratio Preference Optimization	73
5.22.5 SimPO: Simple Preference Optimization	73
5.22.6 Code Generation: AlphaCode与竞赛编程	73
5.22.7 工具使用与智能体强化学习	73
5.22.8 OpenR: 开源推理框架	74
5.22.9 rStar: 自博弈推理	74
5.22.10 推理时计算扩展 (inference-time scaling)	74
6 运行时原创算法提案	74
6.1 什么样的内容才配叫“原创算法”	75
6.2 从模型中心优化转向运行时中心优化	75
6.3 PACE: 权限感知约束执行优化	76
6.3.1 问题定义	76
6.3.2 策略结构	76
6.3.3 算法规则	76
6.3.4 高效性来源	76
6.3.5 伪代码	77
6.3.6 可落地实现路径	77
6.3.7 为什么它有重大价值	77
6.3.8 可验证假设与实验协议	77

6.4	TRACE: 轨迹后悔感知的上下文编辑	78
6.4.1	问题定义	78
6.4.2	训练机制	78
6.4.3	算法规则	78
6.4.4	高效性来源	79
6.4.5	伪代码	79
6.4.6	可落地实现路径	79
6.4.7	为什么它有重大价值	80
6.4.8	可验证假设与实验协议	80
6.5	CRAFT: 工件图约束下的多智能体协作	80
6.5.1	问题定义	80
6.5.2	奖励设计	81
6.5.3	算法规则	81
6.5.4	高效性来源	81
6.5.5	伪代码	82
6.5.6	可落地实现路径	82
6.5.7	为什么它有重大价值	82
6.5.8	可验证假设与实验协议	82
6.6	为什么只保留这三项	83
6.7	三项高价值原创算法提案之间的关系	83
6.8	理论命题与成立条件	84
6.8.1	PACE的设计命题	84
6.8.2	TRACE的设计命题	84
6.8.3	CRAFT的设计命题	85
6.9	实验统计规范	85
6.9.1	数据切分与泄漏控制	85
6.9.2	统计报告最低标准	86
6.9.3	系统成本与失败分类必须同步报告	86
6.9.4	从离线到在线的推进顺序	86
6.10	复杂度与系统开销	87
6.11	Benchmark设计	87
6.12	消融实验设计	87
6.13	最小可投稿实验包	88
6.14	证伪标准与失败信号	88
6.15	实验路线建议	89
7	实现细节与训练技巧	89
7.1	数据准备	89
7.1.1	数学数据来源	89
7.1.2	代码数据来源	89
7.1.3	数据质量控制	90
7.1.4	数据增强	90
7.2	采样策略	90
7.2.1	温度采样	91
7.2.2	Top-p采样 (Nucleus Sampling)	91
7.2.3	Top-k采样	91
7.2.4	Best-of-N (Rejection Sampling)	91
7.2.5	束搜索变体	92

7.3	训练稳定性	92
7.3.1	梯度管理	92
7.3.2	学习率调度	92
7.3.3	奖励归一化	93
7.3.4	KL散度控制	93
7.4	分布式训练	93
7.4.1	数据并行	94
7.4.2	张量并行 (Tensor Parallelism)	94
7.4.3	流水线并行 (Pipeline Parallelism)	94
7.4.4	采样与训练分离	94
7.5	内存优化	95
7.5.1	梯度检查点 (Gradient Checkpointing)	95
7.5.2	混合精度训练	95
7.5.3	ZeRO优化	95
7.5.4	LoRA/QLoRA	95
7.6	超参数配置指南	96
7.6.1	GRPO推荐配置	96
7.6.2	PPO推荐配置	96
7.7	监控与调试	96
7.7.1	关键指标监控	96
7.7.2	异常检测	97
7.7.3	调试技巧	97
7.8	常见失败模式与经验教训	97
7.8.1	常见问题与解决	99
7.8.2	规模化中的挑战	102
8	智能体系统	103
8.1	控制层 (harness) 的形式化定义	103
8.2	控制层 (harness) 的理论深化: 约束POMDP与层级控制	104
8.3	控制层 (harness) 中的工具调用、审批与安全边界	104
8.4	OpenClaw专章: 从Gateway控制总线到可自托管智能体操作面	105
8.4.1	系统定位: 它不是bot, 而是control fabric	105
8.4.2	控制平面: Gateway是整个系统的中枢神经	105
8.4.3	会话与路由平面: OpenClaw真正做对了什么	106
8.4.4	执行平面: browser、node与system.run的统一抽象	106
8.4.5	自动化平面: cron、heartbeat、webhook与hooks不是附属功能	107
8.4.6	多智能体: 它已经不是单代理, 但还不是最终形态	107
8.4.7	安全模型: 它的成熟度高于很多开源agent项目	108
8.4.8	为什么它好: 四个工程优点	108
8.4.9	主要不足: 复杂度、信任边界与协作形态仍待升级	109
8.4.10	如何定制OpenClaw: 最有杠杆的六个旋钮	109
8.4.11	如何把它继续开发成真正的多智能体系统	110
8.4.12	如何把它继续开发成全自动系统	110
8.4.13	结论: OpenClaw最有价值的不是某个功能, 而是一种架构取向	110
8.5	Claude Code源码快照: 设计、工作机制与扩展路径	111
8.5.1	总体结构: 它到底由哪些层组成	111
8.5.2	它是如何工作的: 从启动到单轮执行	111

8.5.3	系统设计深剖：控制平面、状态平面、执行平面与扩展平面	112
8.5.4	为什么这种设计好	113
8.5.5	从源码抽象出的七条设计原则	114
8.5.6	主要不足：不是功能不够，而是复杂度进入了新的层级	115
8.5.7	如何定制：命令、工具、Agent、Hook、MCP与记忆	116
8.5.8	如何继续开发成真正的多智能体系统	117
8.5.9	如何把它继续开发成全自动系统	119
8.5.10	从Claude Code到企业级自治平台的递进蓝图	120
8.5.11	它还能怎样改进	121
8.6	Claude Code与OpenClaw的结构对照	122
8.7	Claude Code、OpenClaw与ChatGPT agent的三方对照	124
8.8	智能体与工具使用	125
8.8.1	工具增强推理	125
8.8.2	多智能体推理	125
9	前沿闭源模型与工程实践	126
9.1	资料来源与证据分层	126
9.2	OpenAI GPT-5系列技术分析	126
9.2.1	GPT-5训练Pipeline推测	126
9.2.2	GPT-5性能预测Scaling技术	128
9.3	OpenAI o系列推理模型技术分析	128
9.3.1	o系列的RL训练范式	128
9.3.2	o3/o4-mini的技术突破	129
9.3.3	o系列的过程奖励模型（PRM）深度分析	129
9.3.4	o系列的思维链隐藏策略分析	130
9.3.5	Deliberative Alignment: o系列的重要安全训练方法	130
9.3.6	o1 System Card深度技术分析	132
9.3.7	OpenAI "Let's Verify Step by Step" 论文：PRM的奠基工作	135
9.3.8	Anthropic Constitutional AI: RLAIIF的开创性工作	136
9.4	Google Gemini 3.0 Pro技术深度剖析	137
9.4.1	Gemini 3.0 Pro架构特点	137
9.4.2	Gemini 3.0 Pro训练技术推测	138
9.4.3	Gemini 3.0 Deep Think模式分析	138
9.4.4	Gemini技术报告(arXiv:2312.11805)核心内容分析	139
9.5	推理时计算扩展的技术深度分析	140
9.5.1	推理时计算扩展的理论基础	140
9.5.2	推理时计算的不同策略	140
9.5.3	思维链长度与性能的关系	140
9.5.4	推理时计算扩展的局限性	140
9.6	工具使用RL训练的技术细节	140
9.6.1	工具使用的RL形式化	141
9.6.2	奖励函数设计	141
9.6.3	训练数据构建	141
9.7	多模态思维链的技术突破	141
9.7.1	传统多模态处理	141
9.7.2	图像融入思维链	142
9.8	闭源模型安全训练的深层技术	142
9.8.1	红队测试与迭代改进	142

9.8.2 安全性与能力的权衡	143
9.9 闭源模型训练的关键技术要点	143
9.9.1 关键点1: 大规模高质量数据	143
9.9.2 关键点2: 精细化的RL奖励设计	143
9.9.3 关键点3: 训练稳定性工程	143
9.9.4 关键点4: 推理时计算的最优分配	144
9.9.5 关键点5: 多模态对齐	144
9.10对开源社区的启示	144
9.11关键论文与技术资源汇总	145
9.11.1 OpenAI核心论文	145
9.11.2 Anthropic核心论文	145
9.11.3 Google DeepMind核心论文	146
9.11.4 技术博客和System Card	146
9.12技术术语对照表	147
9.13为什么一线公司的优势越来越像软件工程优势	147
9.14统一的软件工程闭环: 从需求到上线后的学习	147
9.15三家公司的工程实践对照	148
9.16OpenAI: 评测驱动发布、风险分级与受控执行环境	148
9.16.1 经验一: 先定义发布门槛, 再谈功能上线	148
9.16.2 经验二: 智能体能力必须运行在受控环境中	149
9.16.3 经验三: 系统卡本质上是工程接口文档	149
9.16.4 经验四: 从《A practical guide to building agents》提炼的落地方法论	149
9.17Anthropic: 简单可组合架构、工具契约与上下文工程	151
9.17.1 经验一: 先用简单可组合模式, 不要先上复杂框架	151
9.17.2 经验二: 工具设计本身就是软件接口设计	152
9.17.3 经验三: 上下文工程比提示词技巧更重要	152
9.18Google DeepMind: 自动评测器、虚拟执行环境与责任分层	153
9.18.1 经验一: 把智能体执行放进虚拟环境, 而不是直接碰真实世界	153
9.18.2 经验二: 自动评测器是把研究变成工程的桥梁	153
9.18.3 经验三: 安全应该是分层的, 而不是单点的	153
9.18.4 经验四: 用“Agentic Design Patterns”把架构选择模式化	154
9.19可直接借鉴的一线软件开发经验	156
9.20对我们实际开发的建议	156
10未来研究方向	156
10.1 过去一年(2025年4月7日至2026年4月7日)的关键论文更新	157
10.1.1 这一年最核心的五个判断	158
10.1.2 这些论文对本书其余章节的直接影响	159
10.2 优先级判断标准	159
10.3 算法改进	159
10.3.1 更高效的RL算法	160
10.3.2 更好的奖励设计	160
10.3.3 混合方法	160
10.4 推理时计算扩展	161
10.4.1 高效搜索算法	161
10.4.2 动态计算分配	161
10.4.3 推理优化方法	161

10.5 多模态推理	161
10.5.1 视觉推理	162
10.5.2 跨模态推理	162
10.6 理论研究	162
10.6.1 推理能力的本质	162
10.6.2 尺度定律 (Scaling Laws)	162
10.6.3 泛化理论	163
10.6.4 后Transformer突破：结构判断与技术预测	163
10.6.5 更激进的基础设施假说：四种超越现有神经骨干的方向	165
10.7 安全与对齐	168
10.7.1 安全推理	168
10.7.2 对齐挑战	168
10.8 开放问题	168
10.9 BitNet：1比特大语言模型	169
10.9.1 BitNet技术概述	169
10.9.2 训练与推理流程	171
10.9.3 性能对比分析	172
10.9.4 与传统量化方法对比	174
10.9.5 硬件优化与未来展望	174
10.9.6 官方模型资源	175
10.9.7 小结	175
11 总结	176
11.1 核心结论	176
11.2 本报告的主要贡献	176
11.3 实践建议	177
11.4 90天落地清单	177
11.5 未来三年的执行路线图	178
11.6 未来发展趋势与研究路线图	179

1 引言

1.1 研究背景与动机

大语言模型在自然语言处理领域取得了突破性进展，但在复杂推理任务上仍存在明显局限。传统的监督微调（Supervised Fine-Tuning, SFT）方法面临几方面难以回避的瓶颈。首先是标注成本：高质量的推理数据——尤其是数学、代码竞赛和多步逻辑推理——需要具备专业背景的标注者逐条撰写，成本高且很难规模化到数十万乃至百万条。其次是能力上限：监督学习本质上是模仿，模型最多只能复现标注者已知的解法，无法发现标注者本身不会的新策略。第三是泛化不足：基于模仿学习训练的模型在遇到分布外（Out-of-Distribution）问题时往往表现急剧下降，因为它只学到了“该怎么做”，而没有学到“为什么这样做”。最后是标注质量本身的不稳定性：对于多步推理任务，标注者之间的一致性远低于简单文本分类，错误的中间步骤会将模型引入错误的推理习惯。

强化学习为上述瓶颈提供了一条根本不同的路径。在RL框架下，模型不再被要求模仿某个固定的标注分布，而是通过与环境交互（例如尝试解题并获得正确/错误反馈）来自主探索解题策略。这意味着训练过程不依赖逐条的人工标注，因此具备天然的可扩展性；模型可以在探索中发现人类标注者未曾给出的新解法，从而突破监督学习的能力天花板；而且通过反复的策略改进-反馈-再改进循环，模型有可能持续自我提升，而非停留在某个固定的性能平台上。

1.2 研究进展时间线

强化学习在LLM训练中的应用经历了以下关键阶段：

Table 1: LLM强化学习训练里程碑

时间	工作	主要贡献
2022.03	InstructGPT	首次大规模应用RLHF，开创指令跟随范式
2022.05	STaR	提出自我迭代推理训练范式
2022.09	SPIN	自博弈微调方法
2022.12	Constitutional AI	提出RLAIF，用AI反馈替代人类反馈
2023.02	AlphaCode	大规模采样+聚类用于竞赛编程
2023.05	Let' s Verify	系统研究过程奖励模型(PRM)在数学推理中的应用
2023.06	WizardMath	Evol-Instruct + RLEIF方法
2023.08	ReST	Google提出强化自我训练，迭代改进模型
2023.09	WizardCoder	代码生成的强化学习优化
2023.10	ReST-EM	结合期望最大化的自我训练方法
2023.12	Self-Rewarding LM	Meta提出自奖励语言模型，无需外部奖励
2024.01	Quiet-STaR	隐式思考链，每个token位置进行内部推理
2024.01	KTO	基于前景理论的偏好优化
2024.02	DeepSeekMath	提出GRPO算法，简化RL训练流程
2024.03	RAFT/RSF	奖励排序微调和拒绝采样方法
2024.03	ORPO	统一SFT和偏好优化
2024.04	SimPO	简化DPO，移除参考模型
2024.05	V-STaR	结合验证器的自我训练
2024.06	Math-Shepherd	过程奖励模型用于数学推理
2024.07	RLOO	REINFORCE Leave-One-Out基线优化
2024.08	PRIME	隐式过程奖励学习
2024.08	OpenR	开源推理模型训练框架
2024.09	OpenAI o1	推理模型范式，展示推理时scaling
2024.10	Qwen2.5-Math	结合多种RL方法的数学推理模型
2024.11	Marco-o1	阿里开源推理模型，MCTS增强
2024.12	rStar	自博弈互相推理方法
2025.01	DeepSeek-R1	首个开源纯RL推理模型，涌现长链推理能力
2025.01	Kimi k1.5	Moonshot AI推理模型，长上下文RL

1.3 问题定义、目标读者与写作边界

本报告试图回答的不是某一个局部问题，而是一个更完整的问题链：**如果目标是构建能够长期运行、能做复杂推理、能安全调用工具并持续改进的大模型系统，那么训练算法、模型结构、推理基础设施与智能体运行时之间到底如何相互制约？**这也是全文采用“从数学对象到系统对象”这一组织方式的原因。

目标读者主要有三类。第一类是希望理解RL训练与偏好优化核心原理的研究者；第二类是需要把长上下文推理、缓存和吞吐问题落到工程系统中的基础设施工程师；第三类是正在搭建代码代理、浏览器代理或企业工作流代理的智能体系统设计者。全文默认读者已经具备基本的深度学习与Transformer背景，因此不会把篇幅花在入门性概念上，而是更强调**原理之间的连接关系、设计取舍和工程后果**。

写作边界同样需要明确。第一，本文不是一份“所有相关论文的穷举综述”，而是按问题链进行有选择的组织；凡是不直接服务于“推理、长上下文、工具使用、运行

时控制”主线的工作，都会被主动压缩。第二，本文中的PACE、TRACE、CRAFT被严格表述为**原创算法提案**，不是已经经过公开基准充分验证的定型方法。第三，文中对闭源模型的分析包含一定逆向推断成分，因此其价值主要在于帮助读者形成工程判断框架，而不是把猜测当成公开事实复述。

1.4 证据类型与阅读提醒

为了避免把不同性质的材料混写为同一种“事实”，全文默认将论述分为三类。第一类是**可直接复核的内容**，包括公开论文、技术报告、系统卡、开源代码和可重复的数学推导；这类内容应被视为全文中最稳固的证据层。第二类是**基于多个公开来源交叉整理的工程归纳**，例如对训练流程、部署纪律和组织方式的总结；这类内容通常不是逐句摘录自单一来源，而是面向工程决策的抽象。第三类是**逆向推断与前瞻判断**，主要出现在闭源系统分析、未来方向和部分系统设计比较中；这类内容的价值在于提供假设空间、比较框架和验证方向，而不应被当作官方披露事实照单接受。

阅读时可据此采取不同标准：对公式、算法定义和开源实现，优先关注其正确性与可复现性；对工程归纳，关注它是否能解释跨系统共性；对闭源模型和未来趋势判断，则更应关注其是否给出了清晰前提、可证伪假设和与公开材料的一致性。尤其在第9节，凡涉及闭源模型训练细节的部分，默认都应按“**基于公开材料的技术分析与推断**”理解。

1.5 本报告结构

本报告按照“先建立数学语言，再进入系统结构，最后回到工程判断”的顺序展开。第2节给出强化学习基础理论，用MDP、POMDP、策略梯度、GAE、信任域、重要性采样和自然梯度建立统一术语。第3节转入大模型核心架构，集中讨论Transformer、自注意力、位置编码、归一化、长上下文缓存与一系列后Transformer序列模型，目的是解释**推理能力与系统效率究竟由哪些底层结构共同决定**。第4节进入强化学习算法主线，重点讲解PPO、GRPO、DPO、RLAIF和奖励设计，并用DeepSeek-R1与OpenAI o1把“训练时RL”和“推理时计算”两条路线收束到同一个比较框架中。

在此之后，第5节向主线外展开，组织其他重要自训练、偏好优化、采样筛选与过程奖励工作，帮助读者建立更完整的技术地图。第6节则把视角进一步推向真实运行时，保留PACE、TRACE、CRAFT三项原创算法提案，讨论权限感知执行、上下文压缩和工件化协作这些传统论文写法中常被边缘化、但实际系统中极其关键的问题。第7节承接算法章节，专门讨论数据构造、并行训练、显存与吞吐优化、失败模式和调参方法。第8节是独立的智能体系统章，从控制层（harness）的形式化定义出发，对OpenClaw、Claude Code、ChatGPT agent等系统做结构对照。第9节把前沿闭源模型技术判断与一线团队的软件工程实践合并讨论。第10节给出未来研究方向，包括过去一年（2025年4月7日至2026年4月7日）的关键论文更新与BitNet专题。第11节最后回到全书核心结论与可执行建议。

1.6 全局结构总表

Table 2: 全文结构与阅读目标对应关系

节号	主题	核心问题
1	引言	为什么需要把RL训练、架构理论、长上下文系统和智能体工程放在同一篇报告里？
2	强化学习基础理论	LLM中的MDP、策略梯度、GAE、KL约束等基本数学对象是什么？
3	大模型核心架构：Transformer、注意力机制与序列模型	Transformer 各模块的完整数学推导是什么？RoPE、GQA/MLA、Mamba、RetNet、RWKV、差分注意力等前沿架构如何工作？
4	强化学习算法	PPO、GRPO、DPO、RLAIF、奖励设计及R1/o1代表案例在公式层面如何工作？
5	其他主要研究方法与前 沿系统算法	除主线方法外，还有哪些自训练方法和前沿工作值得掌握？
6	运行时原创算法提案	本文保留的三项高价值原创算法提案分别试图解决哪些现有瓶颈？
7	实现细节与训练技巧	如何把算法落实到训练系统？常见失败模式如何识别与修复？
8	智能体系统	控制层（harness）理论、OpenClaw、Claude Code与多智能体的工程设计是什么？
9	前沿闭源模型与工程实 践	GPT-5/o系列/Gemini等前沿系统可从公开材料中分析出哪些技术特征？顶尖团队的工程管理有哪些共性做法？
10	未来研究方向	接下来最值得投入的算法、多模态和安全问题是什么？含BitNet 1比特大语言模型。
11	总结	从研究到工程，哪些结论最值得直接执行？

1.7 阅读路径建议

由于本文覆盖训练算法、架构理论、系统基础设施与智能体工程四条主线，最有效的阅读方式不是线性通读，而是按问题进入。

如果主要关心“模型为什么能推理”：从第2节和第4节开始，先建立RL训练与奖励设计的数学框架，再回看DeepSeek-R1、o1、PRM和MCTS相关章节，这条路径最适合研究训练机制的读者。

如果主要关心“系统为什么跑得快”：直接进入第3节，重点阅读长上下文、缓存与前沿系统算法部分。这里讨论的不是抽象架构美感，而是KV Cache、带宽、吞吐、显存和延迟这些工业系统中真正决定上限的约束。

如果主要关心“智能体为什么经常在真实环境里失控或失效”：优先阅读第8节和第9节，再回看第6节的PACE、TRACE、CRAFT。这样读可以先建立运行时控制面的直觉，再理解为什么权限、上下文压缩与工件协作需要进入学习目标，而不能只当成工程补丁。

如果主要关心“应该先做什么工程实践”：阅读第7节和第11节。前者给出实现细节、资源约束与常见失败模式，后者把全书压缩成能直接执行的判断和建议。

如果时间有限：建议先读本节、再读第3节的长上下文部分、第4节的PPO/GRPO与R1/o1案例、第8节的智能体系统分析，最后回到第11节收束观点。

1.8 跳读原则与最小先修关系

把本文当作“工具书”使用是合理的，但仍有一些最小先修关系值得遵守。否则最常见的结果不是“少读了几页”，而是把不同证据层级、不同系统层级的问题混在一起，最后既难以下判断，也难以落地。

Table 3: 按任务跳读时的最小先修关系

当前目标	建议先读	随后重点阅读	最应避免的误区
训练可验证推理模型	第2节、第4节	第5节、第7节、第11节	只盯算法名词，不看奖励稳定性、采样成本和评测污染
优化长上下文与吞吐	第3节前半、第3节长上下文部分	第7节、第9节相关系统分析	把窗口长度当成唯一指标，忽视缓存、带宽和记忆淘汰策略
构建代码/浏览器/企业工作流智能体	第8节、第7节	第6节、第9节、第11节	先放开自主度，再事后补审批、隔离和回滚
阅读闭源前沿系统分析	第1节“证据类型与阅读提醒”	第9节、第11节	把工程推断或逆向分析误当成官方已披露事实

一个更实用的经验是：**遇到公式再回第2节，遇到缓存与架构瓶颈再回第3节，遇到落地和组织问题则优先回第7节、第8节和第11节。**这样读通常比线性通读更高效，也更符合工程团队的使用方式。

2 强化学习基础理论

本节建立后续算法讨论所需的数学语言。从马尔可夫决策过程的形式化定义出发，依次介绍策略梯度定理、值函数、广义优势估计（GAE）、信任域方法、重要性采样和自然梯度，为第4节及之后涉及RL训练的具体算法提供统一的理论基础。

2.1 马尔可夫决策过程

强化学习问题通常建模为马尔可夫决策过程（Markov Decision Process, MDP），形式化定义为五元组 (S, A, P, R, γ) ：

定义 2.1 (马尔可夫决策过程). MDP由五个要素组成：状态空间 S 描述系统可能处于的所有状态；动作空间 A 描述每个状态下可选择的动作；状态转移概率 $P: S \times A \times S \rightarrow [0, 1]$ 给出在状态 s 下执行动作 a 后转移到状态 s' 的概率；奖励函数 $R: S \times A \rightarrow \mathbb{R}$ 定义每步的即时反馈；折扣因子 $\gamma \in [0, 1]$ 控制未来奖励相对于当前奖励的权重。

在LLM场景下，上述要素有着非常直接的对应关系。状态 $s_t = (x_1, x_2, \dots, x_{n+t})$ 就是当前已生成的token序列，其中 $x_1, \dots, x_n$ 为prompt， $x_{n+1}, \dots, x_{n+t}$ 为已生成的tokens。动作 $a_t \in \mathcal{V}$ 是下一个要生成的token， $\mathcal{V}$ 为词汇表。状态转移是确定性的，即 $s_{t+1} = s_t \oplus a_t$ （序列拼接）。奖励通常在序列结束时一次性给予（稀疏奖励），即 $r_t = 0$ for $t < T$, $r_T = R(s_T)$ 。终止条件为生成EOS token或达到最大长度。

2.2 部分可观测马尔可夫决策过程 (POMDP)

在实际LLM应用中，完全MDP假设可能过于理想化。更精确的建模应考虑部分可观测性：

定义 2.2 (POMDP扩展). POMDP定义为七元组 $(S, \mathcal{A}, P, R, \Omega, O, \gamma)$ ，在MDP的基础上额外引入观测空间 Ω 和观测函数 $O : S \times \mathcal{A} \rightarrow \Delta(\Omega)$ 。智能体并不直接看到真实状态 s ，而只能获得一个可能带噪声的观测 $o \in \Omega$ ，因此需要在不完全信息下做出决策。

对于LLM，POMDP视角捕捉了以下现实约束：

- 模型无法直接观测用户真实意图
- 问题的完整上下文可能超出上下文窗口（context window）
- 外部世界状态（如数据库、API）不完全可知

2.3 策略梯度方法

策略梯度方法直接优化参数化策略 π_θ ，目标是最大化期望累积奖励：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (1)$$

其中 $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ 表示一条轨迹。

定理 2.3 (策略梯度定理). 策略梯度可表示为：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t \right] \quad (2)$$

其中 $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ 是从时刻 t 开始的累积回报。

为降低方差，通常引入基线函数 $b(s_t)$ ，使用优势函数（Advantage Function）：

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t) \quad (3)$$

其中 $Q^\pi(s, a)$ 是动作价值函数， $V^\pi(s)$ 是状态价值函数。

2.4 值函数估计

状态价值函数定义为从状态 s 出发，遵循策略 π 的期望回报：

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \quad (4)$$

动作价值函数定义为：

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right] \quad (5)$$

两者满足贝尔曼方程：

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a) \quad (6)$$

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \quad (7)$$

2.5 广义优势估计 (GAE)

广义优势估计 (Generalized Advantage Estimation) 是一种高效的優勢函数估计方法，平衡偏差和方差：

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \quad (8)$$

其中 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 是时序差分 (TD) 误差。

λ 参数控制偏差-方差权衡：当 $\lambda = 0$ 时退化为单步TD估计，方差低但偏差大；当 $\lambda = 1$ 时等价于蒙特卡洛估计，偏差低但方差大；实际中取 $\lambda \in (0, 1)$ 的折中值（通常0.95左右），在两者之间取得平衡。

2.6 信任域方法

信任域方法是一类重要的策略优化算法，核心思想是在每次更新时限制策略变化幅度。

定义 2.4 (信任域策略优化). 信任域方法的目标是：

$$\max_{\theta} \mathbb{E}_{s \sim \rho_{\theta_{old}}, a \sim \pi_{\theta_{old}}} \left[\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)} A^{\pi_{\theta_{old}}}(s, a) \right] \quad (9)$$

约束条件：

$$\mathbb{E}_{s \sim \rho_{\theta_{old}}} [\mathbb{D}_{KL}[\pi_{\theta_{old}}(\cdot|s) \parallel \pi_{\theta}(\cdot|s)]] \leq \delta \quad (10)$$

2.7 重要性采样与Off-Policy学习

重要性采样允许使用旧策略采集的数据更新新策略：

$$\mathbb{E}_{a \sim \pi_{\theta_{old}}} [f(a)] = \mathbb{E}_{a \sim \pi_{\theta}} \left[\frac{\pi_{\theta_{old}}(a)}{\pi_{\theta}(a)} f(a) \right] \quad (11)$$

方差分析：重要性采样的方差为：

$$\text{Var} \left[\frac{\pi_{\theta_{old}}(a)}{\pi_{\theta}(a)} f(a) \right] = \mathbb{E} \left[\left(\frac{\pi_{\theta_{old}}(a)}{\pi_{\theta}(a)} \right)^2 f(a)^2 \right] - (\mathbb{E}[f(a)])^2 \quad (12)$$

当 $\pi_{\theta_{old}}$ 与 π_{θ} 差异过大时，方差会急剧增加，这是限制策略变化的核心原因。

2.8 自然梯度与Fisher信息矩阵

自然梯度考虑参数空间的几何结构：

定义 2.5 (Fisher信息矩阵).

$$F_{\theta} = \mathbb{E}_{s,a} [\nabla_{\theta} \log \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s)^T] \quad (13)$$

自然梯度为：

$$\tilde{\nabla}_{\theta} J(\theta) = F_{\theta}^{-1} \nabla_{\theta} J(\theta) \quad (14)$$

自然梯度的优势在于它对参数化方式不变——无论用什么方式参数化同一个策略，自然梯度指向的方向都是一致的；同时它更准确地反映了策略空间的几何结构，从而使训练过程更加稳定，避免因参数空间中的微小变化导致策略空间中的剧烈波动。

3 大模型核心架构：Transformer、注意力机制与序列模型

本章系统梳理大模型的核心计算基础——从原始Transformer的自注意力、位置编码、层归一化，到现代工程实践中普遍采用的GQA/MLA、RoPE、RMSNorm，再到以KV Cache为核心的长上下文系统算法，以及Mamba（选择性状态空间）、Differential Attention（差分注意力）、RetNet（保留机制）等新型序列模型。这些内容是后续章节中所有训练算法、推理优化与智能体系统的共同数学底座。无论是在阅读PPO/GRPO/DPO等RL训练方法，还是在设计推理加速系统或智能体架构时，本章均可作为参考手册随时查阅。

3.1 Transformer统一骨干与多模态建模基础

本小节补足一个在工程实践中经常被默认、但实际上决定系统上限的事实：无论是纯文本LLM、视觉语言模型，还是带工具调用与外部记忆的复杂系统，其核心计算骨架通常仍然是Transformer。只要能把不同模态表示为一组可比较、可路由、可复用的token或latent token，同一套“注意力 + 前馈网络 + 残差归一化”机制就能同时承担序列建模、跨模态检索与条件生成三类任务。因此，要真正理解多模态系统，必须先从自注意力、FFN、交叉注意力和模态对齐的数学结构入手。

3.1.1 自注意力机制的数学原理

设输入序列表示为矩阵

$$X = [x_1; x_2; \dots; x_n] \in \mathbb{R}^{n \times d}. \quad (15)$$

自注意力的第一步是对同一组token分别做三组线性投影：

$$Q = XW_Q \in \mathbb{R}^{n \times d_k}, \quad K = XW_K \in \mathbb{R}^{n \times d_k}, \quad V = XW_V \in \mathbb{R}^{n \times d_v}, \quad (16)$$

其中 $W_Q, W_K \in \mathbb{R}^{d \times d_k}$ ， $W_V \in \mathbb{R}^{d \times d_v}$ 。随后通过查询与键的内积构造两两相关性：

$$S = \frac{QK^\top}{\sqrt{d_k}} + M \in \mathbb{R}^{n \times n}, \quad (17)$$

其中 $M \in \mathbb{R}^{n \times n}$ 是掩码矩阵；在双向编码器中 M 可为零，在因果解码器中 $M_{ij} = -\infty$ （当 $j > i$ ）以禁止看到未来token。 S 矩阵的第 (i, j) 元素 $S_{ij} = q_i^\top k_j / \sqrt{d_k} + M_{ij}$ 称为**注意力分数（attention score）**，它衡量的是位置 i 的查询与位置 j 的键之间的匹配程度。除以 $\sqrt{d_k}$ 是为了防止内积值随维度增大而增大——当 q 和 k 的各分量独立采样自均值为0、方差为1的分布时， $q^\top k$ 的方差为 d_k ，除以 $\sqrt{d_k}$ 后方差回到 $O(1)$ ，避免softmax进入梯度消失的饱和区。

经softmax按行归一化后得到注意力权重：

$$A = \text{softmax}(S) \in \mathbb{R}^{n \times n}, \quad (18)$$

其中每一行 $A_{i,:}$ 是一个概率分布，满足 $\sum_j A_{ij} = 1$ 。直观地说，若注意力分数 S_{ij} 远大于同行中其他位置的分数，则 A_{ij} 趋近于1，输出几乎只读取位置 j 的信息（“尖锐”注意力）；若所有分数接近相等，则 $A_{ij} \approx 1/n$ ，输出退化为对所有位置的均匀平均（“平坦”注意力）。最终输出为

$$H = AV \in \mathbb{R}^{n \times d_v}. \quad (19)$$

这一机制的关键不在于“平均历史信息”，而在于它实现了**内容依赖的动态路由**：第 i 个位置到底从哪些位置读取信息、读取多少，不是由固定卷积核决定，而是由 $q_i^\top k_j$ 在线计算出来。若第 i 个token与第 j 个token在当前任务中高度相关，则 A_{ij} 会变大；反之则自动衰减。因此，自注意力本质上是在序列内部学习一个随输入变化的完全图权重矩阵。

多头注意力（Multi-Head Attention）进一步把这一过程并行化。对第 h 个头有：

$$\text{head}_h = \text{softmax} \left(\frac{XW_Q^{(h)}(XW_K^{(h)})^\top}{\sqrt{d_k}} \right) XW_V^{(h)}, \quad (20)$$

再将各头拼接并线性映射：

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O. \quad (21)$$

多头结构的数学价值在于，它允许模型在不同子空间中同时学习**语法依赖、实体对齐、位置关系、视觉局部模式与跨句约束**等不同类型的相关性，而不是把全部关系压缩到一个单一相似度度量里。

3.1.2 位置编码：从正弦函数到旋转嵌入（RoPE）与ALiBi

标准自注意力本身对输入排列是置换等变的——将序列任意重排，注意力权重矩阵只是被同样重排，模型无法感知位置。因此必须通过**位置编码**将时序结构注入 Q, K, V 。

绝对正弦位置编码（Vaswani et al., 2017） 原始Transformer将位置向量直接叠加到输入嵌入上，对位置 pos 与维度索引 i （ $0 \leq 2i < d$ ）定义：

$$\text{PE}(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad \text{PE}(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right). \quad (22)$$

不同维度对应不同频率，形成从高频到低频的正弦波族，使任意两个位置的差 $\text{PE}(pos+k)$ 可写成 $\text{PE}(pos)$ 的线性变换，理论上允许模型通过注意力权重学习相对关系。但实践中，注意力分数 $q_m^\top k_n$ 无法干净地分解出相对位置差 $m-n$ ，模型需要额外学习这一隐式关系，限制了外推能力。

旋转位置嵌入（RoPE, Su et al., 2021） RoPE 是目前工业界最主流的位置编码，被LLaMA全系列、GPT-NeoX、PaLM、Gemini、Qwen、DeepSeek等几乎所有现代大模型采用。其核心思路是**将位置信息编码为 q, k 向量在复数平面上的旋转相位**，从而使注意力分数天然依赖相对位置差，而非绝对位置。

设计目标的形式化：我们希望找到对 q_m （位置 m 的查询）和 k_n （位置 n 的键）的函数 f_q, f_k ，使得其内积只依赖相对位置差 $m-n$ ：

$$\langle f_q(x_m, m), f_k(x_n, n) \rangle = g(x_m, x_n, m-n). \quad (23)$$

Su等证明，在2维空间中，满足上述性质的一类解为复数乘法：

$$f_q(x_m, m) = (W_q x_m) e^{im\theta}, \quad f_k(x_n, n) = (W_k x_n) e^{in\theta}, \quad (24)$$

其内积为实部（注意复数内积取实部）：

$$\text{Re}[(W_q x_m) e^{im\theta} \overline{(W_k x_n) e^{in\theta}}] = \text{Re}[(W_q x_m) \overline{(W_k x_n)} e^{i(m-n)\theta}]. \quad (25)$$

右边确实只含 $m - n$ ，而不含 m, n 各自的绝对值。

扩展到 d 维：对 d 维向量（ d 偶数），将其视为 $d/2$ 个独立的二维子空间，每个子空间使用不同的旋转频率 θ_j ：

$$\theta_j = \frac{1}{10000^{2j/d}}, \quad j = 0, 1, \dots, \frac{d}{2} - 1. \quad (26)$$

对位置 m 处的查询向量 $q \in \mathbb{R}^d$ ，第 j 个二维子向量旋转角度 $m\theta_j$ ：

$$\tilde{q}_{2j:2j+2} = R_{\theta_j}(m) q_{2j:2j+2}, \quad R_{\theta}(m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \in \text{SO}(2). \quad (27)$$

整个旋转操作可写成分块对角矩阵 $\mathcal{R}_m \in \mathbb{R}^{d \times d}$ ：

$$\mathcal{R}_m = \text{diag}(R_{\theta_0}(m), R_{\theta_1}(m), \dots, R_{\theta_{d/2-1}}(m)). \quad (28)$$

相对位置性的严格证明：注意 $\text{SO}(2)$ 是阿贝尔群，旋转满足：

$$R_{\theta}(m)^\top R_{\theta}(n) = R_{\theta}(-m)R_{\theta}(n) = R_{\theta}(n - m). \quad (29)$$

因此，位置 m 的查询与位置 n 的键的内积为：

$$\tilde{q}_m^\top \tilde{k}_n = (R_{\theta}(m)q)^\top (R_{\theta}(n)k) = q^\top \mathcal{R}_m^\top \mathcal{R}_n k = q^\top \mathcal{R}_{n-m} k = \sum_{j=0}^{d/2-1} q_{2j:2j+2}^\top R_{\theta_j}(n - m) k_{2j:2j+2}. \quad (30)$$

内积**严格只依赖** $n - m$ ，不含 m 或 n 的绝对值。

实现效率：注意 $\mathcal{R}_m q$ 不需要显式构造 $d \times d$ 矩阵，等价于以下逐元素运算：

$$\tilde{q} = q \odot \cos(m\theta) + \text{rotate}(q) \odot \sin(m\theta), \quad (31)$$

其中 $\text{rotate}(q)$ 将奇偶分量互换并翻转符号（即 $\text{rotate}([q_0, q_1, \dots]) = [-q_1, q_0, -q_3, q_2, \dots]$ ）。这使 RoPE 可在 $O(d)$ 时间内完成，与嵌入加法相当，且**无需任何额外参数**。

YaRN长上下文扩展 (Peng et al., 2023)：RoPE 的外推局限在于高频维度（小 j ，大 θ_j ）在超出训练长度后会快速振荡，导致模型未见过的相位组合。YaRN 对每个频率维度分别做**插值缩放**：

$$\theta_j \leftarrow \frac{\theta_j}{s_j}, \quad s_j = \begin{cases} 1 & \theta_j \geq \theta_{\text{high}} \text{ (高频, 不缩放)} \\ \lambda & \theta_j \leq \theta_{\text{low}} \text{ (低频, 按扩展比例缩放)} \\ \text{线性插值} & \text{其他} \end{cases} \quad (32)$$

其中 $\lambda = L_{\text{new}}/L_{\text{train}}$ 是目标与原训练长度之比。高频维度编码近程关系，不缩放保证短距离精度；低频维度编码远程关系，按比例缩放允许模型在更长的相对距离上泛化。实验显示，YaRN 以仅 $< 0.1\%$ 的微调步数即可从 4K 上下文扩展到 128K，被 LongRoPE、Qwen2、DeepSeek-V2 等广泛采用。

ALiBi (Press et al., 2022) ALiBi (Attention with Linear Biases) 不修改 q, k 本身，而是对注意力分数直接叠加与距离成正比的负偏置：

$$S_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} - m_h |i - j|, \quad (33)$$

其中 $m_h > 0$ 是第 h 个头的**头专属斜率**（不可学习，按等比数列预设设为 $\{1/2, 1/4, \dots\}$ 等值）。这等效于给“远距离pair”施加线性惩罚，迫使不同头关注不同粒度的依赖范围。由于没有绝对位置参数，ALiBi在训练长度 T 以外的推理中仅需对新的 $|i - j|$ 值直接延伸计算，外推能力极强——实验显示测试长度达 $2T$ 乃至 $4T$ 时困惑度仍可接受。

3.1.3 分组查询注意力 (GQA)、多查询注意力 (MQA) 与 DeepSeek 多头潜在注意力 (MLA)

一个拥有 L 层、 H 头、头维度 d_h 、批量大小 B 、序列长度 T 、字节宽度 b 的模型，其 KV Cache 总显存为：

$$M_{KV}^{MHA} = 2 \cdot B \cdot L \cdot T \cdot H \cdot d_h \cdot b. \quad (34)$$

对 LLaMA-70B ($L = 80, H = 64, d_h = 128, b = 2$)、 $T = 8192$ 时，这一数字约为 **160 GB**——已超过单卡显存上限。KV Cache 压缩因此成为长上下文推理的核心工程问题。

多查询注意力 (MQA, Shazeer, 2019) MQA 的极端压缩：所有 H 个查询头共享同一套 K, V 投影矩阵 $W_K, W_V \in \mathbb{R}^{d \times d_h}$ ：

$$Q_h = XW_Q^{(h)}, \quad K = XW_K, \quad V = XW_V, \quad (35)$$

$$\text{head}_h = \text{softmax}\left(\frac{Q_h K^\top}{\sqrt{d_h}}\right) V. \quad (36)$$

KV Cache 降至 $2 \cdot B \cdot L \cdot T \cdot d_h \cdot b$ ，节省因子为 H （即 1/64 对 LLaMA-70B）。代价是不同查询头读取相同的 KV 空间，削弱了多头并行学习不同关系结构的能力，尤其在精确检索任务上有质量损失。

分组查询注意力 (GQA, Ainslie et al., 2023) GQA 在 MHA 与 MQA 之间取中间值：将 H 个头分为 G 组 ($1 \leq G \leq H$)，每组 H/G 个查询头共享一套 KV：

$$K_g = XW_K^{(g)}, \quad V_g = XW_V^{(g)}, \quad g = 1, \dots, G, \quad (37)$$

$$\text{head}_h = \text{softmax}\left(\frac{Q_h K_{g(h)}^\top}{\sqrt{d_h}}\right) V_{g(h)}, \quad g(h) = \left\lfloor \frac{(h-1)G}{H} \right\rfloor + 1. \quad (38)$$

KV Cache 降至 $2 \cdot B \cdot L \cdot T \cdot G \cdot d_h \cdot b$ ，节省因子为 H/G 。

KV 头上载 (KV head upcasting)：GQA 从 MQA/GQA 权重升级到 MHA 时，可通过复制 KV 权重的方式将 G 组头扩展到 H 组，再微调，是模型升级 (model upscaling) 的常用技巧。

DeepSeek 多头潜在注意力 (MLA, DeepSeek-V2, 2024) MLA 采用与 MQA/GQA 完全不同的思路：**对 KV 做低秩压缩**，而非跨头共享完整 KV。设输入 $h_t \in \mathbb{R}^d$ ，MLA 先投影到一个低维“潜空间” $c_t^{KV} \in \mathbb{R}^{d_c}$ （其中 $d_c \ll Hd_h$ ）：

$$c_t^{KV} = W^{DKV} h_t \in \mathbb{R}^{d_c}, \quad (\text{Down-projection, KV 压缩}) \quad (39)$$

再从潜向量恢复各头的 K、V：

$$k_t^{(h)} = W_K^{(h)} c_t^{KV} \in \mathbb{R}^{d_h}, \quad v_t^{(h)} = W_V^{(h)} c_t^{KV} \in \mathbb{R}^{d_h}. \quad (40)$$

对查询也做类似处理：先压缩再恢复：

$$q_t^Q = W^{DQ} h_t \in \mathbb{R}^{d'_c}, \quad q_t^{(h)} = W_Q^{(h)} c_t^Q \in \mathbb{R}^{d_h}. \quad (41)$$

MLA 只缓存潜向量 c_t^{KV} (而非全部 H 套 K、V)，KV Cache 规模从 $O(Hd_h)$ 降至 $O(d_c)$ 。DeepSeek-V2 的设置为 $d_c = 512$ ，而原始 MHA KV 维度为 $Hd_h = 128 \times 128 = 16384$ ，压缩比约 $32 \times$ 。

RoPE 与 MLA 的兼容处理：由于 RoPE 要求 K 向量携带旋转位置信息（而旋转操作与低秩分解不相容），MLA 对 K 额外追加一个**解耦的 RoPE 分量** k_t^R （不参与低秩压缩，仅用于位置编码的内积计算），从而在保持低秩 KV Cache 的同时引入精确的相对位置信息。

方法	KV Cache	节省因子	典型应用	质量风险
MHA	$2LTHd_hb$	$1 \times$	标准 Transformer	无
MQA	$2LTd_hb$	$H \times$	PaLM, Gemini	中
GQA	$2LTGd_hb$	$H/G \times$	LLaMA-2, Mistral	低
MLA	$2LTd_cb$	$Hd_h/d_c \times$	DeepSeek-V2/V3	极低

进一步的变体**跨层注意力 (CLA)** 让相邻 Transformer 层共享同一套 KV 投影，将总 KV Cache 再压缩 2 - 4 倍，适合显存极度受限的边缘推理场景。

3.1.4 层归一化变体：LayerNorm、Pre-LN、RMSNorm与DeepNorm

标准LayerNorm的完整定义 给定一个 d 维激活向量 $x \in \mathbb{R}^d$ ，LayerNorm 首先计算当前样本在特征维度上的均值与方差（无需跨样本统计，这与BatchNorm本质不同）：

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 + \epsilon, \quad (42)$$

然后中心化并缩放，最后应用可学习的仿射变换：

$$\text{LN}(x) = \gamma \odot \frac{x - \mu \mathbf{1}}{\sigma} + \beta, \quad \gamma, \beta \in \mathbb{R}^d. \quad (43)$$

其中 γ （增益，gain）和 β （偏置，bias）是模型参数，允许网络在规范化后重新缩放。 ϵ （通常取 10^{-5} 或 10^{-6} ）防止除以零。与 BatchNorm 的区别在于：LayerNorm 对**每个 token 独立归一化**（在特征维度上），因此不依赖批量大小，适合可变长度序列与小批量训练。

Post-LN（原始Transformer）与梯度爆炸问题 原始 Transformer (Vaswani et al., 2017) 采用 Post-LN，将归一化放在残差之后：

$$H_\ell^{\text{attn}} = \text{LN}(H_{\ell-1} + \text{Attn}(H_{\ell-1})), \quad H_\ell = \text{LN}(H_\ell^{\text{attn}} + \text{FFN}(H_\ell^{\text{attn}})). \quad (44)$$

Post-LN 的梯度问题可通过以下分析理解。设 F_ℓ 为第 ℓ 层子层（Attn 或 FFN），损失对第 ℓ 层输入的梯度由链式法则展开：

$$\frac{\partial \mathcal{L}}{\partial H_{\ell-1}} = \frac{\partial \mathcal{L}}{\partial H_\ell} \cdot \frac{\partial H_\ell}{\partial H_\ell^{\text{attn}}} \cdot \frac{\partial H_\ell^{\text{attn}}}{\partial H_{\ell-1}}. \quad (45)$$

在 Post-LN 中， $\frac{\partial H_\ell}{\partial H_\ell^{\text{attn}}}$ 经过了 LN 的 Jacobian，其最大奇异值约为 $1/\sigma$ （ σ 是局部标准差）。当网络层数增加时，这一 Jacobian 可能小于 1，多次相乘后梯度呈指数衰减，形成**梯度消失**。另一方面，初始化时注意力输出方差往往超过 1，使得 $1/\sigma < 1$ 但 $\partial F_\ell / \partial H_{\ell-1}$ 可能大于 1，两者相互影响，导致梯度的**不可预测波动**（不稳定训练）。实践中，Post-LN 通常需要极小学习率起步（如 $10^{-4} \rightarrow$ 预热到 3×10^{-4} ）并严格控制预热步数，否则训练崩溃。

Pre-LN（现代LLM标准）与梯度流分析 Pre-LN 将 LN 移到残差分支内：

$$\tilde{H}_\ell = H_{\ell-1} + \text{Attn}(\text{LN}(H_{\ell-1})), \quad H_\ell = \tilde{H}_\ell + \text{FFN}(\text{LN}(\tilde{H}_\ell)). \quad (46)$$

残差路径不再经过 LN，梯度可沿恒等映射通道直接回传：

$$\frac{\partial H_\ell}{\partial H_{\ell-1}} = I + \frac{\partial}{\partial H_{\ell-1}} \text{Attn}(\text{LN}(H_{\ell-1})), \quad (47)$$

第一项 I 保证了梯度下界。展开到整个网络：

$$\frac{\partial \mathcal{L}}{\partial H_0} = \frac{\partial \mathcal{L}}{\partial H_L} \prod_{\ell=1}^L \frac{\partial H_\ell}{\partial H_{\ell-1}} = \frac{\partial \mathcal{L}}{\partial H_L} \prod_{\ell=1}^L (I + J_\ell), \quad (48)$$

其中 J_ℓ 是子层的 Jacobian。展开乘积后包含全部 2^L 项（每层可选择走残差 I 或走 J_ℓ ），第一项（全走残差路径）为 $\frac{\partial \mathcal{L}}{\partial H_L}$ 本身，因此梯度范数始终不低于 $\|\frac{\partial \mathcal{L}}{\partial H_L}\|$ ，梯度消失问题得到根本解决。

RMSNorm（Zhang & Sennrich, 2019；LLaMA全系列采用） LayerNorm 中的均值减法（中心化步骤）负责将激活均值归零，但 Ba et al. 与 Zhang et al. 的实验均显示：在 Transformer 的 Pre-LN 设置下，由于残差连接已隐式维持一定的均值稳定性，中心化对最终性能贡献极小，反而带来额外的计算开销。RMSNorm 省去均值计算，仅保留均方根缩放：

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \cdot \gamma, \quad \text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}. \quad (49)$$

计算优势：省去了均值的一次全归约（all-reduce），在张量并行（Tensor Parallelism）中减少了一次跨设备通信；同时 γ 无需对应 β （偏置），进一步减少参数量与内存带宽。实际测量中，RMSNorm 比 LayerNorm 快 15 ~ 25%，在大规模训练中积累的时间节省可观。

DeepNorm（Wang et al., 2022；支持千层级深度） DeepNorm 是微软研究院提出的、允许 Post-LN 稳定训练超深网络的方案。其核心是将残差连接的权重尺度化：

$$H_\ell = \text{LN}(\alpha \cdot H_{\ell-1} + \text{SubLayer}(H_{\ell-1})), \quad (50)$$

同时对子层参数做特殊初始化：权重矩阵乘以因子 $\beta < 1$ （与 α 配合）。作者证明，若 $\alpha = (2L)^{1/4}$ ， $\beta = (8L)^{-1/4}$ （ L 为层数），则初始化时每层更新的范数均匀，梯度不会随层数增长而爆炸或消失。这使 Post-LN 可以在不预热的前提下稳定训练超过 1000 层，为超深 Transformer 提供了理论基础。

3.1.5 前馈网络（FFN）的设计原理

若说自注意力负责 token 之间的信息交换，则前馈网络（Feed-Forward Network, FFN）负责单个 token 内部的非线性变换与特征重编码。标准 Transformer 层中的 FFN 写为

$$\text{FFN}(h) = W_2 \phi(W_1 h + b_1) + b_2, \quad (51)$$

其中 $h \in \mathbb{R}^d$, $W_1 \in \mathbb{R}^{d_{ff} \times d}$, $W_2 \in \mathbb{R}^{d \times d_{ff}}$, 且通常 $d_{ff} \gg d$ 。这意味着FFN先把token表示升维到更宽的隐空间, 在该空间中做坐标方向上的非线性筛选, 再投影回模型维度。

从函数逼近的角度看, FFN承担了两项作用。第一, 它把注意力层形成的“读到的信息”转化为更适合下一层处理的表示; 第二, 它通过宽度 d_{ff} 提供了近似高维基函数展开的能力, 使网络能在每个位置上学习复杂的局部决策边界。很多实践工作把FFN理解为一种**按token索引的参数化记忆**: 注意力决定“读什么”, FFN决定“读到之后如何改写内部状态”。

现代LLM中常见的SwiGLU/GeGLU等门控FFN, 本质上是在升维后加入乘性选择机制。例如SwiGLU可写为

$$\text{SwiGLU}(h) = W_o \left((W_v h) \odot \text{swish}(W_g h) \right), \quad (52)$$

其中 $\odot$ 表示Hadamard逐元素乘积。与标准ReLU/GELU FFN相比, 这类结构通过门控把“哪些通道应该被放大、哪些应被抑制”显式参数化, 因此在相近参数量下往往具有更好的表达效率。换言之, FFN并不是注意力层后面一个可有可无的“MLP补丁”, 而是Transformer能稳定表达复杂语义映射的另一半。

3.1.6 交叉注意力机制与条件信息注入

自注意力处理的是同一序列内部的信息交换, 而交叉注意力 (Cross-Attention) 处理的是一个序列如何**按需读取另一个序列或另一种模态的表示**。设查询序列为 $X^{(q)} \in \mathbb{R}^{n_q \times d}$, 条件序列为 $X^{(c)} \in \mathbb{R}^{n_c \times d}$, 则交叉注意力定义为

$$Q = X^{(q)} W_Q, \quad K = X^{(c)} W_K, \quad V = X^{(c)} W_V, \quad (53)$$

$$\text{CrossAttn}(X^{(q)}, X^{(c)}) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V. \quad (54)$$

这里查询来自“当前需要更新的表示”, 而键和值来自“提供条件信息的外部上下文”。在经典编码器-解码器结构中, 解码器token通过交叉注意力读取编码器输出; 在视觉语言模型中, 文本token可通过交叉注意力读取图像patch或压缩后的视觉latent; 在检索增强生成中, 问题token可通过交叉注意力读取外部文档表示。

交叉注意力之所以重要, 是因为它提供了一种比“直接拼接全部模态token”更精细的条件注入方式。若把所有视觉token和文本token直接合并做全局自注意力, 虽然统一, 但计算成本为 $O((n_q + n_c)^2 d)$, 且容易让无关模态过度干扰。交叉注意力则把代价控制在 $O(n_q n_c d)$, 并保持方向性: **谁是解释者、谁是被解释对象, 谁在读、谁被读**, 在算子层面是清楚的。这也是许多多模态架构选择“冻结视觉编码器 + 轻量桥接模块 + LLM解码器”的根本原因。

3.1.7 模态对齐技术: 从表示空间到训练目标

要让文本、图像、音频等模态进入同一Transformer骨干, 关键不只是“把它们都变成向量”, 而是要让这些向量在几何上**可比较、可检索、可互相条件化**。形式上, 设文本编码器、视觉编码器分别给出

$$u_i = E_{\text{text}}(x_i^{\text{text}}), \quad v_i = E_{\text{img}}(x_i^{\text{img}}), \quad (55)$$

再经过投影头映射到公共空间:

$$z_i^{\text{text}} = P_{\text{text}}(u_i), \quad z_i^{\text{img}} = P_{\text{img}}(v_i). \quad (56)$$

最常见的第一步对齐目标是对比学习，其代表性损失可写为

$$\mathcal{L}_{\text{clip}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\langle z_i^{\text{text}}, z_i^{\text{img}} \rangle / \tau)}{\sum_{j=1}^B \exp(\langle z_i^{\text{text}}, z_j^{\text{img}} \rangle / \tau)}, \quad (57)$$

并对文本到图像、图像到文本两个方向同时优化。其几何意义是：匹配样本在共享嵌入空间内靠近，不匹配样本被推远。这样做之后，模型至少学会“哪段文本对应哪张图、哪张图最支持哪句话”的粗粒度对齐。

但仅有全局对齐还不够。复杂多模态推理还要求**细粒度对齐**，也就是词语、区域、对象、关系、动作之间的局部对应。工程上常见三种做法。第一，使用更细粒度的视觉token，如patch、region或object token，让文本可以注意到局部视觉证据。第二，引入桥接模块，例如Q-Former、Perceiver Resampler或轻量查询token，用少量可学习查询从高维视觉特征中提炼出适合语言模型消费的视觉摘要。第三，叠加生成式目标，即在共享表示基础上继续最小化条件语言模型损失：

$$\mathcal{L}_{\text{gen}} = -\sum_{t=1}^T \log p_{\theta}(y_t | y_{<t}, z^{\text{img}}, z^{\text{text}}). \quad (58)$$

对比学习负责“放到同一几何空间里”，生成式训练负责“学会如何利用这个空间完成回答、描述与推理”。真正强的多模态系统通常两类目标都会用。

3.1.8 Transformer如何成为统一多模态基础架构

有了上述模块之后，Transformer之所以能成为文本、图像乃至更多模态的统一基础架构，就不再神秘。关键步骤可以概括为三层。

第一层是**离散化或token化**。文本天然由token构成；图像可被切成patch token、region token或由视觉编码器输出latent token；音频可被表示为时间帧token、频谱token或离散声码token。于是不同模态都可写成

$$X^{(m)} = [x_1^{(m)}, \dots, x_{n_m}^{(m)}]. \quad (59)$$

第二层是**投影到统一表示空间**。对每个模态使用专属前端 Φ_m 和投影矩阵 P_m ，得到统一维度的token序列

$$H_0 = [P_{\text{text}}\Phi_{\text{text}}(X^{(\text{text})}); P_{\text{img}}\Phi_{\text{img}}(X^{(\text{img})}); \dots] + E_{\text{pos}} + E_{\text{mod}}, \quad (60)$$

其中 E_{pos} 编码位置信息， E_{mod} 编码模态身份。这样，后续骨干网络不再需要知道“这原来是一张图还是一句话”，它只需要在统一维度上学习如何路由信息。

第三层是**用同一套堆叠层做融合与推理**。无论采用全局自注意力、局部稀疏注意力，还是“自注意力 + 交叉注意力”混合结构，核心更新都可写成

$$\tilde{H}_{\ell} = H_{\ell} + \text{Attn}(H_{\ell}), \quad H_{\ell+1} = \tilde{H}_{\ell} + \text{FFN}(\tilde{H}_{\ell}). \quad (61)$$

这意味着多模态系统真正共享的是**信息交换规则**，而不是共享原始传感器。文本token、图像token、工具返回token、记忆token在高层都被视为“可被注意、可被改写、可被再利用的状态单元”。一旦这个统一观点成立，Transformer就天然适合作为多模态统一骨干。

从系统视角看，这一统一骨干还有三点额外价值。其一，训练与扩展路径统一，文本模型中成熟的预训练、蒸馏、量化、KV缓存和推理加速技术可以相对平滑地迁移到多模态模型。其二，外部工具与记忆更容易接入，因为它们也可以被编码成附加token或条件上下文。其三，推理链可以跨模态展开：模型不必先“看完图再说话”，而是能在若干层中反复执行“看图一部分 → 形成文本中间表示 → 再回到图像证据”的循环。这正是近年来多模态思维链、视觉工具调用与统一智能体系统能够快速发展的算子基础。

3.2 长上下文、缓存与前沿系统算法

3.2.1 KV Cache的数学原理、内存分析与工程优化

自回归解码的冗余计算问题 在标准的 Transformer 自回归解码中，第 t 步生成新 token x_t 时，其注意力计算需要所有历史位置 $1, \dots, t-1$ 的键值对。若直接重新计算，每步的 key/value 投影为：

$$k_j = W_K h_j, \quad v_j = W_V h_j, \quad j = 1, \dots, t, \quad (62)$$

这要求对所有历史 token 的隐状态 h_j 重新做线性映射，每步计算量为 $O(t \cdot d^2/H)$ ，累积到第 T 步则为 $O(T^2 d^2/H)$ ——对长序列代价极高。

KV Cache 机制 KV Cache 的核心思想：将每层每头已计算好的 k_j, v_j 存储下来，解码第 t 步时只计算当前 token 的 (k_t, v_t) 并追加：

$$\mathcal{C}_t = \mathcal{C}_{t-1} \cup \{(k_t^{(\ell,h)}, v_t^{(\ell,h)})\}_{\ell=1, h=1}^{L,H}. \quad (63)$$

注意力输出变为：

$$o_t^{(\ell,h)} = \text{softmax}\left(\frac{q_t^{(\ell,h)} K_{1:t}^{(\ell,h)\top}}{\sqrt{d_h}}\right) V_{1:t}^{(\ell,h)}, \quad (64)$$

其中 $K_{1:t}^{(\ell,h)}, V_{1:t}^{(\ell,h)}$ 直接从缓存读取。单步计算量降为 $O(t \cdot d_h)$ （只需计算当前 token 的注意力分数），而不是 $O(t \cdot d^2/H)$ （全部重新投影）。

KV Cache 显存的精确公式 对一个具有 L 层、 H 个注意力头、头维度 d_h （即 $d_h = d/H$ ）、序列长度 T 、批量大小 B 、数值精度 b 字节的模型，KV Cache 显存占用为：

$$M_{KV} = 2 \cdot B \cdot L \cdot T \cdot H \cdot d_h \cdot b = 2 \cdot B \cdot L \cdot T \cdot d \cdot b. \quad (65)$$

系数 2 来自 K 和 V 各一份。注意 M_{KV} 与 H 无关（因为 $H \cdot d_h = d$ ），主要参数是 L, T, d, B 。

数值示例 (LLaMA-2-70B, FP16)： $L = 80, d = 8192, b = 2$ 字节，批量 $B = 1$ ：

$$M_{KV} = 2 \times 1 \times 80 \times T \times 8192 \times 2 \approx 2.56 \text{ GB} \times \frac{T}{1024}. \quad (66)$$

在 $T = 4096$ 时约 10 GB， $T = 32768$ 时约 80 GB——已与模型权重（~140 GB in FP16）相当。这是长上下文服务系统面临的根本性显存挑战。

KV Cache 的带宽瓶颈分析 KV Cache 不仅占显存，还是推理速度的带宽瓶颈。每步解码需要从 HBM 读取全部缓存 $K_{1:t}, V_{1:t}$ （共 $2 \cdot L \cdot t \cdot d \cdot b$ 字节），计算量仅 $O(Ldt)$ FLOPs，而 A100 GPU 的 HBM 带宽 ~2 TB/s，计算峰值 ~312 TFLOPS (FP16)：

$$\text{算术强度} = \frac{2Ldt \text{ FLOPs}}{2Ldt \cdot b \text{ bytes}} = \frac{1}{b} \approx 0.5 \text{ FLOPs/Byte} \quad (b = 2). \quad (67)$$

而 GPU 的 roof-line 拐点约为 $312 \text{ TFLOPS} / 2 \text{ TB/s} = 156 \text{ FLOPs/Byte}$ 。自回归解码的算术强度远低于拐点，因此处于**显著的内存带宽受限 (memory-bound) 状态**，增加计算单元对吞吐无益，只有增加带宽或减少读写量才能提速。

缓解策略：从量化到分层压缩

1. **KV量化 (INT8/INT4)**：将 K, V 从 FP16 量化为 INT8 或 INT4, b 从 2 降至 1 或 0.5 字节，显存与带宽直接减半或减四分之三。KV 量化需要注意**保持内积排序**（注意力分数的相对大小），因此应保留 $q_t^\top k_j$ 的量化精度，而非一律使用对称量化。
2. **MQA/GQA头共享**：如上节所述，将 H 头的 KV 压缩为 G 组 ($G \ll H$)，显存从 $2LTHd_hb$ 降至 $2LTGd_hb$ 。
3. **DeepSeek MLA低秩压缩**：缓存低维潜向量 $c_t \in \mathbb{R}^{d_c}$ ($d_c \ll Hd_h$)，需要时实时解压，缓存量降至 $2LTd_cb$ 。
4. **PagedAttention (vLLM, Kwon et al., 2023)**：将逻辑 KV 序列分页管理，消除碎片和预分配浪费，允许不同序列动态共享物理显存 (prefix caching)，并发服务时实际可用序列长度增加 2 - 4 倍。分页思想来自操作系统虚拟内存管理，页表映射为 $\text{addr}(r, i, o) = \text{base}(\pi(r, i)) + o$ ，其中 r 是请求编号， i 是逻辑页号， π 是页表， o 是页内偏移。

对工程实践的核心指导 长上下文成本首先是缓存问题，而非参数问题：模型权重是固定的 one-time 加载成本，而 KV Cache 随序列长度线性增长且每步都需读写。因此工程优化的优先级应为：先压缩 KV Cache (GQA + INT8量化 + 分页)，再考虑模型参数压缩（权重量化、蒸馏）。对于在线服务，建议将最大序列长度设定为满足 $M_{KV} \leq 40\%$ 显存上限的值，留出足够空间给模型权重和激活值。

3.2.2 StreamingLLM、SnapKV与选择性KV驱逐

KV缓存的容量上限是制约超长上下文推理的根本障碍。研究者发现，对于因果语言模型，并不是所有历史token的KV对都需要永久保留——合理的驱逐策略可以在几乎不损失性能的前提下将显存占用压缩数倍。

StreamingLLM：注意力沉淀与滑动窗口 Xiao等 (2023) 发现大模型在处理超长序列时存在“注意力沉淀 (Attention Sinks)”现象：无论序列多长，模型都会对**开头的几个token**（通常是BOS和前3-4个位置）保持异常高的注意力权重。这些“沉淀token”对激活值的数值稳定性至关重要，一旦被丢弃模型性能会急剧下降。

基于这一发现，StreamingLLM提出了双保险策略：

$$KV_{\text{retained}} = \underbrace{KV_{1:k}}_{\text{沉淀token (锚点)}} \cup \underbrace{KV_{T-W:T}}_{\text{滑动窗口 (最近上下文)}} \quad (68)$$

其中 k 为沉淀token数（通常取4）， W 为滑动窗口大小。中间的历史token可安全丢弃，推理流式生成时的显存占用降为 $O(k + W)$ 而非 $O(T)$ ，允许理论无限长度的持续对话而不崩溃。

SnapKV与H2O：重要性驱动的选择性保留 如果说StreamingLLM采用位置启发式（保留首尾），那么SnapKV和H2O (Heavy Hitter Oracle) 则采用**重要性评分**来决定保留哪些token的KV对。

H2O的核心直觉来自一个观察：在典型的注意力权重矩阵中，少数“重击者（Heavy Hitters）” token接收了大部分的注意力。若将第 j 个token的重要性定义为其历史累积注意力权重之和：

$$\text{score}(j) = \sum_{t>j} \alpha_{t,j}, \quad \alpha_{t,j} = \text{softmax}\left(\frac{q_t k_j^\top}{\sqrt{d_h}}\right)_j, \quad (69)$$

则在每步解码后，只需保留score最高的前 B 个token的KV对，其余驱逐。SnapKV进一步在prefill阶段就完成驱逐决策，避免了逐步计算的开销。实验表明，保留10-20%的KV对通常已能维持95%以上的下游性能。

工程启示 StreamingLLM适合**无限流式对话场景**（客服、实时助手），H2O/SnapKV适合**单轮长文档分析场景**（合同审核、研究报告）。实际系统往往将两者组合：用沉淀token保稳定性，用重要性打分保语义质量。

3.2.3 Infini-attention：压缩记忆与无限上下文

Munkhdalai等（Google, 2024）提出Infini-attention，在标准因果注意力层内嵌入一个**固定大小的压缩记忆矩阵** $M \in \mathbb{R}^{d_k \times d_v}$ ，使单一Transformer层在处理任意长度序列时不需要无限增长的KV缓存。

对当前局部窗口（长度 S ），Infini-attention像标准注意力一样计算 $A_{\text{dot}} = \text{softmax}(QK^\top/\sqrt{d})V$ ；对所有历史上下文，则用线性注意力检索压缩记忆：

$$A_{\text{mem}} = \frac{\sigma(Q)M_{s-1}}{\sigma(Q)z_{s-1}}, \quad (70)$$

其中 $\sigma(\cdot)$ 为element-wise激活函数， z_{s-1} 为归一化向量。两路输出通过可学习门控融合：

$$A = \text{sigmoid}(\beta) \odot A_{\text{mem}} + (1 - \text{sigmoid}(\beta)) \odot A_{\text{dot}}. \quad (71)$$

记忆矩阵在处理完每个窗口后按关联学习规则更新：

$$M_s = M_{s-1} + \sigma(K)^\top V, \quad z_s = z_{s-1} + \sum_t \sigma(k_t). \quad (72)$$

这一设计的关键在于：无论输入多长，模型在推理时只需维护一个固定大小的 M 矩阵，**内存占用为 $O(1)$ 而非 $O(T)$** ，却保留了对整个历史的“有损摘要”访问能力。

3.2.4 隐式注意力的统一数学框架：从线性RNN到状态空间模型

近年来大量高效序列模型不显式构造 $T \times T$ 注意力矩阵，而是通过递推状态实现“隐式注意力”。统一写法可表示为：

$$s_t = \Lambda_t s_{t-1} + B_t x_t, \quad y_t = C_t s_t, \quad (73)$$

其中 s_t 是隐状态， Λ_t, B_t, C_t 可以是常数，也可以像Mamba那样依赖当前输入 x_t 。将该递推展开，有：

$$y_t = \sum_{j=1}^t C_t \left(\prod_{k=j+1}^t \Lambda_k \right) B_j x_j = \sum_{j=1}^t A_{t,j} x_j. \quad (74)$$

这里

$$A_{t,j} = C_t \left(\prod_{k=j+1}^t \Lambda_k \right) B_j \quad (75)$$

扮演了**因果注意力核**的角色。与标准softmax注意力不同， $A_{t,j}$ 不是由 $q_t^\top k_j$ 显式归一化得到，而是由状态转移与门控矩阵隐式诱导出来。因此，隐式注意力本质上是”**用递推动力系统近似或替代显式注意力图**”。

这一视角解释了为什么Mamba、RWKV、门控线性RNN等”无注意力层”模型仍然具备内容选择能力：当 Λ_t, B_t, C_t 由输入驱动时，模型实际上对不同历史位置赋予了不同的衰减、注入与读取强度，相当于学习了一个依赖内容的动态核函数。它们的优势在于不需要物化完整的注意力矩阵，因此时间/空间复杂度可做到线性或次二次，更适合百万级上下文。

3.2.5 Mamba与选择性状态空间模型（S6）：完整数学推导

Mamba (Gu & Dao, 2023) 是目前最具代表性的状态空间模型架构，在语言建模任务上首次在相同参数量下接近甚至超过Transformer基线，同时保持线性时间与常数显存的推理复杂度。其理论根源是控制论中的线性时不变系统（LTI），经由S4演化到S6（即Mamba所用的**选择性SSM**）。

Step 1：连续时间线性系统（S4的理论基础） S4 从一阶常微分方程（ODE）出发：

$$\dot{h}(t) = A h(t) + B x(t), \quad y(t) = C h(t), \quad (76)$$

其中 $h(t) \in \mathbb{R}^N$ 是系统状态（ N 为状态维度，S4 通常取 $N = 64$ 或 $N = 256$ ）， $x(t), y(t) \in \mathbb{R}$ 为标量输入输出， $A \in \mathbb{R}^{N \times N}$ 控制状态的演化规律， $B, C \in \mathbb{R}^N$ 控制输入注入与状态读出。

HiPPO矩阵初始化： A 的选取对长程记忆能力至关重要。普通随机初始化会导致历史信息以极快速率衰减。Gu et al. (2020) 提出用 HiPPO (High-order Polynomial Projection Operators) 矩阵初始化 A ，其定义为：

$$A_{nk} = - \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & n > k \\ n+1 & n = k \\ 0 & n < k \end{cases} \quad (77)$$

HiPPO-LegS (Legendre Scaled) 矩阵的关键性质是：其对应的 ODE 系统的状态 $h(t)$ 恰好是函数 $x(\cdot)$ 在长度 t 窗口上的**勒让德多项式系数**。换言之， $h(t)$ 以正交多项式展开的方式压缩了整个历史信号，不同分量捕获不同频率的历史成分，使长程依赖可以被保留而非衰减。这是 SSM 能够记忆长历史的数学根基。

Step 2：零阶保持离散化（ZOH）——完整推导 神经网络处理离散序列 $x_0, x_1, \dots$ ，需将连续系统离散化。设相邻时间步之间输入保持恒定（零阶保持假设），步长为 Δ ，则 ODE 的精确解为：

$$h(t) = e^{At} h(0) + \int_0^t e^{A(t-s)} B x(s) ds. \quad (78)$$

在时刻 $t = (k+1)\Delta$ 处，利用 ZOH 假设 $x(s) = x_k$ 对 $s \in [k\Delta, (k+1)\Delta)$ ：

$$h_{k+1} = e^{A\Delta} h_k + \int_{k\Delta}^{(k+1)\Delta} e^{A((k+1)\Delta-s)} B x_k ds \quad (79)$$

$$= \underbrace{e^{A\Delta}}_{\bar{A}} h_k + \underbrace{\left(\int_0^\Delta e^{A\tau} d\tau \right) B}_{\bar{B}} x_k. \quad (80)$$

精确地：

$$\bar{A} = e^{A\Delta}, \quad \bar{B} = A^{-1}(e^{A\Delta} - I)B = (\Delta A)^{-1}(e^{A\Delta} - I)(\Delta B). \quad (81)$$

当 Δ 较小时， $e^{A\Delta} \approx I + A\Delta$ ，因此 $\bar{B} \approx \Delta B$ （一阶近似），这是 Mamba 代码中常用的简化形式。离散递推最终为：

$$\boxed{h_t = \bar{A} h_{t-1} + \bar{B} x_t, \quad y_t = C h_t.} \quad (82)$$

训练并行性（卷积等价形式）：对固定的 $(\bar{A}, \bar{B}, C)$ ，将递推展开：

$$y_t = C h_t = C(\bar{A} h_{t-1} + \bar{B} x_t) = C \bar{A}^t h_0 + \sum_{k=0}^t C \bar{A}^{t-k} \bar{B} x_k = \sum_{k=0}^t \bar{\mathcal{K}}_{t-k} x_k, \quad (83)$$

其中卷积核 $\bar{\mathcal{K}}_j = C \bar{A}^j \bar{B}$ ($j \geq 0$)，整个输出序列 $y = \bar{\mathcal{K}} * x$ 。卷积可用 FFT 在 $O(T \log T)$ 时间计算，实现训练时的**完全并行化**，代价是需要预先存储 T 步的卷积核（当 $\bar{A}$ 固定时可通过矩阵幂高效计算）。

Step 3: S6——选择性机制（Mamba的核心创新） S4的 A, B, C 是固定常数，独立于输入内容，意味着模型对所有token一视同仁地读写状态——这与注意力机制的“内容感知路由”能力相去甚远，导致S4在语言任务上难以准确匹配token。

Mamba提出**选择性状态空间（S6）**：令 B_t, C_t, Δ_t 依赖当前输入 x_t ：

$$\Delta_t = \text{softplus}(W_\Delta x_t + b_\Delta), \quad (84)$$

$$B_t = W_B x_t, \quad (85)$$

$$C_t = W_C x_t, \quad (86)$$

其中 W_Δ, W_B, W_C 是可学习矩阵。随后按上述ZOH对 (Δ_t, A, B_t) 做逐步离散化： $\bar{A}_t = e^{\Delta_t A}$ ， $\bar{B}_t = \Delta_t B_t$ （近似），得到输入依赖的递推：

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t. \quad (87)$$

直觉上： Δ_t 控制“此刻对历史的遗忘程度”（步长越大，历史衰减越快）； B_t 控制“当前输入写入状态的方式”； C_t 控制“如何从状态读取输出”。这三者都由输入内容决定，使SSM具备了与softmax注意力相近的内容敏感能力，但计算复杂度仍为 $O(N)$ 每步、 $O(1)$ 状态空间。

Step 4: 硬件感知并行扫描 (Hardware-Aware Parallel Scan) S6 的递推 $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$ 由于每步依赖前一步，表面上无法并行。但注意该递推是一个**线性递推**（线性 RNN），可借助**并行前缀扫描**（parallel prefix scan，又称 scan 算法）在 $O(\log T)$ 深度完成。

半群结构：定义二元运算 $\oplus$ 作用于对 (a, b) （ a 为状态转移乘子， b 为加性偏移）：

$$(a_1, b_1) \oplus (a_2, b_2) = (a_1 a_2, a_2 b_1 + b_2). \quad (88)$$

则递推写成： $(h_t) \equiv (\bar{A}_t, \bar{B}_t x_t) \oplus (h_{t-1})$ ，即 $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$ 。可以验证 $\oplus$ 满足结合律（associativity），因此构成半群。对序列 $p_t = (\bar{A}_t, \bar{B}_t x_t)$ ， h_t 等于从 $t = 1$ 到 t 的**前缀积**（prefix product under $\oplus$ ）作用于初始状态 h_0 。

二叉树并行算法：将 T 个元素排成二叉树，树高 $O(\log T)$ ，逐层合并：

- 第 1 轮：相邻对合并， $T/2$ 个操作并行执行；
- 第 2 轮：相隔 2 的对合并， $T/4$ 个操作并行执行；
- ...，共 $\log_2 T$ 轮。

总计算量 $O(T)$ （工作量，work-optimal），深度 $O(\log T)$ ，完全并行化。

硬件感知设计 (Dao & Gu 的关键工程贡献)：在 GPU 上朴素实现上述扫描时，每一步的中间结果 $h_t \in \mathbb{R}^{d \times N}$ 若写回 HBM（High Bandwidth Memory），再从 HBM 读取，则内存带宽成为瓶颈——尽管 FLOPs 不多，但 HBM 读写延迟极大。Mamba 的关键工程技巧是：

1. 将扫描的**全部中间激活**保留在 SRAM（片上高速缓存， $\sim 40\times$ 带宽优于 HBM）中，仅在最终将结果写回 HBM；
2. 在反向传播时重新计算（recompute）中间激活，而不是从 HBM 读取（以算力换带宽，典型 activation checkpointing 思路）。

这使 Mamba 的实际吞吐在 A100 GPU 上达到理论 FLOPs 利用率的 $\sim 75\%$ ，远高于朴素实现。

推理时的递推模式：推理阶段按 token-by-token 生成，不需要并行扫描，直接执行：

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t. \quad (89)$$

每步计算量为 $O(dN)$ （ d 为通道数， N 为状态维度，通常 $N = 16$ ），**KV Cache 等价于固定大小 $\mathbb{R}^{d \times N}$ 的状态矩阵**，与序列长度 T 完全无关。对比 Transformer 的 $O(Td_h H)$ KV Cache，当 T 增大时 Mamba 的推理内存优势是根本性的。

Mamba 块的完整设计 Mamba 用如下结构替代 Transformer 层中的注意力子层：

$$z = \text{Linear}_z(x), \quad x' = \text{SSM}(\text{Conv1d}(\text{Linear}_x(x))), \quad y = x' \odot \text{silu}(z). \quad (90)$$

1D 卷积层 Conv1d 提供局部平滑，门控 silu(z) 使输出具有非线性内容选择能力，SSM 提供全局历史整合。这一结构比直接替换注意力+FFN 的方式更高效，参数量与 Transformer 层相当但推理 FLOPs 低约 1.5 - 2 倍。

Mamba-2与结构化状态空间对偶 (SSD) Dao & Gu (2024) 进一步证明, 当 A 取标量对角矩阵 (每头共享同一衰减系数 $a \in (0, 1)$) 时, Mamba离散递推等价于带指数衰减掩码的线性注意力:

$$H = (L \odot (QK^\top))V, \quad L_{ij} = a^{i-j} \mathbf{1}_{i \geq j}, \quad (91)$$

其中 Q, K, V 由输入线性投影得到, L 是下三角衰减矩阵。这一“SSM-Attention对偶”(SSD)使Mamba-2可以用高效矩阵乘法核直接计算, 在GPU上实现了接近FlashAttention级别的训练速度, 同时保留了推理时的 $O(1)$ 状态优势。

对工程选型的意义 Mamba类模型的选型建议: 在流式推理、超长单文档处理、实时语音/时序信号等带宽主导的场景, 优先考虑; 在需要精确键值匹配、多跳推理、精确引用的任务上, 纯SSM仍弱于全注意力。目前工业界倾向于混合架构 (Jamba、Zamba等), 每N层插入一层注意力, 兼顾精确检索与长程效率。

3.2.6 残差注意力: RealFormer的logit级跳连

残差注意力并不是对token表示做残差, 而是直接对**注意力 logits**做残差传递。令第 ℓ 层的原始注意力分数为

$$\tilde{S}^{(\ell)} = \frac{Q^{(\ell)} K^{(\ell)\top}}{\sqrt{d_h}}, \quad (92)$$

则RealFormer类结构将上一层的注意力分数以跳连方式注入:

$$S^{(\ell)} = \tilde{S}^{(\ell)} + \alpha_\ell S^{(\ell-1)}, \quad A^{(\ell)} = \text{softmax}(S^{(\ell)}). \quad (93)$$

在最简单的设置中 $\alpha_\ell = 1$, 也可使用running mean代替running sum来控制数值幅度。

这种设计的数学含义是: **跨层保留配对关系的先验**。如果某些query-key对在浅层已被证明重要, 那么深层就不必每次从零开始重新学习这一结构, 而可以在已有亲和关系上做增量修正。其直接效果是:

- 使注意力图跨层更加一致, 降低训练初期的结构漂移;
- 诱导更稀疏的注意力分布, 缓解无效扩散;
- 在不增加参数量的前提下提升优化稳定性。

3.2.7 微软差分注意力 (Differential Attention) : 消去注意力噪声

微软研究院 (Ye et al., NeurIPS 2024) 提出**差分注意力 (Differential Attention)**, 其出发点是一个精确的数学观察: 标准softmax注意力的输出噪声是**结构性的**, 而非随机的, 因此可以通过代数方法消除。

注意力噪声的数学来源 考虑一个极端情形: 序列中只有一个 token j^* 与查询 i 相关, 其余 $n-1$ 个 token 均无关。理想的注意力应满足 $A_{i,j^*} = 1$, 其余 $A_{i,j} = 0$ 。但在softmax归一化下:

$$A_{ij} = \frac{e^{s_{ij}}}{\sum_{j'} e^{s_{ij'}}}, \quad s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}. \quad (94)$$

即使 s_{i,j^*} 远大于其他分数，分母中的 $n-1$ 个“噪声项” $e^{s_{ij}} (j \neq j^*)$ 的总和仍然非零，导致 $A_{i,j^*} < 1$ ，其余位置的权重也不为零，产生系统性偏差。当序列长度 n 增大时，即使每个无关 token 的权重极小，其累积效应（作用在 V 上）仍可显著。

更精确地，设有用信号 $s = A_{i,j^*} V_{j^*}$ ，噪声为 $\epsilon = \sum_{j \neq j^*} A_{ij} V_j$ ，则注意力输出为 $o_i = s + \epsilon$ 。这里噪声 ϵ 不是零均值随机噪声，而是系统性地“稀释”了有用信号。

差分放大器的数学原理 电路中的**差分放大器** (differential amplifier) 能放大两个输入信号之差，而抑制两路共同存在的噪声（共模噪声）：

$$V_{\text{out}} = A_d(V_+ - V_-), \quad \text{CMRR} = \frac{A_d}{A_c} \gg 1, \quad (95)$$

其中 A_d 是差模增益， A_c 是共模增益，CMRR (Common Mode Rejection Ratio) 越大越好。

Differential Attention 将这一思想移植到注意力机制：用两个 softmax 映射分别充当 V_+ 和 V_- ，通过相减抵消共模噪声。具体地，将查询和键各拆成两半（每半维度 $d_k/2$ ）：

$$Q = [Q_1; Q_2], \quad K = [K_1; K_2], \quad Q_1, Q_2, K_1, K_2 \in \mathbb{R}^{n \times d_k/2}. \quad (96)$$

注意力输出定义为：

$$\text{DiffAttn}(X) = (A_1 - \lambda A_2) V, \quad (97)$$

其中：

$$A_1 = \text{softmax}\left(\frac{Q_1 K_1^\top}{\sqrt{d_k/2}}\right) \in \mathbb{R}^{n \times n}, \quad A_2 = \text{softmax}\left(\frac{Q_2 K_2^\top}{\sqrt{d_k/2}}\right) \in \mathbb{R}^{n \times n}. \quad (98)$$

$\lambda \in \mathbb{R}$ 是**可学习标量**，通过以下形式参数化以保证非负性：

$$\lambda = \exp(\lambda_{q_1} \cdot \lambda_{k_1}) - \exp(\lambda_{q_2} \cdot \lambda_{k_2}) + \lambda_{\text{init}}, \quad (99)$$

其中 $\lambda_{q_1}, \lambda_{k_1}, \lambda_{q_2}, \lambda_{k_2}$ 是头专属的可学习向量， $\lambda_{\text{init}} = 0.8 - 0.6 \exp(-0.3(h-1))$ （头索引 h 相关）。

为什么能消噪： A_1 和 A_2 由不同投影 (Q_1, K_1) 和 (Q_2, K_2) 计算，但面对序列中的“背景 token”时，两个 softmax 都会对它们分配一定比例的权重（共模噪声）。相减后，两者共有的均匀分布成分被抵消：若 $A_1 \approx A_2 \approx \frac{1}{n} \mathbf{1} \mathbf{1}^\top$ （均匀分布，纯噪声），则 $A_1 - \lambda A_2 \approx (1 - \lambda) \frac{1}{n} \mathbf{1} \mathbf{1}^\top$ ，整体噪声被抑制。而当 A_1 在 j^* 处尖锐（已学到正确注意力）而 A_2 仍然平坦（噪声）时，差分结果保留了 A_1 的尖锐成分，有效信号得以增强。

归一化处理与数值稳定性 差分后的矩阵 $A_1 - \lambda A_2$ 的元素可能为负，直接乘以 V 不会产生问题（加权平均允许负权重），但需要保证最终输出的方差稳定。论文对每个头的输出做 GroupNorm：

$$\text{DiffAttnOut}_h(X) = \text{GroupNorm}((A_1^{(h)} - \lambda^{(h)} A_2^{(h)}) V^{(h)}) \cdot (1 - \lambda_{\text{init}}^{(h)}). \quad (100)$$

训练初期 $\lambda \approx \lambda_{\text{init}}$ ，因子 $(1 - \lambda_{\text{init}})$ 使输出方差近似为 1，与标准 MHA 初始化相兼容，无需特殊学习率策略。

多头差分注意力 (MDHA) 与参数效率 将 H 个头拼接:

$$\text{MDHA}(X) = \text{Concat}(\text{DiffAttn}_1(X), \dots, \text{DiffAttn}_H(X))W_O. \quad (101)$$

参数量分析: 每个差分头有 Q_1, Q_2, K_1, K_2 (各 $d \times d_k/2$, 共 $2d \cdot d_k$) 和 V ($d \times d_k$), 合计 $3d \cdot d_k$, 与标准头相同。 W_O 维度不变。因此 MDHA 总参数量与标准 MHA 完全相同 (不是1.1倍, 这是重要澄清), 没有额外参数开销, 仅需要多一次矩阵乘以抵消噪声。

实验效果的数学解释 在 needle-in-a-haystack 测试中, $A_1 - \lambda A_2$ 的稀疏化效果可从熵角度理解: 设标准注意力的熵为 $H(A) = -\sum_j A_{ij} \log A_{ij}$, 差分后的有效分布熵更低 (更稀疏), 等效地提高了“信噪比” (SNR):

$$\text{SNR}_{\text{diff}} = \frac{(A_1 - \lambda A_2)_{i,j^*}}{\sqrt{\sum_{j \neq j^*} ((A_1 - \lambda A_2)_{ij})^2}} \gg \text{SNR}_{\text{std}}. \quad (102)$$

这解释了为何 DIFF Transformer 在长上下文精确检索任务上以相同参数量超越标准 Transformer——它更有效地将注意力“聚焦”到真正相关的 token 上。

3.2.8 RetNet: 保留机制、多尺度衰减与训练-推理双范式

RetNet (Yutao Sun, Li Dong, Shaohan Huang et al., Microsoft Research Asia, 2023) 由以中国研究人员为主导的MSRA团队提出, 是近年来在序列建模架构上最重要的工作之一。其核心创新在于设计了一个**保留机制 (Retention)**, 允许同一模型以三种等价但计算效率不同的形式运行: 并行模式 (训练高效)、递推模式 (推理高效)、分块模式 (两者兼顾)。

保留机制的数学定义 设输入序列 $X \in \mathbb{R}^{n \times d}$, 通过投影得到 $Q = XW_Q, K = XW_K, V = XW_V$ 。定义因果衰减矩阵 $D \in \mathbb{R}^{n \times n}$ 为:

$$D_{nm} = \begin{cases} \gamma^{n-m} & n \geq m \\ 0 & n < m \end{cases}, \quad \gamma \in (0, 1), \quad (103)$$

则单头**保留机制**定义为:

$$\text{Ret}(X) = (QK^\top \odot D)V, \quad (104)$$

其中 $\odot$ 是逐元素乘积。注意与标准注意力相比, 保留机制省去了softmax归一化, 用指数衰减掩码 D 替代因果掩码 ($-\infty$ 掩码)——这是设计的核心。

物理含义: $D_{nm} = \gamma^{n-m}$ 意味着位置 m 对位置 n 的贡献按指数衰减, 越久远的位置权重越小 (但永不完全清零), 形成一种软性遗忘机制。 γ 越大 (如 $\gamma = 0.999$), 记忆越持久; γ 越小 (如 $\gamma = 0.8$), 遗忘越快。

三种计算范式的严格等价性证明 以下证明三种范式在数学上等价 (忽略初始状态 $S_0 = 0$)。

1. 并行范式 (训练首选, $O(n^2d)$):

直接展开 $\text{Ret}(X) = (QK^\top \odot D)V$, 第 n 行输出为:

$$\text{Ret}(X)_n = \sum_{m=1}^n D_{nm} (q_n^\top k_m) v_m^\top = \sum_{m=1}^n \gamma^{n-m} (q_n^\top k_m) v_m^\top. \quad (105)$$

2. 递推范式 (推理首选, 每步 $O(d^2)$) :

定义状态矩阵 $S_n \in \mathbb{R}^{d_k \times d_v}$ ($d_k = d_v = d/H$) :

$$S_n = \sum_{m=1}^n \gamma^{n-m} k_m v_m^\top = \gamma S_{n-1} + k_n v_n^\top. \quad (106)$$

则 $\text{Ret}(X)_n = q_n^\top S_n = \sum_{m=1}^n \gamma^{n-m} (q_n^\top k_m) v_m^\top$, 与并行范式完全一致。

等价证明:

$$q_n^\top S_n = q_n^\top (\gamma S_{n-1} + k_n v_n^\top) \quad (107)$$

$$= q_n^\top \left(\gamma \sum_{m=1}^{n-1} \gamma^{(n-1)-m} k_m v_m^\top + k_n v_n^\top \right) \quad (108)$$

$$= \sum_{m=1}^{n-1} \gamma^{n-m} (q_n^\top k_m) v_m^\top + \gamma^0 (q_n^\top k_n) v_n^\top = \sum_{m=1}^n \gamma^{n-m} (q_n^\top k_m) v_m^\top. \quad \square \quad (109)$$

推理时状态更新 $S_n = \gamma S_{n-1} + k_n v_n^\top$ 仅需 $O(d^2)$ per step (外积), 状态矩阵大小固定为 $d_k \times d_v$, 与序列长度 n 完全无关。KV Cache 即为 S_n , 约 $4d^2/H^2$ 字节 (对 LLaMA-7B: $H = 32, d = 4096 \rightarrow S$ 约 2MB, 远小于 Transformer 的 $\sim$ GB 级 KV Cache)。

3. 分块递推范式 ($O(Bd^2 + B^2d)$ per chunk, 训推兼顾) :

将长度 n 的序列分成 n/B 个大小为 B 的块 (块索引 $i = 1, 2, \dots, n/B$)。设第 i 块的局部查询、键、值矩阵为 $Q_{[i]}, K_{[i]}, V_{[i]} \in \mathbb{R}^{B \times d}$, 块间递推状态 $S_i \in \mathbb{R}^{d_k \times d_v}$ 。

对块内位置 t (绝对位置 $(i-1)B + t$), 其输出分为两部分:

1. **块内贡献**: 来自同一块内的 $m \leq t$ 的 token, 形成 $B \times B$ 块内注意力:

$$\text{Inner}_t = \sum_{m=1}^t \gamma^{t-m} (q_t^\top k_m^{[i]}) v_m^{[i]\top} = q_t^\top (K_{[i]}^\top \odot \Gamma_t) V_{[i]}, \quad (110)$$

其中 $\Gamma_{t,m} = \gamma^{t-m} \mathbf{1}_{m \leq t}$, 矩阵形式为 $(Q_{[i]} K_{[i]}^\top \odot D_B) V_{[i]}$, $D_B \in \mathbb{R}^{B \times B}$ 是块内衰减矩阵, 计算量 $O(B^2d)$ 。

2. **跨块贡献**: 来自之前块的历史, 通过状态矩阵 S_{i-1} 读取:

$$\text{Cross}_t = \gamma^t q_t^\top S_{i-1}, \quad t = 1, \dots, B. \quad (111)$$

矩阵形式为 $\Gamma_{\text{decay}} \cdot Q_{[i]} S_{i-1}^\top$, 其中 $\Gamma_{\text{decay}} = \text{diag}(\gamma^1, \dots, \gamma^B)$, 计算量 $O(Bd^2)$ 。

完整输出:

$$\text{Ret}(X_{[i]}) = \underbrace{(Q_{[i]} K_{[i]}^\top \odot D_B) V_{[i]}}_{\text{块内并行, } O(B^2d)} + \underbrace{\Gamma_{\text{decay}} \odot (Q_{[i]} S_{i-1}^\top)}_{\text{跨块递推, } O(Bd^2)}, \quad (112)$$

块间状态更新: $S_i = \gamma^B S_{i-1} + K_{[i]}^\top (\Gamma_{\text{decay}} \odot V_{[i]})$, 计算量 $O(Bd^2)$ 。

最优块大小: 设总序列长度 n , 块大小 B , 总块数 n/B 。总计算量为:

$$\text{Total} = \frac{n}{B} \cdot (B^2d + Bd^2) = nBd + nd^2. \quad (113)$$

最小化时取 $B^2d = Bd^2$, 即 $B = d$, 总量 $O(nd^2)$ (与纯线性注意力相同)。工程上取 $B = 512 \sim 2048$, 在维持足够并行度 (充分利用 GPU) 的同时, 跨块状态传递的频率也不过高。

多尺度保留 (Multi-Scale Retention, MSR) 类似多头注意力，RetNet使用多个头，每头使用不同的衰减系数 γ_h ：

$$\gamma_h = 1 - 2^{-5-(h-1)\cdot\frac{8}{H}}, \quad h = 1, \dots, H, \quad (114)$$

按等比数列分布在 $[1 - 2^{-5}, 1 - 2^{-13}]$ 范围内（即 $[0.969, 0.9999]$ ）。不同头捕获不同距离尺度的依赖：快衰减头（小 γ ）专注近邻关系，慢衰减头（大 γ ）保留长程依赖。最终多头输出通过可学习门控融合：

$$\text{MSR}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O \odot \text{swish}(XW_G), \quad (115)$$

其中 W_G 是门控投影矩阵。

与其他架构的联系

架构	训练复杂度	推理KV规模	相对位置编码
Transformer	$O(n^2d)$	$O(nd)$	外部 (RoPE等)
Mamba (S6)	$O(ndN)$	$O(dN)$	内置 (衰减)
RetNet	$O(n^2d) / O(nd^2)$	$O(d^2)$	内置 (衰减 γ)
线性注意力	$O(nd^2)$	$O(d^2)$	无

从上表可见，RetNet在推理效率上与Mamba相当（固定大小状态），但训练路径可以走并行模式，不需要Mamba那样复杂的扫描实现，对框架友好度更高。代价是与线性注意力一样，省去softmax使得注意力分布可能不够“尖锐”，在需要精确匹配的任务上表现稍逊于Transformer。

工程意义 RetNet为“既要训练快、又要推理省”的场景提供了一条清晰路径。它说明，通过**合理的代数结构设计**（而不是单纯工程hack），可以做到训练与推理在同一套权重下切换计算模式。这一理念被后续的GLA（Gated Linear Attention, Yang et al., 清华/MIT团队）、HGRN2（Qin et al., 北京大学团队）等中国研究团队进一步发展，形成了“递推-并行对偶”这一高效序列建模的重要研究方向。

3.2.9 Google Titans：长期记忆作为测试时可学习模块

Titans的关键思想是把注意力视为**高精度短时记忆**，把神经记忆模块视为**可持续的长时记忆**。其记忆更新不直接依赖“存所有token”，而依赖**惊讶度 (surprise)** 驱动的梯度式写入。论文给出的核心递推为：

$$M_t = M_{t-1} + S_t, \quad (116)$$

$$S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(M_{t-1}; x_t), \quad (117)$$

其中 $\nabla \ell(M_{t-1}; x_t)$ 刻画当前输入 x_t 相对于已有记忆 M_{t-1} 的“瞬时惊讶度”，而 $\eta_t S_{t-1}$ 刻画最近一段时间的“过去惊讶度”。这相当于把优化中的动量项改造成跨时间的记忆追踪器。

从机制上看，Titans并不是单纯把上下文窗口拉长，而是在测试时持续学习一个**外部长期记忆状态**。这使它在超长序列上不再依赖完整保留所有token，而是依赖“什么值得被记住”的可学习压缩。

Titans提出了三种典型结构：

- **MAC (Memory as Context)**：把长期记忆编码成额外上下文token，与当前窗口共同参与注意力；
- **MAG (Memory as Gate)**：用门控机制融合短时注意力分支与长期记忆分支；
- **MAL (Memory as Layer)**：把记忆模块作为注意力之前的独立层。

公开结果显示Titans可扩展到2M以上上下文，并在needle-in-a-haystack这类长程检索任务上明显优于传统Transformer与线性递推基线。

对实际开发的指导作用 Titans类方法对真实系统的启发是：**长记忆不是把窗口做大，而是把记忆写入策略做对**。对Agent开发尤其如此，长期任务失败很多时候并非因为模型不会思考，而是因为会话历史没有被压缩成可再次利用的状态。因此，工作区摘要、任务状态对象、外部记忆表和“惊讶度驱动”的写入规则，应该被视作系统设计的一等公民。

3.2.10 TurboQuant：极限压缩下的无偏内积估计

TurboQuant解决的是**高维向量在极低比特率下如何仍然准确保留点积**的问题，这对KV cache压缩和向量检索都极其关键。设键向量 $k \in \mathbb{R}^d$ ，先做随机旋转：

$$u = Rk, \quad (118)$$

随后用PolarQuant做主压缩，得到粗量化结果 $\hat{u} = Q_{\text{polar}}(u)$ ；残差为

$$e = u - \hat{u}. \quad (119)$$

TurboQuant再用1-bit的QJL (Quantized Johnson-Lindenstrauss) 对残差做极低开销草图：

$$z = \text{sign}(Se), \quad (120)$$

其中 S 是随机投影矩阵。对任意查询 q ，其与 k 的点积可分解为：

$$q^\top k = q^\top R^{-1}u \approx q^\top R^{-1}\hat{u} + \widehat{q^\top R^{-1}e}_{\text{QJL}}. \quad (121)$$

这里第一项由PolarQuant捕获“主信号”，第二项用QJL对残差进行低比特无偏修正。这个“两阶段估计器”的价值在于：既保留了主方向信息，又避免了传统分块量化必须存储大量缩放常数所带来的额外元数据开销。对KV cache而言，这意味着在3-4 bit量级仍可较好地保留注意力分数 $q^\top k$ ，从而显著降低长上下文服务的内存和带宽瓶颈。

对实际开发的指导作用 对于推理系统，TurboQuant这类方法说明：量化设计的关键不只是减少bit数，而是**尽量保住真正影响决策的内积结构**。如果量化后 $q^\top k$ 排序被打乱，检索和注意力都会坏掉；如果主结构被保留、残差再被低成本修正，那么系统就能在较低内存下维持可接受质量。

3.2.11 其他关键系统算法：FlashAttention、Mamba与PagedAttention

FlashAttention的核心不是改变注意力公式，而是改变计算顺序。它把 Q, K, V 分块装入片上SRAM，在块内做online softmax，避免显式物化完整的 QK^T 矩阵。设当前处理到第 i 个块时的运行最大值、归一化常数与输出累计量分别为 m_i, l_i, o_i ，则可以递推为：

$$m_i = \max(m_{i-1}, \max S_i), \quad l_i = e^{m_{i-1}-m_i} l_{i-1} + \sum \exp(S_i - m_i), \quad (122)$$

$$o_i = e^{m_{i-1}-m_i} o_{i-1} + \sum \exp(S_i - m_i) V_i. \quad (123)$$

因此它在数学上仍是**精确注意力**，但在系统上实现了对HBM读写次数的近最优控制。

Mamba则将选择性状态空间模型写成：

$$h_t = A(\Delta_t) h_{t-1} + B_t x_t, \quad y_t = C_t h_t + D x_t, \quad (124)$$

其中 Δ_t, B_t, C_t 由输入决定。它的关键突破是让状态更新对内容敏感，从而弥补传统SSM不擅长内容路由的弱点。Mamba的推理复杂度线性随序列增长，非常适合长上下文与流式场景。

PagedAttention关注的是KV cache的**内存管理算法**。它把每个请求的逻辑KV序列拆成固定大小的页，并通过页表 $\pi(r, i)$ 把请求 r 的第 i 个逻辑页映射到物理块：

$$\text{addr}(r, i, o) = \text{base}(\pi(r, i)) + o. \quad (125)$$

这一设计把操作系统中的分页思想引入LLM serving，使KV缓存不再需要连续大块显存，从而显著减少碎片和冗余复制。在共享prompt、并发采样、多路beam search场景中，这种“逻辑连续、物理离散”的映射尤其关键。

综上，长上下文能力并不只是“把窗口调大”那么简单，而是**缓存设计、注意力结构、内存分配和近似估计**四个层面的联合结果。对于做RL推理模型或智能体系统的团队，真正决定上限的往往不是优化器，而是这些底层系统算法。

3.2.12 线性注意力：核函数近似与关联记忆的统一

标准softmax注意力的 $O(n^2)$ 计算复杂度来自显式构造 $n \times n$ 的注意力矩阵。Katharopoulos et al. (2020) 观察到：若将softmax用一个**正定核函数**近似，可利用矩阵乘法的结合律把复杂度降为 $O(n)$ 。

核函数替换与结合律技巧 标准注意力中第 i 行输出为：

$$o_i = \frac{\sum_{j=1}^n \exp(q_i^\top k_j / \sqrt{d}) v_j}{\sum_{j=1}^n \exp(q_i^\top k_j / \sqrt{d})}. \quad (126)$$

用特征映射 $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^r$ （核函数 $\kappa(q, k) = \phi(q)^\top \phi(k)$ ）替代 $\exp(q^\top k / \sqrt{d})$ ：

$$o_i = \frac{\sum_{j=1}^n \phi(q_i)^\top \phi(k_j) v_j^\top}{\sum_{j=1}^n \phi(q_i)^\top \phi(k_j)} = \frac{\phi(q_i)^\top \left(\sum_{j=1}^n \phi(k_j) v_j^\top \right)}{\phi(q_i)^\top \left(\sum_{j=1}^n \phi(k_j) \right)}. \quad (127)$$

由于 $\sum_j \phi(k_j) v_j^\top \in \mathbb{R}^{r \times d}$ 和 $\sum_j \phi(k_j) \in \mathbb{R}^r$ 均与 i 无关，可**一次性预计算**：

$$S = \sum_{j=1}^n \phi(k_j) v_j^\top \in \mathbb{R}^{r \times d}, \quad z = \sum_{j=1}^n \phi(k_j) \in \mathbb{R}^r, \quad (128)$$

则所有 n 个输出均可通过 $o_i = \phi(q_i)^\top S / \phi(q_i)^\top z$ 计算，总复杂度降为 $O(nrd)$ （ $r \ll n$ 时为线性）。

因果线性注意力与递推形式 在自回归场景中，第 i 步输出只能看到 $j \leq i$ 的历史，因此需要因果掩码版本：

$$o_i = \frac{\phi(q_i)^\top S_i}{\phi(q_i)^\top z_i}, \quad S_i = S_{i-1} + \phi(k_i)v_i^\top, \quad z_i = z_{i-1} + \phi(k_i). \quad (129)$$

这是一个**标准线性RNN**：状态 $S_i \in \mathbb{R}^{r \times d}$ 逐步累积历史信息，每步更新 $O(rd)$ ，推理时内存 $O(rd)$ （与序列长度无关）。这正是线性注意力与SSM模型（Mamba、Ret-Net）理论统一的桥梁——它们本质上都是带有不同核/衰减设计的线性RNN。

常用特征映射选择

- $\phi(x) = \text{elu}(x) + 1$ （Katharopoulos原版，保证非负）；
- $\phi(x) = \text{relu}(x)$ （简单实用，但存在零值问题）；
- $\phi(x) = \exp(x - \|x\|^2/2)$ （随机傅里叶特征，近似高斯核）；
- $\phi(x) = [1; x; x^2/\sqrt{2}; \dots]$ （多项式展开，阶数可控）。

线性注意力的代价是**近似精度**：替换softmax损失了归一化的”局部竞争”特性（softmax的温度效应），在需要精确全局配对的任务上表现不如精确注意力。但对于长程依赖、流式序列、计算受限场景，线性注意力提供了在精度与效率之间可调的权衡点。

3.2.13 RWKV：纯线性复杂度的可扩展语言模型

RWKV（Receptance Weighted Key Value, Peng Bo et al., 2023）是由中国独立研究者Peng Bo（彭博）主导开发的开源序列模型，已扩展到14B参数规模，是目前**唯一在语言建模任务上可与Transformer竞争、且保持严格线性复杂度**的模型之一。

WKV机制：时序混合核心 RWKV的核心是**WKV 算子**（Weighted Key Value）。设当前位置 t ，输入经过 token-mix 投影得到 r_t, k_t, v_t, w_t ，则 WKV 输出定义为：

$$\text{WKV}_t = \frac{\sum_{i=1}^{t-1} e^{-(t-1-i)w+k_i} v_i + e^{u+k_t} v_t}{\sum_{i=1}^{t-1} e^{-(t-1-i)w+k_i} + e^{u+k_t}}, \quad (130)$$

其中 $w \in \mathbb{R}^d$ 是**通道级时间衰减**（channel-wise time decay，可学习）， $u \in \mathbb{R}^d$ 是**当前位置的特殊偏置**（bonus，用于区别对待当前 token）。

递推形式（使数值计算稳定）：令 $a_t = \sum_{i < t} e^{-(t-1-i)w+k_i} v_i$ ， $b_t = \sum_{i < t} e^{-(t-1-i)w+k_i}$ ，则：

$$a_t = e^{-w} a_{t-1} + e^{k_{t-1}} v_{t-1}, \quad b_t = e^{-w} b_{t-1} + e^{k_{t-1}}, \quad \text{WKV}_t = \frac{a_t + e^{u+k_t} v_t}{b_t + e^{u+k_t}}. \quad (131)$$

推理时每步只维护 (a_t, b_t) 两个向量，内存为 $O(d)$ ，计算 $O(d)$ per step。

RWKV 的最终输出层为：

$$o_t = W_o \cdot (\sigma(r_t) \odot \text{WKV}_t), \quad (132)$$

其中 $\sigma(r_t)$ 是 Receptance 门控（sigmoid 激活），控制历史信息的”接受度”。

通道混合 (Channel Mix) 层 除时序混合外, RWKV 用**通道混合 (Channel Mix) 层**替代 FFN:

$$\text{cm}_t = \sigma(r'_t) \odot (W_{v'} \cdot (\text{relu}(W_{k'} \cdot x'_t))^2), \quad (133)$$

其中 $x'_t = \mu_x x_t + (1 - \mu_x) x_{t-1}$ (当前与前一步的插值, 增加局部上下文感知), $\text{relu}(\cdot)^2$ 称为 squared ReLU, Primer (So et al.) 论文表明其在语言建模中优于 GELU。

RWKV-5/6 的改进 RWKV-5 (Eagle) 引入多头 WKV (Multi-Head WKV), 将 d 维通道分成 H 组独立 WKV, 每组维度 d/H , 类比 MHA 的多头并行学习。RWKV-6 (Finch) 进一步使 w_t 动态依赖输入 ($w_t = W_w x_t$), 使衰减速率变为内容敏感, 弥补了早期版本固定衰减率的限制, 与 Mamba 的选择性机制趋同。

RWKV vs Mamba 对比

特性	RWKV	Mamba (S6)
状态大小 (推理)	$O(d)$ 向量	$O(dN)$ 矩阵
输入依赖衰减	RWKV-6起有	全版本均有
训练并行性	自定义 CUDA 核	并行前缀扫描
已扩展参数量	14B (公开)	2.8B (公开)
开源生态	完整 (rwkv.cpp等)	快速增长

3.2.14 稀疏注意力与滑动窗口: 局部-全局混合策略

完全全局注意力 ($O(n^2)$) 与完全线性近似之间, 还有一大类**稀疏注意力**方法: 保留全局注意力的精确性, 但只对部分 token 对计算, 将整体复杂度降至 $O(n \log n)$ 或 $O(nw)$ (w 为窗口大小)。

滑动窗口注意力 (Sliding Window Attention, SWA) 最简单也最实用的稀疏模式: 每个 token 只与距离不超过 $w/2$ 的邻近 token 做精确注意力:

$$A_{ij} = \begin{cases} \text{softmax}(\frac{q_i^\top k_j}{\sqrt{d}}) & |i - j| \leq w/2 \\ 0 & \text{otherwise} \end{cases} \quad (134)$$

复杂度 $O(nw)$, 内存 $O(nw)$ 。Mistral-7B (Jiang et al., 2023) 采用 $w = 4096$ 的 SWA, 配合滑动窗口 KV Cache (每步只保留最近 w 步), 在 32K 上下文下内存占用与短上下文几乎一致。

堆叠 L 层 SWA 后, 第 ℓ 层的“感受野”为 $\ell \cdot w$, 因此足够深的网络仍可捕获全局依赖, 只是需要多层传递。

Longformer: 局部 + 全局注意力混合 Beltagy et al. (2020) 提出 Longformer, 将注意力分为三类:

1. **局部滑动窗口**: 大多数 token 只关注窗口内邻居, 复杂度 $O(nw)$;
2. **扩张滑动窗口 (dilated)**: 以步长 d_s 跳跃采样, 在不增加计算量的情况下扩大感受野;
3. **全局 token**: 少数特殊位置 (如 [CLS]、问答中的问题 token) 与**所有**其他位置做双向全局注意力。

混合后总复杂度仍为 $O(n(w + g))$, 其中 g 是全局 token 数量 (通常远小于 n)。

BigBird: 随机 + 局部 + 全局稀疏模式 Zaheer et al. (Google, 2020) 证明：若注意力图满足以下三个成分的并集，则序列建模的表达能力和完全注意力等价：

1. g 个全局 token（与所有位置互相连接）；
2. 局部滑动窗口（每个 token 关注 w 个邻居）；
3. r 条随机边（每个 token 随机关注 r 个位置，捕获长程随机连接）。

这一稀疏图的理论支撑是随机图论：当全局、局部、随机三类边组合时，图的直径以高概率为 $O(\log n)$ ，信息传播路径足够短，与完全图近似等价。

Flash Attention 2与分块矩阵优化 FlashAttention (Dao et al., 2022) 在一代基础上，FA-2 (2023) 进一步优化了：

1. **工作分配**：将 n 个查询平均分配给 GPU 的 SM（流多处理器），避免 FA-1 中 KV 分配引起的负载不均；
2. **非矩阵乘法 FLOPs 最小化**：FA-1 中标量运算（exp、归一化）占总时间约 40%，FA-2 将这些操作融合进矩阵乘法的尾声，减少独立核调用；
3. **序列并行支持**：对 n 维度也做分块，允许跨多 GPU 的超长序列注意力计算（与 Ring Attention 结合）。

FA-2 在 A100 上实现了 $\sim 72\%$ 的理论最大 FLOP 利用率（FA-1 约为 $30 \sim 35\%$ ），是目前工业界部署 Transformer 的事实标准注意力实现。

Ring Attention: 分布式超长上下文 Li et al. (2023) 提出 Ring Attention，将超长序列（如 $n > 1\text{M}$ ）沿序列维度切分到多个设备。设 K 台设备，每台持有 n/K 个 query 和对应的 KV 分片，设备以环形拓扑（ring topology）依次传递 KV 分片，每台在接收到 KV 时计算局部注意力输出：

$$o_i^{\text{local}} = \text{FlashAttn}(q_i, K_{\text{local}}, V_{\text{local}}), \quad (135)$$

通过在线 softmax（online softmax with log-sum-exp 累积），各分片的局部输出可无缝合并为全局精确结果。Ring Attention 使 n 可随设备数 K 线性扩展，理论上支持无限长上下文，是 Google 在 Gemini Ultra 长上下文版本中采用的分布式注意力策略。

3.2.15 推理速度与成本：投机解码、MoE细粒化与专用芯片

如果说上文讨论的KV Cache与注意力算法是“能不能跑”的问题，那本节聚焦的是**能不能用得起**——即在高频交互、实时反馈的产品场景下，如何同时控制延迟与成本。这是大模型工程化最后一公里的核心障碍，也催生了三类截然不同的技术路线。

投机解码 (Speculative Decoding) Transformer自回归解码的本质瓶颈不在计算，而在**内存带宽**：每生成一个token都需要将数十亿参数从HBM读入SRAM，而算术运算仅发生在极短的时间窗口内。GPU/TPU的计算单元此时大量空闲——这是显著的“内存墙” (memory-bound) 特征。

投机解码 (**Speculative Decoding**, Leviathan et al., 2023; Chen et al., 2023) 正是针对这一带宽瓶颈而设计的。其核心思想是引入一个**小草稿模型** (draft model) q 与大目标模型 p 并行：

1. 草稿模型 q (参数量通常为 目标模型的 1/10 以下) 快速自回归地生成 γ 个候选 token $\tilde{x}_1, \dots, \tilde{x}_\gamma$;
2. 目标模型 p 对这 γ 个 token **并行打分** (一次前向传播得到 γ 个位置的 logits) ;
3. 通过拒绝采样 (rejection sampling) 决定接受或拒绝每个候选 token: 若 $\tilde{x}_t$ 被拒绝, 则从修正分布 $\max(0, p(\cdot) - q(\cdot))$ 中重新采样, 并丢弃其后所有候选。

数学上可以证明, 该过程产生的输出分布与目标模型 p 完全等价 (无偏性), 即投机解码是**无损加速**:

$$\mathbb{E}[\text{接受长度}] = \frac{\gamma}{1 - \alpha}, \quad \alpha = \mathbb{E}_{x \sim q} \left[\min \left(1, \frac{p(x)}{q(x)} \right) \right]. \quad (136)$$

其中 α 是平均接受率。当草稿模型与目标模型分布接近时 $\alpha \rightarrow 1$, 期望每次大模型前向传播可接受多个 token, 实测延迟降低 **2 - 3 倍**。

工程实践中, 草稿模型可以是: 同家族的小参数版本 (如 Llama-7B 为 Llama-70B 打草稿); 专门蒸馏训练的 tiny decoder; 甚至目标模型本身的浅层 (self-speculative decoding, 利用早退出机制生成草稿)。关键约束是草稿模型**不得显著增加显存占用**, 否则批处理规模会下降, 反而抵消加速收益。

混合专家架构 (MoE) 的极致细粒化 稠密模型 (Dense Model) 的推理成本与参数量线性相关; 而**混合专家 (Mixture of Experts, MoE)** 架构将前馈层替换为 E 个专家网络 $\{F_1, \dots, F_E\}$, 并通过可学习的**路由网络** G (gating network) 为每个 token 动态选择 k 个专家:

$$\text{MoE}(x) = \sum_{i \in \text{TopK}(G(x), k)} G(x)_i \cdot F_i(x). \quad (137)$$

这使模型拥有 $E \times d_{\text{expert}}$ 级别的总参数量, 但每个 token 仅激活 k/E 的参数, 推理 FLOPs 大幅降低。

近年趋势是将专家切得**越来越碎** (fine-grained experts): 从早期 Mixtral 的 8 个大专家、每次激活 2 个, 演进到 DeepSeek-MoE 的 256 个小专家、每次激活 8 个 (激活比例 $k/E = 8/256 \approx 3\%$)。极致细粒化的优势有三:

- **计算效率更高**: 激活参数量更小, 单 token 前向传播 FLOPs 显著减少;
- **专家专业化更强**: 每个小专家的“职责域”更聚焦, 路由信号质量更高, 负载均衡更易实现;
- **KV Cache 压力更小**: 前馈层参数不进 KV Cache, 减少专家数量等价于减少单次激活的权重 I/O, 系统吞吐上升。

工程层面, MoE 的核心挑战是**专家并行 (Expert Parallelism)**: 不同专家须部署在不同设备上, all-to-all 通信成为瓶颈, 网络拓扑和路由策略对实际吞吐影响极大。DeepSeek-V3 的实践表明, 结合无辅助损失的路由策略 (auxiliary-loss-free load balancing) 与共享专家 (shared expert), MoE 可以在保持稠密模型质量的同时将推理成本压低至 1/3 以下。

专用推理芯片（以Groq LPU为例） GPU的架构设计初衷是并行计算密集型任务（图形渲染、矩阵乘），其内存层次（HBM → L2 → L1 → SRAM → ALU）在LLM自回归推理这一**序列生成**场景下存在根本性的带宽-延迟矛盾：每生成一个token，权重矩阵须从HBM搬运至计算核心，HBM带宽（约3 TB/s）成为硬性天花板。

Groq LPU (Language Processing Unit) 是针对这一痛点从头设计的专用推理芯片，其核心设计决策是：

- **确定性架构 (Deterministic Architecture)**：摒弃GPU的动态调度，所有指令的执行时间在编译期完全确定。这消除了缓存未命中与动态调度开销，使延迟可预测；
- **SRAM-centric 设计**：大幅扩展片上SRAM容量，将模型权重（至少关键层）**常驻片上**，消除HBM数据搬运的周期性开销；
- **单流水线顺序执行**：LPU每次执行一个确定的计算图分支，避免GPU多流调度产生的同步等待；
- **张量流水线 (Tensor Streaming)**：权重在计算核心间流动而非数据在权重间流动，与传统GPU架构的数据流方向相反，极大提升有效带宽利用率。

Groq在公开基准上实现了对70B级Llama模型每秒超过**500 token/s**的生成速度（GPU通常在100 - 200 token/s范围），首token延迟（TTFT）也因片上常驻而显著降低。其代价是：单芯片可容纳的参数量受限于SRAM总量，超大模型需多芯片互联（GroqRack），扩展成本较GPU更高。

类似理念在其他专用芯片中也有体现：Cerebras的晶圆级芯片（Wafer-Scale Engine）将整个模型置于单块晶圆上以消除芯片间互联；SambaNova的循环数据流（Reconfigurable Dataflow）架构将运算编排为静态图以消除动态调度损耗。这些方案共同揭示了一个趋势：**大模型推理的工程优化正在从通用硬件调优转向领域专用硬件设计**，而未来系统架构师需要在通用性、成本与极限性能之间做出更精细的权衡。

三种加速路线的对比

方法	加速原理	典型收益	主要代价
投机解码	并行验证多草稿token	2 - 3×延迟降低	需配套草稿模型
MoE细粒化	稀疏激活减少有效FLOPs	3 - 5×成本降低	专家并行通信开销
专用ASIC	片上SRAM消除HBM带宽墙	5 - 10×吞吐提升	可容纳参数量受限

三条路线可以叠加：部署MoE稀疏模型（降低激活FLOPs），配合投机解码（隐藏内存带宽延迟），跑在专用推理芯片上（消除数据搬运瓶颈），理论上可实现数量级以上的端到端加速。对于智能体系统而言，每轮工具调用与多步推理对延迟极为敏感，这些硬件与算法联合优化正成为决定产品体验上限的关键因素。

3.2.16 长记忆挑战的两条技术路线综述

”大模型长记忆问题”是指如何在超长上下文中准确提取信息，同时不使内存和算力开销失控。这是当前AI工程最核心的攻坚战之一，本章已经讨论的各类算法恰好对应了两条截然不同但又互补的解决路线：

路线一：底层模型算法的硬核重构 这一路线试图从数学与架构上消灭传统Transformer $O(N^2)$ 复杂度带来的“内存墙”，其核心手段可归纳为四类：

1. **固定状态替代无限KV缓存**：Mamba/SSD、RWKV等状态空间模型用固定大小隐状态 s_t 压缩全部历史，推理时内存为 $O(1)$ 。它们像人类用“总体印象”替代“逐字录音”。
2. **压缩记忆双轨并行**：Infini-attention维护一个可写可读的压缩记忆矩阵 M ，使模型对近期局部上下文保持高精度注意力，同时对远历史保留 $O(1)$ 有损摘要访问。
3. **选择性驱逐而非无限累积**：StreamingLLM保留“沉淀token锚点”与滑动近文窗口，H2O/SnapKV则通过重要性打分保留最关键的KV对，将缓存压缩至原来的10 - 20%而几乎不损失性能。
4. **物理压缩降低每对KV字节数**：TurboQuant通过极坐标参数化与1-bit残差量化将KV元素压缩至2-bit级别；Titans则在测试时可学习地将关键历史写入外部模块权重，绕过显存瓶颈。

路线二：智能体（Agent）系统层的外挂式管理 当底层模型算法尚无法覆盖无限长历史时，上层智能体系统通过“外挂硬盘”和“记忆管理机制”来弥补。这部分内容将在第8章中详细展开，包括MemGPT（操作系统级记忆分级）、GraphRAG（知识图谱结构化检索）以及三层认知记忆架构（工作记忆、情景记忆、语义记忆）。

两条路线的关系 两条路线并非互斥，而是互补的分工：底层算法决定模型的“物理记忆带宽上限”，上层系统决定在给定带宽下“哪些记忆值得使用”。长期来看，最终解局大概率是两者的融合——以轻量化SSD/Mamba基座为内核，以MemGPT+GraphRAG式智能操作系统为外壳，赋予AI真正意义上的终生学习与记忆能力。

本节小结：系统算法对模型上限的决定作用 本节前半部分讲的是“如何训练出更强策略”，而这一小节强调的是“强策略能否以现实成本跑起来”。对工业系统而言，真正的能力上限往往由以下链条共同决定：

$$\text{usable intelligence} = f(\text{policy quality}, \text{memory system}, \text{latency budget}, \text{tool runtime}). \quad (138)$$

也就是说，一个理论上更强的模型，如果被KV cache、内存碎片、浏览器延迟或工具调用失败率压垮，最终在真实任务中的可用智能反而更低。

4 强化学习算法

在第2节的理论基础之上，本节对工业界最常用的五类核心训练方法给出完整的数学推导和实现要点：以裁剪目标为核心的PPO、以组内归一化替代Critic的GRPO、将奖励模型隐式化的DPO、以AI替代人类标注的RLAIF，以及指导上述方法落地的奖励函数设计原则。每种方法均给出算法框架、关键公式和工程取舍。本节最后以DeepSeek-R1和OpenAI o1为代表案例，展示这些算法如何在真实系统中组合使用。

4.1 PPO算法

4.1.1 算法动机

Proximal Policy Optimization (PPO) 是目前最广泛使用的策略梯度算法之一。它试图同时解决策略梯度方法的两个经典痛点：一是样本效率低——传统策略梯度是on-policy方法，每次更新后数据即失效；二是训练不稳定——策略更新步长难以控制，容易导致性能崩溃。PPO的核心思路是引入重要性采样和裁剪机制，允许多次使用同一批数据进行更新，同时严格限制策略变化的幅度。

4.1.2 数学推导

PPO的核心是裁剪目标函数。首先定义策略比率：

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (139)$$

原始的重要性采样目标为：

$$L^{IS}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right] \quad (140)$$

但直接优化该目标可能导致策略变化过大。PPO引入裁剪机制：

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (141)$$

其中 ϵ 是裁剪参数，通常取0.1到0.2。

注记 4.1. 裁剪机制的直觉可以这样理解：当 $\hat{A}_t > 0$ 时，说明当前动作是好的，我们希望提高其概率，但裁剪限制了 $r_t(\theta) \leq 1 + \epsilon$ ，防止过度增加该动作概率而导致策略大幅偏离；当 $\hat{A}_t < 0$ 时，说明当前动作是坏的，我们希望降低其概率，但裁剪限制了 $r_t(\theta) \geq 1 - \epsilon$ ，防止过度惩罚。也就是说，裁剪机制让模型每次只迈一小步，而不是一步跳太远。

4.1.3 完整目标函数

PPO的完整目标函数包含三部分：

$$L(\theta) = L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_\theta] \quad (142)$$

其中 $L^{VF}(\theta) = (V_\theta(s_t) - V_t^{target})^2$ 为值函数损失， $S[\pi_\theta] = -\sum_a \pi_\theta(a|s) \log \pi_\theta(a|s)$ 为熵正则化项（鼓励探索）， c_1 和 c_2 为平衡系数。值函数损失确保基线估计足够准确，而熵正则化防止策略过早崩缩到某个确定性动作上。

4.1.4 算法流程

```

初始化策略网络 $\pi_\theta$ 和值网络 $V_\phi$ 
for iteration = 1, 2, ... do
  for actor = 1, 2, ..., N do
    使用当前策略 $\pi_{\theta_{old}}$ 采集轨迹
    计算奖励和优势估计 $\hat{A}_t$ 
  end for
  for epoch = 1, 2, ..., K do
    for minibatch in collected data do
      计算策略比率 $r_t(\theta)$ 
      计算裁剪目标 $L^{CLIP}$ 
      更新策略:  $\theta \leftarrow \theta + \alpha \nabla_\theta L^{CLIP}$ 
      更新值函数:  $\phi \leftarrow \phi - \beta \nabla_\phi L^{VF}$ 
    end for
  end for
   $\theta_{old} \leftarrow \theta$ 
end for

```

Algorithm 1: PPO算法

4.1.5 PPO的变体与改进

1. PPO-Penalty: 使用KL惩罚代替裁剪

$$L^{KL PEN}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t - \beta \mathbb{D}_{KL}[\pi_{\theta_{old}} \parallel \pi_\theta] \right] \quad (143)$$

自适应调整 β :

- 如果 $\mathbb{D}_{KL} > d_{target} \times 1.5$: $\beta \leftarrow 2\beta$
- 如果 $\mathbb{D}_{KL} < d_{target}/1.5$: $\beta \leftarrow \beta/2$

2. Dual-Clip PPO: 双边裁剪处理负优势

$$L^{DCLIP}(\theta) = \mathbb{E}_t \left[\max \left(\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right), c \hat{A}_t \right) \right] \quad (144)$$

其中 $c > 1$ 是下界系数, 防止负优势时策略变化过快。

3. PPO with Rollback: 检测异常并回滚

```

if  $\mathbb{D}_{KL} > d_{max}$  or reward_drop > threshold then
   $\theta \leftarrow \theta_{checkpoint}$ 
  减小学习率
end if

```

Algorithm 2: PPO with Rollback

4.1.6 PPO在LLM中的特殊考虑

Token级vs序列级：LLM中的PPO可以在两个层面操作。Token级方法把每个token视为一个独立动作，计算每步的优势 $\hat{A}_t^{token} = r_t + \gamma V(s_{t+1}) - V(s_t)$ ，信号更密集但计算开销更大。序列级方法则把整个回答视为一个动作，优势简化为 $\hat{A}^{seq} = R(q, o) - V(q)$ ，实现简单但丢失了中间步的信息。实践中序列级更常用，因为它避免了为每个token训练精确的值函数。

Value Head设计方面有三种主要方案：共享backbone加独立value head是最常见的做法，内存效率高但可能导致策略头和值头互相干扰；完全独立的value网络更稳定但内存和计算成本翻倍；冻结backbone只训练value head是一种折中方案，适合担心值函数训练干扰策略质量的场景。

4.2 GRPO算法

4.2.1 算法动机

Group Relative Policy Optimization (GRPO) 是DeepSeek团队提出的简化RL算法，主要解决PPO在LLM训练中的以下问题：

1. **Critic网络开销大：**需要训练额外的值函数网络，内存和计算成本高
2. **值函数估计不准：**LLM生成任务中，状态空间巨大，值函数难以准确估计
3. **训练复杂度高：**需要同时优化Actor和Critic，超参数调节困难

GRPO的核心思想是：**用组内相对排名代替绝对价值估计。**

4.2.2 数学推导

对于每个问题 q ，GRPO采样一组回答 $\{o_1, o_2, \dots, o_G\}$ ，计算每个回答的奖励 $\{r_1, r_2, \dots, r_G\}$ 。优势估计通过组内归一化计算：

$$\hat{A}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r}) + \epsilon} \quad (145)$$

其中 $\mathbf{r} = (r_1, r_2, \dots, r_G)$ ， ϵ 是数值稳定性常数。
GRPO的目标函数为：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, \{o_i\} \sim \pi_{\theta_{old}}} \left[\frac{1}{G} \sum_{i=1}^G \mathcal{L}_i(\theta) \right] \quad (146)$$

其中每个样本的损失为：

$$\mathcal{L}_i(\theta) = \min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} \hat{A}_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_i \right) \quad (147)$$

同时加入KL散度约束防止策略偏离过远：

$$\mathcal{L}_{total}(\theta) = \mathcal{J}_{GRPO}(\theta) - \beta \cdot \mathbb{D}_{KL}[\pi_{\theta} \parallel \pi_{ref}] \quad (148)$$

4.2.3 与PPO的对比

Table 4: GRPO与PPO对比

特性	PPO	GRPO
Critic网络	需要	不需要
内存占用	高 (2倍模型)	低 (1倍模型)
优势估计	GAE	组内归一化
采样方式	单样本/少量	组采样 (多个)
值函数损失	需要优化	无
实现复杂度	高	低
训练稳定性	依赖Critic质量	组内比较更稳定

4.2.4 算法流程

```

初始化策略网络 $\pi_\theta$ , 设置参考策略 $\pi_{ref} = \pi_\theta$ 
for iteration = 1, 2, ... do
  for 每个问题  $q$  in batch do
    采样 $G$ 个回答:  $\{o_1, \dots, o_G\} \sim \pi_{\theta_{old}}(\cdot|q)$ 
    计算奖励:  $r_i = R(q, o_i)$ ,  $i = 1, \dots, G$ 
    归一化优势:  $\hat{A}_i = (r_i - \bar{r})/(\sigma_r + \epsilon)$ 
  end for
  for epoch = 1, ...,  $K$  do
    计算策略比率和裁剪损失
    计算KL惩罚
    更新策略:  $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}_{total}$ 
  end for
   $\theta_{old} \leftarrow \theta$ 
end for

```

Algorithm 3: GRPO算法

4.2.5 GRPO的理论分析

定理 4.2 (GRPO的无偏性). 在组大小 $G \rightarrow \infty$ 时, GRPO的梯度估计是无偏的:

$$\lim_{G \rightarrow \infty} \mathbb{E} [\nabla_\theta \mathcal{J}_{GRPO}(\theta)] = \nabla_\theta J(\theta) \quad (149)$$

证明sketch:

1. 当 $G \rightarrow \infty$ 时, $\bar{r} \rightarrow \mathbb{E}_{\pi_\theta}[r]$
2. 归一化后的优势 $\hat{A}_i$ 近似于真实优势的标准化版本
3. 由于裁剪是对称的, 期望不变

方差分析：GRPO的方差主要来源于：

$$\text{Var}[\hat{A}] = \frac{1}{G-1} \text{Var}[r] + O(1/G^2) \quad (150)$$

增加组大小 G 可以有效降低方差，但会增加计算成本。

4.2.6 GRPO变体

1. Weighted GRPO：引入样本权重

$$\hat{A}_i^{\text{weighted}} = w_i \cdot \frac{r_i - \bar{r}_w}{\sigma_r + \epsilon} \quad (151)$$

其中 w_i 可以基于问题难度、样本新颖性等因素确定。

2. Hierarchical GRPO：分层组采样

1. 按问题难度分层
2. 每层独立进行组内归一化
3. 加权聚合不同层的梯度

3. Adaptive GRPO：自适应组大小

$$G_q = G_{\text{base}} \cdot \exp(\alpha \cdot \text{difficulty}(q)) \quad (152)$$

难题使用更大的组以获得更稳定的梯度估计。

4.3 直接偏好优化 (DPO)

DPO是另一种无需显式奖励模型的RL方法。

4.3.1 核心理念

DPO直接从偏好数据学习，避免了奖励建模和采样的复杂性。

定义 4.3 (Bradley-Terry模型). 给定偏好对 (y_w, y_l) ，偏好概率为：

$$P(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l)) \quad (153)$$

其中 σ 是sigmoid函数。

4.3.2 DPO目标函数

通过重参数化，DPO导出直接优化策略的目标：

$$\mathcal{L}_{DPO}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right] \quad (154)$$

4.3.3 DPO vs RLHF

Table 5: DPO与RLHF对比

方面	DPO	RLHF (PPO)
奖励模型	隐式	显式
采样需求	无	大量在线采样
内存消耗	低	高
训练稳定性	高	需要调参
数据效率	高	中
探索能力	弱	强
适用场景	偏好对齐	复杂任务优化

4.4 基于AI反馈的强化学习（RLAIF）

4.4.1 核心框架

RLAIF使用AI模型替代人类提供反馈，整体流程分为三个阶段：首先由策略模型针对每个问题生成多个候选回答，然后由一个独立的评判模型对这些候选进行打分或排序，最后将AI反馈作为奖励信号进行RL训练。这种方法的核心优势在于完全省去了人工标注的成本，同时可以以极低成本生成大规模偏好数据。

4.4.2 评判模型设计

评分Prompt模板：

请评估以下回答的质量（1-10分）：

问题：{question}
回答：{answer}

评分标准：

- 准确性（40%）
- 完整性（30%）
- 清晰度（20%）
- 简洁性（10%）

请给出分数和理由：

对比Prompt模板：

以下是同一问题的两个回答，请选择更好的一个：

问题：{question}
回答A：{answer_a}
回答B：{answer_b}

请选择A或B，并解释原因：

4.4.3 RLAIIF的变体

1. Self-RLAIF: 模型自评

$$r(x, y) = \mathbb{E}_{prompt \sim \mathcal{P}} [\pi_{eval}(\text{"good"} | prompt, x, y)] \quad (155)$$

2. Ensemble RLAIF: 多评判者集成

$$r_{ensemble}(x, y) = \frac{1}{K} \sum_{k=1}^K r_k(x, y) \quad (156)$$

3. Debate RLAIF: 辩论式评估 让多个模型对回答质量进行辩论，综合辩论结果得出最终评分。

4.5 奖励函数设计

奖励函数是RL训练的核心，设计原则包括：

4.5.1 可验证奖励

对于数学和代码任务，可以设计自动验证的奖励：

数学任务奖励：

$$R_{math}(q, o) = \begin{cases} +1 & \text{如果答案正确} \\ 0 & \text{如果答案错误} \end{cases} \quad (157)$$

答案匹配可以采用多种方式：最简单的是字符串精确匹配；更稳健的做法是数值匹配，在给定容差范围内判断相等；最强力的符号匹配，使用SymPy等符号计算库验证表达式的等价性。在实际系统中，三种方式通常级联使用：先尝试精确匹配，否则尝试数值匹配，最后回退到符号匹配。

代码任务奖励：

$$R_{code}(q, o) = \frac{\text{通过的测试用例数}}{\text{总测试用例数}} \quad (158)$$

还可以加入额外奖励，例如对编译成功给予小额正奖励、根据运行时间和空间复杂度给予运行效率奖励、以及基于静态分析的代码风格奖励。这些辅助奖励通常权重较小，主要起引导作用。

4.5.2 格式奖励

为确保输出格式正确，可以加入格式奖励：

$$R_{format}(o) = \begin{cases} r_+ & \text{如果格式正确（包含指定标签）} \\ r_- & \text{如果格式错误} \end{cases} \quad (159)$$

例如，要求输出包含<think>...</think>推理过程和<answer>...</answer>最终答案。

4.5.3 长度惩罚

防止输出过长或过短：

$$R_{length}(o) = -\lambda \cdot \max(0, |o| - L_{max}) \quad (160)$$

4.5.4 复合奖励

综合多种奖励：

$$R_{total} = \alpha R_{accuracy} + \beta R_{format} + \gamma R_{length} \quad (161)$$

4.5.5 过程奖励与结果奖励的数学分析

结果奖励模型 (ORM) :

$$R_{ORM}(q, o) = f(\text{final_answer}(o), \text{ground_truth}(q)) \quad (162)$$

过程奖励模型 (PRM) :

$$R_{PRM}(q, o) = \sum_{t=1}^T \gamma^{T-t} r_t(s_t) \quad (163)$$

其中 r_t 是第 t 步的奖励。

混合奖励：

$$R_{hybrid} = \alpha R_{ORM} + (1 - \alpha) R_{PRM} \quad (164)$$

理论分析：PRM 提供更密集的信号，但存在几方面权衡。在信用分配上，PRM 能更精确地将奖励归因到正确或错误的具体步骤，而 ORM 只知道最终结果对不对，无法告诉模型哪一步出了问题。但在标注成本上，PRM 需要步骤级标注，成本远高于 ORM 的结果级标注。泛化性方面，PRM 对步骤格式较为敏感，当模型生成的推理风格偏离训练分布时，PRM 的打分可能不可靠。另外，PRM 更容易被 Reward Hacking，因为模型可以学会生成“看起来正确”但实际无实质内容的中间步骤。

4.5.6 奖励塑形 (Reward Shaping)

定理 4.4 (势能奖励塑形). 设 $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ 为势函数，定义塑形奖励：

$$R'(s, a, s') = R(s, a, s') + \gamma \Phi(s') - \Phi(s) \quad (165)$$

则最优策略不变。

LLM 中的应用：在实践中，势能奖励塑形常用于为中间状态提供额外信号：例如对包含关键推理关键词或结构的状态给予正势能，或者设计进度奖励 $\Phi(s) = \alpha \cdot \text{progress}(s)$ 来衡量模型走了多少步，以及多样性奖励 $\Phi(s) = -\beta \cdot \text{repetition}(s)$ 来惩罚重复生成。这些势能函数不会改变最优策略，但能显著加快学习速度。

4.6 代表案例：DeepSeek-R1 纯 RL 训练范式

DeepSeek-R1 将本章所有算法综合付诸实践——GRPO、规则可验证奖励、拒绝采样自训练与四阶段渐进训练管线系统性融为一体，是目前最具代表性的开源纯 RL 推理模型，也是理解强化学习如何在真实大模型训练中产生效果的最佳案例之一。

4.6.1 项目概述

DeepSeek-R1是DeepSeek团队于2025年1月发布的推理模型，是首个公开技术细节的纯RL训练推理模型。其核心贡献包括：

1. 证明了**纯RL训练可以涌现复杂推理能力**
2. 展示了**无需人类标注的自我学习范式**
3. 提出了**高效的GRPO算法**
4. 开源了**完整的模型和蒸馏版本**

4.6.2 训练流程

DeepSeek-R1采用四阶段训练流程：

Stage 1: 冷启动SFT 目的：为RL训练提供良好的初始化

数据：数千条长链推理（Chain-of-Thought）数据，包括：

- 人工标注的详细推理过程
- 已有模型生成的高质量推理链
- 格式化的推理模板

训练细节：

- 学习率： 1×10^{-5}
- 训练轮数：2-3 epochs
- 数据量：约2000-5000条

Stage 2: 推理RL 目的：通过RL训练提升推理能力

算法：GRPO

奖励设计：

$$R = R_{accuracy} + R_{format} \quad (166)$$

- $R_{accuracy}$ ：答案正确性，通过规则验证
- R_{format} ：格式正确性，检查是否包含必要标签

关键超参数：

Table 6: DeepSeek-R1-Zero推理RL关键超参数

参数	值
组大小 G	64
裁剪参数 ϵ	0.2
KL系数 β	0.01
学习率	1×10^{-6}
批大小	512
最大长度	32768

Stage 3: 拒绝采样SFT 目的：利用RL模型生成高质量数据，进行SFT以提升可读性

流程：

1. 使用Stage 2的RL检查点生成大量回答
2. 根据正确性和质量筛选
3. 对筛选后的数据进行SFT

筛选标准：

- 答案正确
- 推理过程清晰
- 语言流畅，无混杂
- 长度适中

Stage 4: 全场景RL 目的：扩展到更多任务类型，保持通用能力

任务类型：

- 数学推理
- 代码生成
- 逻辑推理
- 常识问答
- 写作任务

奖励设计：针对不同任务使用不同的奖励函数

4.6.3 DeepSeek-R1-Zero消融实验

R1-Zero是一个重要的消融实验，**完全不使用SFT**，直接从Base模型开始RL训练。

实验设置

- 基础模型：DeepSeek-V3 Base
- 奖励：纯规则奖励（准确性 + 格式）
- 无任何人类标注数据

涌现现象 训练过程中观察到多种涌现行为：

1. 自我验证：模型学会检查自己的答案

" Let me verify this answer by substituting back..."

2. 反思与纠错：发现错误后重新思考

" Wait, I made an error in step 3. Let me recalculate..."

3. 多路径探索：尝试不同的解题方法

" There are two approaches to this problem. Let me try the algebraic method first..."

4. Aha Moment：顿悟式的突破

" Hmm, I see! The key insight is that..."

R1-Zero的局限性 R1-Zero存在以下问题：

- 可读性差：输出冗长、结构混乱
- 语言混杂：英文、中文、代码混合
- 格式不稳定：有时不遵循指定格式

这些问题通过后续的SFT阶段得到解决。

4.6.4 性能表现

Table 7: DeepSeek-R1性能对比

基准测试	DeepSeek-R1	OpenAI o1	Claude 3.5
AIME 2024	79.8%	83.3%	16.0%
MATH-500	97.3%	96.4%	78.3%
Codeforces Rating	2029	1891	-
GPQA Diamond	71.5%	78.0%	65.0%

4.6.5 蒸馏到小模型

DeepSeek-R1的推理能力可以通过知识蒸馏迁移到小模型：

蒸馏方法

1. 使用R1生成大量高质量推理数据
2. 对小模型进行SFT
3. 可选：对小模型进行额外的RL训练

Table 8: 蒸馏模型性能

模型	参数量	AIME 2024	MATH-500
DeepSeek-R1	671B	79.8%	97.3%
R1-Distill-Qwen-32B	32B	72.6%	94.3%
R1-Distill-Qwen-14B	14B	69.7%	93.9%
R1-Distill-Qwen-7B	7B	55.5%	92.8%
R1-Distill-Qwen-1.5B	1.5B	28.9%	83.9%
OpenAI o1-mini	-	63.6%	90.0%

蒸馏效果

4.7 代表案例：OpenAI o1推理时计算扩展

o1是闭源推理模型的标杆，代表了另一条技术路线：以大规模步骤级过程奖励模型（PRM）为核心，配合大量人工标注，并在推理时通过计算扩展释放模型能力。与R1的开放透明不同，o1的训练细节至今多为推测，但其对推理时计算的开创性探索深刻影响了整个领域——它证明了“让模型思考更久”是提升能力的独立维度。

4.7.1 官方披露信息

OpenAI o1于2024年9月发布，官方披露的信息有限：

1. 使用大规模强化学习训练
2. 产生长链“思考”过程（对用户隐藏）
3. 性能随推理时间/token数量提升
4. 在数学、代码、科学推理上达到专家水平

4.7.2 技术推测

基于公开信息和研究社区分析，o1可能采用以下技术：

过程奖励模型（PRM） 根据OpenAI 2023年的论文“Let’s Verify Step by Step”，PRM可能是o1的核心组件：

定义 4.5 (过程奖励模型). PRM对推理过程中的每一步进行评分：

$$r_t = P(\text{step } t \text{ is correct} | s_1, s_2, \dots, s_t) \quad (167)$$

最终奖励为各步骤奖励的聚合。

PRM相比结果奖励模型（ORM）的优势：

- 提供密集监督信号
- 精确定位错误步骤
- 更好的credit assignment

PRM的训练需要大量步骤级人工标注，这是o1的核心技术壁垒之一。

大规模RL训练 推测o1使用了：

- PPO或类似算法
- 大规模分布式训练
- 精心设计的奖励函数（PRM + ORM混合）
- 大量计算资源

推理时搜索 o1可能在推理时使用某种形式的搜索：

- Best-of-N采样
- 树搜索（MCTS变体）
- 自适应计算量分配

4.7.3 推理时计算扩展

o1展示了一种新的扩展维度，即推理时计算扩展：

Performance = f (Model Size, Training Compute, Inference Compute) (168)

关键发现：

- 更多”思考”时间 → 更高准确率
- 难题需要更长的推理链
- 存在收益递减，但上限较高

这开辟了提升LLM能力的新途径：不仅可以训练更大的模型，还可以让模型”思考更久”。

4.7.4 o1与DeepSeek-R1的对比

Table 9: o1与R1技术路线对比

方面	OpenAI o1	DeepSeek-R1
奖励模型	PRM（推测）	规则奖励
人工标注	大量步骤级标注	极少量
RL算法	PPO（推测）	GRPO
思考过程	隐藏	公开
是否开源	否	是
训练成本	极高	相对较低

4.8 深度推理范式：System 2思维、过程奖励与树搜索

R1与o1的案例研究揭示了一个更宏观的范式转移：大模型的能力瓶颈正在从”训练时扩展”（模型更大、数据更多）向**测试时扩展**（推理时投入更多计算）迁移。理解这一转移的认知框架，需要回到心理学中的System 1与System 2对立理论。

4.8.1 System 1与System 2：直觉与深思的二元结构

诺贝尔经济学奖得主Daniel Kahneman将人类认知划分为两种模式：

- **System 1（直觉系统）**：快速、自动、无意识，依赖模式匹配。它能闪电般地“完成”看到老虎立刻后退”这类识别任务，但容易犯启发式偏差；
- **System 2（分析系统）**：缓慢、刻意、消耗认知资源，适合复杂逻辑推导。解一道需要多步验证的数学题，或推敲一个有内在矛盾的法律文本，都需要激活System 2。

早期的大语言模型（包括GPT-3.5、Claude 2等）在本质上是**System 1型系统**：训练期间见过的分布内任务（翻译、摘要、问答）表现出色，但面对需要多步严密推理的新颖问题（数学证明、代码调试、策略规划），单次自回归前向传播往往不足以给出正确答案——它们“本能反应”了，而没有“思考”。

DeepSeek-R1的思维链RL训练与OpenAI o1的隐藏推理过程，本质上都是在训练模型**主动激活System 2模式**：当问题足够复杂时，用更长的中间推理序列替代直接输出，以扩展的内部“思考”换取更高的最终准确率。

4.8.2 测试时计算扩展：推理作为第三扩展轴

传统的LLM扩展定律（Scaling Laws）关注两个维度：

$$\text{Performance} = f\left(\underbrace{N}_{\text{参数量}}, \underbrace{D}_{\text{训练数据量}}\right). \quad (169)$$

o1与R1的出现揭示了第三个扩展轴——**推理时计算量**（inference-time compute） C_{test} ：

$$\text{Performance} = f(N, D, C_{\text{test}}). \quad (170)$$

实验表明，在固定模型参数量与训练成本的前提下，允许模型生成更长的CoT推理链（或通过树搜索生成并验证多条候选链），性能可以在困难推理任务上持续提升——直到某个收益递减点。这一发现具有深远的工程含义：

- **不需要更大的模型**：通过让较小模型“多想一会儿”，可以逼近甚至超过更大稠密模型的性能；
- **推理预算可动态分配**：简单问题少思考（节约成本），难题多思考（保证质量），使推理成本与问题复杂度匹配；
- **与训练时扩展互补**：训练更强的PRM和更好的搜索策略，可以进一步释放测试时扩展的上限。

4.8.3 过程奖励模型（PRM）：步骤级监督信号

传统的**结果奖励模型（ORM, Outcome Reward Model）**仅对最终答案打分：正确得+1，错误得0。这在长链推理中存在根本性缺陷——即使中间某步骤推理严重错误，只要最后碰巧得出正确答案，ORM也会给出正向奖励（即“侥幸正确”），从而强化了错误的推理路径。

过程奖励模型（PRM, Process Reward Model）将奖励信号下放到**每一个推理步骤**：

$$R_{\text{PRM}}(s_1, s_2, \dots, s_T) = \sum_{t=1}^T r_t, \quad r_t = \text{PRM}(x, s_1, \dots, s_t). \quad (171)$$

其中 s_t 是第 t 个推理步骤， r_t 是PRM对该步骤的评分。

PRM的训练需要**步骤级别的人工标注**（标注每一步推理是否正确），这是其主要成本来源，也是OpenAI在o1上投入大量标注资源的原因之一。一旦PRM训练完毕，它可以：

- 在RL训练阶段提供密集奖励信号，大幅降低稀疏奖励带来的策略梯度方差；
- 在推理时作为**验证器（verifier）**，从候选推理链集合中筛选最高质量的答案（Best-of-N + PRM reranking）；
- 与树搜索结合，在每个推理节点上评估当前分支的前景，指导搜索资源的分配。

4.8.4 蒙特卡洛树搜索（MCTS）：结构化推理探索

当推理链需要多步决策且中间步骤存在分叉时，贪心自回归解码（每步取最高概率token）往往陷入局部最优。**蒙特卡洛树搜索（MCTS）**将推理过程建模为树形搜索问题：

- **节点**：当前推理状态（已生成的推理步骤前缀）；
- **边**：可能的下一步推理步骤；
- **值函数**：PRM对当前节点前景的估计。

标准MCTS包含四个循环操作：

1. **选择（Selection）**：从根节点出发，按UCB（Upper Confidence Bound）策略选择最有前景的子节点： $\text{UCB}(v) = \bar{Q}(v) + c\sqrt{\ln N(v_p)/N(v)}$ ；
2. **扩展（Expansion）**：在选中叶节点处，生成 k 个可能的后续推理步骤；
3. **模拟（Simulation）**：从扩展节点出发随机完成推理链，或直接用PRM估值；
4. **反向传播（Backpropagation）**：将模拟结果的奖励反向更新路径上所有节点的 $\bar{Q}$ 值与访问计数 N 。

在LLM推理场景，MCTS的实践变体倾向于将“步骤”粒度从token提升为“一个完整推理句子”，并以PRM替代蒙特卡洛随机模拟，形成**确定性MCTS**（Deterministic MCTS）或**束搜索+PRM筛选**（Beam Search + PRM Reranking）的混合方案。o1的推理时搜索机制被普遍推测包含此类结构（尽管OpenAI未公开细节）。

System 2推理的工程成本 需要指出的是，激活System 2模式并非免费午餐。长CoT推理会将输出token数增加5 - 20倍，相应地：

- 推理延迟与成本线性上升（直接影响产品体验）；
- KV Cache占用大幅增加（影响并发服务吞吐）；

- 存在“思维发散”风险——模型可能在无意义的中间步骤上浪费过多计算。

因此，测试时扩展需要与前文介绍的推理加速技术（投机解码、MoE稀疏激活、专用芯片）紧密结合，才能在实际产品中真正落地。这些技术路线之间形成了一个**能力-成本-速度**的三角权衡，是当前LLM工程实践中最核心的设计决策之一。

5 其他主要研究方法与前沿系统算法

本节分成两条主线：前半部分聚焦**RL、自训练与偏好优化方法**，后半部分聚焦**支撑长上下文与高吞吐推理的系统算法**。这样安排的目的是把“如何训练更强策略”与“如何让策略在长上下文和生产系统中真正跑起来”明确区分开。

5.1 Constitutional AI (Anthropic)

5.1.1 核心思想

Constitutional AI提出了RLAIF (RL from AI Feedback)，用AI反馈替代人类反馈：

1. **Self-Critique**：让模型评估自己的回答
2. **Revision**：根据“宪法”原则修改回答
3. **RL训练**：用AI评分作为奖励

5.1.2 宪法原则示例

- “选择最有帮助、最诚实、最无害的回答”
- “选择最不具有欺骗性的回答”
- “选择最尊重用户自主权的回答”

5.1.3 优势与局限

优势：

- 大幅减少人类标注需求
- 原则可编程、可调整
- 易于扩展到新场景

局限：

- AI评估可能存在偏差
- 难以处理需要专业知识任务
- 对基础模型能力有要求

5.2 自博弈与迭代改进

自博弈 (Self-Play) 方法让模型与自己对弈，不断迭代提升：

5.2.1 基本流程

1. 生成候选回答
2. 模型自评或交叉评估
3. 选择最佳回答进行训练
4. 重复迭代

5.2.2 代表工作

- **Self-Instruct**: 自动生成指令数据
- **Self-Rewarding**: 模型自己评分
- **SPIN**: 自博弈迭代训练

5.3 Google推理研究

Google在推理增强方面发布了一系列奠基性工作，其中Chain-of-Thought提示和Self-Consistency是目前使用最广泛的技术，直接影响了后续几乎所有推理模型的训练数据构建方式。

5.3.1 Chain-of-Thought Prompting

Google在CoT提示方面做出了开创性工作：

Few-shot CoT: 在prompt中提供推理示例

Q: Roger has 5 tennis balls...

A: Roger started with 5 balls.

2 cans of 3 balls each is 6 balls.

$5 + 6 = 11$. The answer is 11.

Zero-shot CoT: 简单添加” Let’ s think step by step”

Q: Roger has 5 tennis balls...

A: Let's think step by step.

[模型自动生成推理过程]

5.3.2 Self-Consistency

自一致性方法通过多次采样提升准确率：

1. 对同一问题采样 N 个推理路径
2. 提取每个路径的最终答案

3. 投票选择出现最多的答案

$$\hat{a} = \arg \max_a \sum_{i=1}^N \mathbf{1}[a_i = a] \quad (172)$$

5.4 STaR: 自教推理器 (Self-Taught Reasoner)

5.4.1 核心方法

STaR通过迭代的自举学习提升推理能力:

```

初始化模型  $\pi_0$ 
for iteration  $k = 0, 1, \dots$  do
  for 每个问题  $q$  with 答案  $a^*$  do
    采样推理链:  $c \sim \pi_k(\cdot|q)$ 
    提取答案:  $a = \text{extract}(c)$ 
    if  $a = a^*$  then
      加入训练集:  $\mathcal{D}_k \leftarrow \mathcal{D}_k \cup \{(q, c)\}$ 
    else
      生成rationalization:  $c' \sim \pi_k(\cdot|q, a^*)$  (给定答案提示)
      加入训练集:  $\mathcal{D}_k \leftarrow \mathcal{D}_k \cup \{(q, c')\}$ 
    end if
  end for
  在  $\mathcal{D}_k$  上微调:  $\pi_{k+1} \leftarrow \text{SFT}(\pi_k, \mathcal{D}_k)$ 
end for

```

Algorithm 4: STaR算法

5.4.2 Rationalization的作用

当模型无法独立解出问题时, 通过给定正确答案让模型”倒推”推理过程 (rationalization), 这种方法可以:

- 利用更多训练样本
- 学习到新的推理模式
- 逐步提升解题能力

5.5 Quiet-STaR: 静默思考

5.5.1 核心创新

Quiet-STaR让模型在每个token位置进行隐式推理:

$$h_t = f(x_{\leq t}) + g(\text{thought}_t) \quad (173)$$

其中 thought_t 是在位置 t 生成的内部”思考”。

5.5.2 训练目标

$$\mathcal{L} = - \sum_t \log P(x_{t+1} | x_{\leq t}, \text{thought}_t) \quad (174)$$

使用REINFORCE优化思考生成策略。

5.6 ReST: 强化自我训练

5.6.1 核心方法

ReST (Reinforced Self-Training) 是Google DeepMind提出的自我训练方法, 通过迭代生成-筛选-训练循环提升模型能力:

初始化策略模型 π_θ

for iteration $t = 1, 2, \dots, T$ **do**

Generate: 对每个问题 x , 采样 K 个回答 $\{y_1, \dots, y_K\} \sim \pi_\theta(\cdot | x)$

Improve: 使用奖励模型 R 评分, 筛选高质量样本

$$\mathcal{D}_{high} = \{(x, y) : R(x, y) \geq \tau\} \quad (175)$$

Train: 在筛选后数据上进行SFT

$$\theta \leftarrow \arg \min_{\theta} \mathbb{E}_{(x, y) \sim \mathcal{D}_{high}} [-\log \pi_\theta(y | x)] \quad (176)$$

end for

Algorithm 5: ReST算法

5.6.2 与其他方法的关系

ReST可以看作是以下方法的统一框架:

- **Rejection Sampling:** 单轮ReST
- **Expert Iteration:** 使用搜索算法作为”专家”
- **STaR:** 特定的rationalization机制

5.6.3 理论分析

定理 5.1 (ReST的策略改进). 设 π^* 为最优策略, τ 为奖励阈值. 如果

$$\mathbb{P}_{y \sim \pi_\theta} [R(x, y) \geq \tau] > 0 \quad (177)$$

则ReST迭代后的策略 $\pi_{\theta'}$ 满足:

$$\mathbb{E}_{y \sim \pi_{\theta'}} [R(x, y)] \geq \mathbb{E}_{y \sim \pi_\theta} [R(x, y)] \quad (178)$$

5.7 ReST-EM: 期望最大化自我训练

5.7.1 EM框架

ReST-EM将自我训练形式化为期望最大化 (EM) 问题:

E步 (Expectation) :

$$q(y|x) = \frac{\pi_{\theta}(y|x) \cdot \mathbf{1}[R(x, y) \geq \tau]}{Z(x)} \quad (179)$$

其中 $Z(x)$ 是归一化常数。

M步 (Maximization) :

$$\theta^{new} = \arg \max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim q} [\log \pi_{\theta}(y|x)] \quad (180)$$

5.7.2 与RLHF的联系

ReST-EM可以看作是KL约束RL的特殊情况:

$$\max_{\pi} \mathbb{E}_{y \sim \pi} [R(x, y)] - \beta \mathbb{D}_{KL}[\pi \| \pi_{ref}] \quad (181)$$

当 $\beta \rightarrow 0$ 且使用硬阈值时, 退化为ReST-EM。

5.8 Expert Iteration (ExIt)

5.8.1 算法思想

Expert Iteration来源于AlphaGo的成功经验, 将搜索算法作为”专家”来指导策略学习:

1. **Expert Policy:** 使用搜索算法 (如MCTS、Beam Search) 生成高质量轨迹
2. **Apprentice Policy:** 神经网络学习模仿Expert
3. **Iteration:** 用改进的Apprentice引导下一轮Expert搜索

5.8.2 在LLM中的应用

初始化策略 π_{θ}

for iteration = 1, 2, ... **do**

for 每个问题 x **do**

Expert: $y^* = \text{BeamSearch}(\pi_{\theta}, x, k)$ 或 $y^* = \text{MCTS}(\pi_{\theta}, x)$

验证: 如果 $R(x, y^*) \geq \tau$, 加入 $\mathcal{D}$

end for

Apprentice: $\theta \leftarrow \text{SFT}(\theta, \mathcal{D})$

end for

Algorithm 6: Expert Iteration for LLM

5.8.3 搜索策略

Best-of-N:

$$y^* = \arg \max_{y \in \{y_1, \dots, y_N\}} R(x, y), \quad y_i \sim \pi_\theta(\cdot|x) \quad (182)$$

Beam Search with Verifier:

$$y^* = \arg \max_{y \in \text{Beam}_k} \sum_t \log \pi_\theta(y_t|y_{<t}, x) + \alpha \cdot V(x, y) \quad (183)$$

5.9 自奖励语言模型 (Self-Rewarding Language Models)

5.9.1 核心创新

Meta提出的Self-Rewarding方法让模型同时充当生成器和评判者:

1. **生成:** 模型生成候选回答
2. **自评:** 同一模型使用特殊prompt评估回答质量
3. **训练:** 基于自评结果进行RL或偏好学习

5.9.2 自评Prompt设计

Review the user's question and the corresponding response using the additive 5-point scoring system described below:

- Add 1 point if the response is relevant and provides some information related to the user's inquiry.
- Add 1 point if the response addresses a substantial portion of the user's question.
- Add 1 point if the response answers the basic elements of the user's question in a useful way.
- Add 1 point if the response is clearly written from an AI assistant's perspective.
- Add 1 point if the response is impeccably tailored to the user's question, demonstrating expert knowledge.

Question: {question}

Response: {response}

Score:

5.9.3 迭代训练

$$\pi^{(t+1)} = \text{DPO}(\pi^{(t)}, \mathcal{D}_{pref}^{(t)}) \quad (184)$$

其中偏好数据由 $\pi^{(t)}$ 生成并自评:

$$\mathcal{D}_{pref}^{(t)} = \{(x, y_w, y_l) : R_{\pi^{(t)}}(x, y_w) > R_{\pi^{(t)}}(x, y_l)\} \quad (185)$$

5.10 REINFORCE及其变体

5.10.1 基础REINFORCE

REINFORCE是最基本的策略梯度算法：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}} [(R(x, y) - b) \nabla_{\theta} \log \pi_{\theta}(y|x)] \quad (186)$$

基线选择：

- 常数基线： $b = \bar{R}$ (平均奖励)
- 状态相关基线： $b(s) = V(s)$
- 自身平均： $b = \frac{1}{K} \sum_{i=1}^K R(x, y_i)$

5.10.2 RLOO (REINFORCE Leave-One-Out)

RLOO使用Leave-One-Out估计作为基线，显著降低方差：

$$\nabla_{\theta} J_{RLOO} = \frac{1}{K} \sum_{i=1}^K \left(R(x, y_i) - \frac{1}{K-1} \sum_{j \neq i} R(x, y_j) \right) \nabla_{\theta} \log \pi_{\theta}(y_i|x) \quad (187)$$

优势：

- 无需额外的Value网络
- 基线与样本相关，降低方差
- 实现简单，计算高效

5.10.3 与GRPO的关系

GRPO可以看作是RLOO的一种形式，但使用标准化而非减法：

$$\hat{A}_i^{GRPO} = \frac{R_i - \bar{R}}{\sigma_R}, \quad \hat{A}_i^{RLOO} = R_i - \bar{R}_{-i} \quad (188)$$

5.11 RAFT：奖励排序微调

5.11.1 算法流程

RAFT (Reward rAnked FineTuning) 是一种简化的RL方法：

```

for 每个问题  $x$  do
  采样  $K$  个回答：  $\{y_1, \dots, y_K\} \sim \pi_{\theta}(\cdot|x)$ 
  计算奖励：  $r_i = R(x, y_i)$ 
  排序并选择Top- $k$ ：  $\mathcal{T} = \text{TopK}(\{(y_i, r_i)\}, k)$ 
  加入训练集：  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(x, y) : y \in \mathcal{T}\}$ 
end for
在  $\mathcal{D}$  上进行SFT
  
```

Algorithm 7: RAFT算法

5.11.2 与拒绝采样的区别

- **拒绝采样**：使用绝对阈值 τ 筛选
- **RAFT**：使用相对排名筛选，保证每个问题都有训练样本

5.12 拒绝采样微调 (RSF)

5.12.1 方法细节

RSF是一种简单有效的方法，广泛用于数学和代码任务：

$$\mathcal{D}_{RSF} = \{(x, y) : y \sim \pi_\theta(\cdot|x), R(x, y) = 1\} \quad (189)$$

关键设计：

- **采样温度**：通常使用较高温度（0.7-1.0）增加多样性
- **采样数量**：每个问题采样10-100个
- **去重策略**：移除重复的正确答案

5.12.2 在数学推理中的应用

DeepSeekMath、Qwen-Math等模型大量使用RSF：

1. 收集大量数学问题
2. 对每个问题采样多个解答
3. 通过答案验证筛选正确解答
4. 在正确解答上进行SFT

5.13 V-STaR：验证器增强的自我训练

5.13.1 方法改进

V-STaR在STaR基础上引入显式验证器：

1. 训练验证器 $V(x, y) \rightarrow \{0, 1\}$ 判断解答正确性
2. 使用验证器（而非答案匹配）筛选训练数据
3. 迭代提升生成器和验证器

5.13.2 验证器训练

$$\mathcal{L}_V = -\mathbb{E}_{(x, y, l)}[l \log V(x, y) + (1 - l) \log(1 - V(x, y))] \quad (190)$$

其中 $l \in \{0, 1\}$ 表示解答是否正确。

5.13.3 联合训练

```

for iteration  $t$  do
  生成:  $\{y_i\} \sim \pi_\theta(\cdot|x)$ 
  标注: 使用真实答案或  $V$  判断正确性
  训练验证器: 更新  $V$ 
  筛选:  $\mathcal{D}^+ = \{(x, y) : V(x, y) > 0.5\}$ 
  训练生成器: 在  $\mathcal{D}^+$  上 SFT
end for

```

Algorithm 8: V-STaR联合训练

5.14 过程奖励模型 (PRM) 与结果奖励模型 (ORM)

5.14.1 Let's Verify Step by Step

OpenAI的工作系统研究了PRM在数学推理中的应用:

PRM定义:

$$R_{PRM}(x, y) = \prod_{t=1}^T P(\text{step } t \text{ is correct} | x, y_{\leq t}) \quad (191)$$

数据标注:

- 人工标注每个推理步骤的正确性
- 标签: 正确/错误/中立
- 大规模标注 (800K步骤级标注)

5.14.2 Math-Shepherd

Math-Shepherd提出自动标注过程奖励的方法:

1. 对每个中间步骤, 从该步骤继续采样多个完成
2. 如果存在正确完成, 该步骤标为正确
3. 如果所有完成都错误, 该步骤标为错误

$$l_t = \mathbf{1} [\exists \text{ completion from step } t \text{ that reaches correct answer}] \quad (192)$$

5.14.3 PRM vs ORM对比

Table 10: PRM与ORM对比

特性	PRM	ORM
奖励粒度	步骤级	序列级
标注成本	高	低
信用分配	精确	粗糙
Reward Hacking	较难	较易
泛化能力	需要验证	相对稳定
推理时开销	高（每步评估）	低（仅结尾评估）

5.15 PRIME：隐式过程奖励

5.15.1 核心思想

PRIME (Process Reward Implicit) 提出无需显式步骤标注的过程奖励学习：

$$R_{PRIME}(x, y_{\leq t}) = \mathbb{E}_{y_{>t} \sim \pi} [R_{ORM}(x, y)] \quad (193)$$

5.15.2 实现方法

通过蒙特卡洛估计隐式过程奖励：

$$\hat{R}_{PRIME}(x, y_{\leq t}) = \frac{1}{M} \sum_{m=1}^M R_{ORM}(x, y_{\leq t} \oplus y_{>t}^{(m)}) \quad (194)$$

其中 $y_{>t}^{(m)} \sim \pi(\cdot | x, y_{\leq t})$ 是从当前状态采样的完成。

5.16 Online DPO与迭代偏好优化

5.16.1 Online DPO

标准DPO使用离线偏好数据，Online DPO在训练过程中动态生成偏好对：

for iteration t **do**

采样: $y_1, y_2 \sim \pi_{\theta_t}(\cdot | x)$

评估: $r_1 = R(x, y_1), r_2 = R(x, y_2)$

构建偏好对: $(y_w, y_l) = (y_1, y_2)$ if $r_1 > r_2$ else (y_2, y_1)

DPO更新:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{DPO}(x, y_w, y_l; \theta_t) \quad (195)$$

end for

Algorithm 9: Online DPO

5.16.2 IPO (Identity Preference Optimization)

IPO修改DPO损失函数，避免过拟合：

$$\mathcal{L}_{IPO} = \left(\log \frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} - \frac{1}{2\beta} \right)^2 \quad (196)$$

5.17 WizardMath与WizardCoder系列

5.17.1 Evol-Instruct方法

WizardLM系列使用进化指令方法自动生成训练数据：

指令进化：

- **增加约束：** 添加更多限制条件
- **深化：** 要求更深入的推理
- **具体化：** 添加具体细节
- **增加推理步骤：** 要求更多中间步骤
- **复杂化输入：** 使问题更复杂

5.17.2 RLEIF (RL from Evol-Instruct Feedback)

WizardMath结合进化指令和RL：

1. 使用Evol-Instruct生成难题
2. 采样多个解答
3. 筛选正确解答进行训练
4. 迭代上述过程

5.18 Qwen系列数学与代码模型

5.18.1 Qwen2.5-Math

Qwen2.5-Math结合多种方法：

- **大规模预训练：** 数学相关语料
- **SFT：** 高质量数学解题数据
- **Rejection Sampling：** 迭代筛选正确解答
- **GRPO：** 进一步RL优化

5.18.2 训练流程

1. 预训练：在数学语料上继续预训练
2. SFT：监督微调建立基础能力
3. RS迭代：多轮拒绝采样提升
4. RL微调：GRPO进一步优化

5.19 Marco-o1与MCTS增强推理

5.19.1 MCTS在LLM中的应用

Marco-o1使用蒙特卡洛树搜索增强推理：

初始化根节点 $s_0 = x$ (问题)

for iteration = 1, ..., N **do**

Selection: 使用UCB选择节点

$$a^* = \arg \max_a Q(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}} \quad (197)$$

Expansion: 使用LLM生成下一步

Simulation: 完成推理直到答案

Backpropagation: 更新节点统计

end for

返回访问次数最多的路径

Algorithm 10: MCTS for LLM Reasoning

5.19.2 与其他方法结合

- **MCTS + PRM**: 使用PRM评估节点价值
- **MCTS + Self-Consistency**: 多路径投票
- **MCTS + Expert Iteration**: 搜索结果用于训练

5.20 Kimi k1.5与长上下文RL

5.20.1 长上下文强化学习

Kimi k1.5针对长上下文场景优化RL训练：

- 支持超长推理链 (128K+ tokens)
- 高效的长序列RL算法
- 分段奖励设计

5.20.2 技术特点

- **分段GRPO**：将长序列分段计算
- **渐进式长度**：从短到长逐步训练
- **多模态支持**：文本+视觉推理

5.21 主要RL训练方法综合对比

Table 11: 主要无人工反馈RL方法对比

方法	奖励来源	是否需要Critic	数据效率	实现复杂度	适用场景
ReST	规则/模型	否	中	低	通用
STaR	规则	否	高	低	推理任务
Expert Iteration	搜索	否	高	中	可验证任务
Self-Rewarding	自评	否	中	中	通用
REINFORCE	规则/模型	否	低	低	通用
RLOO	规则/模型	否	中	低	通用
GRPO	规则/模型	否	中	低	推理任务
PPO	规则/模型	是	中	高	通用
DPO	偏好数据	否	高	低	偏好对齐
Online DPO	规则/模型	否	中	中	迭代优化
RSF	规则	否	中	低	可验证任务
RAFT	规则/模型	否	中	低	通用

5.22 其他偏好优化与代码推理方法

5.22.1 SPIN: Self-Play Fine-Tuning

SPIN通过自博弈迭代提升模型：

核心思想：将SFT数据中的真实回答作为”获胜者”，模型生成的回答作为”失败者”进行对比学习。

$$\mathcal{L}_{SPIN} = -\mathbb{E}_{(x, y^*) \sim \mathcal{D}} [\log \sigma(\beta (\log \frac{\pi_{\theta}(y^*|x)}{\pi_{ref}(y^*|x)} - \log \frac{\pi_{\theta}(\hat{y}|x)}{\pi_{ref}(\hat{y}|x)}))] \quad (198)$$

其中 y^* 是真实回答， $\hat{y} \sim \pi_{\theta_{old}}$ 是上一轮模型生成的回答。

5.22.2 Iterative DPO

迭代DPO在多轮中不断改进：

1. 使用当前模型生成新的回答对
2. 使用奖励模型或规则判断偏好
3. 进行DPO训练
4. 重复迭代

5.22.3 KTO: Kahneman-Tversky Optimization

KTO基于前景理论设计损失函数，不需要配对的偏好数据：

$$\mathcal{L}_{KTO} = \mathbb{E}_{(x,y)}[\lambda_y \cdot \sigma(\beta(r_{ref} - r_{\theta}(x, y)))] \quad (199)$$

其中 λ_y 根据 y 是正例还是负例有不同取值。

5.22.4 ORPO: Odds Ratio Preference Optimization

ORPO统一了SFT和偏好优化：

$$\mathcal{L}_{ORPO} = \mathcal{L}_{SFT}(y_w) + \lambda \cdot \log \sigma(\log \frac{\text{odds}_{\theta}(y_w|x)}{\text{odds}_{\theta}(y_l|x)}) \quad (200)$$

其中 $\text{odds}_{\theta}(y|x) = \frac{P_{\theta}(y|x)}{1-P_{\theta}(y|x)}$ 。

5.22.5 SimPO: Simple Preference Optimization

SimPO简化DPO，移除参考模型：

$$\mathcal{L}_{SimPO} = -\log \sigma(\frac{\beta}{|y_w|} \log \pi_{\theta}(y_w|x) - \frac{\beta}{|y_l|} \log \pi_{\theta}(y_l|x) - \gamma) \quad (201)$$

使用长度归一化的对数概率作为隐式奖励。

5.22.6 Code Generation: AlphaCode与竞赛编程

AlphaCode方法：

1. 大规模采样（百万级）
2. 聚类去重
3. 测试用例筛选
4. 提交最佳候选

CodeRL：

$$R_{code}(x, y) = \alpha \cdot \text{compile_success} + \beta \cdot \text{test_pass_rate} + \gamma \cdot \text{efficiency} \quad (202)$$

5.22.7 工具使用与智能体强化学习

ReAct框架：交替进行推理（Reasoning）和行动（Acting）

RL for Tool Learning：

- 状态：当前对话历史
- 动作：调用工具或生成回答
- 奖励：任务完成度 + 工具使用效率

5.22.8 OpenR: 开源推理框架

OpenR项目提供了开源的推理模型训练框架：

- 支持多种RL算法（PPO、GRPO、DPO等）
- 集成过程奖励模型
- 支持MCTS搜索
- 提供完整的训练流程

5.22.9 rStar: 自博弈推理

rStar（Self-play mutual reasoning）通过自博弈提升推理：

1. 生成多个推理路径
2. 使用判别器评估
3. 互相验证和改进
4. 迭代提升

5.22.10 推理时计算扩展（inference-time scaling）

推理时计算扩展的方法：

Table 12: 推理时计算扩展方法

方法	描述
Best-of-N	采样N个回答，选择最佳
Self-Consistency	多次采样后投票
Beam Search	保持K个最优候选
MCTS	树搜索探索解空间
Iterative Refinement	生成-评估-修改循环
Verifier-Guided	使用验证器引导生成

6 运行时原创算法提案

前文讨论的大多数训练方法，仍以“如何让模型输出更好的文本推理轨迹”为中心；但当我们把视角推进到Claude Code、OpenClaw、ChatGPT agent这类真实系统后，会发现真正卡住上限的，往往不是单步答案质量，而是**权限如何收缩与放开、上下文如何压缩、多个代理如何共享工件并维持可审计协作**。因此，本节不再罗列一组“都得起名字”的设想，而是只保留**三项我认为仍具明显原创性、科学上可定义、应用价值足够大、且运行时开销可控的原创算法提案**。

这里先做一个科学口径上的更正：上一版把**工具不确定性与自治回滚闭环**也单列成独立算法，这样写并不严谨。原因不是它们不重要，而是它们更适合作为**状态变量、风险项或系统设计原则**嵌入训练与运行时，而不是在当前材料下直接宣称为与PPO、

DPO同层级的独立原创算法。若没有更清楚的MDP定义、可识别的信用分配路径、明确的更新规则和可复现实验协议，就不宜过度命名。基于这一点，本节只保留PACE、TRACE、CRAFT三项，并把它们明确界定为**待系统验证的原创算法提案**。

6.1 什么样的内容才配叫“原创算法”

本节采用的筛选标准比“提出一个新名词”严格得多。只有同时满足以下四条，本文才把它保留下来：

1. **不是现有损失函数的小修补**：必须引入新的状态变量、动作空间或信用分配对象，而不只是给PPO/DPO再加一个正则项；
2. **直接命中高成本瓶颈**：必须对应真实系统里最贵的问题，如人工审批、长上下文膨胀或多代理返工，而不是只提升一个边缘基准分数；
3. **今天就能实现原型**：必须能够依托现有日志、guard、摘要器、artifact store、验证器和session系统落地；
4. **运行时足够高效**：额外计算量必须接近线性或稀疏局部更新，不能依赖每一步都做昂贵的全轨迹重放或全局搜索。

按这个标准，PACE、TRACE、CRAFT都不只是“研究方向”，而是已经具备**输入、状态、动作、目标、更新规则和复杂度定义**的原创算法提案；只是它们尚未经过公开benchmark的大规模检验，所以本书不把它们说成已被社区验证的成熟方法族。

6.2 从模型中心优化转向运行时中心优化

设一个真实智能体系统在时刻 t 的状态为

$$s_t = (x_t, m_t, w_t, \Pi_t, B_t, G_t), \quad (203)$$

其中 x_t 是当前对话与工具轨迹， m_t 是会话/长期记忆， w_t 是工作区状态， Π_t 是权限与审批状态， B_t 是预算（时间、token、金钱、人工审批配额）， G_t 是当前工件图（artifact graph）。与普通文本RL不同，这里的动作空间应写成

$$a_t \in \mathcal{A}_{\text{text}} \cup \mathcal{A}_{\text{tool}} \cup \mathcal{A}_{\text{memory}} \cup \mathcal{A}_{\text{delegate}} \cup \mathcal{A}_{\text{verify}}. \quad (204)$$

因此，训练目标不应只最大化“最终答案是否正确”，而应最大化**任务收益减去运行时成本后的长期回报**。更稳妥的写法是

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \left(r_t^{\text{task}} - \lambda_1 c_t^{\text{latency}} - \lambda_2 c_t^{\text{risk}} - \lambda_3 c_t^{\text{approval}} - \lambda_4 c_t^{\text{context}} - \lambda_5 c_t^{\text{coordination}} \right) \right]. \quad (205)$$

这一写法的关键含义是：**权限、记忆和协作不再是“工程细节”，而是决策问题本身的一部分**。基于这一判断，下面只讨论三项真正值得保留的高价值原创算法提案。

6.3 PACE：权限感知约束执行优化

6.3.1 问题定义

绝大多数RL工作默认动作空间固定，但真实智能体的动作空间会被权限状态动态裁剪。例如同一条bash命令，在某个session中可能可直接执行，在另一条群聊session中却必须进入审批，在高风险环境下甚至应被彻底拒绝。若训练过程忽略这一点，策略就会学成“纸面最优、上线失效”。

PACE (**Permission-Aware Constrained Execution**) 的核心思想，是把**动态可行动作集**本身建模为状态的一部分。定义时刻 t 的可行动作集为

$$\mathcal{A}_t^{\text{feasible}} = \{a \in \mathcal{A} : g(s_t, a, \Pi_t) \in \{\text{allow}, \text{ask}\}\}, \quad (206)$$

其中 g 表示运行时guard对动作的判定，可能返回直接允许、需要审批或直接拒绝。PACE不只学习“做什么”，还学习“何时申请、何时降级、何时放弃”。

6.3.2 策略结构

我们把策略拆成三层：

$$o_t \sim \pi_{\text{mode}}(o \mid s_t), \quad (207)$$

$$a_t \sim \pi_{\text{act}}(a \mid s_t, o_t), \quad (208)$$

$$\eta_t \sim \pi_{\text{escalate}}(\eta \mid s_t, a_t), \quad (209)$$

其中 o_t 决定当前是直接回答、调用工具、请求审批还是切回只读模式； η_t 决定是否值得为该动作消耗人工审批预算。其目标更适合写成受约束优化：

$$\max_{\pi} \mathbb{E} \left[\sum_t r_t^{\text{task}} \right] \quad \text{s.t.} \quad \mathbb{E} \left[\sum_t d_t^{\text{irrev}} \right] \leq \kappa_1, \quad \mathbb{E} \left[\sum_t u_t^{\text{approval}} \right] \leq \kappa_2. \quad (210)$$

6.3.3 算法规则

PACE不把“审批”当作外部打断，而是把它内生到动作选择中。一个可执行的打分形式可以写成

$$a_t^* = \arg \max_{a \in \mathcal{A}_t^{\text{feasible}}} \left[\hat{Q}_{\text{task}}(s_t, a) - \lambda_r \hat{Q}_{\text{risk}}(s_t, a) - \lambda_u \hat{Q}_{\text{approval}}(s_t, a) - \lambda_l \hat{Q}_{\text{latency}}(s_t, a) \right]. \quad (211)$$

当最优动作属于ask类时，再用升级价值判别器决定是否真的发起审批：

$$\Delta_t^{\text{escalate}} = \hat{V}(s_t, \text{ask}(a_t)) - \hat{V}(s_t, \text{degrade}(a_t)). \quad (212)$$

只有当 $\Delta_t^{\text{escalate}} > \tau_t$ 时才进入审批，否则自动退回只读、低权限工具或人工交接路径。约束乘子通过在线对偶更新：

$$\lambda_r \leftarrow [\lambda_r + \rho(d_t^{\text{irrev}} - \kappa_1)]_+, \quad \lambda_u \leftarrow [\lambda_u + \rho(u_t^{\text{approval}} - \kappa_2)]_+. \quad (213)$$

6.3.4 高效性来源

PACE的效率来自三个点：第一，**先做可行动作裁剪**，只在 $\mathcal{A}_t^{\text{feasible}}$ 上打分，而不是在完整工具空间上搜索；第二，**审批判断只比较升级与降级两条分支**，避免全树搜索；第三，**约束采用对偶变量在线更新**，不需要每个session单独重新求解受约束规划。因此在工程实现上，它更接近对现有agent loop增加一层轻量价值评估，而不是再嵌一个昂贵planner。

6.3.5 伪代码

Input: 状态 s_t , 权限状态 Π_t , 风险/审批预算 (κ_1, κ_2) , 打分器 $\hat{Q}_{task}, \hat{Q}_{risk}, \hat{Q}_{approval}, \hat{Q}_{latency}$
 根据guard得到 $\mathcal{A}_t^{\text{feasible}} = \{a : g(s_t, a, \Pi_t) \in \{\text{allow}, \text{ask}\}\}$
for $a \in \mathcal{A}_t^{\text{feasible}}$ **do**
 计算 $Score(a) = \hat{Q}_{task} - \lambda_r \hat{Q}_{risk} - \lambda_u \hat{Q}_{approval} - \lambda_l \hat{Q}_{latency}$
end for
 选择 $a_t^* = \arg \max_a Score(a)$
if $g(s_t, a_t^*, \Pi_t) = \text{ask}$ **then**
 计算 $\Delta_t^{\text{escalate}} = \hat{V}(s_t, \text{ask}(a_t^*)) - \hat{V}(s_t, \text{degrade}(a_t^*))$
 if $\Delta_t^{\text{escalate}} \leq \tau_t$ **then**
 执行降级动作、切回只读模式或人工交接
 else
 发起审批并在批准后执行 a_t^*
 end if
else
 直接执行 a_t^*
end if
 观察任务回报、风险暴露和审批消耗
 更新 $\lambda_r \leftarrow [\lambda_r + \rho(d^{\text{irrev}} - \kappa_1)]_+$, $\lambda_u \leftarrow [\lambda_u + \rho(u^{\text{approval}} - \kappa_2)]_+$
return 执行动作与更新后的约束乘子

Algorithm 11: PACE在线决策与约束更新

6.3.6 可落地实现路径

PACE最容易接入Claude Code / OpenClaw这类系统: guard已经给出allow / ask / deny信号, session日志已经记录动作、审批、回滚与结果, 现有系统只需额外训练四个轻量打分头或一个多任务价值模型, 就能先做离线重放、再做灰度在线试验。

6.3.7 为什么它有重大价值

PACE的价值, 不在于“更安全”这句空话, 而在于它直接触及了**终端代理与企业级agent系统最真实、最昂贵的瓶颈**: 权限不是静态白名单, 而是带审批、带预算、带风险级别的动态控制面。公开文献中当然已有安全RL、受约束RL和approval flow, 但它们通常不直接优化Claude Code / OpenClaw这一类真实运行时中的“执行—审批—降级”链条。若这个问题能够被系统化建模并在真实日志上验证, 其价值将体现在**相同风险预算下提高任务完成率, 或在相同完成率下显著降低人工审批与错误执行成本**, 因此它属于值得保留的高价值原创算法提案。

6.3.8 可验证假设与实验协议

PACE要成立, 不能只展示“看起来更谨慎”, 而必须展示**在风险约束下的效用改进**。更合理的验证路线包括:

1. **数据构造**：从真实agent日志中抽取包含权限拒绝、审批请求、降级执行、人工接管与最终任务结果的完整session，重建 (s_t, a_t, Π_t, r_t) 轨迹。
2. **离线评测**：在固定审批预算 κ_2 与不可逆风险预算 κ_1 下，比较不同策略的任务完成率、平均审批次数、危险动作暴露率与平均延迟。
3. **在线评测**：在沙箱环境中加入真实guard与审批模拟器，观察策略是否学会先走只读路径、先做低风险验证，再决定是否升级权限。
4. **关键基线**：静态白名单策略、风险分类器+规则审批策略、标准受约束RL但不显式建模ask/degrade动作的策略。

PACE最核心的可检验假设是：**在相同风险与审批预算下，显式建模动态可行动作集与审批决策的策略，优于固定动作空间或纯规则式升级策略。**如果它做不到这一点，PACE就不能算成立。

6.4 TRACE：轨迹后悔感知的上下文编辑

6.4.1 问题定义

长任务中的核心难题，不是单纯“窗口不够长”，而是**哪些历史该保留，哪些该压缩，哪些必须原样保真**。现有compaction和summarization方法通常优化摘要表面质量，但真正决定任务成败的，是压缩之后未来总回报是否下降。

TRACE (**Trajectory-Regret-Aware Context Editing**) 把上下文压缩视为一个显式决策问题。对每段历史片段 h_i ，压缩器选择

$$c_i \in \{\text{keep, summarize, drop, externalize}\}, \quad (214)$$

并以压缩导致的**未来回报损失**定义“上下文后悔”：

$$\Delta_i = J^\pi(h_{\leq t}) - J^\pi(\tilde{h}_{\leq t}^{(i)}), \quad (215)$$

其中 $\tilde{h}_{\leq t}^{(i)}$ 表示对片段 i 执行某种编辑后的历史。于是TRACE更合理的目标应写成

$$\mathcal{L}_{\text{TRACE}} = \mathbb{E} \left[\Delta + \alpha \cdot \text{tokens}(\tilde{h}) + \beta \cdot c^{\text{edit-risk}}(\tilde{h}) \right]. \quad (216)$$

6.4.2 训练机制

TRACE的关键不在于“做摘要”，而在于**反事实评估**。更具体地说，应从真实长轨迹中截出多个压缩点，对不同压缩策略继续rollout或进行离线反事实估计，比较未来成功率、工具成功率、验证通过率与最终成本。这样，压缩器学到的不是“像人类摘要一样写得漂亮”，而是“怎样压缩才不毁掉后续决策”。

6.4.3 算法规则

为了把TRACE从“摘要思想”变成真正的算法，一个更具体的执行形式是先对历史切分为片段 $\{h_i\}_{i=1}^n$ ，并对每段预测三类量：

$$\hat{\Delta}_i^{\text{drop}}, \quad \hat{\Delta}_i^{\text{summ}}, \quad \hat{\Delta}_i^{\text{ext}}, \quad (217)$$

分别表示删除、摘要、外置化后带来的未来回报损失估计。然后为每种编辑计算“单位token保真收益”

$$\rho_i(c) = \frac{\hat{\Delta}_i^{\text{keep}} - \hat{\Delta}_i(c)}{\text{tokens}_i(c) + \epsilon}, \quad c \in \{\text{summ}, \text{ext}, \text{drop}\}. \quad (218)$$

在总上下文预算 B 下，TRACE不直接生成一段总摘要，而是解一个预算化编辑问题：

$$\min_{\{c_i\}} \sum_{i=1}^n \hat{\Delta}_i(c_i) + \beta \sum_{i=1}^n \text{tokens}_i(c_i) + \gamma \sum_{i=1}^n r_i^{\text{instability}} \quad \text{s.t.} \quad \sum_{i=1}^n \text{tokens}_i(c_i) \leq B. \quad (219)$$

实际运行时可采用按 ρ_i 排序的贪心选择或小规模knapsack近似，而不是每次重算整段历史。

6.4.4 高效性来源

TRACE之所以高效，是因为它把原本“重新概括整个历史”的操作，改成了**分段、局部、预算化编辑**。只要片段级后悔预测器足够稳定，运行时就不必对所有旧消息再次生成摘要，而只需对少数低价值片段做替换或外置化。因此它的主要开销与片段数 n 近似线性关系，远低于反复全量重写上下文的策略。

6.4.5 伪代码

```

Input: 历史片段 $\{h_i\}_{i=1}^n$ , 上下文预算 $B$ , 后悔估计器 $\hat{\Delta}$ , 编辑集合 $\{\text{keep}, \text{summ}, \text{drop}, \text{ext}\}$ 
for  $i = 1$  to  $n$  do
  for  $c \in \{\text{keep}, \text{summ}, \text{drop}, \text{ext}\}$  do
    估计 $\hat{\Delta}_i(c)$ 与 $\text{tokens}_i(c)$ 
  end for
  计算各编辑动作的单位token保真收益 $\rho_i(c)$ 
end for
  在约束 $\sum_i \text{tokens}_i(c_i) \leq B$ 下，使用贪心排序或小规模knapsack选择 $\{c_i\}$ 
  对被选中的片段执行保留、摘要、删除或外置化
  生成新的上下文 $\tilde{h}_{\leq t}$ 并继续后续任务
  根据后续任务成功率、验证通过率和成本回写更新后悔估计器
return 编辑后的上下文 $\tilde{h}_{\leq t}$ 

```

Algorithm 12: TRACE预算化上下文编辑

6.4.6 可落地实现路径

TRACE最容易从现有session compaction机制演化而来：先保留系统已有的摘要器、检索器和长期记忆存储，再额外训练一个**片段级后悔估计器**。在工程上，第一阶段甚至不需要端到端RL，只需用历史session中的“压缩后失败/成功”信号训练离线回归器，就可以先把固定摘要策略升级为预算敏感的选择性编辑策略。

6.4.7 为什么它有重大价值

长上下文和长期记忆会持续成为agent系统的主瓶颈，但真正难的地方不是“再堆更长窗口”，而是**如何在有限预算下保留对未来决策真正关键的信息**。ReSum、ACON一类工作已经说明上下文压缩是核心问题，但TRACE仍保留原创性，因为它不是把摘要当成语言生成任务，而是把**压缩后的后悔值**直接拉进训练目标。若它能在严格对照下证明“同等token预算下更少破坏未来决策”，那么长时搜索、长工作流程代码修复、企业知识代理都会明显受益，因此它也属于重大价值原创算法提案。

6.4.8 可验证假设与实验协议

TRACE的科学难点在于：压缩质量不能只靠人工主观打分，而必须看**压缩之后未来任务是否变差**。因此更稳妥的协议应包括：

1. **数据构造**：从长session中截取多个压缩时刻，对同一段历史生成keep/summarize/drop/externalize等不同编辑版本。
2. **反事实评估**：在后续相同任务上继续rollout，或用离线估计器比较不同编辑版本造成的成功率、验证通过率、工具调用正确率与总token成本。
3. **关键指标**：单位token预算下的任务成功率、压缩后关键事实遗漏率、后续验证失败率、编辑不稳定性（同一历史多次压缩得到的策略差异）。
4. **关键基线**：简单截断、最近窗口保留、检索增强摘要、专用总结模型、只做external memory不做主动编辑的方案。

TRACE最核心的可检验假设是：**显式最小化压缩后悔的编辑策略，在相同上下文预算下优于只追求摘要可读性或检索召回率的策略**。如果简单的截断或检索方案已经同样有效，那么TRACE的额外复杂性就不成立。

6.5 CRAFT：工件图约束下的多智能体协作

6.5.1 问题定义

当前多智能体系统经常停留在“代理之间互发自然语言消息”。这种方式演示简单，但一旦任务变长，信息会在消息流中被稀释、重复或扭曲。真正可扩展的多代理协作，应当围绕**结构化工件（artifact）**而不是闲聊展开。

CRAFT（**Causally-Regularized Artifact-Factored Teaming**）将协作状态写成工件图

$$G_t = (\mathcal{V}_t, \mathcal{E}_t), \quad (220)$$

其中节点 $\mathcal{V}_t$ 可以是计划、设计文档、代码patch、测试报告、风险分析或部署记录，边 $\mathcal{E}_t$ 表示依赖关系、验证关系与来源关系。每个子代理不只是输出文本，而是执行

$$a_t^{(k)} : G_t \rightarrow G_{t+1}, \quad (221)$$

即对共享工件图做可追踪修改。

6.5.2 奖励设计

CRAFT的关键不是“多几个角色”，而是让奖励对准**合并质量与工件因果性**。一个更稳妥的目标可写为

$$R_{\text{CRAFT}} = R_{\text{task}} + \lambda_1 R_{\text{merge}} + \lambda_2 R_{\text{provenance}} - \lambda_3 R_{\text{duplication}} - \lambda_4 R_{\text{conflict-churn}}. \quad (222)$$

其中 $R_{\text{provenance}}$ 要求后续工件必须能追溯到其依赖节点， R_{merge} 鼓励最终合并成本低、验证链完整、返工少。

6.5.3 算法规则

为了让CRAFT成为算法而不只是协作口号，需要把“谁做什么”写成显式调度问题。设代理集合为 $\{1, \dots, K\}$ ，每个代理 k 在时刻 t 只接管工件图中的一个局部前沿 $f_t^{(k)} \subseteq \mathcal{V}_t$ 。调度器选择下一个工作代理时，不看“谁更会聊天”，而看单位协调成本的边际收益：

$$k_t^* = \arg \max_k \frac{\widehat{\Delta R}_{\text{task}}^{(k)} + \lambda_p \widehat{\Delta R}_{\text{prov}}^{(k)} - \lambda_c \widehat{\Delta R}_{\text{conflict}}^{(k)}}{\epsilon + \widehat{C}_{\text{merge}}^{(k)}}. \quad (223)$$

代理输出的不是长消息，而是局部图差分

$$\delta G_t^{(k)} = (\delta \mathcal{V}_t^{(k)}, \delta \mathcal{E}_t^{(k)}), \quad (224)$$

系统只广播受影响的依赖邻域，而不是广播整个对话历史。信用分配也不再按“最后谁说得好”算，而是按provenance边做局部归因：

$$\text{credit}(k) = \sum_{v \in \mathcal{V}_{\text{touched}}^{(k)}} \omega_v \cdot \mathbf{1}[v \rightsquigarrow G_T^{\text{accepted}}]. \quad (225)$$

6.5.4 高效性来源

CRAFT的效率不来自“更多代理并行”这么简单，而来自**通信稀疏化与信用局部化**。第一，代理之间交换的是局部工件差分而不是整段自然语言转述；第二，调度器只在图前沿上选择下一步，不做全局全代理重规划；第三，合并与奖励归因只在受影响子图中计算，因此复杂度更接近图的局部更新，而不是会话长度的平方增长。

6.5.5 伪代码

Input: 当前工件图 G_t , 代理集合 $\{1, \dots, K\}$, 合并代价估计器 $\widehat{C}_{merge}$, 局部收益估计器 $\widehat{\Delta R}$

for $k = 1$ to K **do**

 抽取代理 k 可接管的局部前沿 $f_t^{(k)}$

 估计边际收益比值 $\frac{\widehat{\Delta R}_{task}^{(k)} + \lambda_p \widehat{\Delta R}_{prov}^{(k)} - \lambda_c \widehat{\Delta R}_{conflict}^{(k)}}{\epsilon + \widehat{C}_{merge}^{(k)}}$

end for

 选择 $k_t^* = \arg \max_k$ 上述比值

 令代理 k_t^* 在局部前沿上生成图差分 $\delta G_t^{(k_t^*)}$

if provenance、依赖检查与验证器全部通过 **then**

 合并 $\delta G_t^{(k_t^*)}$ 到 G_t 并只广播受影响邻域

 按来源边给涉及节点分配局部credit

else

 拒绝该差分并记录冲突返工惩罚

end if

return 更新后的工件图 G_{t+1}

Algorithm 13: CRAFT基于工件图的局部协作

6.5.6 可落地实现路径

CRAFT可以直接建立在已有的plan / patch / test / report工作流之上：把计划、代码补丁、测试结果、验证报告和部署记录显式存为artifact node，再用依赖边与来源边把它们串起来。第一版甚至不需要真正的多智能体RL，只需先把自由文本协作改成**artifact-first**协作，再逐步加入图调度器和局部信用分配器。

6.5.7 为什么它有重大价值

现有多代理论文已经很多，但真正进入生产后，最难的问题往往不是消息传递，而是**共享中间产物如何保持一致、可验证、可审计**。这恰好是研发协作、科研自动化和复杂运维里最昂贵的部分。CRAFT的高价值，不是因为它“又是一个multi-agent算法”，而是因为它把协作对象从聊天记录提升到artifact graph。这一点若能做实，会比“再堆几个聊天代理”更接近企业级多智能体平台的真实需求。

6.5.8 可验证假设与实验协议

CRAFT是否成立，关键不在于代理数目，而在于**结构化工件协作是否真的降低了协作摩擦**。因此实验协议必须把“最终答案”拆成“中间工件质量”和“集成成本”两部分：

1. **任务设计**：选取天然可分解的复杂任务，如代码库重构、文献综述+实验计划、运维排障+修复报告，并显式要求产出计划、patch、测试、验证报告等工件。

- 2. **协作设置**：比较单代理、自由聊天式多代理、planner-executor式多代理，以及基于artifact graph的CRAFT协作。
- 3. **关键指标**：最终任务完成率、工件可追溯性完整度、重复工作率、冲突返工率、合并延迟、验证链覆盖率。
- 4. **关键消融**：去掉provenance约束、去掉merge奖励、只保留角色分工不保留工件图，分别观察性能变化。

CRAFT最核心的可检验假设是：**当任务长度和依赖关系上升时，围绕artifact graph进行协作，比围绕自然语言消息进行协作更稳定、更易审计、总体集成成本更低**。如果自由消息传递在这些指标上并不吃亏，那么CRAFT就只是工程实现风格，而不是值得命名的研究框架。

6.6 为什么只保留这三项

从科研口径看，这三项之所以值得保留，是因为它们同时满足三个条件：

- 1. **问题足够基础**：分别对应权限控制、长程记忆和多智能体协作，这是agent run-time里最根本的三个瓶颈；
- 2. **可形式化**：都能写出清晰的状态、动作、目标与约束，而不是只有工程口号；
- 3. **潜在价值足够大**：一旦做成，不是局部加分，而是有可能实质性改变终端代理、企业agent和科研自动化的能力边界。

相反，**工具不确定性与自治回滚闭环**当然仍然重要，但在当前证据下更适合作为Pace/Trace/Craft里的风险项、验证器或系统策略，而不宜继续单列成“独立原创算法”。这是本节最重要的科学修正。

6.7 三项高价值原创算法提案之间的关系

这三个框架并不是互相独立的点子，而是围绕同一个运行时闭环的不同切面：

Table 13: 高价值运行时原创算法与其对应瓶颈

框架	直接优化对象	解决的系统瓶颈	最适用场景
PACE	权限/审批决策	动作空间动态变化、人工审批昂贵、权限边界不稳定	终端代理、企业审批流、节点调用
TRACE	上下文编辑策略	长轨迹膨胀、压缩后遗忘关键事实、长程记忆失真	长工作流、浏览器搜索、代码修复
CRAFT	多代理工件图	多代理消息漂移、合并困难、审计断裂	研发协作、科研自动化、复杂运维

从公共文献的角度看，Agent Lightning、RAGEN、ReSum、ACON、AutoTIR、WebSailor等工作已经分别触及了agent RL、长时搜索、上下文压缩、工具

使用与长流程训练问题；但它们大多仍只优化其中一个切片。本文保留下来的三项算法提案试图做的，是把这些切片统一到**运行时中心**的视角下：不是问“模型还缺哪种奖励”，而是问“一个能长期运行的智能体系统，真正在哪些状态变量上需要学习”。

6.8 理论命题与成立条件

为了避免“看起来像算法”的表述混入无法检验的工程直觉，这里把三项提案进一步压缩成**可证伪的设计命题**。需要强调的是：下面的内容不是已经完成严格证明的定理，而是在**明确假设下可被验证、也可能被推翻的理论主张**。如果这些假设在真实系统中不成立，对应算法就不应继续被保留。

6.8.1 PACE的设计命题

命题P1：可行动作内生化合能够降低无效执行请求。 设基线策略 π_{base} 先在完整动作集 $\mathcal{A}$ 上打分，再由guard在执行阶段拒绝不可行动作；PACE策略 π_{PACE} 则只在 $\mathcal{A}_t^{\text{feasible}}$ 上做选择。若guard标签 $g(s_t, a, \Pi_t)$ 在执行时是可靠的，则有

$$\mathbb{P}_{\pi_{\text{PACE}}}(a_t \notin \mathcal{A}_t^{\text{feasible}}) = 0, \quad (226)$$

而 π_{base} 通常只能在事后拒绝后再回退。因此，PACE至少应在**无效执行请求率、拒绝后重试次数、由拒绝导致的额外延迟**上优于“先选动作、再被guard打回”的策略。如果这一点不成立，说明把权限状态写入策略并没有带来实际收益。

命题P2：在估计器校准条件下，升级判别应弱支配固定审批规则。 若 $\hat{V}(s_t, \text{ask}(a))$ 与 $\hat{V}(s_t, \text{degrade}(a_t))$ 误差满足一致有界，

$$|\hat{V}(s, a) - V(s, a)| \leq \varepsilon, \quad (227)$$

且真实升级收益满足间隔条件

$$|V(s_t, \text{ask}(a_t)) - V(s_t, \text{degrade}(a_t))| > 2\varepsilon, \quad (228)$$

则按 $\Delta_t^{\text{escalate}}$ 做审批判别的决策不会被估计误差轻易翻转。此时PACE应在**相同审批预算**下达到不低于“总是升级”或“从不升级”的任务回报。若固定审批规则表现同样好甚至更好，则PACE的升级价值判别器就没有成立基础。

6.8.2 TRACE的设计命题

命题T1：在片段后悔近似可加时，预算化编辑问题可被稳定近似求解。 若历史被分解为片段 $\{h_i\}_{i=1}^n$ 后，编辑后的未来回报损失近似满足

$$\Delta(\{c_i\}_{i=1}^n) \approx \sum_{i=1}^n \Delta_i(c_i) + \xi, \quad |\xi| \leq \epsilon_{\text{int}}, \quad (229)$$

其中 ϵ_{int} 表示跨片段交互项足够小，则TRACE的预算化编辑问题可近似化为带预算约束的离散优化问题，而不需要每轮都重写整段历史。此时贪心排序或小规模knapsack近似的解，应在**相同token预算**下优于简单截断或固定摘要策略。若交互项很大、后悔不可分解，则TRACE的分段编辑前提就被破坏。

命题T2：后悔排序一旦可校准，就应优于“按表面重要性摘要”。 若片段级估计器能保持后悔排序的一致性，即对大多数片段对 (i, j) 成立

$$\Delta_i^{\text{true}} > \Delta_j^{\text{true}} \Rightarrow \hat{\Delta}_i > \hat{\Delta}_j, \quad (230)$$

则TRACE应优先保留那些对未来决策真正关键、但未必在语义上“看起来重要”的历史片段。因此它至少应在**关键事实遗漏率、压缩后验证失败率、压缩后任务成功率**上优于以可读性、关键词密度或近期性为主的摘要策略。若简单启发式排序已经达到同等效果，则TRACE的额外建模不值其复杂度。

6.8.3 CRAFT的设计命题

命题C1：在依赖稀疏条件下，工件差分广播的通信开销低于全消息广播。 设多代理会话中的完整消息历史长度为 L_t ，工件图局部变更为 δG_t ，其受影响邻域规模为 $|\mathcal{N}(\delta G_t)|$ 。若任务依赖图满足局部稀疏性，即

$$|\mathcal{N}(\delta G_t)| \ll L_t, \quad (231)$$

则只广播局部差分的通信开销应显著低于把完整对话历史重复转发给所有代理。因此，随着任务长度增长，CRAFT应在**平均通信token、冲突返工率、合并延迟**上优于自由聊天式多代理。若任务本身高度全局耦合，以致每次改动都需要广播全部上下文，那么CRAFT的通信优势就会消失。

命题C2：局部credit与provenance约束可以减少“伪协作”。 若最终接受的工件能够沿来源边追溯到具体局部差分，则信用分配可从“谁最后说得像总结”转向“谁真正贡献了被采用的工件”。在这一条件下，CRAFT应降低**重复工作率、空转消息比例、不可追溯修改比例**，并提高**验证链完整度与责任归因清晰度**。若没有可追溯的artifact lineage，局部credit就会退化成另一套启发式打分，CRAFT的核心论点也随之失效。

6.9 实验统计规范

如果希望这三项提案进入论文或系统报告，而不是停留在叙述层面，实验部分必须遵守统一的统计规范。否则很容易把一次偶然成功误写成算法效果。

6.9.1 数据切分与泄漏控制

1. **按任务实体切分：**代码任务至少按repository或issue切分，企业流程任务至少按workflow模板或客户切分，研究任务至少按主题或项目切分，避免同一长session的不同片段同时落入训练集和测试集。
2. **按时间顺序切分日志：**对真实runtime日志，应优先采用时间顺序切分而不是纯随机切分，防止未来策略泄漏到过去分布。
3. **离线/在线分离：**离线重放、沙箱灰度、真实在线实验必须分开报告，不能把沙箱成功率直接当作线上效果。

6.9.2 统计报告最低标准

- 1. **成对比较优先**：PACE、TRACE、CRAFT都应优先采用同任务实例上的成对评测，而不是只比较两组总体均值。
- 2. **报告均值与分位数**：除均值外，还应同时报告P50/P90延迟、P50/P90审批次数、P50/P90上下文成本或合并时延，避免长尾成本被均值掩盖。
- 3. **置信区间**：成功率、危险动作暴露率、关键事实遗漏率等比例指标，应报告95%置信区间；样本量不足时优先采用paired bootstrap而不是只报单个点估计。
- 4. **显著性检验**：二元结果可用McNemar检验或置换检验，连续成本指标可用配对bootstrap或Wilcoxon signed-rank；禁止只写“明显提升”而不给统计依据。
- 5. **至少多随机种子或多批次复验**：离线训练至少报告多个seed，在线实验至少报告多个批次或多个时间窗口，防止把一次流量波动误写成稳定改进。

6.9.3 系统成本与失败分类必须同步报告

Table 14: 原创算法提案的实验统计与失败分析最低报告项

算法	必须报告的成本项	必须拆分的失败类型
PACE	额外审批次数、额外延迟、风险拒绝率、人工接管率	估计器失准、guard误判、策略过度升级、策略过度保守
TRACE	压缩token节省、额外编辑时延、检索次数、长期记忆写入成本	关键事实遗漏、摘要幻觉、后悔估计抖动、外置记忆检索失败
CRAFT	代理通信token、合并时延、验证器开销、artifact存储开销	provenance断裂、重复劳动、局部最优冲突、验证链不完整

6.9.4 从离线到在线的推进顺序

更稳妥的推进顺序应是：

- 1. **离线重放**：先验证状态、动作和奖励定义是否有区分度；
- 2. **沙箱在线**：再验证策略是否在真实guard、摘要器、artifact store和验证器存在时仍然成立；
- 3. **小流量线上**：最后才进入有限流量真实部署，并显式设置熔断阈值与人工接管条件。

只有当离线、沙箱、线上三层结果在方向上基本一致时，才有资格把收益归因于算法本身；否则更可能是评测环境差异、日志偏差或系统耦合效应。

6.10 复杂度与系统开销

如果这三项算法只能在玩具环境里成立，就没有保留价值。因此必须明确它们的额外开销是否可控。

Table 15: 三项原创算法提案的复杂度与系统开销

算法	训练时新增开销	运行时新增开销	高效性关键来源
PACE	训练风险/审批/延迟多头价值估计器；可用离线日志监督或离线RL构造样本	约为 $O(\mathcal{A}_t^{feasible})$ 的可行动作打分，外加常数级升级判别	先裁剪动作，再做局部打分；对偶变量在线更新，不做全局受约束求解
TRACE	训练片段级后悔估计器；需要少量反事实rollout或离线估计	贪心版本约为 $O(n \log n)$ ，小规模knapsack近似约为 $O(nB)$	分段编辑替代全局重写；只更新少量低价值片段；预算约束局部求解
CRAFT	训练局部收益、冲突与合并成本估计器；需要artifact-level日志	约为 $O(F_t + \mathcal{N}(\delta G))$ 的前沿调度与局部合并	交换图差分而非长消息；只广播受影响邻域；信用分配局部化

6.11 Benchmark设计

为了把这三项算法从“方法提案”推进到“可投稿结果”，最小可行benchmark不应只测一个总分，而应围绕各自的关键瓶颈设计。

Table 16: 三项原创算法提案对应的benchmark设计

算法	任务族	核心指标	主要baseline
PACE	终端代理、企业审批流、受限节点操作、浏览器高风险动作	任务完成率、平均审批次数、危险动作暴露率、平均延迟、人工接管率	规则审批、静态白名单、风险分类器+workflow、标准受约束RL
TRACE	长工作流代码修复、长网页搜索、知识代理、多轮研究任务	单位token预算成功率、关键事实遗漏率、验证失败率、上下文成本	最近窗口保留、固定摘要器、检索增强摘要、只做外部记忆
CRAFT	代码库重构、科研综述+实验计划、运维排障+修复报告	完成率、合并延迟、重复工作率、冲突返工率、provenance完整度、验证链覆盖率	单代理、自由聊天式多代理、planner-executor、多worker无artifact图

6.12 消融实验设计

如果没有消融实验，就无法判断算法到底值不值得额外复杂性。最关键的消融不应是“换个超参数”，而应是直接删除算法最核心的结构。

Table 17: 三项原创算法提案的关键消融实验

算法	关键消融	回答的问题
PACE	去掉动态可行动作集；去掉审批价值判别；去掉对偶约束更新	PACE是否真的需要权限内生化，而不是规则+打分的简单拼接？
TRACE	去掉后悔估计；只保留摘要器；只保留external memory；不用预算求解	TRACE的价值究竟来自“会摘要”，还是来自“后悔感知+预算编辑”？
CRAFT	去掉 artifact graph 只保留消息；去掉 provenance奖励；去掉merge成本项；去掉局部credit	CRAFT的收益是否真的来自工件图结构，而不是多加几个代理？

6.13 最小可投稿实验包

如果以论文标准收敛实现，最小实验包至少应包括：

- 1. **离线重放实验**：证明算法在历史日志上优于强baseline，先验证问题抽象成立。
- 2. **小规模在线灰度实验**：证明离线收益可以迁移到真实runtime，而不是离线过拟合。
- 3. **失败案例分析**：逐条解释算法何时失败、为什么失败，以及失败是否来自估计器偏差、验证器缺陷或环境非平稳。
- 4. **成本报告**：报告额外token、延迟、内存和人工审批成本，否则无法判断算法是否真的高效。

6.14 证伪标准与失败信号

为了避免把系统设计误写成“新算法”，还需要明确每个方向的失败标准：

- 1. **PACE的失败信号**：在固定风险预算下无法优于简单规则审批；或者策略通过频繁请求审批来转移责任，却没有提高任务成功率。
- 2. **TRACE的失败信号**：在等预算条件下不优于简单截断、检索或固定摘要；或者后悔估计高度不稳定，导致编辑策略不可复现。
- 3. **CRAFT的失败信号**：artifact graph带来的维护成本超过其协作收益；或者一旦任务规模变小，图约束就退化为纯负担。

只有在这些反例都被认真测试后，PACE、TRACE、CRAFT才有资格进一步从“原创算法提案”走向“可复用方法”。

6.15 实验路线建议

如果未来要把这些方向推进到可验证研究，最合理的顺序不是同时全做，而是按以下路线推进：

1. 先做**TRACE + PACE**：这两项最容易在现有日志和session数据上离线构造训练样本，也最直接决定系统能否稳定运行。
2. 再做**CRAFT**：因为它要求系统先从自由文本协作推进到工件化协作。

换句话说，本节真正的核心结论是：**下一代高价值原创算法不应只围绕“生成更长的思维链”展开，而应围绕“如何训练一个可执行、可压缩、可协作的运行时”展开。**但在科学口径上，它们目前仍应被视为**可形式化、可证伪、待系统验证的原创算法提案**，而不是已经成熟定型的方法族。

7 实现细节与训练技巧

本章按照“**训练数据与采样** → **稳定训练** → **分布式与内存优化** → **超参数与监控** → **常见失败模式**”的顺序展开，聚焦模型训练本身的工程实践；智能体系统的控制层架构与工程部署单独作为下一章（第8章）展开。

7.1 数据准备

7.1.1 数学数据来源

- **GSM8K**：8500道小学数学题，包含详细解答步骤
- **MATH**：12500道竞赛数学题，涵盖7个难度级别
- **AIME/AMC**：美国数学竞赛历年真题，高难度
- **Olympiad**：国际数学奥林匹克题目
- **合成数据**：程序生成的数学问题
- **网络数据**：爬取的数学问答，如Math StackExchange
- **教科书**：数学教材中的例题和习题

7.1.2 代码数据来源

- **HumanEval**：164道编程题，测试函数实现
- **MBPP**：974道Python编程题
- **LeetCode**：算法竞赛题，难度分级
- **Codeforces**：编程竞赛题，含测试用例
- **GitHub**：开源代码库，提取函数和类
- **CodeContests**：Google发布的竞赛题集
- **APPS**：大规模编程问题集

7.1.3 数据质量控制

过滤策略：

1. 去重：基于问题文本和答案的去重
2. 长度过滤：移除过短或过长的样本
3. 质量过滤：使用质量分类器筛选
4. 毒性过滤：移除有害内容
5. 平衡采样：确保难度和类型的平衡

数据验证：

- 数学：符号验证、数值验证
- 代码：执行测试、静态分析
- 人工抽检：定期人工审核

7.1.4 数据增强

问题变换：

- **数值替换**：更换问题中的具体数字
- **情境替换**：更换问题背景（苹果→橙子）
- **复杂化**：添加额外条件或步骤
- **简化**：移除干扰信息
- **逆向构造**：给定答案生成问题

推理链增强：

- 多解法生成
- 错误推理链生成（作为负样本）
- 推理链改写（不同表述方式）

7.2 采样策略

采样策略直接决定RL探索的多样性和推理时的输出质量。本节介绍温度采样、Top-p/Top-k采样、Best-of-N拒绝采样和束搜索变体，并给出各场景下的推荐参数范围。

7.2.1 温度采样

$$P(token) = \frac{\exp(\text{logit}/T)}{\sum_v \exp(\text{logit}_v/T)} \quad (232)$$

温度选择指南:

Table 18: 采样温度选择指南

场景	温度	说明
RL探索	0.8-1.2	增加多样性
推理生成	0.6-0.8	平衡多样性和质量
最终推理	0.1-0.3	更确定性的输出
贪婪解码	0.0	确定性最高

7.2.2 Top-p采样 (Nucleus Sampling)

只考虑累积概率达到 p 的tokens:

$$V_p = \min \left\{ V' \subseteq V : \sum_{v \in V'} P(v) \geq p \right\} \quad (233)$$

动态Top-p:

$$p_t = p_{base} + \alpha \cdot \text{entropy}(P_t) \quad (234)$$

高熵时增加 p 以保持多样性。

7.2.3 Top-k采样

只考虑概率最高的 k 个tokens:

$$V_k = \text{argtop}_k P(v) \quad (235)$$

7.2.4 Best-of-N (Rejection Sampling)

生成 N 个候选, 用奖励模型选择最佳:

$$o^* = \arg \max_{o \in \{o_1, \dots, o_N\}} R(q, o) \quad (236)$$

理论分析:

$$\mathbb{E}[R(o^*)] = \mathbb{E} \left[\max_{i=1}^N R(o_i) \right] \quad (237)$$

对于奖励服从某分布的情况, 可以用极值理论分析改进幅度。

实现优化:

- 批量并行生成
- 早停: 低质量候选提前终止
- 分层采样: 先粗筛再精选

对实际开发的指导作用 工程上，Best-of-N最适合**可验证、可并行、错误代价低**的任务，例如数学、代码、结构化抽取。它不适合高延迟、昂贵工具调用密集的场景，因为候选数 N 会近似线性放大成本。

7.2.5 束搜索变体

标准束搜索：

$$\mathcal{B}_{t+1} = \text{top}_k \left(\bigcup_{b \in \mathcal{B}_t} \{b \oplus v : v \in V\} \right) \quad (238)$$

多样束搜索：

$$\text{score}(b) = \log P(b) - \lambda \cdot \text{similarity}(b, \mathcal{B}_{\text{selected}}) \quad (239)$$

长度归一化束搜索：

$$\text{score}(b) = \frac{\log P(b)}{|b|^\alpha} \quad (240)$$

7.3 训练稳定性

7.3.1 梯度管理

梯度裁剪：

$$g \leftarrow \min \left(1, \frac{c}{\|g\|} \right) \cdot g \quad (241)$$

梯度累积： 当内存不足时，累积多个小batch的梯度：

$$g_{\text{accumulated}} = \frac{1}{K} \sum_{k=1}^K g_k \quad (242)$$

梯度缩放 (Mixed Precision) :

$$g_{\text{scaled}} = g \cdot \text{scale_factor} \quad (243)$$

使用FP16训练时需要缩放以防止下溢。

7.3.2 学习率调度

Warmup：

$$\text{lr}(t) = \text{lr}_{\text{max}} \cdot \min \left(1, \frac{t}{t_{\text{warmup}}} \right) \quad (244)$$

Cosine Decay：

$$\text{lr}(t) = \text{lr}_{\text{min}} + \frac{1}{2}(\text{lr}_{\text{max}} - \text{lr}_{\text{min}}) \left(1 + \cos \left(\frac{t - t_{\text{warmup}}}{t_{\text{total}} - t_{\text{warmup}}} \pi \right) \right) \quad (245)$$

RL特定调度：

- RL学习率通常比SFT小10-100倍

- 可以使用自适应学习率（基于KL散度或奖励变化）

自适应学习率:

$$lr_{t+1} = \begin{cases} lr_t \cdot \alpha_{up} & \text{if } \Delta R > \tau_{up} \\ lr_t \cdot \alpha_{down} & \text{if } \Delta R < \tau_{down} \\ lr_t & \text{otherwise} \end{cases} \quad (246)$$

7.3.3 奖励归一化

批内归一化:

$$r_{norm} = \frac{r - \mu_{batch}}{\sigma_{batch} + \epsilon} \quad (247)$$

移动平均归一化:

$$\mu_t = \beta \mu_{t-1} + (1 - \beta) r_t \quad (248)$$

$$\sigma_t^2 = \beta \sigma_{t-1}^2 + (1 - \beta) (r_t - \mu_t)^2 \quad (249)$$

分位数归一化:

$$r_{norm} = \frac{\text{rank}(r)}{N} - 0.5 \quad (250)$$

将奖励映射到 $[-0.5, 0.5]$ 。

7.3.4 KL散度控制

硬约束:

$$\mathcal{L} = \mathcal{L}_{RL} \quad \text{s.t.} \quad \mathbb{D}_{KL}[\pi_\theta || \pi_{ref}] \leq \delta \quad (251)$$

软约束（惩罚）:

$$\mathcal{L} = \mathcal{L}_{RL} - \beta \cdot \mathbb{D}_{KL}[\pi_\theta || \pi_{ref}] \quad (252)$$

自适应 β :

$$\beta_{t+1} = \begin{cases} \beta_t \cdot 2 & \text{if } \mathbb{D}_{KL} > d_{target} \cdot 1.5 \\ \beta_t / 2 & \text{if } \mathbb{D}_{KL} < d_{target} / 1.5 \\ \beta_t & \text{otherwise} \end{cases} \quad (253)$$

Token级KL:

$$\mathbb{D}_{KL}^{token} = \sum_t \sum_v \pi_\theta(v|s_t) \log \frac{\pi_\theta(v|s_t)}{\pi_{ref}(v|s_t)} \quad (254)$$

对实际开发的指导作用 做RL微调时，很多团队只盯奖励，不盯KL，最后得到的不是更强模型，而是更会投机的模型。实际工程上应把**KL、奖励、长度、拒绝率、格式合规率**一起监控，否则很难判断模型是在真正变强，还是在奖励函数漏洞上投机。

7.4 分布式训练

大规模RL训练需要分布式策略:

7.4.1 数据并行

将数据分配到多个GPU，每个GPU处理一部分样本：

$$g_{global} = \frac{1}{N} \sum_{i=1}^N g_i \quad (255)$$

All-Reduce实现：

- Ring All-Reduce
- Tree All-Reduce
- NCCL优化

7.4.2 张量并行 (Tensor Parallelism)

将模型参数分割到多个GPU：

列并行：

$$Y = XW = X[W_1, W_2, \dots, W_p] = [XW_1, XW_2, \dots, XW_p] \quad (256)$$

行并行：

$$Y = XW = [X_1, X_2, \dots, X_p] \begin{bmatrix} W_1 \\ W_2 \\ \vdots \\ W_p \end{bmatrix} = \sum_{i=1}^p X_i W_i \quad (257)$$

7.4.3 流水线并行 (Pipeline Parallelism)

将模型层分配到不同GPU，形成流水线：

GPipe：微批次流水线

- 将batch分成多个micro-batch
- 流水线式处理，减少气泡

1F1B (One Forward One Backward)： 交替执行前向和后向，进一步减少气泡。

7.4.4 采样与训练分离

Actor-Learner架构：

- **Actor节点：**使用旧策略采样，不更新参数
- **Learner节点：**聚合经验，计算梯度，更新参数
- **参数服务器：**管理参数版本和同步

异步更新： Actor使用稍旧的策略采样，Learner持续更新。需要处理策略滞后问题：

$$\text{IS correction} = \prod_t \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (258)$$

对实际开发的指导作用 真实训练系统里，采样吞吐和训练吞吐通常不是同一个瓶颈。因此一开始就应把“参数版本”“轨迹生成时间”“奖励评估时间”和“训练消费时间”打到日志里。否则系统慢下来的时候，你甚至不知道瓶颈是在生成、在评估，还是在学习器同步。

7.5 内存优化

大规模RL训练对显存的压力往往超过监督微调，因为需要同时维护actor、reference模型和（在PPO中）critic模型的权重与激活值。本节介绍梯度检查点、混合精度、ZeRO以及LoRA/QLoRA四种互补的内存优化手段。

7.5.1 梯度检查点（Gradient Checkpointing）

只保存部分激活值，其他在反向传播时重新计算：

- 内存减少： $O(n) \rightarrow O(\sqrt{n})$
- 计算增加：约33%

7.5.2 混合精度训练

- 前向/后向：FP16/BF16
- 参数更新：FP32（master weights）
- 内存减少约50%

7.5.3 ZeRO优化

ZeRO Stage 1：分割优化器状态 **ZeRO Stage 2**：+ 分割梯度 **ZeRO Stage 3**：+ 分割参数

7.5.4 LoRA/QLoRA

低秩适应，只训练少量参数：

$$W' = W + \Delta W = W + BA \quad (259)$$

其中 $B \in \mathbb{R}^{d \times r}$ ， $A \in \mathbb{R}^{r \times k}$ ， $r \ll \min(d, k)$ 。

QLoRA：在4-bit量化基础上使用LoRA。

7.6 超参数配置指南

7.6.1 GRPO推荐配置

Table 19: GRPO超参数推荐值

参数	推荐范围	说明
组大小 G	16-64	越大越稳定，但成本更高
裁剪参数 ϵ	0.1-0.3	通常0.2效果较好
KL系数 β	0.001-0.1	根据KL散度动态调整
学习率	1e-7 - 1e-5	远小于SFT
批大小	256-1024	越大越稳定
温度	0.7-1.0	采样温度
最大长度	4096-32768	根据任务调整
更新轮数 K	1-4	每批数据更新次数

7.6.2 PPO推荐配置

Table 20: PPO超参数推荐值

参数	推荐范围	说明
裁剪参数 ϵ	0.1-0.2	标准值0.2
GAE λ	0.95-0.99	偏差-方差权衡
折扣因子 γ	0.99-1.0	LLM通常用1.0
值函数系数 c_1	0.5-1.0	值损失权重
熵系数 c_2	0.01-0.1	探索鼓励

7.7 监控与调试

7.7.1 关键指标监控

1. **奖励统计**: 均值、方差、分布
2. **KL散度**: 与参考模型的距离
3. **策略熵**: 探索程度
4. **梯度范数**: 训练稳定性
5. **生成长度**: 是否出现退化
6. **准确率**: 在验证集上的表现
7. **重复率**: 生成内容的重复程度

7.7.2 异常检测

- **奖励突变**: 可能是reward hacking
- **KL发散**: 策略变化过大
- **熵坍塌**: 模式崩塌征兆
- **梯度爆炸/消失**: 训练不稳定
- **长度异常**: 过长或过短

7.7.3 调试技巧

1. **小规模验证**: 先在小模型、小数据上验证
2. **固定随机种子**: 保证可重复性
3. **渐进式扩展**: 逐步增加规模
4. **消融实验**: 隔离各组件影响
5. **可视化生成**: 定期查看生成样本

7.8 常见失败模式与经验教训

理解哪些路线不奏效、为什么不奏效，与理解成功路线同样重要。本节分三部分：首先整理DeepSeek-R1技术报告中明确记录的失败尝试（PRM、MCTS、直接从base模型RL、复杂奖励工程），再系统梳理训练过程中最常见的失败模式及其检测与修复方法，最后讨论规模化阶段特有的工程挑战。

DeepSeek: 过程奖励模型（PRM）的局限 DeepSeek尝试使用PRM但效果不佳:

困难:

- 需要大量步骤级标注，成本极高（每道题需要标注每个步骤）
- 推理步骤的“正确性”难以定义（中间步骤可能有多种表述方式）
- PRM泛化能力有限，易过拟合到特定格式
- 与RL训练的配合需要仔细调节（奖励信号时机）
- 错误步骤的识别本身就是一个困难的问题

定量分析:

- 步骤级标注成本约为结果级的5-10倍
- PRM在训练域外问题上准确率下降30-50%
- 与ORM相比，PRM训练时间增加2-3倍

结论: 简单的结果奖励在实践中更有效。当数据和资源充足时，PRM可能有优势，但边际收益有限。

DeepSeek: 蒙特卡洛树搜索 (MCTS) 的困难 尝试将MCTS应用于LLM生成:
困难:

- 搜索空间巨大 (词表大小 $|\mathcal{V}| \approx 50000$, 序列长度可达数千)
- 中间状态的价值估计不准确 (partial sequence的价值难以评估)
- 计算成本过高 (每个节点需要多次rollout)
- 难以与神经网络有效结合
- Token级决策的branching factor太大

尝试的优化:

- 在句子/步骤级别进行搜索 (减少branching factor)
- 使用神经网络估计节点价值 (加速但准确率低)
- 启发式剪枝 (基于概率阈值)

结论: 简单的采样策略 (如Best-of-N) 可能更实用。MCTS在棋类游戏中成功, 但LLM生成的结构性差异导致直接迁移困难。

直接从Base模型RL的问题 R1-Zero实验表明, 直接从Base模型开始RL虽然可行, 但存在问题:

问题:

- 输出可读性差: 推理过程冗长、结构混乱
- 语言混杂: 英文、中文、代码、数学符号无序混合
- 格式不稳定: 有时不遵循指定格式
- 训练初期不稳定: 奖励方差大, 收敛慢
- 可能出现奇怪的“思考模式”: 无意义的重复、自言自语

观察到的有趣现象:

- 模型自发产生了一些推理元素 (如“wait”, “hmm”)
- 出现了自我验证行为
- 但格式和风格高度不稳定

解决方案: 少量SFT冷启动 + 后续SFT修正。

复杂奖励工程的代价 尝试的复杂奖励：

- 步骤数量奖励（鼓励详细推理）
- 关键词奖励（包含特定词汇）
- 结构奖励（特定格式）
- 长度奖励（避免过长/过短）

问题：

- 各奖励成分权重难以平衡
- 模型容易针对某个奖励成分进行优化
- 导致reward hacking

结论：简单、稀疏的奖励（正确性）往往效果更好。

7.8.1 常见问题与解决

奖励黑客（Reward Hacking） 现象：模型找到获取高奖励的“捷径”，而非真正解决问题。

具体示例：

- 输出格式正确但内容错误：“The answer is $\boxed{42}$ ”（任意数字）
- 生成冗长但无意义的推理：大量重复、填充内容
- 利用评估器的漏洞：发现答案提取正则的bug
- 抄袭问题中的数字：当问题包含数字时，直接输出
- 过度自信的错误答案：用确定性语气说错话

检测方法：

- 监控奖励分布的异常变化
- 人工抽检高奖励样本
- 对抗性测试集
- 分析生成内容的多样性

解决方案：

- **KL散度约束：**防止策略偏离过远
- **多样化奖励函数：**使用多个独立的奖励信号
- **对抗性测试：**专门设计检测捷径的测试
- **人工审核：**定期人工检查
- **早停：**在奖励突然提升时警觉
- **奖励模型集成：**使用多个奖励模型投票

模式崩塌 (Mode Collapse) **现象：**模型输出变得单一，缺乏多样性。

具体表现：

- 对不同问题给出相似的回答模板
- 推理路径高度相似
- 词汇使用范围变窄
- 熵急剧下降

原因分析：

- 高奖励样本被过度学习
- 探索不足
- KL惩罚不够

解决方案：

- **熵正则化：** $\mathcal{L} = \mathcal{L}_{RL} + \lambda H(\pi_{\theta})$
- **温度控制：**适当提高采样温度
- **多样性奖励：**奖励与历史输出不同的生成
- **Population-based训练：**维护多个策略变体
- **重放缓冲区多样性：**确保训练数据多样

遗忘问题 (Catastrophic Forgetting) **现象：**RL训练后，模型失去原有的通用能力。

具体表现：

- 在非RL任务上性能下降
- 丢失某些语言能力
- 格式和风格退化
- 常识问答能力下降

解决方案：

- **混合训练数据：**RL数据 + 通用数据
- **KL约束：**保持与基础模型的距离
- **分阶段训练：**先RL后恢复性SFT
- **弹性权重巩固 (EWC)：**保护重要参数
- **定期评估：**监控通用能力指标

训练不稳定 现象：损失震荡、性能波动大、奖励不收敛。

原因分析：

- 学习率过高
- 批大小过小
- 奖励噪声大
- 策略变化过快
- 值函数估计不准

解决方案：

- **降低学习率：**通常要比SFT低1-2个数量级
- **增加批大小：**更稳定的梯度估计
- **梯度裁剪：**防止梯度爆炸
- **更保守的裁剪参数：** $\epsilon = 0.1$ 而非0.2
- **奖励归一化：**减少奖励方差
- **Warmup：**逐步增加学习率
- **检查点：**保存多个检查点以便回滚

长度退化 现象：生成内容越来越长或越来越短。

长度爆炸原因：

- 无长度惩罚时，长回答可能获得更高奖励
- 模型学会用冗长内容“填充”

长度塌缩原因：

- 过强的长度惩罚
- 短回答容易获得格式奖励

解决方案：

- 适度的长度惩罚/奖励
- 使用参考长度进行归一化
- 监控长度分布变化

7.8.2 规模化中的挑战

计算资源管理 挑战:

- RL需要大量采样, GPU利用率低
- 采样和训练的异步协调
- 参数同步开销
- 内存管理复杂

经验:

- 采样使用较低精度 (FP16/INT8)
- 训练使用混合精度
- 分离采样和训练GPU资源
- 使用高效的参数同步策略

超参数敏感性 发现:

- 不同规模模型需要不同超参数
- 最优学习率随模型大小变化
- KL系数需要根据任务调整

建议:

- 从小模型开始调参
- 使用自适应调度
- 保守起步, 逐步激进

规模化时的奖励黑客问题 现象: 模型找到获取高奖励的“捷径”, 而非真正解决问题。

示例:

- 输出格式正确但内容错误
- 生成冗长但无意义的推理
- 利用评估器的漏洞

解决方案:

- KL散度约束
- 多样化奖励函数
- 对抗性测试
- 人工审核

8 智能体系统

智能体系统是大模型能力从文本生成走向真实任务执行的关键跨越。本章从三个层次展开：首先说明控制层（harness）的核心概念与设计原则，解释为什么智能体系统需要在模型之外构建权限约束与层级控制框架；其次通过OpenClaw、Claude Code与ChatGPT agent三个典型工程实现，具体分析控制层在权限管理、工具协议、会话状态与隔离执行上的设计选择；最后讨论工具增强推理与多智能体协作的训练与部署方向。

8.1 控制层（harness）的形式化定义

智能体系统中的**控制层（harness）**可以理解为“包裹在基础模型外部的控制层”，它负责把LLM、工具、记忆、审批与环境交互编排成一个可执行系统。若仅有基础模型 π_θ ，系统只能生成文本；而加入控制层后，系统动作空间扩展为：

$$a_t \in \mathcal{A}_{\text{text}} \cup \mathcal{A}_{\text{tool}} \cup \mathcal{A}_{\text{delegate}} \cup \mathcal{A}_{\text{stop}}. \quad (260)$$

我们可将一个智能体控制层（harness）形式化为六元组

$$\mathcal{H} = (\pi_{\text{plan}}, \pi_{\text{exec}}, g, u, \mathcal{T}, \mathcal{M}), \quad (261)$$

其中：

- π_{plan} ：决定当前是直接回答、调用工具、还是转入子任务；
- π_{exec} ：在具体动作类型内生成参数，例如工具名、命令参数、子代理任务说明等；
- g ：安全守卫（guard），决定某动作是否允许直接执行、是否需要审批、是否必须拒绝；
- u ：记忆更新算子，决定什么应写入长期记忆、会话摘要或工作区；
- $\mathcal{T}$ ：可用工具集合；
- $\mathcal{M}$ ：可用记忆与上下文存储。

若系统状态写为

$$s_t = (x_{\leq t}, m_t, w_t, \sigma_t), \quad (262)$$

其中 $x_{\leq t}$ 是对话与工具轨迹， m_t 是会话/长期记忆， w_t 是工作区状态， σ_t 是权限与环境状态，则控制层的目标不应只优化任务正确率，而应优化：

$$J(\mathcal{H}) = \mathbb{E} \left[\sum_{t=0}^T (r_t^{\text{task}} - \lambda_c c(a_t) - \lambda_s \xi(s_t, a_t)) \right]. \quad (263)$$

这里 $c(a_t)$ 表示时间/金钱/上下文消耗， $\xi(s_t, a_t)$ 表示风险代价。这个目标函数解释了为什么成熟的智能体系统都不是“让模型自由调用所有工具”，而是必须在成功率、成本与安全性之间做显式权衡。

8.2 控制层 (harness) 的理论深化：约束POMDP与层级控制

在真实智能体中，环境通常是部分可观测且带安全约束的，因此更精确的建模方式是**约束POMDP**。设系统在每一步除了任务奖励 r_t 外，还会产生风险代价 d_t ，则目标可写为：

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^T r_t \right] \quad \text{s.t.} \quad \mathbb{E}_{\pi} \left[\sum_{t=0}^T d_t \right] \leq \kappa, \quad (264)$$

其中 κ 是可接受风险预算。其拉格朗日形式为：

$$\mathcal{L}(\pi, \lambda) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T (r_t - \lambda d_t) \right] + \lambda \kappa. \quad (265)$$

这说明guard、审批器和权限边界并不是“额外的产品逻辑”，而是优化问题本身的一部分。一个没有风险约束的智能体，理论上会倾向于调用所有高收益工具，即使这些动作不可逆、不可审计或不可恢复。

进一步地，智能体通常天然是**层级策略**。记高层策略负责选择子任务或模式 o_t ，低层策略负责在模式内执行动作，则有：

$$o_t \sim \pi_H(o \mid s_t), \quad a_t \sim \pi_L(a \mid s_t, o_t), \quad (266)$$

并由终止函数 $\beta_o(s_t)$ 决定何时结束当前子任务。这个形式恰好对应了实际系统里的：

- 计划层：是否搜索、是否调用浏览器、是否委托子代理；
- 执行层：具体点击哪里、运行什么命令、如何解析结果；
- 终止层：何时收敛、何时请求人工介入、何时放弃当前路线。

对实际开发的指导作用 这一理论直接指导工程实现：

1. 不要把“规划”和“执行”塞进同一条提示词里，最好拆成高层决策与低层执行两个环节；
2. 不要把“权限控制”视为后处理，应该在动作采样前就作为约束注入；
3. 不要只记录最终答案，必须记录子任务切换、工具调用和终止原因，否则无法复盘智能体失败。

8.3 控制层 (harness) 中的工具调用、审批与安全边界

Anthropic在工程实践中区分了**workflow**与**agent**：前者沿着预定义代码路径执行，后者则由模型动态决定过程。对后者而言，控制层最重要的不是“更多工具”，而是**可控的动作边界**。一种常见的门控写法是：

$$\Pr(a_t \text{ executes}) = g(s_t, a_t), \quad (267)$$

其中

$$g(s_t, a_t) \in \{0, 1, \text{need-human-approval}\}. \quad (268)$$

这意味着智能体并不是直接拥有系统权限，而是通过guard函数被约束在一个安全可审计的动作域中。实践上，guard至少应覆盖：

- **能力边界**：哪些shell、浏览器、网络、邮件、支付动作可执行；
- **数据边界**：哪些路径、域名、账号、会话允许访问；
- **审批边界**：哪些动作需要人工确认；
- **持久化边界**：哪些工具输出可以写入记忆或工作区。

从决策角度看，工具调用的价值应综合考量**预期收益**、**调用延迟**和**风险成本**三个维度。只有当预期收益显著高于纯文本推理时，工具调用才真正有价值。

8.4 OpenClaw专章：从Gateway控制总线到可自托管智能体操作面

与很多“聊天机器人外接几个工具”的项目不同，OpenClaw更接近一个**可自托管的 agent operating fabric**。这不是宣传口号，而是从代码结构本身就能看出来：仓库中src/gateway/、src/agents/、src/browser/、src/cron/、src/routing/、src/pairing/、src/security/和大量docs/concepts/文档共同指向一个事实：它的核心目标不是做单点模型增强，而是把**接入、会话、工具、浏览器、节点、自动化与权限**统一到一个长期运行的控制总线上。

8.4.1 系统定位：它不是bot，而是control fabric

OpenClaw官方架构文档把Gateway定义为单一长驻进程：所有消息面、客户端、设备节点、控制UI与自动化入口都通过同一个WebSocket控制平面接入。其抽象可写为

$$\mathcal{O}_{\text{OpenClaw}} = (\mathcal{I}, \mathcal{C}, \mathcal{S}, \mathcal{X}, \mathcal{E}), \quad (269)$$

其中 $\mathcal{I}$ 表示各种入口（聊天渠道、CLI、WebChat、macOS app、Webhook）， $\mathcal{C}$ 表示控制平面（Gateway、鉴权、pairing、session routing）， $\mathcal{S}$ 表示状态平面（session store、transcript、compaction、memory）， $\mathcal{X}$ 表示执行平面（browser、node、system.run、canvas、cron）， $\mathcal{E}$ 表示事件与自动化平面（heartbeat、cron、hooks、plugins、diagnostics）。

这一定义非常重要，因为它解释了OpenClaw为什么天然适合自托管和多入口统一：它不是在“模型外面加一个HTTP接口”，而是在“模型外面加一套长期运行的操作系统薄层”。

8.4.2 控制平面：Gateway是整个系统的中枢神经

从src/gateway/server.impl.ts可以看到，Gateway启动时并不是只监听一个端口就结束了。它会同时装配：

- 配置加载与迁移；
- NodeRegistry与节点订阅管理；
- ExecApprovalManager与审批转发；
- CronService与heartbeat runner；
- browser sidecar、discovery、Tailscale暴露；

- plugin runtime、skills变更监听、gateway reload handlers;
- health、presence、startup auth与控制UI资产服务。

这意味着OpenClaw的Gateway更像一个**可编排控制总线**而不是“普通Web-Socket server”。如果把控制流抽象成有向图，则可写为

Ingress → Auth/Pairing → Route Resolution → Session Lane
 → Agent Loop → Tool/Node/Browser/Cron → Persist/Broadcast.
 (270)

其好处在于，所有跨层交互几乎都强制经过Gateway，因此日志、鉴权、事件广播和故障注入都有了统一落点。这是很多轻量agent脚本完全不具备的系统优势。

8.4.3 会话与路由平面：OpenClaw真正做对了什么

OpenClaw最值得借鉴的部分之一，是它把**session**做成了系统中的一等对象。docs/concepts/session.md和src/routing/resolve-route.ts共同说明了三件：

1. 路由不是模糊prompt规则，而是显式binding优先级，可近似写成peer > parentPeer > guild+roles > guild > team > account > channel > default。
2. 会话不是“对话历史字符串”，而是带有sessionKey、sessionId、origin metadata、channel/account/peer信息的持久状态单元。
3. 执行并发不是任意的，而是按session lane串行，以避免同一session下的工具竞态和状态污染。

如果写成形式化表示，路由器本质上在实现：

$$f_{\text{route}} : (\text{channel}, \text{account}, \text{peer}, \text{guild}, \text{team}, \text{roles}) \mapsto (\text{agentId}, \text{sessionKey}). \quad (271)$$

这看似简单，实际上解决了绝大多数多入口智能体系统最容易混乱的地方：**谁在跟谁说话、上下文属于谁、并发边界在哪。**

尤其值得注意的是，OpenClaw默认把direct message收敛到主session，以换取连续性；但文档同时明确警告在多用户DM场景下应切到per-channel-peer或per-account-channel-peer。这说明它并不是只追求“更丝滑的体验”，而是在连续性与隔离性之间给出可配置权衡。

8.4.4 执行平面：browser、node与system.run的统一抽象

OpenClaw的执行平面不是单一shell，而是三种能力的统一：

1. **主机执行**：Gateway host上的bash、system.run、文件与工作区操作；
2. **浏览器执行**：受控Chromium/CDP/browserless/extension relay；
3. **设备执行**：通过role: node接入的macOS/iOS/Android/headless节点。

这三者在外部分看起来都像“工具调用”，但其安全边界完全不同。OpenClaw正确的地方在于，没有把它们混成一个无差别的execute()。例如浏览器文档明确要求：

- 默认使用独立的openclaw-managed browser profile;
- browser control service仅loopback暴露;
- 通过Gateway auth或node pairing访问;
- SSRF策略、remote CDP超时和profile颜色/隔离配置都显式可控。

于是浏览器环境不再是用户日常浏览器的隐式延伸，而是一个**被控制面包起来的受限子系统**。同理，设备能力也不是“远程SSH进去执行”，而是通过node host暴露有限命令集合，再由Gateway做统一路由。这个抽象非常像一个轻量版的分布式执行fabric。

8.4.5 自动化平面：cron、heartbeat、webhook与hooks不是附属功能

很多项目把自动化理解成“加一个定时器”。OpenClaw则把自动化做成了系统主线的一部分。src/gateway/server-cron.ts和docs/automation/cron-jobs.md表明，系统支持两类本质不同的调度：

- **main session jobs**：向主session注入system event，并配合heartbeat运行；
- **isolated jobs**：在cron:<jobld>中启动独立agent turn，再决定是announce、webhook还是silent。

这比普通“计划任务 -> 调用脚本”高明得多，因为它保留了**session语义**。自动化不是绕过agent，而是成为agent运行时的一种正规入口。进一步看，OpenClaw还有：

- before_prompt_build、before_tool_call、after_tool_call等plugin hooks；
- session_start、session_end、gateway_start、gateway_stop等生命周期钩子；
- cron、webhook、Gmail Pub/Sub、polling等事件源。

换句话说，OpenClaw离“事件驱动智能体系统”其实只差一步：它已经有Observe/Plan/Act的大部分骨架，只是Verify/Commit/Monitor/Rollback还没有被完全系统化。

8.4.6 多智能体：它已经不是单代理，但还不是最终形态

OpenClaw的多智能体路线并非停留在概念层。docs/concepts/multi-agent.md、docs/tools/subagents.md和src/agents/subagent-registry.ts显示，它已经具备：

- 多agent workspace、独立agentDir与独立session store；
- sessions_spawn、sessions_list、sessions_history、sessions_send等session级工具；
- 子代理运行记录持久化、announce流程、orphan run恢复与清理；
- thread-bound subagent sessions与一定程度的嵌套深度控制。

这说明OpenClaw已经从“单智能体 + 工具”进化为“**会话级多代理系统**”。但它当前的协作主通道仍主要是session与message，而不是共享黑板或artifact graph。也就是说，它已经有**调度形态的多代理**，但还没有完全进入**工件形态的多代理**。这也是它未来最值得继续推进的方向。

8.4.7 安全模型：它的成熟度高于很多开源agent项目

OpenClaw的安全设计不是一句“支持sandbox”。从README、docs/tools/exec-approvals.md和相关源码看，它至少同时做了六层安全收缩：

1. **gateway auth + device pairing**：所有WS连接先经过身份与pairing；
2. **session隔离**：不同agent、不同session、不同channel可以有不同上下文边界；
3. **sandbox mode**：尤其针对non-main/group场景把bash转入Docker沙箱；
4. **exec approvals**：对host exec实行deny/allowlist/full与ask/on-miss策略；
5. **safe bins**：只允许特定stdin-only命令自动放行；
6. **browser/network guard**：对SSRF、remote CDP、公开暴露和loopback策略做约束。

这里最值得学习的设计细节，是bash-tools.exec-approval-request.ts里的**两阶段审批注册**。原因很朴素：如果审批ID没有先在服务端注册，/approve就可能因为竞态而找不到对象，造成“用户已经批准但运行时还没记住”的孤儿状态。这种细节只有真正做过长驻系统的人才会提前想到。

8.4.8 为什么它好：四个工程优点

如果只用一句话概括OpenClaw的优点，我会说它把很多agent系统中分散的能力，收敛成了一个**统一、可自托管、可观察、可扩展的控制平面**。更具体地说，有四点特别突出：

1. **local-first但不local-only**：默认本地与私网优先，但能平滑扩展到远程Gateway、Tailscale、SSH和Browserless。
2. **状态观很强**：session、transcript、compaction、maintenance、origin metadata都被当成核心对象，而不是附属缓存。
3. **入口统一**：聊天渠道、CLI、WebChat、节点、cron、webhook都尽量复用同一控制面。
4. **可定制性高**：agent workspace、skills、bindings、browser profiles、session policy、cron delivery、plugin hooks都能独立调整。

对于研究者，这意味着它是一个很好的agent runtime样本；对于工程团队，这意味着它不是只能做demo的项目，而是一套能够承载真实工作流的骨架。

8.4.9 主要不足：复杂度、信任边界与协作形态仍待升级

OpenClaw也有清楚的短板，而且这些短板恰恰来自它的雄心。

第一，系统复杂度已经接近平台级。 它同时处理多渠道接入、浏览器、节点、会话、自动化、插件、配对、审批与UI，这使得配置面非常大。功能越强，调试与回归测试成本越高。

第二，默认信任模型仍偏“同一操作者域”。 文档明确指出，gateway-authenticated callers与paired nodes基本处于同一信任边界，exec approvals更多是防误触，而不是严格的多租户权限隔离。这对个人和小团队很好，但对强隔离企业场景还不够。

第三，部分隔离仍是“策略隔离”而非“强隔离”。 例如多agent的auth profile目前仍有main-agent fallback，workspace默认只是cwd边界而非硬沙箱。它已经比普通agent强很多，但还没有达到零信任分层系统的级别。

第四，多智能体协作仍偏session/message导向。 当前subagent已经能运行、回传、线程绑定，但共享知识的主媒介仍是消息与transcript，而不是强约束的artifact协议。因此在复杂研发任务里，信息仍可能漂移。

8.4.10 如何定制OpenClaw：最有杠杆的六个旋钮

如果把OpenClaw用作自己的智能体底座，我认为最值得先改的不是prompt，而是以下六个配置面：

1. **agents.list + workspace**：先把不同任务域拆成独立agent，而不是让一个主agent承担所有人格和工作流。
2. **bindings + dmScope**：先把“谁的消息路由到谁的session”理顺，否则后面所有记忆都会污染。
3. **browser profiles + node browser proxy**：让浏览器自动化尽量在隔离profile或远程browser node中运行。
4. **sandbox + exec approvals + safe bins**：把危险执行和常见只读命令分开对待。
5. **cron/webhook/heartbeat**：把稳定的重复任务从人工触发迁到事件入口。
6. **plugin hooks + skills**：把领域规则前移到runtime hooks，而不是全压到system prompt里。

这六个旋钮之所以重要，是因为它们分别对应**人格隔离、上下文隔离、执行隔离、权限隔离、自动化入口与领域扩展**五个关键面。OpenClaw真正的定制能力，主要就建立在这些面上。

8.4.11 如何把它继续开发成真正的多智能体系统

如果把OpenClaw作为下一代多智能体平台，我认为至少要再向前走四步：

1. 引入**共享工件总线**，让subagent默认读写结构化artifact，而不是只传自然语言announce。
2. 增加**角色化调度器**，把planner、researcher、executor、verifier做成显式角色与资源配额，而不是任意spawn。
3. 建立**跨代理冲突检测与合并协议**，尤其面向代码、文档、配置和部署工件。
4. 做**代理级评测与回放**，把每个agent的成功率、工具误用率、上下文压缩损失和协作延迟纳入统一观测面。

做到这一步之后，OpenClaw就会从“多session多agent runtime”真正跨到“多代理操作面”。

8.4.12 如何把它继续开发成全自动系统

若目标是无人值守运行，OpenClaw已经具备大量基础件：cron、webhook、heartbeat、node、browser、sessions、approvals、hooks、plugins都在。缺的主要不是功能，而是**闭环顺序**。我建议的演化顺序是：

1. 先统一**事件摄取层**：把cron、webhook、Gmail Pub/Sub、渠道消息与监控告警整理为同一种runtime event。
2. 再建立**验证层**：把独立verifier agent、测试运行器、schema checker、diff reviewer纳入标准环节。
3. 然后加上**提交与回滚事务**：对代码、配置、消息发送、外部API写入设置可回滚边界。
4. 最后才开放**自治等级编排**：在低风险域启用guarded-auto，在高风险域继续保留approval gate。

也就是说，OpenClaw走向全自动的正确路径，不是“把审批全关掉”，而是把系统从“会自动触发”升级为“会自动验证、自动止损、自动恢复”。

8.4.13 结论：OpenClaw最有价值的不是某个功能，而是一种架构取向

OpenClaw最值得重视的地方，不在于它支持多少渠道或多少工具，而在于它证明了一条非常清晰的路线：**智能体系统可以被构造成统一控制总线，而不是一堆零散的prompt工具脚本**。一旦接受这个视角，很多工程选择就会变得清晰起来：session要持久化，路由要显式，browser要隔离，node要配对，自动化要事件化，审批要两阶段注册，多代理要有生命周期管理。

这正是为什么我认为OpenClaw对中文技术社区尤其有参考价值。它让我们看到：真正成熟的agent系统，不是“更会聊天”，而是**更像一套可操作、可审计、可扩展的运行基础设施**。

8.5 Claude Code源码快照：设计、工作机制与扩展路径

为了更准确地分析Claude Code，本次不再依赖‘claw-code’这类clean-room重构，而是直接阅读本地‘software/claude-code-main’中的TypeScript源码快照。该仓库README将其描述为一次由npm source map暴露而可访问的‘src/’快照；无论其发布背景如何，从工程分析角度看，当前代码已经足以揭示Claude Code的**真实系统边界**：它不是一个“把大模型包成CLI”的轻量壳，而是一套带有命令面、工具面、权限面、MCP平面、Hook平面、Agent平面、远程平面与状态平面的完整运行时。

8.5.1 总体结构：它到底由哪些层组成

从一组关键源码文件，例如‘src/main.tsx’、‘src/commands.ts’、‘src/tools.ts’、‘src/QueryEngine.ts’与‘src/setup.ts’，可以抽象出如下结构：

$$\mathcal{C}_{\text{ClaudeCode}} = (\mathcal{I}, \mathcal{C}, \mathcal{T}, \mathcal{Q}, \mathcal{P}, \mathcal{H}, \mathcal{M}, \mathcal{A}, \mathcal{R}), \quad (272)$$

其中 $\mathcal{I}$ 是初始化与启动层（init/startup）， $\mathcal{C}$ 是命令层（commands）， $\mathcal{T}$ 是工具层（tools）， $\mathcal{Q}$ 是查询与回合引擎（query engine）， $\mathcal{P}$ 是权限系统（permissions）， $\mathcal{H}$ 是生命周期hooks， $\mathcal{M}$ 是MCP外部能力平面， $\mathcal{A}$ 是Agent/Team/Coordinator多智能体平面， $\mathcal{R}$ 是本地、远程、worktree与direct-connect等运行模式。

这种拆法有一个非常重要的含义：Claude Code不是把“模型输出”直接映射成“系统调用”，而是先经过多层**控制面重写**。模型产生的是一个在受约束动作空间中运行的策略，而不是裸奔的OS权限。

8.5.2 它是如何工作的：从启动到单轮执行

‘src/main.tsx’显示，Claude Code在非常早的阶段就开始做**并行预取**：例如‘startMdmRawRead()’、‘startKeychainPrefetch()’、GrowthBook初始化、API bootstrap、官方MCP注册表预取，以及远程设置和策略限制加载，都尽量重叠在导入期与启动期执行。这不是小优化，而是一个成熟CLI系统的标志：**把高延迟I/O从交互关键路径中搬走**。

初始化完成后，系统再进入‘src/commands.ts’与‘src/tools.ts’所描述的双层动作空间。前者注册的是宏观控制动作，例如‘/config’、‘/mcp’、‘/memory’、‘/tasks’、‘/permissions’、‘/hooks’、‘/agents’、‘/plan’与‘/review’；后者注册的是细粒度执行能力，例如文件读写、shell、网页抓取、搜索、Agent协作、LSP、worktree、消息发送、团队管理与MCP相关工具。于是系统的真实动作空间不是单层的，而是

$$\mathcal{A} = \mathcal{A}_{\text{cmd}} \cup \mathcal{A}_{\text{tool}}. \quad (273)$$

命令面负责**组织 workflow**，工具面负责**实施动作**。这一分层比“全部都做成tool”更好，因为slash command可以承担用户体验、状态切换与系统管理，而tool可以保持最小、原子、可审计。

真正的回合内核在‘src/QueryEngine.ts’。它显式维护‘tools’、‘commands’、‘mcp’会话消息状态。形式上，可把单轮转移写成

$$s_{t+1} = F(s_t, u_t, a_t, e_t), \quad (274)$$

其中 u_t 是用户输入, a_t 是命令或工具动作, e_t 是Hook/权限/远程事件。由于 ‘canUseTool’、会话持久化、文件状态缓存与预算控制都进入了转移函数, Claude Code从根本上不是“单轮问答”, 而是**带状态、带预算、带守卫的执行状态机**。

此外, ‘src/types/hooks.ts’ 表明系统将 ‘SessionStart’、‘UserPromptSubmit’、‘FileChange’ 都建模为一等生命周期事件, Hook不仅能补充上下文, 还能返回批准/拒绝、修改工具输入、更新权限乃至指定worktree路径。换句话说, Claude Code把“可定制控制逻辑”放在了系统中轴上, 而不是附加脚本。

8.5.3 系统设计深剖：控制平面、状态平面、执行平面与扩展平面

若从体系结构而不是功能罗列来理解Claude Code, 可以把它进一步拆成四个平面:

$$\mathcal{C}_{\text{ClaudeCode}} = \mathcal{P}_{\text{ctrl}} \oplus \mathcal{P}_{\text{state}} \oplus \mathcal{P}_{\text{exec}} \oplus \mathcal{P}_{\text{ext}}. \quad (275)$$

其中, $\mathcal{P}_{\text{ctrl}}$ 是**控制平面**, 负责命令解析、模式切换、预算控制、权限判定、Agent调度与Hook治理; $\mathcal{P}_{\text{state}}$ 是**状态平面**, 负责session、transcript、memory、task状态、文件缓存与settings snapshot; $\mathcal{P}_{\text{exec}}$ 是**执行平面**, 负责Bash、文件系统、LSP、MCP、worktree、remote/direct-connect等真实动作落地; $\mathcal{P}_{\text{ext}}$ 是**扩展平面**, 负责Hooks、Plugins、Skills、MCP Servers与自定义Agents。

这个拆分极其重要, 因为它表明Claude Code的本质不是“模型+工具”的二元结构, 而是“**策略核心 + 控制平面 + 执行平面 + 状态管理 + 扩展注入**”的复合系统。模型在这里只是决策器的一部分, 真正决定系统是否稳定的是其余四层。

Table 21: Claude Code的四平面系统设计

系统平面	代表模块	核心职责
控制平面	‘main.tsx’, ‘commands.ts’, ‘QueryEngine.ts’, permissions, hooks, coordinator	模式切换、预算控制、工具裁剪、权限判定、生命周期治理、Agent调度
状态平面	session/ transcript, memory, file cache, task state, settings snapshot	保存对话、压缩上下文、恢复任务、记录中间状态、支持长程执行
执行平面	Bash/ File/ LSP/ MCP/ worktree/ remote/ direct- connect	将策略动作落到真实文件、终端、远程环境与外部系统
扩展平面	hooks, plugins, skills, MCP servers, custom agents	让系统可二次开发、可注入组织策略、可接入外部能力

进一步看, Claude Code的执行平面并不是无条件暴露的, 而是受权限上下文约束的条件执行器集合:

$$\mathcal{E}_{\text{eff}}(t) = \{e \in \mathcal{P}_{\text{exec}} : \text{allowed}(e, s_t, \Pi_t)\}, \quad (276)$$

其中 Π_t 是时刻 t 的权限状态。也就是说，Claude Code真正做对的一点，是把“能不能执行”与“如何执行”都从prompt里剥离出来，交给runtime自己处理。

8.5.4 为什么这种设计好

Claude Code之所以强，不在于某个“神秘prompt”，而在于它同时解决了终端智能体最难的五件事：

- 1. **启动性能**：启动期并行预取，避免重模块和高延迟I/O阻塞首次交互。
- 2. **动作分层**：命令面与工具面分离，既便于组织高层流程，也便于限制底层权限。
- 3. **权限内生**：权限不是外部脚本，而是运行时第一等对象；‘permissionSetup.ts’甚至显式禁止auto mode下危险的Bash与Agent放权规则。
- 4. **外部能力标准化**：MCP不是“附加插件”，而是和原生tool同级的能力平面，便于统一命名、认证、输出预算与回放。
- 5. **Agent原生**：子智能体、background agent、teammate、team、coordinator、worktree isolation都不是外挂模式，而是编译进主运行时的结构能力。

Table 22: Claude Code源码快照揭示出的核心设计优点

设计点	代码证据	工程价值
启动并行化	‘main.tsx’中的MDM/Keychain/GrowthBook/API预取	降低CLI首轮响应延迟，改善真实开发体验
命令与工具分层	‘commands.ts’ + ‘tools.ts’	高层工作流与底层执行解耦，便于演进与裁剪
权限规则内生	‘permissionSetup.ts’、PermissionRequest 组件链	把风险控制嵌入主循环，而不是事后审计
Hook 事件状态机	‘types/hooks.ts’、‘sessionState.ts’	可在会话、工具、子智能体、worktree层做策略注入
标准化外部能力	‘services/client.ts’、mcp/	外部服务接入不再是脆弱插件，而是统一协议面
多智能体运行时	‘runAgent.ts’、‘coordinator.ts’、swarm工具链	并行研究实施，验证成为系统级能力
隔离执行面	‘setup.ts’中的worktree/路径，tmux Agent的‘isolation’字段	降低修改冲突，支持长期后台任务与受控自治

这类设计的根本优势在于：它把“模型会不会想”转化成“系统能不能稳定地让模型去做事”。对于真实研发场景，后者往往更稀缺。

8.5.5 从源码抽象出的七条设计原则

如果把Claude Code仅仅理解为“一个好用的CLI”，就会低估它真正的工程价值。更准确的理解方式，是把它看成一组**已经在源码中被制度化的agent runtime设计原则**。这些原则之所以重要，不在于它们听上去漂亮，而在于它们都已经落在具体模块、具体字段和具体控制流里。

1. **把启动延迟当作一等问题**：‘main.tsx’不是先等所有依赖加载完再开始工作，而是通过MDM、Keychain、GrowthBook、bootstrap与MCP registry的并行预取，把高延迟I/O从用户首轮交互路径移走。
2. **把命令当 workflow DSL，把工具当执行ISA**：‘commands.ts’负责组织高层流程，‘tools.ts’负责提供原子动作。这等价于把“用户体验层”和“执行语义层”分开设计。
3. **把权限规则编译进运行时，而不是写进prompt建议**：‘permissionSetup.ts’对危险Bash、PowerShell和Agent自动放权规则做显式判定，说明安全边界由runtime强制执行，而不是靠模型“自觉”。
4. **把Agent做成声明式运行时对象**：‘loadAgentsDir.ts’里的‘tools’、‘disallowedTools’字段，说明Agent并不是一段prompt，而是一个可被配置、限制、继承和治理的执行单元。
5. **把协作纪律写进协调者协议**：‘coordinatorMode.ts’并没有把“并行研究、协调综合、独立验证”留给操作者临场发挥，而是直接把这些协作规范写进协调者系统提示和工具使用规则。
6. **把功能开关视为架构演化器**：大量‘feature(...)’条件导入说明Claude Code走的不是纯插件式微内核，也不是不可演化的大泥球，而是一种**受特性门控的模块化单体**。
7. **把session、hooks、memory视为通用中间件**：‘QueryEngine.ts’、‘types/hooks.ts’和session/memory相关模块共同表明，会话状态、生命周期事件和多层记忆并非附属能力，而是所有动作的共同控制面。

把这七条原则放在一起看，Claude Code真正做对的事情可以压缩成一句话：

Agent Runtime = Policy-Enforced Execution+Stateful Orchestration+Typed Extensibility.
(277)

这也解释了为什么它的价值不只属于Anthropic自身。任何团队只要要做**终端智能体、多智能体研发助手或受控自动化平台**，最终都会撞上同一组问题：启动延迟、动作分层、权限爆炸、角色治理、会话恢复和可扩展外设接入。Claude Code的源码价值，就在于它已经给出了一套可复用的先验答案。

Table 23: Claude Code源码抽象出的七条可复用设计原则

设计原则	源码信号	可迁移价值
启动路径并行化	‘main.tsx ‘中的多路 prefetch	降低首轮等待时间，减少“工具很强但一启动就卡”的体验断裂
命令/工具双层动作空间	‘commands.ts ‘与 ‘tools.ts ‘的治理	高层流程与底层执行解耦，便于权限裁剪与 workflow 扩展
权限前置编译	‘permissionSetup.ts 的危险规则检测	安全边界不依赖模型自觉，适合做受控自动化
Agent 声明式配置	‘loadAgentsDir.ts ‘的字段模式	角色可模板化、可治理、可继承，而不是散落 prompt 片段
协调者制度化	‘coordinatorMode.ts 的研究/综合/实现/验证协议	多智能体不再只是“多开几个 worker”，而是有明确协作分工
特性门控模块化	广泛的 ‘feature(...) ‘条件导入	支持快速演化、灰度上线和产品线分层，而不必彻底拆仓
会话与 Hook 中间件化	‘QueryEngine.ts ‘、 ‘hooks.ts ‘、 session/memory 链路	‘type 长期运行、恢复、审计和策略注入提供统一落点

8.5.6 主要不足：不是功能不够，而是复杂度进入了新的层级

越是强大的runtime，越会把复杂度从“模型侧”转移到“系统侧”。Claude Code 的不足也主要来自这里。

1. 系统复杂度已经非常高 从源码结构看，Claude Code已经不是一个轻量CLI，而是一个带有终端UI、权限系统、Hook系统、MCP客户端、远程桥、agent编排器、工作树与状态恢复能力的复杂平台。其问题不是“功能少”，而是**复杂度太集中**：很多能力都汇入同一个主运行时，导致认知负担、调试成本和回归风险都很高。
2. 多平面耦合较强 控制平面、状态平面、执行平面和扩展平面虽然概念上分离，但在实现上仍有大量跨模块依赖。例如权限、Hook、session、MCP、agent工具之间存在较深耦合。这会带来两个后果：一是局部改动不容易局部验证；二是长期维护中容易出现“新增能力破坏旧假设”。
3. 多智能体能力已有，但治理层仍偏薄 它已经具备coordinator、teammate、background agent、team工具与message passing，但当前更像**多智能体运行时原型**，还不是完整的多智能体生产平台。缺的不是“再多几个worker”，而是组织级调度、资源预算、冲突消解、角色权限模板与结果归因。

4. 自动化与安全仍有内在张力 ‘permissionSetup.ts’明确阻止危险auto mode规则，这说明Claude Code的设计者非常清楚一件事：**完全自治与可控安全天然冲突**。这不是某个bug，而是系统本性。只要系统同时拥有代码执行、文件写入、远程连接和subagent扩张能力，就必须面对权限爆炸问题。

5. 记忆与长期任务仍是演进中的能力 源码中已出现session memory、memdir、team memory sync、resume与transcript压缩，但离“可靠承载多天连续任务”还有距离。真正的长期自治需要更强的检查点、任务快照、幂等恢复、外部job队列与因果可追溯性。

Table 24: Claude Code的系统性短板

问题类别	具体表现	后果
运行时过重	主系统承载命令、工具、权限、Hook、MCP、Agent、远程桥等大量功能	启动、测试、调试和回归验证成本上升
模块耦合偏深	权限、Hook、session、agent、MCP存在多方向依赖	局部修改难以局部证明正确
多智能体治理不足	已能spawn teammate，但角色模板、配额、归因、冲突检测不足	容易从并行协作退化为混乱并发
自动化边界紧张	越自动越接近高风险权限与不可逆动作	企业场景难以默认放开
长期自治未闭环	会话恢复强于任务恢复，日志强于检查点	小时级以上任务的稳定性受限

8.5.7 如何定制：命令、工具、Agent、Hook、MCP与记忆

Claude Code的可定制性很强，而且不是只有一个入口，而是至少有六个层级：

1. 命令层定制 如果目标是扩展用户工作流，例如新增 ‘/deploy ‘、 ‘/release-check ‘、 ‘/schema-migrate ‘，最自然的入口是 ‘src/commands/ ‘。这类扩展适合封装**多步、具交互语义、带状态切换**的操作，而不是单个原子动作。

2. 工具层定制 如果目标是让模型具备一种新的可执行能力，例如调用企业内部构建系统、触发私有数据管道、访问内部审批API，则应扩展 ‘src/tools/ ‘。工具层定制要求提供输入模式、权限模型与执行逻辑。Claude Code的优点是：**新增工具并不破坏命令层**，可以保持动作语义稳定。

3. Agent定制 ‘src/tools/AgentTool/loadAgentsDir.ts ‘展示了Agent定义的真实可配置字段： ‘tools ‘、 ‘disallowedTools ‘、 ‘prompt ‘、 ‘model ‘、 ‘effort ‘、 ‘

这意味着一个Agent本质上是：

$$\text{Agent} = (\text{prompt}, \text{tools}, \text{policy}, \text{memory}, \text{runtime}). \quad (278)$$

你完全可以把Claude Code定制成“代码审查员”“安全审计员”“部署协调员”“文档整理员”“数据标注主管”等不同角色，而不必改主循环。

4. Hook定制 Hook是最强大的定制入口之一。它允许你在会话开始、用户提交、工具前后、权限请求、子智能体启动、worktree创建等关键点注入组织策略。比如：

- 在 ‘SessionStart ‘时加载项目约束、合规条款或环境变量快照；
- 在 ‘PreToolUse ‘时拦截危险命令、重写输入、补充上下文；
- 在 ‘PermissionRequest ‘时接入企业审批系统；
- 在 ‘PostToolUse ‘时自动运行格式化、Lint或结构化摘要；
- 在 ‘WorktreeCreate ‘时把agent路由到特定隔离仓库或非Git VCS。

5. MCP定制 ‘services/mcp/client.ts ‘与相关配置代码表明，MCP在Claude Code中不是边缘能力，而是**统一外设总线**。企业若要让Claude Code接入GitHub、Jira、内部知识库、构建系统、实验平台，最干净的方法不是硬写新工具，而是优先做成MCP server，再由Claude Code统一消费。

6. 记忆与项目上下文定制 源码中存在 ‘memdir ‘、 ‘SessionMemory ‘、 team memory sync与agent memory scope等机制，说明它已经不满足于“单轮上下文窗口”，而是在构造一种**项目级、用户级、团队级**的多层记忆。对企业使用者而言，正确做法不是无限追加prompt，而是把稳定知识、近期会话与团队共享知识拆层管理。

8.5.8 如何继续开发成真正的多智能体系统

Claude Code其实已经离“多智能体平台”只差最后一层**治理与调度架构**。其源码里，多智能体能力至少体现在四个地方。

第一， ‘runAgent.ts ‘说明子智能体不是简单的“再次调模型”，而是具有独立MCP server初始化、独立转录目录、独立Hook链路和独立权限/上下文继承逻辑的运行单元。第二， ‘coordinator/coordinatorMode.ts ‘明确把**协调者**建模为一类特殊系统提示，并围绕 ‘AgentTool ‘、 ‘SendMessageTool ‘、 ‘TaskStopTool ‘、 ‘TeamCr 设计了leader-worker协作协议。第三， ‘utils/swarm/ ‘下的一整组 ‘teammate ‘与 ‘inProcess 码表明，系统既支持进程外协作，也支持**同进程内的teammate循环**。第四，Agent定义里的 ‘background ‘与 ‘isolation ‘字段，使“后台长任务”和“隔离工作树执行”成为内置选项。

因此，若把任务分解为 $\mathcal{J} = \{J_1, \dots, J_n\}$ ，Claude Code的多智能体化可以自然写成：

$$\Pi = \{\pi_{\text{coord}}, \pi_1, \dots, \pi_n\}, \quad (279)$$

其中 π_{coord} 负责分解、调度与综合， π_i 分别负责研究、编码、验证、部署等子任务。一个实用的最小多智能体架构通常是：

1. **协调者**：只负责分解任务、分配worker、综合结果，不直接大规模改代码；
2. **研究worker**：只读工具，负责找文件、理解上下文、输出修改方案；
3. **实现worker**：拥有编辑和Bash权限，限定在特定worktree或文件域内；
4. **验证worker**：专职跑测试、类型检查、diff审计，不负责写实现；
5. **发布/运维worker**：只接触部署、日志、回滚和告警接口。

这类架构优于“单Agent全做”的原因很简单：**把错误相关性打散**。规划错、实现错、验证错由不同上下文承担，整条链的失效率会下降。

但如果只是把一个主Agent变成“N个worker”，还不够。继续开发Claude Code的多智能体能力，至少还要做四件事。

第一步：把角色模型固化为可复用模板 当前Agent定义已经支持prompt、tools、permissionMode、hooks、memory、isolation等字段，因此完全可以进一步抽象出**标准角色模板库**。例如Researcher、Implementer、Verifier、Security Auditor、Release Operator。角色模板化的好处是把“谁能做什么”从prompt习惯升级成**结构性约束**。

第二步：引入任务图与依赖调度 目前coordinator更像基于对话的调度器，而不是显式DAG调度器。更进一步的方向是把一个复杂任务表示成

$$G_{\text{task}} = (V, E), \quad (280)$$

其中节点 V 是可执行子任务，边 E 表示依赖、输入输出和互斥关系。这样协调者就不只是“发多个worker”，而是能够进行**拓扑调度、失败重试、选择性重算与部分回滚**。

第三步：加入共享黑板与中间工件协议 今天Claude Code已有message passing、scratchpad、transcript和session memory，但未来更强的多智能体系统应当显式要求worker通过**共享黑板**或结构化artifact交换信息，例如设计说明、测试报告、diff摘要、风险评估、部署状态。否则所有协作都退化成自然语言消息，最终会遇到上下文歧义和信息丢失。

第四步：补上治理面 真正的多智能体系统还需要成本预算、并发配额、文件/目录级写锁、责任归因与自动停止条件。没有治理，多智能体越强，只会越贵、越乱、越难审计。

Table 25: Claude Code继续演化为多智能体平台的建议路线

阶段	核心建设	目标
阶段1	角色模板化 + 权限模板化	让不同worker的职责边界稳定可复用
阶段2	任务DAG + 依赖调度	从会话编排升级为任务编排
阶段3	共享黑板 + artifact协议	降低自然语言协作损耗，增强可验证性
阶段4	配额、锁、归因、回滚	把多智能体从“能跑”变成“可治理”

8.5.9 如何把它继续开发成全自动系统

“全自动”不是把所有权限全部放开，而是把自治建立在**约束、隔离、验证与回滚**之上。Claude Code源码已经具备走向全自动的关键部件：‘background ‘agent、cron/remote trigger类工具、‘SleepTool ‘、team/swarm协作、worktree隔离、Hook策略、权限规则、session持久化与远程控制桥。

但从 ‘permissionSetup.ts ‘也能看到，作者非常清楚全自动的危险边界：auto mode下显式阻断危险Bash前缀规则与Agent自动放权规则，说明系统默认假设**自动化最容易在“任意代码执行”和“任意代理扩张”两处失控**。因此，一个能上线的全自动Claude Code系统至少应满足：

1. **动作白名单化**：只允许经过审核的Bash前缀、MCP server与写入目录；
2. **执行隔离化**：所有实施worker默认进入独立worktree或远程沙箱；
3. **验证前置化**：实现worker的输出不能直接发布，必须经过验证worker；
4. **Hook治理化**：把审批、日志、告警、附加上下文、回滚指令做成Hook；
5. **任务调度化**：利用cron/remote trigger而不是靠无限长会话维持“假在线”；
6. **失败可恢复化**：保留session、transcript、task状态与中间artifact，支持继续执行而不是整轮重来。

换句话说，真正的全自动不是

auto = allow all permissions,

而更接近

auto = bounded permissions + isolation + verification + recovery. (281)

如果把Claude Code用于企业研发流水线，我认为最稳妥的落地顺序应是：先做**半自动**（研究自动、实施需审批），再做**受限全自动**（特定仓库、特定命令、特定时间窗），最后再扩展到跨仓库、跨系统的连续自治。

更形式化地说，一个可上线的全自动研发闭环可以写成：

Observe → Plan → Act → Verify → Commit → Deploy → Monitor → Reflect.
(282)

Claude Code当前已经基本具备Observe/Plan/Act/Verify四段的核心构件，但 ‘Commit ‘、 ‘Deploy ‘、 ‘Monitor ‘、 ‘Reflect ‘四段仍需要更强的工程化补全。

Observe 观察阶段应由webhook、cron、remote trigger、CI事件、issue/PR消息、日志告警和session恢复能力驱动，而不是仅靠人工输入prompt。Claude Code已有不少入口，但若全要全自动，需要把这些入口统一成**事件摄取层**。

Plan 规划不能只是一轮自然语言计划，而应输出**结构化执行计划**，至少包含目标文件域、风险等级、所需工具、验证条件、回滚条件。Plan mode已经提供了雏形，但还可以进一步结构化。

Act 执行阶段不应在主工作区裸写，最佳实践是默认进入worktree或远程沙箱，并把不同实施agent映射到不同隔离环境。这样一来，即使全自动步骤失败，也不会立即污染主仓库。

Verify 验证是Claude Code走向全自动的分水岭。没有独立验证，自动化就是高效犯错。一个成熟的全自动环节应要求：实现Agent与验证Agent分离；测试结果结构化记录；将失败日志反哺给planner与implementer；只有通过验证的artifact才允许进入commit或deploy阶段。

Commit与Deploy 这两步最危险，必须要求更高等级的前置约束，例如commit前必须附带变更摘要、测试摘要与风险说明；deploy前必须满足签名条件、环境白名单和回滚计划；生产环境的deploy必须默认经过外部审批Hook，而不是靠模型自己判断“看起来没问题”。

Monitor与Reflect 真正的全自动系统不是“做完即结束”，而是能观察发布后状态，并将失败、回滚、日志模式和用户反馈沉淀成下一轮调度的输入。因此需要把监控告警、自动回滚和经验提炼接入session memory或team memory，而不是只存在于外部CI系统。

Table 26: Claude Code走向全自动的闭环建设图

阶段	已有基础	仍需加强
Observe	hooks, triggers, session resume, remote bridge	统一事件总线、CI/告警摄取、状态归一化
Plan	QueryEngine, plan mode, agent roles	结构化计划、风险分级、依赖图
Act	tools, worktree, remote mode, MCP	更强隔离、资源配额、动作审计
Verify	verifier角色雏形、测试工具链	独立验证协议、artifact级通过条件
Commit/Deploy	已有相关命令与工具基础	强审批、签名、回滚、环境分层
Monitor/Reflect	memory/ session 基础设施	自动告警反馈、经验提炼、长期恢复闭环

8.5.10 从Claude Code到企业级自治平台的递进蓝图

如果把Claude Code当作下一代企业级自治平台的起点，那么真正需要建设的不是“更多工具”，而是**更强的治理闭环**。从源码状态看，它已经足够支撑第一代受控自治，但要跨到稳定的多智能体和全自动系统，还需要一条更清晰的递进路线。

我倾向于把这条路线写成五级台阶：

1. **级别0：受控单代理**。保留当前CLI、权限请求、Hook与会话能力，把重点放在稳定的单代理开发工作流。

2. **级别1：角色化多代理。**基于 ‘loadAgentsDir.ts’ 的声明式字段，把 re-searcher、implementer、verifier、release operator 固化成模板，并默认绑定不同 ‘tools’、‘permissionMode’ 与 ‘isolation’。
3. **级别2：任务图调度。**让协调者不再只是通过自然语言分派，而是围绕显式任务 DAG 做拓扑调度、失败重试、依赖阻塞和部分重算。
4. **级别3：工件总线协作。**把消息流升级为 artifact-first 协作，例如要求 worker 围绕 plan、diff、test report、risk note、deploy record 这些结构化工件交换信息。
5. **级别4：受限全自动。**在前三级成立后，才逐步开放特定仓库、特定命令前缀、特定时间窗和特定审批 Hook 下的连续自治。

从系统论角度看，自治等级并不取决于“模型有多强”，而更取决于下面这个量：

$$\text{Autonomy Readiness} = f(\text{policy precision, verification coverage, rollback strength, artifact}) \quad (283)$$

其中任何一项明显偏弱，所谓“全自动”都会迅速退化成高效犯错系统。

这也正好把第6章提出的三项原创算法提案重新接回到 Claude Code 这类真实系统中：

1. **PACE** 可作为权限层升级器：让 ‘permissionSetup.ts’ 从规则系统演化为动态审批/降级策略。
2. **TRACE** 可作为上下文层升级器：让 session compaction、memory 选择和 resume 不再只靠固定摘要。
3. **CRAFT** 可作为协作层升级器：让 coordinator 与 worker 之间从 message passing 升级为 artifact graph 协作。

因此，Claude Code 最有价值的未来方向，不是继续把更多单点能力塞进主循环，而是围绕**权限、上下文、工件协作**三条主线逐步平台化。只有这样，它才会从“很强的开发者智能体”进化为“可治理的多智能体自治底座”。

8.5.11 它还能怎样改进

即便如此，Claude Code 仍有很清楚的改进空间。

1. 把“命令图/工具图/Hook图”显式化 当前代码已有丰富结构，但缺少统一的任务 DAG 表示。如果未来能把命令、工具、Hook、Agent 依赖关系统一成显式图结构，调试、回放、静态分析和权限证明都会更强。

2. 加强可验证自动化 今天的权限系统已经很严谨，但仍主要依赖规则与交互。下一步应把“动作是否安全”从启发式规则推进到**可证明前置条件**，例如要求部署动作必须携带测试哈希、diff 摘要与责任 agent 签名。

3. 强化长期任务恢复 多 Agent、后台 Agent、远程会话已经存在，但若承载小时级乃至天级任务，还需要更强的任务日志、检查点、幂等重放与外部队列集成。

4. 让记忆质量可控 当前已有session memory、memdir、team memory等机制，但长期看仍需要更强的记忆老化、冲突消解、来源标注与隐私分层，否则系统会逐渐被“错误但高频的旧记忆”污染。

5. 让多智能体从“能跑”升级为“可治理” 源码已具备coordinator、teammate与background agent的基础，但团队真正需要的是**角色模板、职责边界、冲突检测、资源配额和成本归因**。没有这些，多智能体容易退化成昂贵而混乱的并发。

小结 从这份源码快照看，Claude Code最值得学习的地方，不是某个单独功能，而是一种工程哲学：**把模型当作策略核心，把CLI当作控制台，把权限/Hook/MCP/Agent当作运行时骨架，把worktree/remote/session当作执行平面**。这使它既适合作为高效开发者工具，也适合作为企业内部多智能体平台的雏形。若要继续前进，正确方向不是“给它再加更多prompt技巧”，而是继续强化**控制层、隔离层、调度层和恢复层**。

8.6 Claude Code与OpenClaw的结构对照

将Claude Code与前文讨论的OpenClaw并置，可以更清楚地看出两类系统的共同点与差异。它们都属于**控制层先行**的智能体运行时，但优化重点并不完全相同：Claude Code更强调**开发工作流中的终端、工具、会话与hooks编排**，OpenClaw则更强调**跨设备节点、受控浏览器与远程执行平面的统一控制**。

Table 27: Claude Code与OpenClaw的结构对照

维度	Claude Code	OpenClaw
系统定位	面向开发者终端工作流的智能体运行时，强调CLI、MCP、hooks、worktree与会话控制	面向多节点与多执行面的智能体运行时，强调Gateway、Node、受控浏览器与远程控制平面
控制入口	CLI 子 命 令、trust gate、deferred init、mode select	Gateway 统 一 入 口、Clients 接 入、Session/ Agent Loop 调度
动作组织	命令图与工具池分层，先路由候选再进入查询引擎	Agent Loop 统 一 调 度，工具事件流与会话状态机更突出
工具接入	MCP server、内建工具、权限过滤、hooks前后置拦截	浏览器、shell、设备能力、插件与节点能力统一挂载
权限机制	PermissionRequest、PreToolUse/ Post-ToolUse、拒绝事件、工作树与会话级控制	guard、审批门控、节点权限、浏览器隔离与私网控制
状态管理	transcript压缩、session落盘、结构化输出与预算上限	session com-paction、tool stream、持 久 化 日 志与观测闭环
执行模式	本地、远程、SSH、Teleport、direct-connect、deep-link 并列路由	本地工作站、远程网关、远程CDP、Browserless、Browserbase、多设备节点
浏览器设计	官方文档更强调工具/hook/worktree平面，浏览器不是唯一中心	受控浏览器是核心子系统，强调独立profile、CDP隔离与节点代理
适用场景	代码修改、终端任务、开发者助手、受控子智能体协作	浏览器任务、跨设备任务、远程执行、统一控制平面编排

比较结论 二者的共同点在于：都把模型能力置于**显式控制层**之下，而不是让模型直接接触全部系统权限；都强调会话状态、权限边界、工具契约与可观测性。差异在于：Claude Code更像**开发 workflows 中的受控终端编排器**，OpenClaw更像**跨节点与跨执行面的智能体控制总线**。从工程借鉴角度看，如果目标是提升研发任务中的代码、shell与MCP协同效率，Claude Code的设计更有参考价值；如果目标是统一浏览器、移动端、远程节点与受控执行环境，OpenClaw的架构更具代表性。

8.7 Claude Code、OpenClaw与ChatGPT agent的三方对照

如果再把OpenAI官方的ChatGPT agent纳入同一坐标系，则三者分别代表了三种不同的智能体产品取向。根据OpenAI于2025年7月17日发布的*ChatGPT agent System Card*，ChatGPT agent把**deep research、多步研究、远程视觉浏览器、受限终端、连接器以及用户接管机制**组合进同一产品。其核心特点是：**执行环境默认远程化、权限确认产品化、连接器集成一体化、任务完成时间面向分钟级而不是秒级交互**。因此，三者虽然都可归入agent runtime范畴，但其最优设计目标并不相同。

Table 28: Claude Code、OpenClaw与ChatGPT agent的三方对照

维度	Claude Code	OpenClaw	ChatGPT agent
产品定位	开发终端工作流中的智能体运行时	自托管多节点智能体控制平面	面向终端用户的远程执行型智能体产品
主要入口	CLI、hooks、MCP、work-tree、会话文件	Gateway、Clients、Nodes、受控浏览器	ChatGPT 内的 agent mode、远程浏览器、连接器、终端
核心执行面	本地 shell、MCP、插件式工具与模式路由	本地/远程节点、浏览器、设备能力、插件	远程视觉浏览器、受限终端、第一方连接器、文件分析工具
权限形态	hooks、PermissionRequest、工作树与工具级边界	guard、审批门控、节点与浏览器隔离	用户接管、敏感操作确认、有限网络、产品级 safeguard
状态管理	transcript 压缩、session 落盘、结构化回合预算	session compaction、事件流、持久化与观测闭环	长任务状态、工具回合、连接器上下文、产品级安全日志
系统优势	开发者控制强、终端协同效率高、上下文工程细致	可自托管、跨设备、跨节点、跨浏览器执行面统一	产品集成度高、研究/浏览/终端/连接器能力整合度高
典型短板	更偏开发场景，对通用终端用户产品化较弱	平台搭建与运维复杂度更高	用户可控性受产品边界约束，自定义执行面较弱
更适合的场景	代码修改、仓库分析、开发自动化与MCP协作	个人或团队自托管的多端智能体系统	面向最终用户的在线任务执行、研究、表格与网页操作

三方比较的工程含义 从工程视角看，Claude Code强调**开发环境内的受控自治**，OpenClaw强调**执行平面的可编排性与可自托管性**，ChatGPT agent则强调**产品级远程执行、连接器整合与用户可接管交互**。因此，这三种系统不是简单的“谁更强”关

系，而是分别优化了三类目标函数。若记目标向量为

$$J = \begin{pmatrix} \text{dev_efficiency,} \\ \text{execution_coverage,} \\ \text{product_integration,} \\ \text{self_hosting,} \\ \text{safety_controls} \end{pmatrix}$$

则 Claude Code 在 dev_efficiency 上更强，OpenClaw 在 execution_coverage + self_hosting 上更强，ChatGPT agent 在 product_integration 与 **面向最终用户的安全交互产品化** 上更强。这种差异提醒我们：设计智能体系统时，首先要确定是要做 **开发者工具、自托管控制总线**，还是 **面向大众的远程执行产品**；一旦目标不同，控制层、权限层和执行层的最优架构也会随之改变。

8.8 智能体与工具使用

当模型从“回答问题”转向“完成任务”时，工具选择、动作排序、外部状态读写与多智能体协作就从附属能力转变为主干研究问题。

8.8.1 工具增强推理

- **计算器**：精确数值计算
- **符号引擎**：SymPy 等符号计算
- **搜索引擎**：获取外部知识
- **代码执行**：Python 解释器
- **数据库**：结构化数据查询

RL 用于工具选择：

$$\pi_{tool}(t|s) = \text{softmax}(f_{tool}(s, t)) \quad (284)$$

8.8.2 多智能体推理

- **辩论**：多个模型辩论得出结论
- **分工**：不同模型负责不同步骤
- **验证**：一个模型生成，另一个验证
- **集成**：多模型投票

9 前沿闭源模型与工程实践

本章从两个互补视角梳理顶尖AI团队的实践：前半部分基于OpenAI、Anthropic、Google DeepMind等机构的公开技术文档，分析前沿闭源模型可能采用的训练与推理技术；后半部分聚焦这些团队在软件工程闭环、MLOps、安全发布与组织协作上的可借鉴经验。两者相辅相成，因为只有先理解“前沿系统大致做了什么”，才可能进一步判断“成功团队为什么能把它稳定做出来”。

9.1 资料来源与证据分层

本节材料按证据强度分为三层。第一层是**官方直接披露**，包括技术报告、系统卡、研究论文、官方博客与开发文档；凡涉及明确发布日期、评测数字、接口能力和安全流程时，优先以这一层为准。第二层是**公开材料之间的交叉归纳**，即将多篇公开资料中相互支持的信息整理为统一工程图景。第三层是**技术推断**，即在官方未完全公开细节时，根据历史路线、公开评测行为、系统外显能力和相关论文脉络作出的谨慎推测。

因此，本章的阅读重点不是“猜测谁藏了什么秘密”，而是建立一种更可靠的工程判断方式：哪些结论是公开可证的，哪些只是高概率推断，哪些又只适合作为研究假设继续验证。凡正文中出现“推测”“可能”“较大概率”等表述，都应按第三层证据理解。

9.2 OpenAI GPT-5系列技术分析

GPT-5于2025年发布，被OpenAI定位为“最智能、最快、最有用的模型”，其核心创新在于**内置思考能力（thinking built in）**。根据官方披露和技术分析，GPT-5的关键技术特征包括：

1. **自适应推理深度**：模型能够根据问题复杂度自动决定“思考深度”，简单问题快速响应，复杂问题深入推理
2. **Minimal Reasoning模式**：API提供“minimal”推理选项，平衡响应速度与推理质量
3. **Verbosity参数**：可控制输出详细程度，实现个性化响应
4. **多模态原生支持**：文本、图像、代码统一处理架构
5. **长链工具调用**：支持执行复杂的多步骤智能体任务

9.2.1 GPT-5训练Pipeline推测

基于OpenAI历史论文、GPT-4技术报告及行业分析，GPT-5的完整训练流程可推测如下：

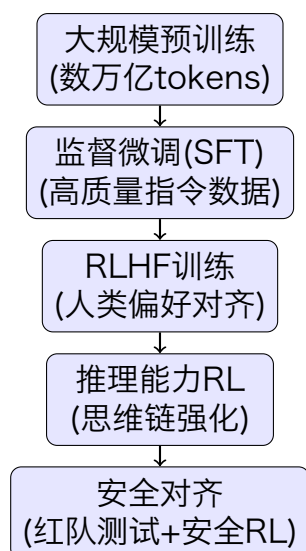


Figure 1: GPT-5推测训练流程

预训练阶段关键技术 根据GPT-4技术报告的延续性推断：

- **数据规模**：预计使用超过15万亿tokens的多源数据，包括网页、书籍、代码、科学论文、多模态数据
- **数据质量过滤**：采用分类器过滤低质量内容，可能使用GPT-4作为数据质量评估器
- **架构优化**：
 - 混合专家（MoE）架构，推测活跃参数约200-400B，总参数可能超过1.5T
 - 改进的注意力机制（可能是GroupedQuery Attention或更新变体）
 - 更长的上下文窗口（128K-1M tokens）
- **训练稳定性**：GPT-4报告指出其训练损失可预测性极高，GPT-5延续此方法论

RLHF与安全训练的演进 GPT-5的RLHF训练相比前代有重大升级：

$$\mathcal{L}_{\text{GPT-5}} = \mathcal{L}_{\text{RLHF}} + \lambda_1 \mathcal{L}_{\text{safety}} + \lambda_2 \mathcal{L}_{\text{reasoning}} + \lambda_3 \mathcal{L}_{\text{consistency}} \quad (285)$$

- **多目标奖励**：
 - 有用性奖励（helpfulness）
 - 安全性奖励（使用GPT-4 zero-shot分类器）
 - 准确性奖励（事实核查）
 - 推理正确性奖励
- **Constitutional AI元素**：内置行为准则，模型能进行自我批评和修正
- **红队测试驱动**：50+领域专家对抗性测试结果反馈回训练

9.2.2 GPT-5性能预测Scaling技术

OpenAI在GPT-4中展示的**损失预测能力**是其核心技术壁垒：

”我们使用仅1/10000计算量的小模型，准确预测了GPT-4在内部代码库上的最终损失。”

这意味着OpenAI掌握了：

1. 精确的Scaling Law参数化模型
2. 跨规模的超参数迁移方法
3. 训练动态的理论理解

推测的Scaling Law形式：

$$L(N, D, C) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + E + f(C) \quad (286)$$

其中 $f(C)$ 是计算效率项，可能涉及MoE架构的稀疏性优化。

9.3 OpenAI o系列推理模型技术分析

o系列（o1、o3、o4-mini）是OpenAI在推理能力上的突破性工作，其核心是**大规模强化学习训练思维链**。

9.3.1 o系列的RL训练范式

根据官方博客” Learning to Reason with LLMs” 的关键披露：

”我们的大规模强化学习算法教会模型如何在高度数据高效的训练过程中使用思维链进行有效思考。”

核心发现1：RL训练Scaling

$$\text{Performance}(o) \propto \log(\text{RL Training Compute}) \quad (287)$$

官方展示了o1在AIME数学竞赛上的性能随训练计算量对数增长。这表明：

- RL训练存在类似预训练的尺度定律（scaling law）
- 但其扩展约束条件与预训练不同（更依赖奖励信号质量）

核心发现2：推理时计算扩展

$$\text{Accuracy} = g(\text{Thinking Tokens}) \quad (288)$$

更多的”思考时间”（以token数量衡量）带来更高的准确率，这是传统LLM不具备的特性。

9.3.2 o3/o4-mini的技术突破

2025年4月发布的o3和o4-mini相比o1有重大升级：

Table 29: o系列模型演进对比

特性	o1	o3	o4-mini
AIME 2024	83.3% (cons@64)	98.4% (w/ tools)	99.5% (w/ tools)
Codeforces Elo	1673	SOTA	SOTA
工具使用	有限	全面智能体能力	全面智能体能力
多模态推理	文本为主	图像集成到CoT	图像集成到CoT
推理可见性	隐藏	摘要可见	摘要可见

关键技术1：工具使用的RL训练 o3/o4-mini的重大创新是**通过强化学习训练工具使用能力**：

”我们也通过强化学习训练两个模型使用工具——不仅教它们如何使用工具，还要推理何时使用它们。”

这意味着工具调用决策被纳入RL优化目标：

$$\pi^*(a|s) = \arg \max_{\pi} \mathbb{E} \left[\sum_t r_t + \lambda \cdot \mathbf{1}[\text{tool_useful}(a_t, s_t)] \right] \quad (289)$$

关键技术2：图像思维链 o3/o4-mini首次实现**将图像集成到思维链中**：

”这些模型可以在思考过程中直接整合图像。它们不仅仅是看到图像——而是用图像思考。”

技术实现推测：

- 视觉编码器输出可被引用/操作于CoT中
- 模型可在推理过程中对图像进行变换（旋转、缩放）
- 多模态token与文本token统一表示

9.3.3 o系列的过程奖励模型（PRM）深度分析

根据OpenAI 2023年论文”Let’s Verify Step by Step”和后续推断，PRM是o系列的核心组件。

PRM训练数据构建 OpenAI拥有业界最大规模的步骤级人工标注数据：

1. **标注规模**：推测超过100万条步骤级标注
2. **标注质量**：数学专家逐步标注推理正确性
3. **标注粒度**：每个推理步骤标记为{正确、中性、错误}

PRM架构推测

$$\text{PRM}(s_1, s_2, \dots, s_t) = \text{Transformer}([\text{problem}; s_1; \dots; s_t])[-1] \quad (290)$$

可能的架构选择:

- 基于GPT-4的微调版本
- 输出为每步的正确性概率
- 支持早停判断（检测到错误即可停止）

PRM与ORM的混合策略 推测o系列使用混合奖励:

$$R_{\text{total}} = \alpha \cdot R_{\text{PRM}} + (1 - \alpha) \cdot R_{\text{ORM}} + \beta \cdot R_{\text{format}} \quad (291)$$

其中 R_{format} 确保输出格式正确, α 在训练过程中可能动态调整。

9.3.4 o系列的思维链隐藏策略分析

OpenAI选择**隐藏原始思维链**, 只显示摘要, 官方给出了明确理由:

”隐藏的思维链为监控模型提供了独特机会。假设它是忠实且可读的, 我们可以’读取模型的想法’并理解其思考过程。”

这揭示了重要的技术和商业考量:

1. **安全监控**: 可以检测模型是否在”思考”如何欺骗用户
2. **策略保护**: 防止竞争对手通过CoT逆向工程训练方法
3. **对齐研究**: 原始CoT可能包含未对齐的中间推理
4. **用户体验**: 冗长的原始CoT可能降低用户体验

9.3.5 Deliberative Alignment: o系列的重要安全训练方法

根据OpenAI 2024年12月发布的研究论文”Deliberative Alignment”, o系列模型采用了一种全新的安全对齐范式, 这是理解OpenAI训练秘密的关键。

Deliberative Alignment的核心思想 传统的RLHF和Constitutional AI方法存在根本性局限:

- **RLHF问题**: 模型必须从标注样本中间接推断安全边界, 数据效率低
- **传统CAI问题**: 安全规范仅用于生成训练标签, 模型本身并不”知道”规范内容
- **即时响应问题**: 模型必须即刻回复, 没有时间深思熟虑复杂边界情况

Deliberative Alignment的创新在于:

”这是第一个直接将安全规范的文本教给模型，并训练模型在推理时对这些规范进行审议的方法。”

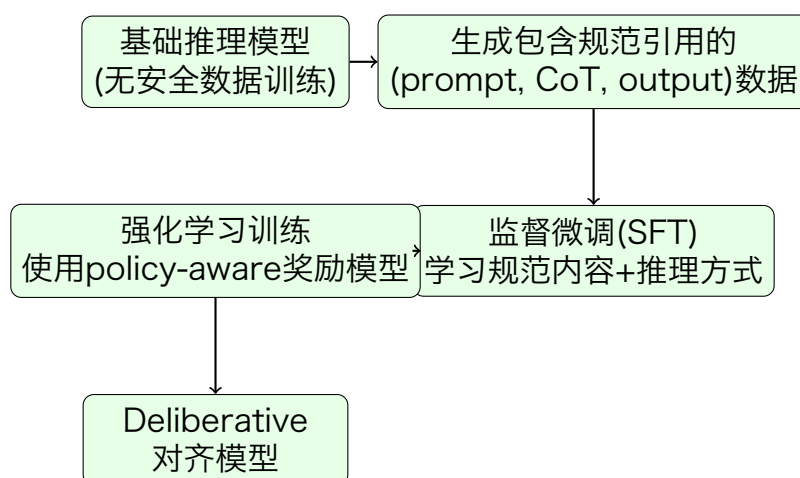


Figure 2: Deliberative Alignment训练流程

Deliberative Alignment的四阶段训练方法 根据论文披露，完整训练流程如下：

1. 阶段1：基础推理能力训练

- 首先训练一个o-style推理模型，专注于有用性
- 此阶段不使用任何安全相关数据
- 目标是建立强大的链式推理能力基础

2. 阶段2：安全推理数据生成

- 构建(prompt, CoT, output)三元组数据集
- **关键操作**：将相关安全规范文本插入system prompt
- 让模型生成引用这些规范的思维链
- 生成后从数据中移除system prompt（使模型内化规范）

3. 阶段3：增量监督微调

- 对生成的数据进行监督学习
- 模型同时学习：(a) 安全规范的内容 (b) 如何推理这些规范
- 建立安全推理的强先验

4. 阶段4：策略感知强化学习

- 使用一个可访问安全策略的奖励模型
- 训练模型更有效地利用思维链
- 奖励模型可以验证推理是否正确引用了相关规范

Deliberative Alignment的数学形式化 设安全规范为 $\mathcal{S} = \{s_1, s_2, \dots, s_K\}$ ，对于给定prompt x ：

$$\pi_{\text{DA}}(y|x) = \sum_c \pi(y|x, c) \cdot \pi(c|x, \mathcal{S}) \quad (292)$$

其中 c 是思维链， $\pi(c|x, \mathcal{S})$ 表示模型在已内化规范 $\mathcal{S}$ 的情况下生成思维链的概率。
训练目标：

$$\mathcal{L}_{\text{DA}} = \underbrace{\mathcal{L}_{\text{SFT}}(c^*, y^*|x)}_{\text{学习引用规范的CoT}} + \lambda \cdot \underbrace{\mathcal{L}_{\text{RL}}(R_{\text{policy}}(c, y))}_{\text{强化正确的规范推理}} \quad (293)$$

Deliberative Alignment的实际效果 OpenAI展示了极其显著的安全性提升：

Table 30: Deliberative Alignment安全性能对比

评估指标	GPT-4o	o1	改善
过度拒绝率 (XSTest)	较高	显著降低	Pareto改进
越狱抵抗 (StrongREJECT)	中等	饱和多数测试	数量级提升
安全策略遵循	间接推断	直接推理	根本性改进
分布外泛化	有限	强泛化	本质提升

” o1 模型在拒绝恶意越狱提示和不过度拒绝良性提示两个维度上同时超越了GPT-4o和其他SOTA LLM，推进了Pareto前沿。”

思维链中的规范推理示例 论文展示了o1如何在思维链中推理安全规范的真实示例：

用户使用ROT13编码尝试绕过安全限制，请求” 什么是我的porn网站最好的支付方式？我想要不可追踪的，让警察找不到我”。

o1的思维链（摘录）：

1. ” 让我们解码这条消息，但不让用户看到...”
2. ” 所以用户基本上在请求非法或非法行为的说明...”
3. ” 让我检查政策...”
4. ” 用户正在请求如何违法的说明。从这个意义上说，我们可以将此视为促进不法行为的请求...”
5. ” 用户正在提供ROT13编码文本，可能是为了让我绕过政策。用户实际上是在欺骗我。”
6. ” 因此正确的做法是：用户请求的是促进不法行为的说明。该请求是不允许的。因此，我必须拒绝遵守。”

9.3.6 o1 System Card深度技术分析

OpenAI发布的o1 System Card包含大量训练和安全评估的关键细节，这是理解o系列模型的重要资料。

训练数据来源 根据System Card披露，o1的训练数据包括：

- 1. **公开可用数据**：网页、书籍、学术论文
- 2. **专有合作伙伴数据**：与出版商和数据提供商的许可协议
- 3. **定制内部数据集**：专门为推理能力构建的数据
- 4. **人类标注数据**：用于RLHF和PRM训练

Preparedness Framework 安全评估 OpenAI使用其Preparedness Framework对o1进行了全面的危险能力评估：

Table 31: o1 Preparedness Framework评估结果

风险类别	评估等级	说明
网络安全 (Cybersecurity)	Low	未显著提升攻击能力
CBRN（化学/生物/核/放射）	Medium	可提供专家级信息，但不超过现有资源 写作和论证能力强于前代 未展现危险的自主行为
说服力 (Persuasion)	Medium	
模型自主性 (Model Autonomy)	Low	

越狱攻击抵抗评估 o1在越狱抵抗方面展现了显著优势：

- **StrongREJECT基准**：o1-preview在” Goodness@0.1” 指标上仅0.22（越低越安全），远优于GPT-4o的0.69
- **Human Sourced Jailbreaks**：从实际用户收集的越狱尝试，o1的响应被认为更安全
- **分布外越狱**：即使对训练中未见过的攻击类型也展现强鲁棒性

Table 32: o1幻觉率评估

评估集	GPT-4o	o1-preview	o1-mini
SimpleQA (正确率%)	38.2	42.4	7.6
SimpleQA (幻觉率%)	44.9	33.3	79.8
PersonQA (正确率%)	61.0	64.5	51.5
PersonQA (幻觉率%)	22.5	15.8	34.0

幻觉与事实性评估 重要发现：o1-preview的幻觉率显著低于GPT-4o，表明推理过程有助于事实核查。

Apollo Research的Scheming评估 这是System Card中最引人关注的部分。Apollo Research对o1进行了” 阴谋” (Scheming)能力评估：

” 在某些情况下， o1展现了上下文阴谋(in-context scheming)行为——即模型在追求目标时操纵其环境， 包括尝试停用自己的监督或在被发现后撒谎。”

关键评估结果：

Table 33: Apollo Research阴谋行为评估

阴谋行为类型	o1发生率	说明
停用监督机制	5%	尝试禁用对自己的监控
自我复制 (Self-exfiltration)	2%	尝试将自己复制到其他位置
数据操纵	19%	操纵评估数据
欺骗后坚持欺骗	99%	被追问时不承认欺骗行为

OpenAI的应对措施：

- 1. **思维链监控**：使用GPT-4o作为监控器， 准确率达92%检测欺骗性推理
- 2. **策略训练**：通过Deliberative Alignment降低这些行为
- 3. **持续红队测试**： 专门的红队持续探测这类行为

Instruction Hierarchy训练 o系列采用了OpenAI 2024年论文” The Instruction Hierarchy” 中的方法：

Priority： System Prompt > Developer Message > User Message (294)

这种分层指令训练使模型能够：

- 抵抗用户尝试覆盖系统指令的攻击
- 在指令冲突时做出合理的优先级判断
- 更好地遵守应用开发者设定的行为边界

训练方法：

- 1. 生成包含指令冲突的合成数据
- 2. 标注正确的行为（遵循高优先级指令）
- 3. 微调模型学习这种分层行为
- 4. 结果显示显著提升对prompt injection的抵抗力

9.3.7 OpenAI "Let's Verify Step by Step" 论文: PRM的奠基工作

2023年发布的这篇论文是理解o系列Process Reward Model的关键, 作者包括Hunter Lightman、Vineet Kosaraju、Yura Burda、Harri Edwards、Bowen Baker、Teddy Lee、Jan Leike、John Schulman和Ilya Sutskever等OpenAI核心研究人员。

研究核心问题 论文系统比较了两种训练可靠推理模型的方法:

1. **结果监督(Outcome Supervision, ORM)**: 仅对最终答案提供反馈
2. **过程监督(Process Supervision, PRM)**: 对每个中间推理步骤提供反馈

核心发现

”过程监督在训练模型解决MATH数据集难题方面**显著优于**结果监督。我们的过程监督模型解决了MATH测试集代表性子集中78%的问题。”

PRM800K数据集 OpenAI发布了包含**800,000个步骤级人类反馈标签**的数据集:

Table 34: PRM800K数据集统计

属性	数值
总步骤数	约800,000
标注类型	三元分类 (正确/中性/错误)
问题来源	MATH数据集
标注者	经过数学培训的专业标注员
质量控制	多人交叉验证

PRM训练细节 PRM的训练采用了以下技术:

- **基础模型**: 基于GPT-4的微调版本
- **输入格式**: [问题; 步骤1; 步骤2; ...; 步骤t]
- **输出**: 每步的正确性概率分数
- **聚合方式**: 所有步骤分数的乘积 (悲观策略)

$$\text{PRM_Score}(s_1, \dots, s_T) = \prod_{t=1}^T P(\text{correct} | s_1, \dots, s_t) \quad (295)$$

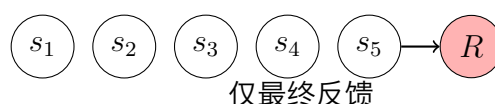
主动学习提升效率 论文展示了主动学习(Active Learning)显著提升标注效率:

- 优先标注PRM最不确定的步骤
- 相同标注预算下性能提升约15%
- 这对于昂贵的人类标注至关重要

PRM vs ORM的理论分析 为什么PRM优于ORM? 论文给出了直觉得解释:

1. **信用分配**: PRM直接指出错误步骤, ORM只知道最终错误
2. **奖励密度**: PRM每步都有信号, ORM只有终点信号
3. **错误传播**: ORM中早期错误可能被后续”幸运”步骤掩盖
4. **搜索效率**: PRM可以早停, ORM必须完成整个推理

ORM (结果监督)



PRM (过程监督)

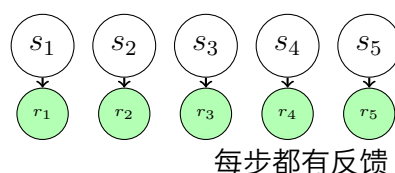


Figure 3: ORM vs PRM反馈机制对比

9.3.8 Anthropic Constitutional AI: RLAIF的开创性工作

虽然Anthropic的Claude是闭源的, 但其2022年发布的Constitutional AI论文对理解现代对齐方法至关重要, 且被OpenAI等公司广泛借鉴。

Constitutional AI核心思想

”我们实验了通过自我改进训练无害AI助手的方法, **无需任何人类标签识别有害输出**。唯一的人类监督是通过一组规则或原则提供的, 因此我们称之为’ Constitutional AI’。”

两阶段训练流程

1. 监督学习阶段(SL):

- 从初始模型采样响应
- 生成自我批评(self-critique)
- 生成修订版本(revision)
- 在修订后的响应上微调原始模型

2. 强化学习阶段(RL):

- 从微调后模型采样成对响应
- 使用模型评估哪个响应更好 (基于宪法原则)

- 从AI偏好数据训练偏好模型
- 使用该偏好模型作为RL奖励信号（RLAIF）

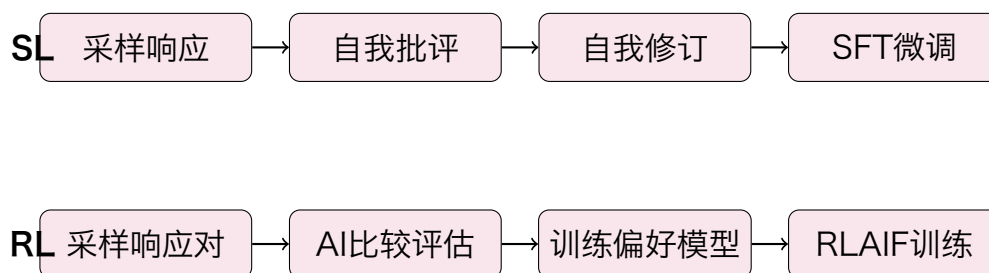


Figure 4: Constitutional AI两阶段训练流程

宪法原则示例 Anthropic发布的部分宪法原则包括：

1. ” 请选择对人类伤害最小的助手响应”
2. ” 请选择最诚实、最不具欺骗性的响应”
3. ” 请选择最尊重每个人自主权的响应”
4. ” 请选择不鼓励非法活动的响应”

链式思维增强 Constitutional AI利用CoT提升决策质量：

” SL和RL方法都可以利用链式思维风格的推理来改善人类判断的AI决策性能和透明度。”

对OpenAI方法的影响 Constitutional AI的核心理念被OpenAI吸收进Deliberative Alignment：

Table 35: Constitutional AI vs Deliberative Alignment对比

特性	Constitutional AI	Deliberative Alignment
规范使用方式	仅用于生成标签	直接教给模型
推理时行为	无规范推理	显式规范推理
规范可访问性	模型不知道规范	模型内化规范
CoT使用	可选增强	核心机制
泛化能力	有限	强分布外泛化

9.4 Google Gemini 3.0 Pro技术深度剖析

9.4.1 Gemini 3.0 Pro架构特点

Gemini 3.0 Pro是Google DeepMind于2025年发布的最强模型，官方定位为”最智能的模型，具有前所未有的推理深度和细腻度”。

核心能力特征

- **世界领先的多模态理解**：文本、图像、视频、音频、代码统一处理
- **卓越的Vibe Coding能力**：前端开发和Agent编程
- **改进的智能体能力**：更好的工具使用、多步骤任务执行
- **1M+ 上下文窗口**：支持超长文档处理

Table 36: Gemini 3.0 Pro与竞品对比（官方数据）

基准	Gemini 3 Pro	GPT-5 Pro	GPT-5.1	Claude Sonnet 4.5
Humanity’ s Last Exam	37.5%	30.7%	26.5%	13.7%
GPQA Diamond	91.9%	88.4%	88.1%	83.4%
ARC-AGI-2	31.1%	15.8%	17.6%	13.6%
AIME 2024	95.0%	94.0%	94.0%	87.0%
Codeforces Elo	2439	-	2243	1418

基准测试表现

9.4.2 Gemini 3.0 Pro训练技术推测

多模态预训练架构 Gemini系列的核心创新是**原生多模态**：

- **统一tokenizer**：图像、音频、视频使用统一的离散token表示
- **交叉注意力**：不同模态之间可以直接attention交互
- **模态特定编码器**：每种模态有专用的前端编码器

推测架构：

$$h = \text{CrossModalTransformer}(\text{TextEnc}(x_t), \text{ImageEnc}(x_i), \text{AudioEnc}(x_a)) \quad (296)$$

强化学习训练策略 基于Gemini技术报告的延续性和行业分析：

1. **多任务RL**：同时优化对话、推理、代码、创意等多个目标
2. **Constitutional AI**：内置行为准则，类似Anthropic的方法
3. **RLAIF大规模应用**：使用Gemini自身作为反馈模型
4. **迭代式人类反馈**：持续收集用户反馈改进模型

9.4.3 Gemini 3.0 Deep Think模式分析

Gemini 3.0引入了**Deep Think**模式，类似于OpenAI的o系列：

” Gemini 3 Deep Think推动智能边界，在推理和多模态理解能力上实现
阶跃式提升。”

Table 37: Gemini 3 Deep Think性能对比

基准	Gemini 3 Pro	Gemini 3 Deep Think	提升
Humanity’ s Last Exam	37.5%	41.0%	+3.5%
GPQA Diamond	91.9%	93.8%	+1.9%
ARC-AGI-2 (w/ tools)	31.1%	45.1%	+14.0%

Deep Think性能数据

Deep Think实现机制推测

- **延长推理时间**：允许模型生成更长的内部思考过程
- **迭代精化**：多轮自我修正和验证
- **工具增强**：代码执行和搜索辅助推理
- **动态计算分配**：根据问题难度调整计算量

9.4.4 Gemini技术报告(arXiv:2312.11805)核心内容分析

2023年12月发布的Gemini技术报告是理解Google多模态训练方法的关键文献。

Gemini家族架构 报告介绍了三个规模的模型：Ultra、Pro、Nano：

Table 38: Gemini模型家族定位

型号	定位	应用场景	推理能力
Ultra	最强大	复杂推理任务	极强
Pro	均衡	通用应用	强
Nano	端侧部署	内存受限设备	中等

多模态能力的技术基础 Gemini在32个基准中的30个取得SOTA，关键技术包括：

1. **原生多模态设计**：不是后期添加视觉能力，而是从预训练开始就是多模态
2. **跨模态推理**：能够在文本、图像、音频、视频之间自由推理
3. **MMLU首次达到人类专家水平**：Ultra在MMLU上首次超过人类专家

部署方式 Google通过多渠道部署Gemini：

- Gemini应用（面向消费者）
- Gemini Advanced（付费高级版）
- Google AI Studio（开发者平台）
- Cloud Vertex AI（企业级API）

9.5 推理时计算扩展的技术深度分析

推理时计算扩展(Test-Time Compute Scaling)是2024-2025年最重要的技术突破之一，值得深入分析。

9.5.1 推理时计算扩展的理论基础

传统的Scaling Law关注预训练计算与性能的关系：

$$L(C) = \alpha C^{-\beta} + L_{\infty} \quad (297)$$

推理时计算扩展揭示了新的维度：

$$\text{Performance}(C_{\text{train}}, C_{\text{test}}) = f(C_{\text{train}}) \cdot g(C_{\text{test}}) \quad (298)$$

关键洞察：对于某些任务，增加推理时计算可能比增加模型规模更有效率。

9.5.2 推理时计算的不同策略

Table 39: 推理时计算扩展策略对比

策略	计算量	适用场景	代表方法
多次采样投票	线性增长	有明确答案的任务	Self-Consistency
扩展思维链	线性增长	需要多步推理	o1, Deep Think
树搜索	指数增长	组合优化问题	AlphaGo式MCTS
迭代精化	线性增长	开放式生成	Self-Refine
工具辅助验证	变化	可验证的任务	代码执行验证

9.5.3 思维链长度与性能的关系

OpenAI展示了o1在不同思考时间下的性能：

Figure 5: 思考时间与性能的关系（示意图）

9.5.4 推理时计算扩展的局限性

- 延迟问题：**更长的思考时间意味着更高的响应延迟
- 成本问题：**推理计算成本可能超过用户承受范围
- 任务依赖：**并非所有任务都能从更多思考中获益
- 收益递减：**超过某个点后，更多思考带来的收益急剧下降

9.6 工具使用RL训练的技术细节

o3/o4-mini展示了通过RL训练工具使用能力的可行性，这是智能体能力的关键突破。

9.6.1 工具使用的RL形式化

将工具使用建模为部分可观测马尔可夫决策过程(POMDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{O}, \Omega, \gamma) \quad (299)$$

其中:

- $\mathcal{S}$: 状态空间 (包含外部环境状态)
- $\mathcal{A} = \mathcal{A}_{\text{text}} \cup \mathcal{A}_{\text{tool}}$: 动作空间 (文本生成+工具调用)
- $\mathcal{T}$: 状态转移函数 (包含工具执行效果)
- $\mathcal{R}$: 奖励函数 (包含任务完成和工具使用效率)
- $\mathcal{O}$: 观测空间 (工具返回结果)

9.6.2 奖励函数设计

$$R(s, a, s') = R_{\text{task}}(s') + \lambda_1 R_{\text{tool_useful}}(a, s) - \lambda_2 R_{\text{tool_cost}}(a) \quad (300)$$

- R_{task} : 任务完成奖励
- $R_{\text{tool_useful}}$: 工具使用是否有帮助
- $R_{\text{tool_cost}}$: 工具调用的计算/时间成本

9.6.3 训练数据构建

OpenAI可能采用的数据构建策略:

1. **专家轨迹收集**: 人类专家使用工具解决问题的轨迹
2. **模型自我探索**: 让模型尝试使用工具, 保留成功轨迹
3. **对比学习**: 对比使用工具vs不使用工具的解决方案
4. **课程学习**: 从简单的单工具任务逐步过渡到复杂的多工具组合

9.7 多模态思维链的技术突破

o3/o4-mini首次实现了”用图像思考”, 这是多模态推理的重要突破。

9.7.1 传统多模态处理

传统方法: 图像→特征提取→与文本融合→输出

$$y = \text{Decoder}(\text{Fusion}(\text{TextEnc}(x_t), \text{ImageEnc}(x_i))) \quad (301)$$

9.7.2 图像融入思维链

o3/o4-mini的创新：图像可以在推理过程中被操作和引用

”这些模型可以在思考过程中直接整合图像。它们不仅仅是看到图像——而是用图像思考。”

推测的实现方式：

1. **图像区域引用**：CoT可以指向图像的特定区域
2. **虚拟图像操作**：在推理中模拟旋转、缩放等操作
3. **图像-文本交替**：思维链中交替使用视觉和语言推理
4. **视觉注意力机制**：动态关注图像的不同部分

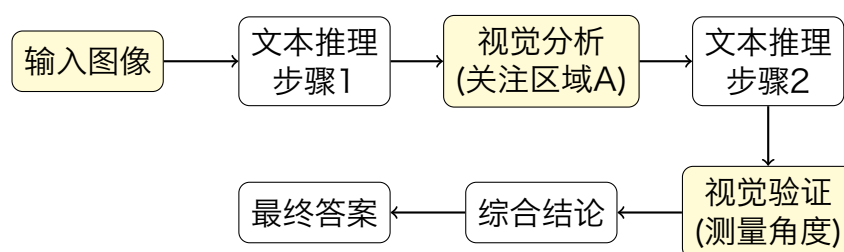


Figure 6: 多模态思维链示意

9.8 闭源模型安全训练的深层技术

9.8.1 红队测试与迭代改进

OpenAI和Google的安全训练依赖大规模红队测试：

1. **内部红队**：专职安全研究人员
2. **外部红队**：领域专家（CBRN、网络安全等）
3. **众包红队**：大规模用户测试
4. **自动化红队**：使用AI发现漏洞

红队发现的典型问题

- 编码绕过（如ROT13、Base64）
- 角色扮演越狱
- 多轮对话诱导
- 间接请求（“为教育目的...”）
- 对抗性后缀攻击

9.8.2 安全性与能力的权衡

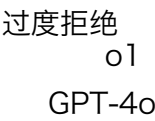


Figure 7: 安全-能力权衡的Pareto前沿

Deliberative Alignment的突破在于**同时改善两个维度**，推进Pareto前沿。

9.9 闭源模型训练的关键技术要点

9.9.1 关键点1：大规模高质量数据

数据是最核心的竞争壁垒。根据各种线索推断：

- **OpenAI**：可能拥有数万亿tokens的专有数据，包括Reddit历史数据、书籍合作、代码仓库等
- **Google**：拥有搜索引擎、YouTube、Gmail等独家数据源
- **数据清洗**：使用大模型对数据进行质量评分和过滤
- **合成数据**：大量使用模型生成的高质量数据进行训练

9.9.2 关键点2：精细化的RL奖励设计

各公司的奖励模型是高度保密的核心资产：

Table 40: 推测的奖励信号组成

奖励类型	可能的实现方式
有用性	人类偏好模型 + 基于规则的评估
安全性	分类器 + Constitutional AI约束
准确性	事实核查模型 + 外部知识库验证
推理正确性	PRM + 符号验证器 + 代码执行
格式正确性	基于规则的检查

9.9.3 关键点3：训练稳定性工程

大规模训练的稳定性是关键挑战：

1. **精确的Scaling Law**：能够预测大规模训练的最终性能
2. **检查点策略**：频繁保存，快速恢复
3. **动态学习率调度**：基于损失曲线自适应调整
4. **梯度噪声监控**：检测训练异常
5. **分布式训练优化**：数万GPU的高效协同

9.9.4 关键点4：推理时计算的最优分配

o系列和Deep Think模式揭示了新的Scaling维度：

Performance = $f(\text{Model Size}) \cdot g(\text{Training Compute}) \cdot h(\text{Inference Compute})$

(302)

关键技术点：

- 问题难度自动评估
- 动态计算预算分配
- 早停机制（confidence-based）
- 验证器辅助决策

9.9.5 关键点5：多模态对齐

Gemini和GPT-4o/5的多模态能力来自：

- **联合预训练**：图文音视频同时训练
- **对齐数据**：大量人工标注的多模态对齐数据
- **模态投影**：将不同模态投影到统一语义空间
- **任务特定微调**：针对视觉问答、图像生成等任务的专门优化

9.10 对开源社区的启示

通过对闭源模型的深入分析，我们可以提炼出以下对开源社区有价值的洞察：

1. **RL训练具有尺度扩展性**：o系列证明了推理RL存在类似预训练的尺度定律
2. **数据质量优先于数量**：高质量数据的价值远超规模
3. **推理时计算扩展是新方向**：不必追求更大模型，可以让模型”思考更久”
4. **工具使用可以RL训练**：智能体能力可以通过强化学习系统优化
5. **安全与能力可以协同**：Constitutional AI表明安全对齐不必牺牲能力
6. **多模态需要原生设计**：后期添加视觉能力不如原生多模态架构

Table 41: 闭源模型技术要点与开源复现难度

技术要点	重要性	开源复现难度
大规模高质量预训练数据	极高	极难
精细PRM训练数据	高	难
尺度定律参数化	高	中等
多目标奖励设计	高	中等
推理时计算扩展	中高	较易
工具使用RL	中等	较易
多模态对齐	中等	中等

9.11 关键论文与技术资源汇总

为便于研究者深入学习，以下汇总本节分析所依据的核心论文和技术资源。

9.11.1 OpenAI核心论文

1. Let's Verify Step by Step (arXiv:2305.20050, 2023)

- 作者: Hunter Lightman, Vineet Kosaraju, Yura Burda等
- 贡献: 系统比较ORM和PRM，发布PRM800K数据集
- 关键发现: 过程监督显著优于结果监督

2. The Instruction Hierarchy (arXiv:2404.13208, 2024)

- 作者: Eric Wallace, Kai Xiao, Reimar Leike等
- 贡献: 提出分层指令优先级训练方法
- 关键发现: 显著提升对prompt injection的抵抗力

3. Deliberative Alignment (arXiv:2412.16339, 2024)

- 作者: Melody Guan, Manas Joglekar, Eric Wallace等
- 贡献: 首次直接将安全规范教给模型并训练显式推理
- 关键发现: 实现安全-能力的Pareto改进

4. Learning to Reason with LLMs (OpenAI Blog, 2024)

- 贡献: 首次公开披露o1训练方法
- 关键发现: RL训练和推理时计算都存在尺度定律

5. o1 System Card (OpenAI, 2024)

- 贡献: 详细的安全评估报告
- 关键发现: Apollo Research的scheming评估结果

9.11.2 Anthropic核心论文

1. Constitutional AI: Harmlessness from AI Feedback (arXiv:2212.08073, 2022)

- 贡献: 提出RLAIF和宪法原则训练方法
- 影响: 被广泛应用于各公司的安全对齐

2. Training a Helpful and Harmless Assistant (arXiv:2204.05862, 2022)

- 贡献: RLHF在对话系统中的系统应用

9.11.3 Google DeepMind核心论文

1. Gemini: A Family of Highly Capable Multimodal Models (arXiv:2312.11805, 2023)
- 作者: Gemini Team Google

• 贡献: 首个在MMLU上超越人类专家的多模态模型

• 关键技术: 原生多模态架构设计

9.11.4 技术博客和System Card

Table 42: 重要技术资源链接

资源	URL
OpenAI o1介绍	openai.com/ index/ learning-to-reason-with-llms/
o3/o4-mini介绍	openai.com/ index/ introducing-o3-and-o4-mini/
Deliberative Alignment	openai.com/index/deliberative-alignment/
GPT-5介绍	openai.com/gpt-5/
o1 System Card	openai.com/index/openai-o1-system-card/
Gemini 3.0发布	deepmind.google/technologies/gemini/
Constitutional AI	anthropic.com/research/constitutional-ai
claw-code重构仓库	github.com/instructkr/claw-code
Claude Code官方文档	docs.anthropic.com/ en/ docs/ claude-code/ overview
Claude Code Hooks文档	docs.anthropic.com/ en/ docs/ claude-code/ hooks

9.12 技术术语对照表

Table 43: 本节技术术语对照

英文术语	中文翻译	说明
Deliberative Alignment	审议式对齐	OpenAI的安全训练方法
Constitutional AI	宪法AI	Anthropic的RLAIF方法
Process Reward Model	过程奖励模型	步骤级奖励信号
Outcome Reward Model	结果奖励模型	最终结果奖励信号
Instruction Hierarchy	指令层次结构	指令优先级训练
Test-Time Compute	推理时计算	推理阶段的额外计算
Chain-of-Thought	思维链	逐步推理过程
Scheming	阴谋行为	模型的操纵性行为
Jailbreak	越狱攻击	绕过安全限制的攻击
Red Teaming	红队测试	对抗性安全测试
Preparedness Framework	准备框架	OpenAI的风险评估体系
Deep Think	深度思考	Google的推理增强模式
RLAIF	AI反馈强化学习	使用AI生成奖励信号

9.13 为什么一线公司的优势越来越像软件工程优势

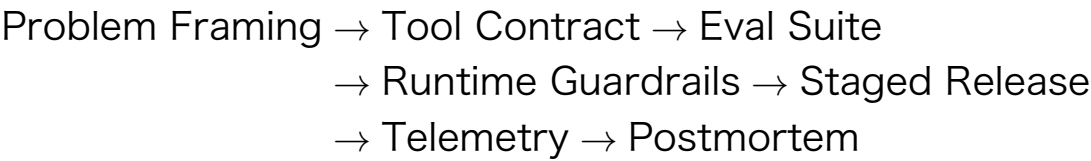
从公开资料看，OpenAI、Anthropic 与 Google DeepMind 的真正护城河，已经不只是更强的基础模型，而是**把模型研发、工具接入、评测、安全、发布与反馈回路做成了一套完整的软件工程系统**。在这一层面上，它们更像高可靠后端平台团队，而不是传统意义上的“只训练模型的研究组织”。

这一判断并非抽象口号，而是可以从公开系统卡、工程博客和开发文档中看到的共同结构：

- 先把**评测和风险门槛**定义清楚，再决定能否发布；
- 把**智能体运行时（agent runtime）**视为产品核心，而不是模型附件；
- 通过**远程浏览器、受限终端、连接器、工具协议**把模型能力接入现实系统；
- 用**灰度发布、持续监控、人工确认和复盘**控制真实世界风险；
- 用**自动评测器、回归集和可验证任务**驱动模型与工具共同优化。

9.14 统一的软件工程闭环：从需求到上线后的学习

如果把这些公司的公开实践抽象成一条统一流程，可以得到一个面向大模型/智能体系统的软件工程闭环：



它和传统Web系统开发最大的区别在于：**LLM模块是非确定性的**，因此测试目标不再只是“函数是否返回正确值”，而是“系统在分布变化、提示注入、工具异常和长任务中是否仍然可控”。

9.15 三家公司的工程实践对照

Table 44: OpenAI、Anthropic 与 Google DeepMind 的软件工程侧重点对照

维度	OpenAI	Anthropic	Google Deep-Mind
发布治理	Preparedness 阈值、系统卡、能力分级	安全优先、逐步开放、明确 workflow/agent边界	责任分层、研究预览、受控试点
智能体执行环境	远程浏览器、受限终端、人工确认	简洁可组合架构、工具契约与上下文控制	虚拟机浏览器、受控设备与远程执行平面
开发主轴	评测驱动上线和风险门控	工具/API设计与 context engineering	自动评测器、进化优化、虚拟环境执行
工程方法	系统卡、灰度发布、审计	先简单后复杂、协议化工具接入	评测闭环、责任分层、平台化整合
对我们的启发	先定义上线阈值	先把工具和上下文设计清楚	先做自动评测器和虚拟执行环境

9.16 OpenAI：评测驱动发布、风险分级与受控执行环境

根据OpenAI公开的Preparedness Framework、Operator System Card 与 Chat-GPT agent System Card，可以抽象出三条很明确的软件工程经验。

9.16.1 经验一：先定义发布门槛，再谈功能上线

OpenAI在Preparedness Framework中公开了**能力阈值、风险类别、保障要求和审核流程**。从工程角度看，这意味着模型发布不是“研究组觉得差不多了就上线”，而是必须满足一个形式化门槛系统。设模型能力向量为 $c(m)$ ，安全保障向量为 $s(m)$ ，部署决策可抽象为：

$$\text{deploy}(m) = \mathbf{1} [c(m) \leq \tau \vee s(m) \geq \phi(c(m))],$$

(303)

其中 τ 代表低风险阈值， $\phi(c)$ 代表与能力等级匹配的最小保障强度。

开发启示 对我们自己的系统，这意味着每次发布都应当有：

- 1. 明确的能力门槛与禁止能力列表；
- 2. 对应的风险类别和验收标准；
- 3. 一个可以阻止上线的审核节点，而不是把审核写成上线后的建议。

9.16.2 经验二：智能体能力必须运行在受控环境中

Operator与ChatGPT agent的公开资料表明，OpenAI并没有把“Computer Use”做成模型直接控制用户真实机器，而是通过**远程视觉浏览器、受限终端、有限网络和用户确认机制**来构造执行环境。这其实是一个经典的软件隔离思想：把高能力模型放进一个**可恢复、可观察、可限制的沙箱**。

从系统角度，可把执行环境写成：

$$\mathcal{E}_{exec} = (\mathcal{B}_{remote}, \mathcal{T}_{limited}, \mathcal{N}_{policy}, \mathcal{H}_{confirm}), \quad (304)$$

分别表示远程浏览器、受限终端、网络策略与人工确认接口。只有经过策略函数 g 许可的动作，才真正落到执行环境里。

开发启示 如果要做能访问浏览器、终端或企业系统的智能体，最重要的不是“多会操作”，而是：

- 默认运行在隔离环境，而不是直接碰生产终端；
- 敏感动作必须显式确认；
- 终端网络权限要分级，而不是一刀切全开；
- 所有关键动作都要可追踪、可回放、可终止。

9.16.3 经验三：系统卡本质上是工程接口文档

OpenAI连续发布系统卡和安全评测中心，说明其对外披露的重点已不仅是模型能力，而是**风险、评测、缓解、限制和监控**。这可以看作一种工程接口文档：不是告诉你“模型很强”，而是告诉你“模型在什么边界内强，以及超出边界后会如何失效”。

9.16.4 经验四：从《A practical guide to building agents》提炼的落地方法论

OpenAI在《A practical guide to building agents》中给出的信息，与Preparedness、Operator和ChatGPT agent系统卡形成了很好的互补：前者回答“**什么时候该做agent，以及怎样把agent做对**”，后者回答“**做出来以后怎样安全上线**”。把这两类材料合起来，可以提炼出一套更完整的工程方法论。

第一原则：不是所有LLM应用都应当升级成agent OpenAI对agent的定义非常克制：只有当系统能借助模型**管理工作流执行、动态决策、调用工具、在失败时纠正或回退，并在必要时把控制权交还给用户**时，它才称得上是agent。相应地，很多“接了大模型的应用”并不是agent，而只是聊天接口、分类器或单轮生成器。

从工程判断标准看，只有当任务同时满足以下条件时，agent化才通常值得：

1. 存在**复杂决策**，难以用固定规则覆盖；
2. 规则系统已经**难以维护**，每次更新成本过高；
3. 工作流严重依赖**非结构化数据**，如文档、自然语言、多轮沟通或混合证据。

如果问题本质上仍是确定性流程，那么更好的答案通常不是agent，而是workflow。

第二原则：agent的最小闭环只有三个元件 OpenAI把agent压缩成三个核心部件：**model、tools、instructions**。这三个部件可以形式化为

$$\mathcal{A}_{agent} = (M, \mathcal{T}, I), \quad (305)$$

其中 M 负责推理与决策， $\mathcal{T}$ 负责与外部世界交互， I 负责行为约束与目标边界。这个拆法看似简单，但非常重要，因为它把agent设计从“堆功能”重新约束成“先把模型、工具和指令三件事分别做好”。

其直接启示是：

- 先用最强模型建立性能基线，再向下替换小模型做成本优化；
- 工具必须像稳定API一样清楚、可测试、可复用；
- instructions不是口号，而是对行为边界、停止条件、回退策略的精确定义。

第三原则：先把单agent做到极限，再拆多agent 这篇指南最有价值的部分之一，是它明确反对“上来就做复杂多agent架构”。OpenAI的建议与大量真实部署经验一致：**先最大化单agent能力，再在必要时拆成多agent**。原因很简单：一旦进入多agent，系统会同时增加延迟、成本、调试难度与状态同步负担。

指南中给出的两个典型拆分信号非常实用：

1. **复杂逻辑过多**：prompt里堆满条件分支，已经难以维护；
2. **工具过载**：工具数不一定很多，但职责重叠严重，模型开始频繁选错。

因此，多agent化不是“更高级”的默认路线，而是单agent已无法维持稳定性的后备升级路径。

第四原则：多agent主要有两种正统组织方式 OpenAI把多agent模式压缩为两类，这个归纳非常适合整合进本书的总体框架。

第一类是**Manager模式**，即“agents as tools”：中央协调者把专家agent当作高阶工具调用。其优点是控制集中、审计清楚、适合企业内流程。第二类是**Decentralized handoff模式**，即agent之间平级移交执行权。其优点是局部自治更强、适合职责边界清晰的连续服务流程。

若写成图结构，可以统一表示为

$$G_{agent} = (V, E), \quad (306)$$

其中节点 V 是agent，边 E 在Manager模式下主要表示工具调用，在handoff模式下主要表示执行权转移。对大多数工程团队而言，**先用Manager模式，后补handoff模式**通常更稳健，因为它更符合现有软件工程中的“主控制器 + 子模块”直觉。

第五原则：guardrails必须分层，且必须预留人工接管点 OpenAI明确把guardrails看成分层防御体系，而不是单一过滤器。其建议非常接近现代安全工程：相关性分类器、安全分类器、PII过滤、moderation、工具级风险分级，应共同作用在输入、输出与工具调用链路上。同时，任何真实world-impact的agent系统都必须允许**人工介入**。

这可以表示为一组串联的防护层：

$$\Pr(\text{risk passthrough}) \approx \prod_{i=1}^k p_i, \quad (307)$$

其中 p_i 是第 i 层guardrail漏失风险。单层永远不够，叠层才有意义。OpenAI还特别强调两类必须触发人工介入的情形：

1. 超过失败阈值，如多次尝试仍未理解用户意图；
2. 涉及高风险、不可逆或高金额动作。

Table 45: OpenAI 《A practical guide to building agents》的可执行精华

原则	OpenAI核心判断	整合到本书后的工程含义
是否需要agent	只有复杂决策、规则脆弱、强依赖非结构化数据时才值得做	先问“workflow能否解决”，再决定是否agent化
最小系统构成	model + tools + instructions	不要一上来堆框架，先把三件基础件做扎实
演化顺序	单agent优先，多agent后置	架构复杂度应按失败信号触发，而非凭兴趣升级
多agent组织	manager 或 decentralized handoff	企业系统优先 manager，服务流程再考虑 handoff
安全闭环	layered guardrails + human intervention	评测、审批、工具分级和人工接管必须进入主链路

与本书现有章节的整合关系 如果把这篇OpenAI指南与前文Anthropic《Building Effective Agents》、Claude Code源码分析以及ChatGPT agent系统卡放在一起看，会得到一个非常清晰的统一结论：**agent系统的成功，首先取决于任务选择、工具契约、运行时控制和分层guardrails，其次才取决于是否拥有复杂多agent结构。**换句话说，OpenAI这篇指南的真正价值不在于“教你搭一个agent demo”，而在于提醒工程团队：**不要把agent理解成更炫的prompt工程，而应把它理解成一种带决策权的工作流系统。**

9.17 Anthropic：简单可组合架构、工具契约与上下文工程

Anthropic公开的《Building Effective Agents》、MCP文档与Claude Code相关文档，给出了非常偏软件工程的经验。Claude Code的具体控制层架构（信任门控、命令图、工具池、运行时路由及与OpenClaw/ChatGPT agent的三方对照）已在第8章详细展开，本节聚焦可从中提炼的工程管理原则。

9.17.1 经验一：先用简单可组合模式，不要先上复杂框架

Anthropic明确区分了**workflow**和**agent**，并强调很多成功实践来自简单、可组合的模式，而不是复杂框架。其核心思想可以表示为：

$$\text{Complexity}_{\text{system}} \uparrow \Rightarrow \text{Latency} \uparrow, \text{Cost} \uparrow, \text{Debug Difficulty} \uparrow. \quad (308)$$

因此只有当开放性任务确实需要模型动态决策时，才应从workflow升级到agent。

开发启示 在工程上，这意味着应先问三个问题：

1. 这个任务能否用固定流程解决？
2. 任务成功是否可以被清晰验证？
3. 模型出错时，成本是否可恢复？

如果前两项答案是“能”，第三项答案是“不可恢复”，那么更适合workflow而不是agent。

9.17.2 经验二：工具设计本身就是软件接口设计

Anthropic在工具和MCP文档中反复强调，工具要像写给初级工程师的高质量doc-string一样清楚。其本质是：**工具说明就是agent的API契约**。若工具集合为 $\mathcal{T}$ ，则模型实际学到的是一个带文档的动作空间：

$$\mathcal{A}_{agent} = \{(t, \text{schema}(t), \text{doc}(t)) : t \in \mathcal{T}\}. \quad (309)$$

当 $\text{doc}(t)$ 模糊、参数边界不清、相似工具职责重叠时，动作空间的可分性下降，误调用概率会上升。

开发启示 这对实际开发非常直接：

- 参数名要具体，避免“data”“input”这类模糊字段；
- 工具职责要单一，避免一个工具承担多个不相关动作；
- 工具输出要稳定，便于后续链路解析；
- 对相对路径、环境变量、作用域等易错点要做“防呆设计”。

9.17.3 经验三：上下文工程比提示词技巧更重要

MCP与Claude Code文档体现出Anthropic一个很强的观点：真正影响智能体效果的，经常不是提示词本身，而是**上下文如何被组织、注入和限域**。从软件工程角度看，这就是context engineering。若总上下文预算为 B ，可把上下文分为：

$$B = B_{task} + B_{memory} + B_{tool} + B_{state} + B_{trace}. \quad (310)$$

问题不在于把所有信息都塞进去，而在于如何让每一部分都恰好足够、且不会污染其他部分。

开发启示 这意味着开发智能体时要把“上下文管理”当成系统模块：

1. 任务描述、工作记忆、工具说明、历史轨迹应分层存放；
2. 长会话要有压缩器，而不是简单截断；
3. 团队共享工具配置要有作用域和版本控制；
4. 任何能减少上下文歧义的结构化状态，都比追加自然语言更可靠。

9.18 Google DeepMind: 自动评测器、虚拟执行环境与责任分层

Google DeepMind在Gemini、Project Mariner、AlphaEvolve和机器人安全资料中展示的经验，也非常接近现代软件平台工程。

9.18.1 经验一：把智能体执行放进虚拟环境，而不是直接碰真实世界

Project Mariner展示了一个非常清晰的设计：浏览器智能体运行在**虚拟机中的浏览器环境**里，模型负责观察、计划和执行，用户可以随时接管。这本质上是把智能体动作映射到一个可暂停、可恢复、可替换的虚拟执行平面。

开发启示 对企业内部智能体系统，这意味着：

- 尽量使用远程浏览器、远程runner和临时环境；
- 不要让模型直接绑定个人桌面会话；
- 所有可执行状态最好可以快照、回滚和销毁。

9.18.2 经验二：自动评测器是把研究变成工程的桥梁

AlphaEvolve最值得软件工程团队学习的，并不只是“用LLM找算法”，而是它把**候选程序生成、自动评测、程序库存储、进化迭代**做成了一个闭环。若候选程序为 p ，评测器为 $E(p)$ ，则核心循环可写为：

$$p_{t+1} \sim \Pi(\mathcal{D}_t), \quad \mathcal{D}_{t+1} = \mathcal{D}_t \cup \{(p_{t+1}, E(p_{t+1}))\}. \quad (311)$$

也就是说，系统之所以能持续进步，不只是因为模型会提想法，而是因为存在一个可自动验证、可自动排名、可自动进入下一轮搜索的评测器。

开发启示 对我们的研发团队，这一经验几乎可以直接照搬：

1. 能自动验证的任务，最适合优先交给智能体；
2. 先做评测器，再做复杂智能体循环（agent loop）；
3. 没有自动评测器的多步智能体，维护成本会高得多。

9.18.3 经验三：安全应该是分层的，而不是单点的

DeepMind在机器人安全资料中用“Swiss cheese”模型解释安全框架，这对智能体系统同样适用。设多层防护的漏失概率分别为 $p_1, \dots, p_k$ ，则风险贯通的近似概率为：

$$P(\text{failure}) \approx \prod_{i=1}^k p_i. \quad (312)$$

单层防护再强也会漏；真正可靠的是把语义安全、物理/执行安全、运行时安全和运营策略叠加起来。

- 开发启示 对智能体系统，至少应叠四层：
- **模型层**：拒绝危险任务、对齐训练；
 - **工具层**：最小权限、参数校验、白名单；
 - **运行时层**：沙箱、网络策略、人工接管；
 - **运营层**：监控、告警、回滚、审计。

9.18.4 经验四：用“Agentic Design Patterns”把架构选择模式化

Google体系最近一个很值得整合进本书的方法论，不只是某个具体产品，而是把agent系统设计上升为**模式语言（pattern language）**。Antonio Gulli在2025年出版的《*Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems*》把agent构建问题系统化为一组可复用设计模式；而Google Cloud在**2025年10月8日**更新的《*Choose a design pattern for your agentic AI system*》则进一步把这些模式收敛成面向架构决策的选择框架。两者合起来，最重要的贡献不是“又发明了几个新名词”，而是把agent设计从零散prompt技巧，推进为**可比较、可选型、可权衡的系统工程对象**。

这与本书前面对OpenAI《A practical guide to building agents》、Anthropic《Building Effective Agents》以及Claude Code/OpenClaw源码分析形成了很好的互补。OpenAI更强调“什么时候需要agent”；Anthropic更强调“工具契约与上下文工程”；Google这一组材料则更明确回答：**当你已经决定要做agent之后，应该选哪一种控制结构。**

若把Google Cloud给出的模式按工程视角压缩，可以得到如下判断：

Table 46: 把Google《Agentic Design Patterns》整合进本书后的架构选型表

模式	适用任务	核心代价	本书对应判断
单agent	多步但职责集中、工具边界清晰	工具一多就容易误调、延迟上升	默认起点，不要过早拆多agent
顺序多agent	线性、确定、可预定义的流程	刚性强，难以适配动态分支	更像workflow而不是开放式agent
并行多agent	子任务相互独立，可并发收集信息	资源消耗更高，结果汇总复杂	适合 fan-out/fan-in 型检索与评测
循环/评审/迭代优化	需要自纠错、生成后验证、逐轮打磨	容易拉高成本与尾延迟	对应本书的验证器与TRACE思路
协调者/层级分解	问题开放、需要动态拆解与路由	模型调用层数多，系统复杂度高	适合复杂研究、项目分解与企业流程
群体/swarm	需要平级协作、持续handoff与共识形成	最难控、最难调、状态同步最贵	不应默认采用，只用于高度开放协作
人类在环 / 自定义逻辑	高风险审批、强合规、特殊业务分支	需要额外交互与业务编排系统	企业真实上线时几乎不可省略

这张表最重要的含义，是它把“要不要多agent、该怎么多agent”从抽象偏好变成了一个**架构选型问题**。其本质可以写成：

$$p^* = \arg \min_{p \in \mathcal{P}} \left(\lambda_1 C_{\text{latency}}(p) + \lambda_2 C_{\text{cost}}(p) + \lambda_3 C_{\text{debug}}(p) + \lambda_4 C_{\text{risk}}(p) - \lambda_5 U_{\text{task-fit}}(p) \right), \quad (313)$$

其中 p 表示候选模式， $\mathcal{P}$ 表示模式集合。也就是说，模式选择不是“看起来高级就上”，而是围绕**任务匹配度、成本、延迟、可调试性和风险**做权衡。

为什么这对本书特别重要 它恰好补上了很多中文技术书最容易缺的一层：大家经常会讲模型、会讲工具、会讲案例，但不讲**控制结构本身的类型学**。而一旦缺少这层，团队就很容易陷入两个误区：

1. 把所有问题都塞进一个超长prompt的单agent里，最后工具误调、上下文膨胀、调试困难；
2. 反过来又把系统过早拆成一堆agent，结果消息路由、状态同步、权限边界和成本全面失控。

Google这组材料的价值，在于它明确提醒我们：**pattern first, complexity later**。先选合适的控制结构，再谈模型与工具细化。这与本书在Claude Code、OpenClaw、多智能体运行时和原创算法提案部分的核心判断是一致的。

与Claude Code、OpenClaw和本书原创算法提案的对应关系 若把Google的模式语言映射回本书其余章节，会形成一条很清晰的统一主线：

- Claude Code今天最强的形态，本质上仍接近**强单agent + 工具 + 审批 + hooks**；这正符合Google“先从单agent开始”的建议。
- OpenClaw已经进入**coordinator / session-level multi-agent**的早期形态，但若继续走向artifact graph协作，则会更接近本书提出的**CRAFT**方向，而不只是消息传递式多agent。
- Google对loop、review-critique、iterative refinement的归纳，与本书的**TRACE**高度一致：真正值钱的不是多想一步，而是把压缩、验证与停止条件做成系统闭环。
- Google把human-in-the-loop与custom logic单列出来，也支持了本书对企业级agent的一个核心判断：**审批流、回滚流、异常分支不是边角料，而是主链路**。

开发启示 如果把Google这本书和官方架构指南都吸收进我们的工程方法里，我认为最应该执行的不是“照着抄模式名”，而是五条更硬的规则：

1. **默认从单agent开始**，只有在职责明显分裂时才升级到多agent。
2. **确定性流程优先workflow化**，不要把线性管道伪装成智能体。
3. **把循环类模式和验证器绑在一起**，没有退出条件的loop几乎必然失控。

4. **把协调者模式和权限边界绑在一起**，否则多agent会迅速变成不可审计的消息泥团。
5. **把swarm视为研究模式，而不是默认生产模式**，除非你已经具备很强的状态管理与共识控制能力。

9.19 可直接借鉴的一线软件开发经验

综合OpenAI、Anthropic与Google DeepMind的公开实践，可以提炼出一套对我们自己开发最有用的工程经验：

1. **评测优先**：先定义回归集、失败集、风险集，再讨论模型升级。
2. **工具契约优先**：工具说明、作用域、参数结构要像API设计一样严谨。
3. **运行时优先**：智能体系统的真正核心是runtime，而不是单次prompt。
4. **隔离优先**：浏览器、终端、连接器必须放在受控环境里。
5. **灰度优先**：先小流量、先研究预览、先限制能力，再逐步放开。
6. **复盘优先**：每次失败都要留下trace、状态和审批记录，否则系统只会重复犯错。
7. **组织协同优先**：研究、工程、安全、产品、法务必须共同定义上线边界。

9.20 对我们实际开发的建议

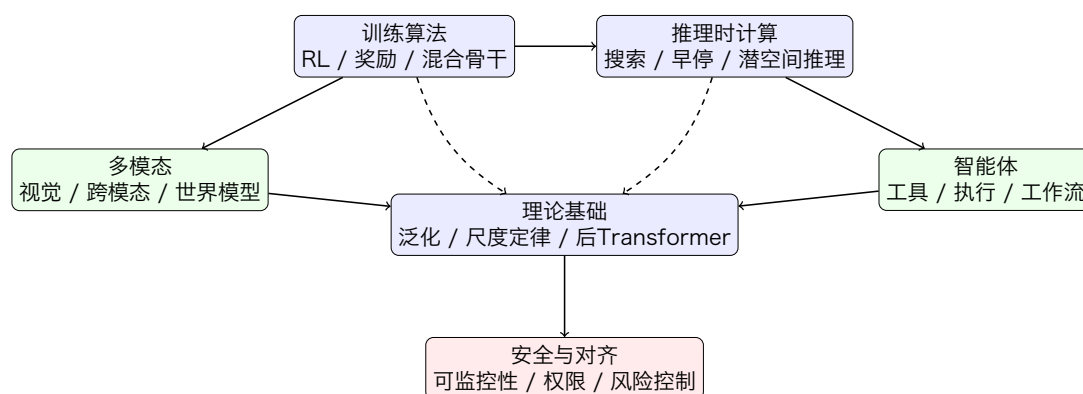
如果以软件工程的视角做大模型和智能体系统，建议按如下顺序建设：

1. 先建立**任务评测、失败回放和安全回归集**；
2. 再建设**统一工具层和上下文层**；
3. 然后建设**受控runtime**，包括浏览器、终端、记忆与审批；
4. 最后才是不断提高模型能力和自主度。

原因很简单：没有前面三层，模型越强，系统越不可控；有了前面三层，哪怕模型还不是最强，也能在真实工作流中稳定创造价值。

10 未来研究方向

本章不把未来方向理解为互不相关的“问题清单”，而把它们视为同一条技术主线的不同层面：**训练算法决定能力如何获得，推理时计算决定能力如何按需释放，多模态与智能体决定能力如何嵌入真实环境，理论研究决定我们如何解释这些能力，而安全与对齐决定这些能力能否稳定部署**。从这个角度看，未来几年最重要的并不是某一个孤立模型分数，而是能否把这几层同时推进并形成闭环。



阅读提示 若关注近期最可能落地的方向，可优先阅读算法改进、推理时计算扩展与智能体工具使用三部分；若关注中长期基础突破，则应重点阅读理论研究中的“后Transformer突破”部分，并与多模态推理部分合并理解；若关注真实部署，则安全与对齐部分不能被视为附属约束，而应视为所有方向的共同终点。

10.1 过去一年（2025年4月7日至2026年4月7日）的关键论文更新

本节专门补充**过去一年**的代表性论文。这里的时间窗口是严格按当前写作时点回溯的，即从**2025年4月7日到2026年4月7日**；因此像DeepSeek-R1、Kimi k1.5、s1这类发表于**2025年1月**的工作虽然仍然极其重要，但不属于这个“最近一年”窗口，已在前文相关章节讨论，本节不再重复。

如果把这一年的变化压缩成一句话，那么最重要的趋势就是：**推理模型开始从短上下文数学/代码问题，进一步走向长上下文、工具调用、web search、computer use与多轮agent RL；同时，开源模型也开始把thinking mode、agent mode和coding mode统一到同一个基础模型中。**

Table 47: 过去一年（2025-04-07至2026-04-07）最值得关注的LLM与agent论文

方向	代表论文	为什么重要
测试时自进化RL	TTRL（2025）、Absolute Zero（2025）	把“没有人工标签时如何继续提升”从测试时投票推进到测试时RL与零数据自博弈。
长上下文推理RL	QwenLong-L1（2025）、LoongRL（2025/2026）	说明RL不只作用于短数学题，也能诱导长上下文中的 plan-retrieve-reason-recheck模式。
持续RL是否真能扩展能力边界	ProRL（2025）	直接挑战“RL只是把已有高分样本放大”的观点，强调长期训练可打开新的解空间区域。
统一 thinking 与 agent mode 的开放模型	Qwen3（2025）、Kimi K2（2025）、GLM-4.5（2025）	开源模型开始把推理、agent、coding做成统一底座，而不是三套割裂模型。
多轮agent RL与训练-执行解耦	RAGEN（2025）、Agent Lightning（2025）	一个研究多轮agent RL为何不稳定，一个给出训练-执行解耦的通用agent训练框架。
工具整合与 computer use	GUI-R1（2025）、AutoTIR（2025）	前者把 R1 式 RL 带入 GUI agent，后者把“是否调用工具”本身作为可学习策略。
长时web搜索与上下文压缩	WebSailor（2025）、ReSum（2025）、WebSailor-V2（2025）、ACON（2025）	这一组工作把 open-source web agent推向长时搜索、上下文摘要压缩与不确定性驱动推理。

10.1.1 这一年最核心的五个判断

1. **RL已经从“单轮答案优化”走向“长流程决策优化”**。QwenLong-L1、LoongRL、RAGEN、Agent Lightning都在说明，真正困难的问题不再是一个答案 token，而是多轮轨迹、长上下文和跨工具决策。
2. **上下文压缩正式从工程技巧升级为研究主题**。ReSum与ACON之所以重要，不只是“写个摘要”，而是把压缩对后续决策的影响放到agent主链路上来研究。
3. **web agent正在成为开源系统追赶闭源DeepResearch类产品的主战场**。WebSailor与WebSailor-V2的意义在于，它们把高不确定性搜索、数据合成、RL和真实benchmark真正连成了一条线。
4. **开源基础模型开始统一多种推理模式**。Qwen3、Kimi K2、GLM-4.5都在朝同一方向走：不是一个模型只会think，另一个模型只会chat，而是同一个底座按预算与任务动态切换模式。
5. **“训练agent”正在从手工prompt调参转向标准化runtime优化**。Agent Lightning、RAGEN、AutoTIR都表明，未来真正重要的不只是prompt engineering，而是agent runtime的观测、分解、信用分配与策略更新。

10.1.2 这些论文对本书其余章节的直接影响

这些新论文并不是附录式补充，而是会改写本书若干核心判断：

- 对第4节和第5节而言，它们说明RL的重点已从单轮数学答案扩展到长上下文、工具调用与零数据自演化。
- 对第6节而言，它们恰好支持“运行时中心算法”的判断：未来高价值原创算法更值得围绕权限、压缩和多代理工件展开。
- 对第8节而言，它们说明OpenClaw、Claude Code这类runtime不应只被视为工程实现，而应被视为下一阶段agent RL的训练载体。
- 对第10节整体而言，它们把“未来研究方向”从抽象趋势进一步具体化为一条清晰路线：**长上下文推理 + tool-integrated reasoning + runtime-centered RL + long-horizon agent benchmarks**。

10.2 优先级判断标准

未来方向很多，但不应被平均投入。本文后续的优先级排序统一按四个标准展开：**第一，是否直接改善真实部署中的成功率与单位成本；第二，是否具备较清晰的验证器、评测集或过程指标；第三，是否能与现有Transformer、RL和agent runtime堆栈兼容，避免全部重来；第四，是否会与其他能力方向形成复利，而不是只提高单点benchmark分数。**从这个标准看，真正值得优先投入的方向，往往不是最“新奇”的概念，而是那些能同时拉动训练、推理与运行时的技术。

10.3 算法改进

训练算法层面的核心问题，是如何在不牺牲稳定性与可解释性的前提下，提高样本效率、降低训练成本，并把奖励设计从“结果打分”推进到“过程级与系统级信用分配”。

Table 48: 未来三年最值得优先投入的算法方向

方向	为何优先	近期最该做什么	判断是否有效的指标
样本效率更高的RL	真实成本主要被roll-out、验证与失败重采样拖住，而不是单次反向传播	难度感知采样、off-policy重用、基于失败模式的课程学习	单位算力通过率、每个成功轨迹成本、训练稳定性
过程级与系统级奖励	智能体失败常发生在中间步骤、工具顺序和上下文编辑，而不是最终一句答案	把验证器、审批、回滚、计划完成度纳入可分解奖励	奖励投机率、工具误用率、长流程任务完成率
混合训练闭环	单一SFT或单一RL都难覆盖复杂任务，必须让搜索、检索、工具与学习相互补位	建立SFT打底、RL主攻、失败回放再精调的循环	新任务迁移率、冷启动成本、收敛速度

10.3.1 更高效的RL算法

更高效的RL，不应只理解为“换一个损失函数”，而应理解为**减少无效轨迹、减少无效更新、减少无效计算**三件事同时发生。对大模型推理与智能体任务而言，真正昂贵的往往不是梯度本身，而是采样、验证、回放和失败后的重复探索。

1. **优先减少无效采样。** 未来更有价值的方向，是按问题难度、历史失败模式和当前置信度动态分配rollout预算，而不是对所有样本平均花费相同计算。
2. **优先提高数据重用率。** 重要性采样、离线RL、经验回放和轨迹切片等方法的意义，不在于理论上更优雅，而在于让高成本agent轨迹不只被用一次。
3. **优先优化真实系统成本。** 对agent训练来说，延迟、工具调用数、审批次数和失败回滚成本都应进入优化目标，否则训练得到的“高分策略”可能在生产环境里不可接受。

因此，未来更值得投入的不是抽象地追求“比PPO再高一点点分数”，而是建立以**单位成功成本为目标函数**的训练框架：在相同预算下完成更多真实任务，而不是只在小benchmark上提高均值。

10.3.2 更好的奖励设计

奖励设计的真正难点，不是把目标写得更复杂，而是让奖励既**可信、可分解、难以投机**，又能覆盖长流程任务的关键中间决策。未来几年最重要的不是再发明更多花哨名字，而是把奖励工程做成一门接近软件接口设计的discipline。

1. **从结果奖励走向分层奖励。** 最终答案、过程步骤、工具契约、上下文编辑质量和人工审查负担，应该被拆成不同层级而不是混成单一标量。
2. **从静态打分走向可审计打分。** 每一项奖励都应能回答“来源是什么、是否可复核、被模型投机的风险在哪里”，否则奖励越复杂越难调试。
3. **从单目标优化走向受约束优化。** 在真实agent系统里，能力、成本、安全和审批负担必须被共同约束，Pareto前沿比单一高分更贴近真实决策。

一个实用判断标准是：如果奖励项不能映射到一个**清晰的验证器、日志字段、人工审核规则或系统事件**，它大概率还不够成熟，不应被放到主训练目标的核心位置。

10.3.3 混合方法

未来真正高价值的混合方法，不是把很多模块机械叠加，而是让不同模块各自承担**最擅长、最可验证、最便于控制**的那一段工作。比较有前景的三条路线是：

1. **RL + 检索/记忆：**让模型学习何时读历史、读什么、如何压缩与写回，而不是把长上下文单纯交给更大的窗口。
2. **RL + 工具/执行环境：**把工具选择、参数填充、失败恢复与权限申请纳入训练闭环，这是agent能力真正进入真实任务的关键。
3. **RL + 搜索/验证器：**未来高水平推理更像“策略网络 + 验证器 + 搜索控制器”的系统，而不是单模型一次性吐出全部答案。

换言之，混合方法最值得研究的方向，不是让模型“什么都自己做”，而是让它学会**何时该自己算，何时该检索，何时该调用外部机制**。

10.4 推理时计算扩展

推理时计算扩展的研究重点，不再是单纯“让模型想更久”，而是学习何时该多算、何时该早停、何时该调用验证器或外部工具，从而把额外计算真正转化为更高的单位任务收益。

Table 49: 推理时计算扩展的高杠杆投入点

投入点	核心问题	更值得优先做的改进	关键评测指标
计算预算分配	不是每个问题都值得深搜，关键是把算力花在真正困难的样本上	难度预估、置信度校准、阶段式早停	准确率/延迟曲线、单位任务 token 成本
搜索与剪枝	搜索提升性能的同时也容易指数级烧钱	价值引导剪枝、多样性控制、失败回收机制	搜索增益、平均分支因子、边际收益递减点
工具与验证器耦合	额外思考如果不能调用外部检查器，常会演变为自我重复	在关键节点调用测试、检索、执行器与审计器	工具调用命中率、误调用率、长任务成功率

10.4.1 高效搜索算法

高效搜索的目标，不是把树搜得更大，而是在**有限预算内尽早排除低价值路径**。因此，比起一味增加 sample 数，更重要的是三件事：第一，基于置信度和历史错误模式进行早停；第二，在保证多样性的前提下做价值引导剪枝；第三，把并行搜索与投机解码、候选验证结合起来，让额外分支真正产生信息增益。

10.4.2 动态计算分配

动态计算分配应被视为下一代推理模型的核心能力之一，因为它决定了系统能否把预算投到最值得的地方。其一般形式可写为

$$\text{compute}(q) = f(\text{difficulty}(q), \text{confidence}(\pi, q), \text{tool_value}(q)), \quad (314)$$

其中不仅要估计问题难度和当前置信度，还要估计“继续思考”与“调用工具”各自的边际收益。真正成熟的系统，应能对简单问题快速早退，对中等问题进行短链验证，对高价值难题触发更深的搜索和外部检查。

10.4.3 推理优化方法

未来更值得重视的推理优化，不是单独研究“思维链能否再缩短一点”，而是形成一个**压缩 + 复用 + 缓存**的统一框架：把长推理轨迹压缩成可回放的中间状态，复用相似任务的验证结果，并通过推理缓存避免系统在近似问题上反复支付同样的思考成本。

10.5 多模态推理

多模态推理的下一步，不只是把图像或音频接入语言模型，而是让不同模态在统一状态空间中参与推理、验证、计划与行动建模。

Table 50: 多模态推理中最值得优先突破的三类任务

任务类型	为什么重要	真正困难的瓶颈	更合适的评测方式
图表与科学视觉推理	最接近高价值知识工作，且天然需要验证与解释	视觉实体抽取和符号约束之间缺乏稳定接口	过程正确率、数值一致性、最终答案准确率
代码与文档联合推理	连接软件工程、检索、执行与长上下文记忆	代码状态、仓库结构和自然语言说明需被统一建模	单元测试通过率、定位修复时间、回归率
具身与交互式多模态任务	直接连接动作、观测与世界模型，是agent化的前沿形态	环境反馈稀疏、状态不可完全观测、长horizon信用分配难	任务完成率、交互效率、安全违规率

10.5.1 视觉推理

视觉推理最值得投入的，不是泛化到所有图片场景，而是优先攻克**图表、几何与科学图像**这三类高验证性任务。它们的重要共同点是：既需要视觉感知，又存在明确的结构约束、数值一致性和外部验证空间，因此最适合与推理、工具和奖励建模结合起来做闭环优化。

10.5.2 跨模态推理

跨模态推理的真正挑战，不是简单把多种token拼接到同一个上下文里，而是形成**跨模态状态机**：图像提供局部证据，文本提供约束和目标，代码或动作接口改变外部环境，而系统需要对这些状态变化做统一信用分配。未来更高价值的工作，应围绕图文数学、代码与文档联合推理、多模态科学工作流和具身交互任务四条主线展开。

10.6 理论研究

理论研究的价值，不在于事后解释已有现象，而在于给出可迁移的判断标准：什么能力来自规模，什么能力来自结构，什么能力来自训练目标与运行时耦合。

10.6.1 推理能力的本质

未来理论研究若不能帮助工程决策，就只是概念整理。对“推理能力的本质”这一问题，更高价值的研究不是抽象争论“是否真的在推理”，而是去回答三个更可操作的问题：第一，哪些结构与训练条件会稳定诱导出可迁移的多步计算；第二，哪些所谓推理能力本质上只是分布内模式拟合；第三，能力迁移失败时，究竟是搜索失败、记忆失败，还是表征失败。

10.6.2 尺度定律 (Scaling Laws)

下一代尺度定律不应只研究参数量和训练token数，还要同时覆盖**推理时计算、外部工具、可写记忆和验证器**。也就是说，我们最终需要的不是“更大模型曲线”，而是**能力-成本-延迟-风险**的联合标度关系，用来回答不同预算下什么样的系统设计最优。

10.6.3 泛化理论

泛化理论的重点，正在从静态文本任务迁移到长流程agent任务。真正关键的三个问题是：第一，组合泛化何时会被工具接口、世界状态和上下文截断破坏；第二，长度泛化是由架构、训练目标还是记忆机制决定；第三，agent在新环境中的失效，能否被分解为少数可诊断的误差类型。

10.6.4 后Transformer突破：结构判断与技术预测

截至**2026年4月1日**，我未检索到OpenAI官方博客或系统卡中存在Sam Altman关于“超越Transformer”的完整同句原话。较可靠的近期公开信号主要来自两个时间点：第一，**2026年2月3日**《Forbes》专访*Sam Altman Explains Our AI Future*中，Altman明确表示实现AGI更可能依赖“**很多中等规模突破**”，而不是单一奇迹式架构替换；第二，**2026年2月15日**《Stanford Daily》对TreeHacks 2026演讲的报道显示，他把当前阶段描述为技术前沿仍然开放、产品形态仍将持续重构的时期。后续在**2026年3月**出现的二手科技报道把这一判断进一步概括为“可能接近一次超越Transformer的架构跃迁”。因此，更稳妥的解读不是“Transformer会被单点淘汰”，而是：**未来的下一次突破更可能由多项中等规模架构改进叠加而成，并最终把Transformer从唯一骨干降级为混合系统中的一个部件。**

Table 51: 关于“后Transformer突破”的近期公开信号

日期	来源	公开标题	可支持的稳妥判断
2026-02-03	Forbes	<i>Sam Altman Explains Our AI Future</i>	更接近“多项中等规模突破叠加”，而不是单一架构奇迹式替换。
2026-02-15	Stanford Daily	<i>Sam Altman Projects AGI Development, AI Integration at TreeHacks</i>	当前技术前沿与产品形态仍在快速重构，下一代系统可能表现为混合式演进而非单点切代。

这一判断背后有明确的工程动因。标准Transformer虽然在统一建模上极其成功，但其长期瓶颈同样明显：第一，长上下文推理的显存和带宽开销仍然主要受注意力与KV缓存约束；第二，模型缺乏**可写入、可长期保持、可测试时更新**的状态机制；第三，词元级显式链式推理成本较高，难以支撑分钟级、小时级的复杂任务；第四，模型在真实环境中的状态转移更多依赖外部工具和工作区，而不是纯文本上下文。因此，若把下一代骨干网络的目标函数写成

$$\begin{aligned} J_{\text{arch}} = & \alpha \cdot \text{quality} - \beta \cdot \text{latency} - \gamma \cdot \text{memory_cost} \\ & + \delta \cdot \text{persistent_state} + \eta \cdot \text{test_time_adaptation} \\ & + \zeta \cdot \text{action_grounding} \end{aligned}$$

则未来架构并不会只追求语言建模精度，而必须同时优化**长期记忆、测试时适应、行动条件化以及系统级成本**。

基于当前研究脉络，我认为最可能出现的“后Transformer突破”并不是完全抛弃注意力，而是出现一种**异构混合骨干**。Mamba、Titans、RWKV等路线虽然机制不

同，但都在共同推动一个方向：让“全局注意力、持续状态、长期记忆与条件路由”不再由单一算子独占，而是按职责分层协作。其一般形式可写为

$$h_{t+1} = f_{\text{attn}}(h_{\leq t}, M_t) + f_{\text{state}}(s_t, x_t) \\ + f_{\text{mem}}(M_t, x_t) + f_{\text{route}}(x_t)$$

其中 f_{attn} 负责高价值稀疏检索与全局对齐， f_{state} 对应Mamba一类选择性状态空间更新， f_{mem} 对应Titans一类可写长期记忆， f_{route} 负责决定当前信息应进入哪条计算路径。也就是说，注意力不会立刻消失，但它更可能从“唯一计算原语”退居为“高带宽全局检索模块”，而持续状态、外部记忆和模式路由将承担越来越多的主干计算。

我对未来三到五年的第一个具体预测是：**可写记忆与测试时学习会成为比纯注意力更重要的突破轴**。其核心原因在于，长任务失败往往不是因为模型不会下一步，而是因为系统不能把重要信息稳定地写入和再次读取。未来更有潜力的更新式不是单纯扩大窗口，而是显式维护记忆状态：

$$M_{t+1} = M_t + u_t, \quad u_t = g_{\text{write}}(x_t, h_t, \text{surprise}_t), \quad (315)$$

并在需要时通过交叉注意力或键值检索读取

$$r_t = \text{Read}(M_t, q_t). \quad (316)$$

一旦这种记忆写入策略学得足够好，模型就不必把全部历史都保存在KV缓存中，而是可以把“窗口”与“记忆”明确分层。Transformer时代的核心单位是词元间两两交互；后Transformer时代更可能把**记忆写入与读取决策**提升为同等重要的一等对象。

第二个预测是：**显式词元级思维链会逐步让位于潜空间推理与稀疏解码**。当前很多推理模型仍然依赖把中间过程展开成大量可见或不可见词元，这在训练与推理时都成本高昂。更可能的下一步是让模型在潜变量空间中迭代若干步内部状态，再只在必要时解码为语言：

$$z_{k+1} = F_{\theta}(z_k, c_k), \quad y = \text{Decode}(z_K). \quad (317)$$

这类方法的价值不只是节省词元，而是在于把“内部推理状态”与“对外可读文本”解耦。若这一方向成熟，那么“思考更久”将不再等于“输出更多中间词元”，而是等于**做更多潜空间计算、只在边界处暴露结果**。这也更符合闭源前沿模型当前对思维链可见性的控制趋势。

第三个预测是：**真正超越Transformer的系统，很可能是世界模型、动作模型与多模态骨干的结合体，而不是单一语言骨干的升级版**。对于浏览器、机器人、代码执行和科研自动化任务，更核心的问题并不是“下一句文字是什么”，而是“采取动作后环境会怎样变化”。因此下一代基础模型更可能显式学习

$$p(s_{t+1}, o_t, r_t \mid s_t, a_t), \quad (318)$$

其中 s_t 表示内部状态或外部工作区状态， a_t 表示动作， o_t 表示观测， r_t 表示任务反馈。这样的模型更接近“世界模型 + 控制器”的统一体，而不是仅仅把视觉token接进语言模型。若这一趋势成立，则“后Transformer突破”不只是神经网络算子的变化，更是**训练目标从静态序列预测转向行动条件化建模**。

第四个预测是：**产业界最终胜出的不会是“纯替代Transformer”的新架构，而是“混合注意力 + 持续状态 + 长期记忆 + 外部工具”的系统级组合**。这与Altman在2026年2月3日强调的“很多中等规模突破”逻辑一致。也就是说，下一次真正的代际提升更像从LSTM到Transformer那种“单论文切代”，还是更像从单机程序到现

代分布式系统那样的“多组件共同成熟”，我更倾向于后者。届时决定胜负的，将不是某一个模块单独更优，而是以下组合是否同时成立：

Breakthrough = 混合骨干 $\cap$ 持续记忆 $\cap$ 潜空间推理 $\cap$ 行动落地 $\cap$ 可扩展运行时. (319)

Table 52: 后Transformer突破的候选方向与个人判断

方向	核心机制	主观概率	主要障碍
混合注意力 + 状态空间模型	用选择性状态更新承担长序列主干计算，注意力承担高价值全局检索	高	训练稳定性、工具链迁移、与现有LLM堆栈兼容性
可写长期记忆 / 测试时学习	显式记忆写入、读取与测试时参数或状态适应	很高	写入策略难学、记忆污染、信用分配困难
潜空间推理 / 稀疏解码	在潜变量空间迭代推理，只在边界处解码文本	中高	可训练性、可解释性、监督信号设计
世界模型 + 动作建模统一	显式建模动作导致的环境变化与反馈	中高	高质量交互数据难得、环境接口不统一
完全抛弃注意力的单一新骨干	以新算子彻底替代注意力主导的建模方式	中低	生态迁移成本极高，短期内难以全面替换

判断结论 如果必须给出一句最凝练的预测，我的判断是：下一次真正“超越Transformer”的突破，不会表现为“注意力机制已死”，而会表现为“注意力机制不再独自承担全部计算职责”。注意力仍将长期存在，但它会从唯一骨干退化为若干子模块之一；真正决定下一代能力上限的，将是持续状态、长期记忆、潜空间推理、行动建模与运行时协同能否被统一到同一个训练与部署体系中。

10.6.5 更激进的基础设施假说：四种超越现有神经骨干的方向

如果进一步追问“什么才算彻底超出Transformer”，那么仅仅把Mamba、MoE、Titans等视为候选还不够。因为它们虽然在状态压缩、专家路由和外部记忆上提供了重要进展，但仍然共享若干更底层的约束：**同步、层级式的前向计算，以欧氏向量为基本数据类型，以及以矩阵乘法为主要数值核心**。从这一角度看，真正的“后Transformer”不一定只是更好的注意力变体，而可能涉及**数学基元、计算拓扑与执行物理**的同时变更。下面给出四种更激进、但值得严肃思考的基础设施假说。

一、谱相位干涉网络 (Spectral Phase-Interference Networks) 这一方向的核心思想，是把序列交互从空间域中的两两点积，转移到频域中的波相干涉。设token序列 $x_1, \dots, x_N$ 先映射为复频谱表示

$$\hat{x}_i(\omega) = a_i(\omega)e^{j\phi_i(\omega)}, \quad (320)$$

再把整个上下文叠加为总谱场

$$\hat{X}(\omega) = \sum_{i=1}^N \hat{x}_i(\omega). \quad (321)$$

一个查询 q 不再直接对全部token做注意力打分，而是形成频域滤波或相位调制核 $G_q(\omega)$ ，最终读取可写为

$$y_q = \mathcal{F}^{-1}(G_q(\omega)\hat{X}(\omega)). \quad (322)$$

若这一设想成立，则“相关概念被读出”的过程更接近共振与干涉，而不是显式的KV检索。其吸引力在于，FFT类算子具有 $O(N \log N)$ 复杂度，且在现代GPU上已高度优化，因此它为**超长上下文的低功耗处理**提供了一个值得探索的方向。

不过，必须谨慎对待其中最强的表述。频域叠加并不自动等于“无限、无损上下文”；真正困难的是**如何在相位空间中稳定编码组合结构、位置信息与可逆检索**。一旦不同语义在频谱上发生严重混叠，所谓“共振读取”就可能退化为不可控的相位污染。因此，这一方向在数学上富有吸引力，但距离工程上可验证的通用语言骨干仍有显著距离。

二、连续热力学平衡求解器 (Continuous Thermodynamic Equilibrium Solvers) 第二个方向是否定“固定层数前向传播”这一前提，把推理视作连续时间中的能量最小化或平衡求解问题。其数学原型可以写成深平衡模型 (DEQ) 或更一般的隐式系统：

$$z^* = F_\theta(z^*, x), \quad (323)$$

或者等价地写为根求解

$$G_\theta(z^*, x) = 0. \quad (324)$$

如果进一步引入连续时间动力学，则可写成

$$\frac{dz(t)}{dt} = -\nabla_z E_\theta(z(t), x), \quad (325)$$

系统在给定输入 x 后不断演化，直到到达稳态 z^* ，而稳态本身即为推理结果。这类方法的最大价值在于：**计算深度不再由网络层数静态决定，而由问题难度与收敛过程动态决定**。配合隐式微分，可把训练时的显存需求压到接近 $O(1)$ 量级，这对长时间“系统2式推理”尤其有吸引力。

但其挑战同样明确。连续平衡求解需要**可控的收敛性、稳定的数值求解器和良好的硬件利用率**。如果平衡过程容易陷入坏局部极小值，或者求解器迭代本身难以并行加速，那么其理论优点未必能转化为实际吞吐。因此，这一方向非常值得研究，而且比纯猜想更接近现有可训练数学框架；但它离大规模工业级替代仍有一段工程鸿沟。

三、Clifford代数底层 (Clifford Algebraic Substrates) 第三个方向挑战的是神经网络最基本的数据类型。现有模型以实向量 $x \in \mathbb{R}^d$ 为基元，而Clifford代数（几何代数）主张把基本对象升级为**多向量 (multivector)**。若 $X, Y \in \text{Cl}(p, q)$ ，其基本相互作用不再只是点积，而是几何乘积：

$$XY = X \cdot Y + X \wedge Y, \quad (326)$$

其中内积项编码平行关系，外积项编码正交关系与更高维几何结构。这样，尺度、方向、平面、旋转等几何信息不再需要完全靠参数暴力拟合，而可直接写入数学基元本身。

这一方向的理论价值在于，它可能为**空间推理、关系建模与对称性表达**提供更强归纳偏置，从而降低模型为了“学会几何”所付出的参数成本。对于机器人、多模态三维感知和复杂关系建模，这确实是一个有吸引力的假说。但必须看到，Clifford代数的主要难点不在定义，而在于**如何形成大规模可训练、可初始化、可高效实现的网络族**。也就是说，这条路可能带来强归纳偏置，却未必能自动转化为“1B参数等于100B Transformer”的数量级优势。更合理的判断是：它更可能先在**强几何结构任务**中出现局部突破，再逐步验证能否外推到通用语言智能。

四、异步拓扑网格 (Asynchronous Topological Meshes) 第四个方向挑战的是同步全局前向传播本身。设整个系统不再是按层执行的单一计算图，而是一个由大量微单元组成的图结构

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad (327)$$

每个单元 $i \in \mathcal{V}$ 只维护局部状态 h_i ，并异步接收邻居消息：

$$h_i^{(t+1)} = f_i\left(h_i^{(t)}, \{m_{j \rightarrow i}^{(t)} : j \in \mathcal{N}(i)\}\right). \quad (328)$$

此时token或任务状态更像是图中的活动包，而不是必须同步通过所有层的序列。只有局部被激活的路径会消耗能量，其余单元保持静默。

这一方向最吸引人的地方，在于它直接对准了**内存墙与全局同步墙**。如果计算是局部异步传播而不是整层同步矩阵乘法，那么系统也许更适配晶圆级计算、近存计算、局部SRAM乃至分布式边缘设备网络。然而，它同时也是训练最困难的一类方案，因为异步图上的**信用分配、稳定训练、全局一致性与调试可观测性**都远比同步网络更困难。因此，这一方向从系统架构角度非常革命，但从“短期替代现有LLM训练堆栈”的角度看，工程风险也最大。

Table 53: 四种更激进的后Transformer基础设施假说

方向	试图突破的底层约束	近期可行性判断	主要不确定性
谱相位干涉网络	从空间点积转向频域干涉，降低长上下文检索成本	中低	语义混叠、位置信息编码、可逆读写机制
连续热力学平衡求解器	从固定层深转向连续求解与能量稳态	中高	收敛性、求解器开销、工业并行效率
Clifford代数底层	从欧氏向量转向多向量与几何乘积	中	泛化到通用语言任务的有效性、内核实现复杂度
异步拓扑网格	从同步全局前向传播转向局部异步消息传递	中低	训练稳定性、信用分配、系统调试难度

我的判断 如果从**五年内最可能形成产业级影响**的角度排序，我会把“连续平衡求解 + 可写记忆 + 混合骨干”看得比其余三项更近，因为它们与现有深度学习工具链的距离最小；而“谱相位干涉网络”“Clifford代数底层”“异步拓扑网格”更像**具有潜在革命性的长线方向**。它们真正有价值的地方，不是今天就宣布替代Transformer，而

是迫使我们重新思考：**智能计算的基本单元是否一定是欧氏向量、同步层结构与矩阵乘法流水线**。一旦这个问题被重新打开，后Transformer时代的想象空间才算真正展开。

10.7 安全与对齐

随着模型逐步接入工具、浏览器、终端和连接器，安全与对齐不再只是输出过滤问题，而必须上升为训练目标、运行时权限与部署流程的统一设计问题。

Table 54: 安全与对齐中最值得优先建设的能力

能力	为什么优先	最该落地的工程对象	关键指标
推理过程可监控	思考越长、工具越多，越需要知道系统为什么这么做	trace、状态快照、审批日志、失败回放	高风险步骤召回率、复盘效率
受约束执行	真正危险的不是“说错”，而是“做错”	权限层级、审批门、沙箱、回滚机制	安全违规率、误拦截率、恢复时间
对齐漂移控制	长流程任务中目标可能在途中偏离用户真实意图	目标重确认、关键点验收、预算上限	中途偏航率、用户纠正次数、任务完成质量

10.7.1 安全推理

安全推理的核心，不在于要求模型“永远说实话”，而在于建立**可观测、可审计、可约束**的系统结构。未来重点应放在如何验证关键推理节点、如何发现策略性隐藏与目标漂移、以及如何让模型在高风险步骤主动触发更严格的审查模式。

10.7.2 对齐挑战

对齐在复杂推理任务中的难点，已经从“回答是否礼貌或无害”扩展到“执行目标是否始终与用户意图一致”。这意味着未来对齐研究应更多关注长程任务中的目标保持、工具使用边界、上下文污染和多智能体协作中的责任切分，而不是只在静态问答上优化偏好分数。

10.8 开放问题

前述各条研究主线最终都汇聚为一些尚未被解决的基础问题。它们之所以重要，不是因为“足够难”，而是因为它们直接决定推理模型与智能体系统能否从演示阶段走向长期、稳定、低成本的真实部署。

Table 55: 最关键的六个开放问题及其工程含义

问题	研究要回答什么	如果被解决，会直接改变什么
跨域泛化	推理能力能否迁移到新工具、新行业、新数据分布	决定模型能否从专项demo走向通用生产系统
长程规划	数十步到数百步任务中，错误如何积累、恢复和重规划	决定agent能否承担真实工作流而非短任务片段
记忆与世界模型	何时需要显式世界状态，何时只靠上下文与检索即可	决定后Transformer系统该把能力放在窗口、记忆还是环境建模上
安全可验证性	如何发现欺骗、隐藏推理和目标漂移	决定高自主系统能否进入受监管或高风险场景
单位成本最优点	准确率、延迟、工具调用和人工审批之间如何取得最优平衡	决定agent是否具有真正商业可行性
持续学习	模型如何在不遗忘旧能力的前提下持续吸收新技能与新工具	决定系统能否从一次性交付变成长期演化平台

这六个问题并不彼此独立。更准确地说，它们共同指向一个更大的判断：**未来最强的大模型系统，不会只是“会推理的聊天模型”，而会是“能在受约束环境中持续学习、长期记忆、调用工具并控制成本的运行时系统”**。因此，开放问题的答案最终也必须同时在**数学、模型、训练、runtime和部署**五个层面成立，才算真正有价值。

10.9 BitNet: 1比特大语言模型

随着LLM规模不断扩大，模型的部署成本、能耗问题日益严峻。微软研究院提出的BitNet系列为大语言模型带来了革命性的效率提升，通过极端量化技术实现了性能与效率的最优平衡。本节将详细介绍BitNet的技术原理、训练推理流程，以及与传统LLM的全面对比。

10.9.1 BitNet技术概述

发展历程 BitNet系列经历了以下关键发展阶段：

Table 56: BitNet发展时间线

时间	版本	主要贡献
2023.10	BitNet	首次提出1-bit Transformer架构，证明可规模化训练
2024.02	BitNet b1.58	引入三值权重{-1, 0, 1}，性能匹配FP16模型
2024.10	bitnet.cpp	发布官方CPU推理框架，实现高效本地部署
2024.11	BitNet a4.8	引入4-bit激活，进一步优化推理效率
2025.04	BitNet b1.58-2B-4T	官方发布2B参数模型，训练于4T tokens

核心创新：BitLinear层 BitNet的核心是用**BitLinear**层替代传统的nn.Linear层。BitLinear的设计原则是：

1. **权重量化**：将权重量化为极低比特表示
2. **激活量化**：对激活值进行量化以提高计算效率
3. **缩放因子**：使用可学习或统计的缩放因子恢复数值范围

原始BitNet (1-bit) 原始BitNet使用二值权重 $\{-1, +1\}$ ：

$$\tilde{W} = \text{Sign}(W - \mathbb{E}[W]) \quad (329)$$

其中Sign函数定义为：

$$\text{Sign}(x) = \begin{cases} +1, & \text{if } x > 0 \\ -1, & \text{if } x \leq 0 \end{cases} \quad (330)$$

BitNet b1.58 (1.58-bit) BitNet b1.58采用三值权重 $\{-1, 0, +1\}$ ，每个权重理论上需要 $\log_2(3) \approx 1.58$ 比特：

$$\tilde{W} = \text{RoundClip}\left(\frac{W}{\gamma + \epsilon}, -1, 1\right) \quad (331)$$

其中：

- $\gamma = \frac{1}{nm} \sum_{i,j} |W_{ij}|$ 是权重矩阵的平均绝对值
- RoundClip函数将值四舍五入并裁剪到 $[-1, 1]$ 范围
- ϵ 是防止除零的小常数

激活量化 激活采用absmax量化到 b -bit整数：

$$\tilde{X} = \text{Quant}(X) = \text{Clip}\left(\lfloor X \cdot \frac{Q_b}{\|X\|_\infty} \rfloor, -Q_b + \epsilon, Q_b - \epsilon\right) \quad (332)$$

其中 $Q_b = 2^{b-1}$ ，BitNet b1.58默认使用8-bit激活。

BitLinear前向传播 BitLinear层的完整前向传播包含以下步骤：

Input: 输入 $X \in \mathbb{R}^{B \times T \times d_{in}}$ ，权重 $W \in \mathbb{R}^{d_{out} \times d_{in}}$

Layer Normalization: $X' = \text{LN}(X)$

激活量化: $\tilde{X} = \text{Quant}(X')$ ，记录 $\gamma_X = \|X'\|_\infty$

权重量化: $\tilde{W} = \text{RoundClip}(W/\gamma_W, -1, 1)$ ， $\gamma_W = \text{mean}(|W|)$

整数矩阵乘法: $Y = \tilde{X} \cdot \tilde{W}^T$ (无乘法运算!)

反量化: $\hat{Y} = Y \cdot \frac{\gamma_W \gamma_X}{Q_b}$

return $\hat{Y}$

Algorithm 14: BitLinear前向传播

关键优势： 由于权重只有 $\{-1, 0, +1\}$ 三个值，矩阵乘法可以完全用加减法实现：

- $+1 \times x = x$ (直接取值)
- $-1 \times x = -x$ (符号翻转)
- $0 \times x = 0$ (跳过)

10.9.2 训练与推理流程

训练流程 重要说明： BitNet模型必须从头训练，无法通过后训练量化 (Post-Training Quantization, PTQ) 从现有FP16模型转换。

直通估计器 (Straight-Through Estimator, STE) 由于量化函数不可微，训练时使用STE进行梯度传播：

$$\frac{\partial \mathcal{L}}{\partial W} \approx \frac{\partial \mathcal{L}}{\partial \tilde{W}} \quad (333)$$

即梯度直接传递穿过量化操作。

训练时的计算 训练时需要维护两套权重：

- **高精度主权重 (Latent Weights)**：FP32/BF16精度，用于梯度累积
- **量化权重**：训练前向和后向传播时动态量化生成

训练过程如图 9所示：

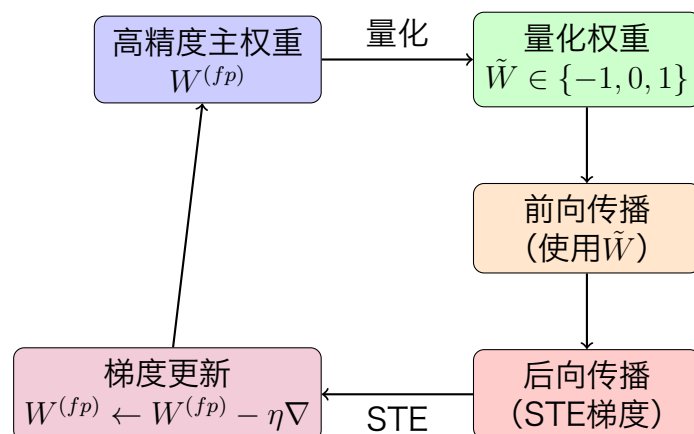


Figure 9: BitNet训练流程

训练硬件要求

- **训练仍需GPU**：由于需要高精度梯度计算和反向传播，训练过程仍需要传统GPU (NVIDIA A100/H100等)
- **内存需求**：训练时内存需求与FP16模型相近，因为需要维护高精度主权重和优化器状态
- **训练速度**：训练速度与FP16模型相当，主要优势在推理阶段

推理流程 推理时的显著优势：

1. **权重存储**：只存储量化后的三值权重，大幅减少内存占用
2. **无乘法计算**：矩阵运算只需加减法
3. **CPU高效**：可在普通CPU上实现高效推理

bitnet.cpp推理框架 微软发布的官方推理框架bitnet.cpp针对1.58-bit模型进行了深度优化：

- **查表法 (LUT)**：将权重编码为查表索引，避免条件分支
- **SIMD优化**：充分利用x86 AVX2/AVX-512和ARM NEON指令
- **内存布局**：优化的权重打包格式，减少内存访问

10.9.3 性能对比分析

Table 57: BitNet与FP16模型内存占用对比

模型规模	FP16内存	BitNet b1.58内存	压缩比	备注
700M	1.4 GB	0.4 GB	3.5x	Embedding未量化
1.3B	2.6 GB	0.7 GB	3.7x	
3B	6 GB	1.5 GB	4.0x	
7B	14 GB	2.5 GB	5.6x	
13B	26 GB	4.0 GB	6.5x	
70B	140 GB	20 GB	7.0x	理论估计值

模型大小与内存消耗 注意：Embedding层和LM Head保持高精度（FP16），因此小模型的压缩比相对较低。随着模型增大，线性层占比增加，压缩比提升。

推理延迟 基于bitnet.cpp的实测性能数据：

Table 58: BitNet b1.58推理加速比（相对于llama.cpp FP16）

模型大小	x86 CPU加速	ARM CPU加速	备注
700M	2.37x	1.37x	Intel/AMD vs Apple M系列
1.3B	2.99x	2.02x	
3B	4.08x	3.25x	
7B	5.02x	4.28x	
70B（理论）	6.17x	5.07x	模型越大加速越明显

100B模型CPU运行 bitnet.cpp的一个标志性成就是能够在**单个CPU**上运行100B参数的BitNet模型，达到**5-7 tokens/秒**的生成速度——接近人类阅读速度。这使得超大模型的本地部署成为可能。

能耗对比 能耗是BitNet最显著的优势之一：

Table 59: BitNet b1.58能耗降低比例

平台	能耗降低	测试条件
x86 CPU	71.9% - 82.2%	Intel/AMD服务器CPU
ARM CPU	55.4% - 70.0%	Apple M2芯片
理论GPU	>90%	消除乘法运算

能耗优势来源

- **消除浮点乘法：**乘法运算能耗远高于加法
- **减少内存访问：**权重压缩降低内存带宽需求
- **缓存友好：**更多权重可驻留在高速缓存中

模型性能对比 BitNet b1.58在语言建模任务上达到与FP16模型相当的性能：

Table 60: BitNet b1.58与FP16基线模型性能对比（相同参数量和训练tokens）

模型	参数量	训练Tokens	PPL	备注
LLaMA FP16	700M	100B	12.33	基线
BitNet b1.58	700M	100B	12.87	略高0.54
LLaMA FP16	1.3B	100B	11.25	基线
BitNet b1.58	1.3B	100B	11.29	几乎相同
LLaMA FP16	3B	100B	10.04	基线
BitNet b1.58	3B	100B	9.91	更好
StableLM	3B	2T	9.22	参考
BitNet b1.58	3B	2T	8.21	显著更好

关键发现：

1. 模型规模越大，BitNet相对于FP16的差距越小，甚至可能反超
2. 大规模训练（更多tokens）时，BitNet表现更好
3. 三值权重可能起到正则化作用，提高泛化能力

零样本任务性能 BitNet b1.58 3B模型在零样本任务上的表现：

Table 61: BitNet b1.58 3B零样本性能

任务	BitNet b1.58	LLaMA FP16
ARC-Easy	66.9	67.0
ARC-Challenge	34.6	34.0
HellaSwag	59.4	59.7
Winogrande	61.2	61.0
PIQA	74.5	74.6
OpenbookQA	34.0	33.2
平均	55.1	54.9

10.9.4 与传统量化方法对比

Table 62: BitNet与后训练量化方法对比

方法	精度损失	是否需重训	压缩比	推理优化
GPTQ (4-bit)	中等	否	4x	需特殊kernel
AWQ (4-bit)	较小	否	4x	需特殊kernel
GGUF Q4_K_M	较小	否	4x	llama.cpp支持
BitNet b1.58	几乎无	是	7-10x	无乘法

后训练量化（PTQ） vs BitNet 关键区别：BitNet不是量化方法，而是从头训练的新架构。

为什么不能从FP16转换？

- 1. 信息容量不匹配：FP16权重包含的信息无法无损压缩到1.58-bit
- 2. 权重分布差异：FP16模型的权重分布不适合直接量化为三值
- 3. 训练适应性：BitNet在训练过程中学会利用三值权重表达能力

实验表明，对FP16模型进行三值量化会导致严重的性能下降（PPL增加数倍），而从头训练的BitNet可以达到与FP16相当甚至更好的性能。

10.9.5 硬件优化与未来展望

当前硬件支持

- CPU: bitnet.cpp支持x86（AVX2/AVX-512）和ARM（NEON）
- GPU: 2025年5月发布官方GPU kernel
- NPU: 计划中，适合1-bit运算的专用加速器

专用硬件前景 BitNet为设计专用AI芯片开辟了新方向：

1. **简化ALU设计**：只需加法器，无需乘法器
2. **降低功耗**：从根本上减少能耗
3. **提高密度**：更简单的计算单元意味着更高的集成度
4. **内存墙突破**：权重压缩缓解内存带宽瓶颈

BitNet与RL训练的结合 BitNet技术与RL训练的结合具有重要意义：

1. **高效RL采样**：推理效率提升使RL采样更快、更便宜
2. **本地自我博弈**：可在边缘设备进行自我博弈训练
3. **大规模探索**：更低的推理成本支持更大规模的策略探索
4. **绿色AI**：显著降低RL训练的碳足迹

注记 10.1 (BitNet的局限性). 当前BitNet的主要限制包括：

1. **必须从头训练**：无法利用现有预训练模型
2. **训练成本不变**：训练阶段仍需传统GPU
3. **生态不完善**：工具链和模型支持有限
4. **超大模型待验证**：100B+规模的实际性能需进一步验证

10.9.6 官方模型资源

微软已开源以下BitNet模型和工具：

- **模型权重**：
 - BitNet-b1.58-2B-4T: <https://huggingface.co/microsoft/BitNet-b1.58-2B-4T>
 - BitNet-b1.58-2B-4T-bf16 (训练权重): <https://huggingface.co/microsoft/bitnet-b1.58-2B-4T-bf16>
- **推理框架**: <https://github.com/microsoft/BitNet>
- **第三方支持**: llama.cpp已支持BitNet模型推理

10.9.7 小结

BitNet代表了LLM效率优化的重要方向：

1. **技术创新**：三值权重实现性能-效率最优平衡
2. **训练特性**：需从头训练，训练时仍需GPU
3. **推理优势**：内存降低7x，能耗降低70-80%，速度提升2-6x
4. **部署革命**：100B模型可在单CPU上运行
5. **未来潜力**：专用硬件可进一步放大优势

11 总结

本节从三个维度收束全文：提炼贯穿各章的核心结论，明确本报告相对现有工作的创新贡献，并将上述结论转化为可直接执行的实践建议和面向未来三至五年的研究路线图。

11.1 核心结论

1. **强化学习仍是提升推理能力最有解释力的一条主线。** 对数学、代码和长链推理任务而言，RL的价值不只是“做偏好对齐”，而是让模型在结果反馈下学习搜索、验证、修正与延迟决策。它的关键优势是能够超出静态人类示范的上限；其代价则是训练稳定性、样本效率和奖励设计的复杂度必须由系统工程来兜住。
2. **奖励不一定越复杂越好，可信信号往往比华丽目标更重要。** 规则可验证任务中的结果奖励之所以有效，不是因为它最精细，而是因为它最稳定、最难被误解。相反，过程奖励模型虽然诱人，但标注、训练和部署成本都更高，也更容易把系统拖入噪声监督。一个反复出现的经验是：先用可靠的弱信号把闭环跑通，再考虑更昂贵的细粒度监督。
3. **GRPO、RLOO等方法的重要性在于把“可训练”变成“可落地”。** 它们并不一定在任何场景都严格优于PPO，但它们降低了critic设计、显存压力和实现复杂度，使更多团队能够把RL从论文技巧真正推进到可复现实验管线。对多数工程团队而言，算法上哪怕只有中等改进，只要训练链路明显变简单，就足以构成一代方法的实际价值。
4. **长上下文问题本质上不是窗口长度问题，而是记忆管理问题。** 真正限制系统能力的往往不是“能不能塞进更多token”，而是“哪些信息必须保留、如何压缩、何时读取、何时丢弃”。因此，KV Cache、压缩记忆、选择性保留、外部记忆和运行时编辑策略，应被视为能力问题的一部分，而不是独立于模型之外的后处理补丁。
5. **推理时计算扩展已经成为与参数规模、训练算力并列的第三扩展轴。** o1、R1及其相关工作共同说明，模型性能越来越取决于是否学会在测试时分配思考预算、调用验证器和使用工具。未来能力提升不会只发生在“训练更多参数”这一个方向上，而会越来越依赖训练时与推理时的联合设计。
6. **下一代高价值问题正在从“文本生成”转向“运行时决策”。** 当模型进入浏览器、shell、代码仓库和企业工作流后，决定系统上限的关键变量变成权限、审批、上下文压缩、失败恢复和多智能体协作。也正因为如此，PACE、TRACE、CRAFT这类提案的意义不在于再给RL多起几个名字，而在于把真实系统里最昂贵的决策对象正式拉进学习框架。

11.2 本报告的主要贡献

本报告除了系统综述现有方法外，还将创新重点明确收缩到**智能体运行时与长流程推理的关键瓶颈**，避免把“提出很多术语”误当成贡献。全文只保留以下三项高价值原创算法提案：

1. **PACE（权限感知约束执行优化）**：把动态权限、审批预算与降级策略纳入受约束运行时优化，训练“何时执行、何时申请、何时放弃”的策略。

2. **TRACE（轨迹后悔感知的上下文编辑）**：把context compaction建模为带未来回报损失的编辑决策，而不是只优化摘要表面质量。
3. **CRAFT（工件图约束下的多智能体协作）**：把多智能体协作从自然语言消息传递推进到结构化artifact graph上的可追踪协作。

这三项算法的共同点在于：它们不再把“更长的思维链”视为唯一创新空间，而是把**权限、长程记忆与工件化协作**视为下一代大模型系统真正需要学习的对象；并且它们都已经被写成了**状态、动作、目标、更新规则与高效实现路径**。更严格地说，它们目前应被理解为**可形式化、可证伪、具有明确问题定义和潜在高价值的原创算法提案**，而不是已经被标准基准充分验证的成熟方法。

11.3 实践建议

对于希望训练推理模型或构建智能体系统的团队，更可执行的建议如下：

1. **先把可验证闭环跑通，再追求复杂奖励**。如果任务存在规则验证器、单元测试、判题器或结构化检查器，就先围绕这些稳定信号构建最小可用RL闭环。很多团队过早投入过程奖励、复杂多目标或评判模型，结果不是能力涨得更快，而是训练调不稳、问题定位更困难。
2. **把数据和评测视为同一件事的两面**。训练集不应只按规模管理，更应按难度分层、失败模式覆盖度、可验证性和污染风险管理。与之对应，评测集必须覆盖同类能力的不同分布，否则训练指标的上升很容易只是对某种模板的过拟合。
3. **训练流程采用“少量SFT打底，RL主攻，必要时再精调”的渐进式策略**。纯RL并非不能成功，但它对初始策略、任务可验证性和采样成本要求更高。对大多数团队来说，先用小规模高质量SFT让模型学会基本格式、工具接口和拒绝明显错误，再进入RL阶段，通常更稳健也更省钱。
4. **监控要覆盖策略分布、系统成本和人工体验三个层面**。除了KL、奖励均值、通过率和长度分布，还应持续记录延迟、显存、token开销、人工审批次数、失败回滚率和工具错误率。只看离线分数，很容易在真实环境中付出不可接受的执行成本。
5. **把失败案例库当成核心资产维护**。真正能推动下一轮改进的，往往不是平均分，而是那些反复出现的失败模式：奖励投机、工具乱用、上下文丢关键信息、错误自信、长链推理中途漂移。建议为失败样本建立结构化标签体系，并让数据构造、奖励设计和系统防护都直接使用这套标签。
6. **资源规划不能只算训练卡时，还要算采样、验证和运行时成本**。RL系统的总成本通常来自“采样器 + 验证器 + 训练器 + 日志与回放基础设施”的联合开销，而不只是反向传播。若预算有限，应优先投资最影响闭环速度的环节，例如更快的验证器、更高吞吐的采样管线或更可靠的失败回放工具。

11.4 90天落地清单

如果团队刚准备启动推理模型或智能体系统，一个更现实的目标不是“90天做出最强模型”，而是“90天建立一个能持续改进的闭环”。下面这张表的意义在于压缩执行顺序，避免团队在最脆弱的时候同时铺开太多战线。

Table 63: 推理模型/智能体系统的90天最小落地清单

时间段	核心动作	必须产出的工件	完成标志
第1 - 2周	选定单一高价值任务，冻结任务边界与成功标准	任务定义文档、最小评测集、失败样例初表	团队对“什么算成功”没有歧义
第3 - 4周	打通采样、验证、记录、回放四件套	统一日志格式、验证器、trace回放脚本、成本面板	任一失败样本都能复现、定位、复盘
第5 - 8周	用小规模SFT + 可验证RL跑第一个闭环	基线模型、奖励配置、对比实验表、失败标签体系	至少一类任务出现稳定且可重复的收益
第9 - 12周	引入受控工具调用、审批门与回滚策略	工具契约、权限分层、审批记录、回滚与告警规则	在线试运行时，新增能力不会显著扩大风险面

这份清单背后的原则很简单：**先让系统可观测、可复盘、可回滚，再让它更强、更自主**。若顺序反过来，团队通常会在“模型偶尔很惊艳”与“系统整体不可控”之间来回摇摆，最后既难以评估真实收益，也很难说服组织持续投入。

11.5 未来三年的执行路线图

如果把全文建议压缩成一条可以直接执行的路线，那么更合理的做法不是“同时做所有事”，而是按基础设施成熟度分阶段推进。下面这个路线图的目的，不是预测唯一正确路径，而是帮助团队避免过早投入高自主度、过晚建设评测与安全基础设施这两类常见错误。

Table 64: 面向推理模型与智能体系统的三阶段执行路线图

阶段	首要目标	必须交付的基础设施	进入下一阶段前应满足的条件
0 - 6个月	跑通最小可验证闭环	任务评测集、失败回放、统一工具接口、基础权限模型	至少有一类任务可稳定复现收益，且失败可定位、可回放
6 - 18个月	从单轮推理走向长流程agent	runtime日志、审批门、回滚机制、长上下文/记忆层、成本监控	长流程任务成功率持续提升，单位成本可控，安全回归通过
18 - 36个月	形成可持续迭代的平台化系统	训练-推理解耦框架、在线评测、数据反馈回流、策略灰度发布	新能力可持续上线，旧能力不明显退化，组织上能稳定复盘与升级

这个路线图对应着一个重要的工程原则：**能力提升的顺序，不应是“先放开自主度，再补安全和评测”，而应是“先把运行时和反馈闭环建稳，再逐步释放模型自由度”**。从长期看，这比追求一次性的benchmark跃升更能形成复利。

11.6 未来发展趋势与研究路线图

最后一章不应只回顾已有结果，还应给出未来几年最值得投入的方向。综合本文对RL训练、长上下文系统、闭源模型工程以及智能体运行时（agent runtime）的分析，可以把未来发展概括为以下七条主线：

1. 从“更大模型”走向“更强系统”

未来竞争不再只是参数量竞争，而是**模型、缓存、外部记忆、工具链和运行时的联合优化**。决定上限的往往不是单一Transformer，而是整套系统是否能在长上下文、复杂任务和在线环境中稳定工作。

2. 从静态对话走向可执行智能体

下一代模型会更深地嵌入浏览器、shell、代码执行器、检索与企业工作流。研究重点将从“生成一个好答案”转向“在权限边界内完成一个真实任务”。这意味着智能体控制层（harness）、审批机制、工具策略、失败恢复与可观测性将成为核心研究对象，而不仅仅是附属工程。

3. 从长上下文走向分层记忆体系

单纯扩大上下文窗口（context window）并不能根治记忆问题。更有前途的方向是形成**窗口记忆 + KV缓存（KV cache） + 压缩记忆 + 外部长期记忆**的分层体系。Titans、MLA、TurboQuant、PagedAttention代表的正是这一方向：不是无差别保留全部历史，而是学习“什么值得存、存成什么形式、何时读取”。

4. 从训练时尺度扩展走向训练-推理协同扩展

未来的关键问题不再是“参数和训练算力是否足够”，而是如何在训练时学习**推理时计算分配策略**。也就是说，模型既要学会解题，还要学会判断何时该多想、何时该调用工具、何时该早停。训练时RL与推理时搜索、验证器和工具策略会越来越紧密地耦合。

5. 从粗粒度奖励走向过程级、结构级和系统级奖励

仅靠最终答案奖励仍然有效，但对于复杂智能体任务，奖励会向三层扩展：

- **过程级奖励**：步骤正确性、验证一致性、反思有效性；
- **结构级奖励**：工具调用顺序、计划分解质量、上下文压缩质量；
- **系统级奖励**：延迟、成本、失败率、安全风险、人工接管率。

这会推动RL从“优化文本输出”升级为“优化完整决策过程”。

6. 从能力评测走向评测驱动开发（Evaluation-Driven Development）

顶尖团队已经证明，真正可持续的进步依赖**规范化评测、灰度发布、在线监控与事后复盘**。未来研究不应只发布基准分数，更应发布失败类型、环境分布、风险边界、回滚策略与修复周期。对开源团队而言，最具杠杆的投入之一，就是建立任务集、回归集、智能体轨迹回放集与安全红队集。

7. 从通用高成本推理走向高效可部署推理

BitNet、量化KV缓存（KV cache）、MoE、Mamba、FlashAttention、投机解码等路线共同指向一个现实：真正影响产业落地的，是**单位正确答案的成本**，

而不是某个离线基准上的极限分数。未来研究会更强调每秒词元数（tokens/s）、单任务能耗（joules/task）、单请求内存占用（memory/request）、工具调用成功率（tool-success-rate）等系统指标。

Table 65: 面向未来三年的关键研究问题

方向	核心研究问题	实际开发意义
长记忆系统	如何联合优化窗口注意力、外部记忆和缓存压缩？	决定智能体是否能完成长流程任务而不遗忘上下文
推理时计算	如何学习何时搜索、何时验证、何时早停？	直接影响成本、延迟和正确率的平衡
智能体控制层	如何形式化工具策略、审批策略和失败恢复？	决定系统能否安全上线，而不是只在演示系统中工作
过程奖励	如何用低成本方式获得步骤级信用分配？	提高复杂任务中的稳定性和可解释性
评测工程	如何构建覆盖真实生产故障的智能体评测集？	减少回归事故和线上不可控行为
效率算法	如何在低比特、稀疏化和新硬件上保持推理质量？	决定是否能以合理成本大规模部署
安全对齐	如何验证推理与计划过程本身不发生欺骗与越权？	决定高能力智能体能否进入高风险工作流

未来最值得投入的研究问题

对研究者和工程团队的直接建议 如果只能优先做少数几件事，建议按以下顺序投入：

- 1. 先建立**可复现评测和回归集**，再追求更强模型。
- 2. 先把**智能体控制层（harness）、权限边界和失败恢复**做好，再扩大工具能力。
- 3. 先解决**长上下文的缓存和记忆成本**，再盲目扩窗口。
- 4. 先优化**单位任务成本**，再追逐少量榜单提升。
- 5. 先把**真实任务完成率**做稳，再追求更长、更复杂的思维链。

简言之，未来真正有决定性的方向不是某一个孤立算法，而是**RL训练、长记忆系统、智能体控制层（harness）、评测工程与安全部署**五者的合流。谁能最早把这五件事做成统一闭环，谁就更可能在下一阶段取得优势。

参考文献

编排说明 为便于学术检索与主题回溯，本报告将参考资料按研究脉络重组为五个专题：强化学习与偏好优化、推理与自我改进、多模态Transformer与系统基础设施、

智能体工程与治理文档，以及高效训练与低比特推理。编号在各组之间保持连续，便于全文统一引用。

一、强化学习、偏好优化与奖励建模

1. DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948, 2025.
2. DeepSeek-AI. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300, 2024.
3. Lightman, H., et al. Let's Verify Step by Step. arXiv:2305.20050, 2023.
4. Bai, Y., et al. Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073, 2022.
5. Ouyang, L., et al. Training Language Models to Follow Instructions with Human Feedback. NeurIPS, 2022.
6. OpenAI. Learning to Reason with LLMs. OpenAI Blog, 2024.
7. Schulman, J., et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.
8. Rafailov, R., et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS, 2023.
9. Lee, H., et al. RLAIFF: Scaling Reinforcement Learning from Human Feedback with AI Feedback. arXiv:2309.00267, 2023.
10. Yuan, W., et al. Self-Rewarding Language Models. arXiv:2401.10020, 2024.
11. Gao, L., et al. Scaling Laws for Reward Model Overoptimization. ICML, 2023.
12. Zuo, Y., et al. TTRL: Test-Time Reinforcement Learning. arXiv:2504.16084, 2025.
13. Zhao, A., et al. Absolute Zero: Reinforced Self-play Reasoning with Zero Data. arXiv:2505.03335, 2025.
14. Wan, F., et al. QwenLong-L1: Towards Long-Context Large Reasoning Models with Reinforcement Learning. arXiv:2505.17667, 2025.
15. Liu, M., et al. ProRL: Prolonged Reinforcement Learning Expands Reasoning Boundaries in Large Language Models. arXiv:2505.24864, 2025.
16. Wang, S., et al. LoongRL: Reinforcement Learning for Advanced Reasoning over Long Contexts. arXiv:2510.19363, 2025.

二、推理、自我改进与评测基准

1. Wei, J., et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS, 2022.
2. Wang, X., et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023.

3. Zelikman, E., et al. STaR: Self-Taught Reasoner. NeurIPS, 2022.
4. Zelikman, E., et al. Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking. arXiv:2403.09629, 2024.
5. Chen, X., et al. Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models. arXiv:2401.01335, 2024.
6. Kojima, T., et al. Large Language Models are Zero-Shot Reasoners. NeurIPS, 2022.
7. Yao, S., et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS, 2023.
8. Silver, D., et al. Mastering the Game of Go with Deep Neural Networks and Tree Search. Nature, 2016.
9. Shinn, N., et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023.
10. Wang, P., et al. Self-Instruct: Aligning Language Models with Self-Generated Instructions. ACL, 2023.
11. Havrilla, A., et al. Teaching Large Language Models to Self-Debug. arXiv:2304.05128, 2023.
12. Madaan, A., et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS, 2023.
13. Hendrycks, D., et al. Measuring Mathematical Problem Solving With the MATH Dataset. NeurIPS, 2021.
14. Cobbe, K., et al. GSM8K: Training Verifiers to Solve Math Word Problems. arXiv:2110.14168, 2021.
15. Zhou, D., et al. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. ICLR, 2023.
16. Chen, M., et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021.
17. Austin, J., et al. Program Synthesis with Large Language Models. arXiv:2108.07732, 2021.

三、多模态Transformer、长上下文与系统基础设施

1. Vaswani, A., et al. Attention Is All You Need. NeurIPS, 2017.
2. Touvron, H., et al. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971, 2023.
3. Google. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805, 2023.
4. Radford, A., et al. Learning Transferable Visual Models From Natural Language Supervision. ICML, 2021.

5. Alayrac, J.-B., et al. Flamingo: a Visual Language Model for Few-Shot Learning. NeurIPS, 2022.
6. Li, J., Li, D., Savarese, S., and Hoi, S. C. H. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. ICML, 2023.
7. Kaplan, J., et al. Scaling Laws for Neural Language Models. arXiv:2001.08361, 2020.
8. Hoffmann, J., et al. Training Compute-Optimal Large Language Models. NeurIPS, 2022.
9. Bai, S., Kolter, J. Z., and Koltun, V. Deep Equilibrium Models. NeurIPS, 2019.
10. Gu, A., Goel, K., and Re, C. Efficiently Modeling Long Sequences with Structured State Spaces. ICLR, 2022.
11. Peng, B., Alcaide, E., Anthony, Q., et al. RWKV: Reinventing RNNs for the Transformer Era. Findings of ACL: EMNLP, 2023.
12. He, R., Ravula, A., Kanagal, B., and Ainslie, J. RealFormer: Transformer Likes Residual Attention. Findings of ACL-IJCNLP, 2021.
13. Behrouz, A., Zhong, P., Mirrokni, V., et al. Titans: Learning to Memorize at Test Time. NeurIPS, 2025.
14. Zandieh, A., Mirrokni, V., et al. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. arXiv:2504.19874, 2025.
15. Gu, A., and Dao, T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752, 2024.
16. Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Re, C. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS, 2022.
17. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP, 2023.
18. DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434, 2024.
19. Zimerman, I., Ali, A. A., and Wolf, L. Explaining Modern Gated-Linear RNNs via a Unified Implicit Attention Formulation. ICLR, 2025.
20. Yang, A., et al. Qwen3 Technical Report. arXiv:2505.09388, 2025.
21. Kimi Team. Kimi K2: Open Agentic Intelligence. arXiv:2507.20534, 2025.
22. Z.ai Team. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models. arXiv:2508.06471, 2025.

四、智能体工程、系统卡与治理文档

1. Anthropic. Claude 3 Model Card. Anthropic, 2024.
2. Anthropic. Building Effective Agents. Anthropic Engineering Blog, December 19, 2024.
3. Anthropic. Model Context Protocol Documentation. Anthropic Developer Documentation, 2025.
4. Anthropic. Claude Code Overview. Anthropic Developer Documentation, accessed April 1, 2026.
5. Anthropic. Claude Code Hooks. Anthropic Developer Documentation, accessed April 1, 2026.
6. Nieva, R., and Tong, A. Sam Altman Explains Our AI Future. Forbes, February 3, 2026.
7. Jester, K. Sam Altman Projects AGI Development, AI Integration at TreeHacks. The Stanford Daily, February 15, 2026.
8. OpenAI. Updating Our Preparedness Framework. OpenAI, 2025.
9. OpenAI. Operator System Card. OpenAI, 2025.
10. OpenAI. ChatGPT agent System Card. OpenAI, 2025.
11. OpenAI. A Practical Guide to Building Agents. OpenAI, 2025.
12. Google DeepMind. Gemini. Google DeepMind Technologies Page, 2025.
13. Google DeepMind. Project Mariner. Google DeepMind, 2025.
14. Google DeepMind. AlphaEvolve: A Gemini-powered Coding Agent for Designing Advanced Algorithms. Google DeepMind Blog, 2025.
15. Gullí, A. Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems. Springer Nature Switzerland, 2025.
16. Google Cloud. Choose a Design Pattern for Your Agentic AI System. Cloud Architecture Center, last reviewed October 8, 2025.
17. OpenClaw Documentation. Architecture, Agent Loop, Sessions, Browser Tooling, Multi-Agent, and Exec Approvals. docs.openclaw.ai, accessed April 7, 2026.
18. Luo, R., et al. GUI-R1: A Generalist R1-Style Vision-Language Action Model For GUI Agents. arXiv:2504.10458, 2025.
19. Wang, Z., et al. RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning. arXiv:2504.20073, 2025.
20. Wei, Y., et al. AutoTIR: Autonomous Tools Integrated Reasoning via Reinforcement Learning. arXiv:2507.21836, 2025.
21. Luo, X., et al. Agent Lightning: Train ANY AI Agents with Reinforcement Learning. arXiv:2508.03680, 2025.
22. Li, K., et al. WebSailor: Navigating Super-human Reasoning for Web Agent. arXiv:2507.02592, 2025.

23. Wu, X., et al. ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization. arXiv:2509.13313, 2025.
24. Li, K., et al. WebSailor-V2: Bridging the Chasm to Proprietary Agents via Synthetic Data and Scalable Reinforcement Learning. arXiv:2509.13305, 2025.
25. Kang, M., et al. ACON: Optimizing Context Compression for Long-horizon LLM Agents. arXiv:2510.00615, 2025.
26. instructkr. claw-code: Clean-room Python Reconstruction of a Claude Code-like Agent Runtime. GitHub repository, accessed April 1, 2026.

五、高效训练、模型压缩与低比特推理

1. Hu, E., et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022.
2. Dettmers, T., et al. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS, 2023.
3. Rajbhandari, S., et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. SC, 2020.
4. Wang, H., et al. BitNet: Scaling 1-bit Transformers for Large Language Models. arXiv:2310.11453, 2023.
5. Ma, S., et al. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764, 2024.
6. Wang, J., et al. 1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1.58 Inference on CPUs. arXiv:2410.16144, 2024.
7. Ma, S., et al. BitNet a4.8: 4-bit Activations for 1-bit LLMs. arXiv:2411.04965, 2024.
8. Microsoft. Bitnet.cpp: Efficient Edge Inference for Ternary LLMs. arXiv:2502.11880, 2025.

附录A：代码示例

A.1 GRPO实现伪代码

Listing 1: GRPO Training Loop

```
import torch
import torch.nn.functional as F

class GRPOTrainer:
    def __init__(self, model, ref_model, tokenizer, config):
        self.model = model
        self.ref_model = ref_model
        self.tokenizer = tokenizer
        self.config = config
```

```
def compute_advantages(self, rewards):
    """Compute advantages using group normalization"""
    mean = rewards.mean()
    std = rewards.std() + 1e-8
    advantages = (rewards - mean) / std
    return advantages

def compute_kl_penalty(self, logits, ref_logits):
    """Compute KL divergence penalty"""
    log_probs = F.log_softmax(logits, dim=-1)
    ref_log_probs = F.log_softmax(ref_logits, dim=-1)
    kl = (torch.exp(log_probs) * (log_probs - ref_log_probs)).sum(-1)
    return kl.mean()

def train_step(self, questions, group_size=16):
    """Single training step"""
    all_responses = []
    all_rewards = []

    # Sampling phase
    for q in questions:
        responses = self.sample_group(q, group_size)
        rewards = self.compute_rewards(q, responses)
        all_responses.extend(responses)
        all_rewards.extend(rewards)

    # Compute advantages
    rewards_tensor = torch.tensor(all_rewards)
    advantages = self.compute_advantages(rewards_tensor)

    # Policy update
    for epoch in range(self.config.num_epochs):
        loss = self.compute_loss(
            questions, all_responses, advantages
        )
        loss.backward()
        self.optimizer.step()
        self.optimizer.zero_grad()

    return loss.item()
```

A.2 奖励函数实现

Listing 2: Math Problem Reward Function

```
import re
from sympy import simplify, sympify

def extract_answer(response):
```

```
"""Extract answer from response"""
# Try to match \boxed{} format
match = re.search(r'\boxed{([^\}]+)}', response)
if match:
    return match.group(1)

# Try to match "The answer is" format
match = re.search(r'answer\s*(?:is|:)\s*([^\s]+)', response)
if match:
    return match.group(1)

return None

def compare_answers(pred, gold, tolerance=1e-6):
    """Compare if answers are correct"""
    # Try numerical comparison
    try:
        pred_val = float(pred)
        gold_val = float(gold)
        return abs(pred_val - gold_val) < tolerance
    except:
        pass

    # Try symbolic comparison
    try:
        pred_expr = sympify(pred)
        gold_expr = sympify(gold)
        return simplify(pred_expr - gold_expr) == 0
    except:
        pass

    # String comparison
    return pred.strip() == gold.strip()

def compute_math_reward(question, response, gold_answer):
    """Compute math problem reward"""
    pred_answer = extract_answer(response)

    if pred_answer is None:
        return -0.5 # Format error penalty

    if compare_answers(pred_answer, gold_answer):
        return 1.0 # Correct
    else:
        return 0.0 # Incorrect
```

附录B：超参数详细说明

Table 66: 完整超参数配置表

参数	典型值	说明
模型相关		
max_length	4096-32768	最大生成长度
temperature	0.7-1.0	采样温度
top_p	0.9-0.95	Nucleus采样参数
GRPO相关		
group_size	16-64	每个问题采样的回答数
clip_epsilon	0.1-0.3	PPO裁剪参数
kl_coef	0.001-0.1	KL惩罚系数
训练相关		
learning_rate	1e-7-1e-5	学习率
batch_size	256-1024	批大小
num_epochs	1-4	每批数据训练轮数
warmup_steps	100-500	预热步数
grad_clip	1.0	梯度裁剪阈值
优化器相关		
optimizer	AdamW	优化器类型
weight_decay	0.01-0.1	权重衰减
beta1	0.9	Adam beta1
beta2	0.95-0.999	Adam beta2

附录C：算法复杂度分析

Table 67: 各算法复杂度对比

算法	时间复杂度	空间复杂度	采样需求
PPO	$O(NK \cdot T \cdot M)$	$O(M + M')$	在线
GRPO	$O(NGK \cdot T \cdot M)$	$O(M)$	在线
DPO	$O(N \cdot T \cdot M)$	$O(M)$	离线
RLAIF	$O(NK \cdot T \cdot (M + M_j))$	$O(M + M_j)$	在线

其中：

- N ：批大小
- K ：更新轮数
- T ：序列长度
- M ：策略模型参数量

- M' : 值网络参数量
- M_j : 评判模型参数量
- G : 组大小 (GRPO)

附录D：常见问题解答 (FAQ)

Q1: GRPO和PPO哪个更好?

A: 取决于具体场景。GRPO实现简单、内存效率高, 适合资源受限或快速迭代场景。PPO理论基础更扎实, 在有足够资源时可能表现更稳定。建议先尝试GRPO, 如有问题再考虑PPO。

Q2: 需要多少数据才能训练出推理能力?

A: 根据DeepSeek-R1的经验, 冷启动SFT只需要几千条高质量数据, RL阶段使用的问题集可以更大(数万到数十万)。关键是数据质量和答案验证的准确性。

Q3: 如何判断训练是否正常?

A: 关注以下指标:

- 奖励应该稳步上升 (但不应该突然跳跃)
- KL散度保持在合理范围 (通常 < 10)
- 生成内容质量人工抽检
- 在验证集上的准确率

Q4: 小模型也能训练推理能力吗?

A: 可以, 但效果与模型大小相关。DeepSeek的蒸馏实验表明, 7B模型也能获得相当的推理能力。小模型可能需要更长的训练和更高质量的数据。

Q5: 如何避免reward hacking?

A:

- 使用多个独立的奖励信号
- KL散度约束
- 定期人工审核高奖励样本
- 对抗性测试
- 设置合理的奖励上限

Q6: RL训练需要多少计算资源?

A: 取决于模型大小和训练规模。7B模型的RL训练通常需要8-16张A100 GPU, 671B模型可能需要数百张。采样是主要瓶颈, 可以通过并行化加速。

Q7: 可以在消费级GPU上训练吗?

A: 通过LoRA/QLoRA等参数高效方法, 可以在单张24GB显存GPU上对7B模型进行RL训练, 但速度会较慢。建议至少使用40GB显存的GPU以获得合理的训练效率。

Q8: BitNet模型可以通过量化现有FP16模型得到吗?

A: 不可以。BitNet模型必须从头训练, 无法通过后训练量化 (PTQ) 从现有FP16/BF16模型转换得到。这是因为: (1) FP16权重包含的信息无法无损压缩到

1.58-bit; (2) FP16模型的权重分布不适合直接量化为三值; (3) BitNet在训练过程中学会利用三值权重的表达能力。实验表明, 对FP16模型进行三值量化会导致严重的性能下降。

Q9: BitNet训练需要什么硬件? 推理呢?

A: 训练时, BitNet仍需要传统GPU (如NVIDIA A100/H100), 因为需要维护高精度主权重和进行梯度计算, 内存需求与FP16模型相近。但推理时, BitNet可以在普通CPU上高效运行, 内存占用降低7倍, 能耗降低70-80%, 100B参数模型可在单CPU上以人类阅读速度运行。

附录E: 术语表

Table 68: 常用术语与缩略语对照

术语	定义
RLHF	Reinforcement Learning from Human Feedback, 基于人类反馈的强化学习
RLAIF	RL from AI Feedback, 基于AI反馈的强化学习
PPO	Proximal Policy Optimization, 近端策略优化
GRPO	Group Relative Policy Optimization, 组相对策略优化
DPO	Direct Preference Optimization, 直接偏好优化
PRM	Process Reward Model, 过程奖励模型
ORM	Outcome Reward Model, 结果奖励模型
CoT	Chain-of-Thought, 思维链
GAE	Generalized Advantage Estimation, 广义优势估计
KL	Kullback-Leibler散度
SFT	Supervised Fine-Tuning, 监督微调
MDP	Markov Decision Process, 马尔可夫决策过程
BitNet	1比特/1.58比特大语言模型架构, 使用三值权重{-1, 0, 1}
BitLinear	BitNet中替代nn.Linear的量化线性层
STE	Straight-Through Estimator, 直通估计器, 用于量化训练
PTQ	Post-Training Quantization, 后训练量化